MACHINE LEARNING / MODULE 1

Supervised learning foundations

Regression, classification and honest evaluation for energy data

Full day · 09:00–17:00

6 parts · 3 labs

Python · scikit-learn

Machine learning for energy applications

Today's plan

09:00	Welcome and framing supervised problems	Lecture
09:30	Data, EDA and preprocessing	Lecture
10:10	Break	Break
10:25	Splits, cross-validation and leakage	Lecture
11:00	Linear models and regularization	Lecture
11:50	Lab 1: regression and regularization	Lab
12:40	Lunch	Lunch
13:30	Classification models	Lecture
14:30	Lab 2: classifiers and a facies split design	Lab
15:25	Break	Break
15:40	Evaluating and selecting models	Lecture
16:15	Lab 3: a tuned pipeline and a locked test	Lab
16:45	Recap and exit check	Discussion

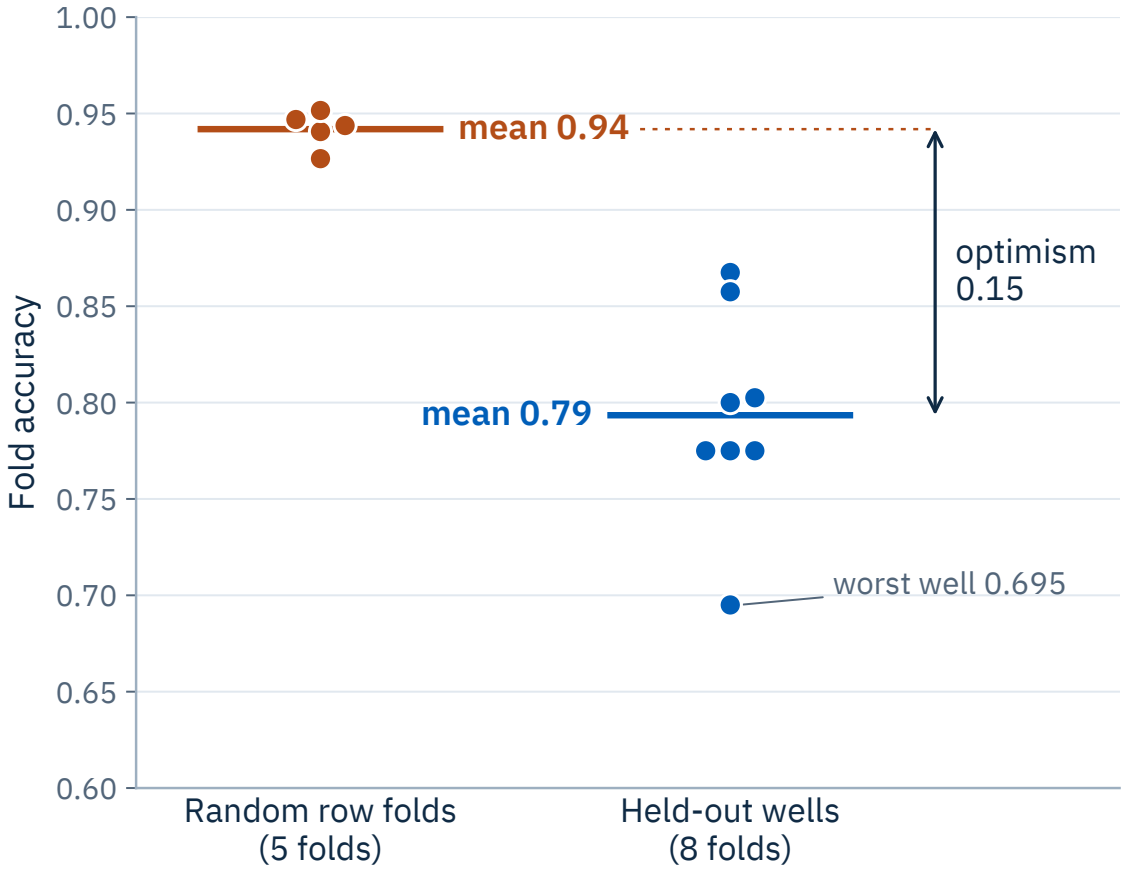
What you will be able to do

By the end of the day you will be able to:

1. Write a prediction contract: target, unit, prediction time, inputs and metric.
2. Audit energy data for null codes, unit mixes, depth shifts and patterned gaps.
3. Choose a split and cross-validation scheme that matches deployment.
4. Fit and regularize linear models and diagnose them with residuals and learning curves.
5. Explain the boundaries of logistic regression, KNN, SVM and decision trees.
6. Evaluate classifiers per class and tune them inside grouped cross-validation.

Same model, same wells, two very different scores

- A 1-nearest-neighbour facies model: 8 synthetic wells, 5 logs
- Random row folds: mean accuracy 0.94
- Held-out wells: mean 0.79; the worst well 0.695
- Which number belongs in the report to the asset team?



Computed on a synthetic 8-well dataset (seed 7); standardized 1-NN classifier. Not field performance.

PART 1 OF 6

01

Framing supervised problems

A model is only as useful as the question it answers.

- Prediction contracts
- Task types and shared vocabulary
- The end-to-end workflow and the energy landscape
- Baselines that make improvement measurable

PRACTICE

3 min · Pairs

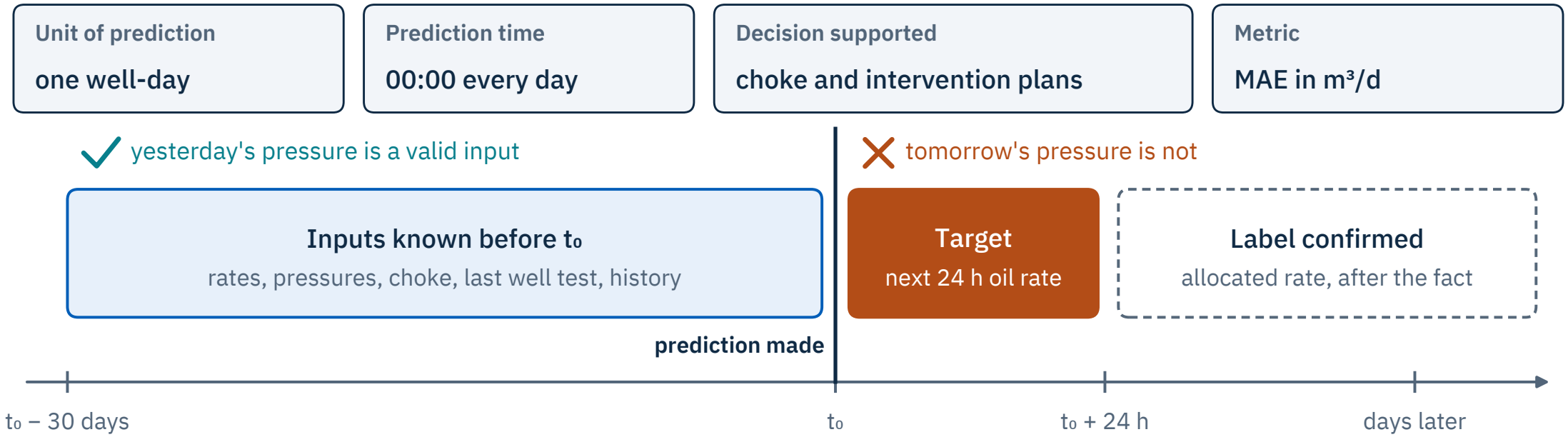
A prediction needs a contract

An operator wants to predict poor production.

1. What exactly is the target, and over what future interval?
2. Is one example a depth sample, a well, or a well-day?
3. Which measurements exist at the moment of prediction?

A contract fixes target, timing, inputs and action

One worked answer: next-day oil rate per well, predicted at midnight.



Schematic prediction contract. The same template applies to any energy forecast or classification.

Supervised learning learns from labelled examples

Regression

- Numeric target
- Porosity, next-day rate, hourly load

Classification

- Class label
- Facies, trip or no trip, fault type

Unsupervised

- No target: structure only
- Groups of production patterns (Modules 3–4)

Reinforcement

- Actions and delayed rewards
- Control policies; background only

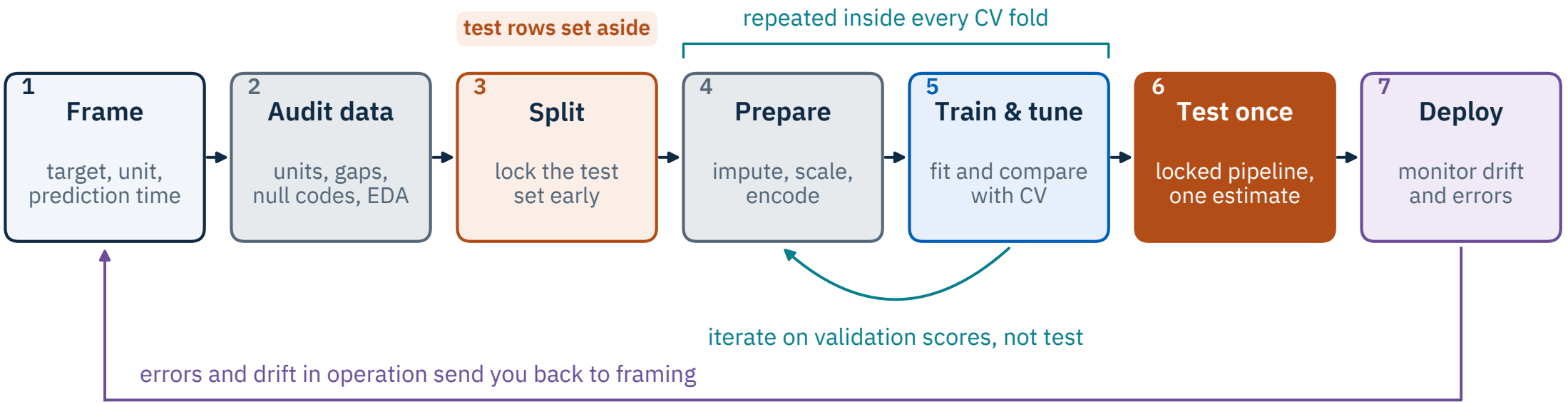
◆ Supervised means every training row carries a known answer: a number or a class.

The vocabulary we will use all week

Term	Meaning	Energy example
Feature	An input known at prediction time	GR and RHOB at one depth
Target	The value or label to predict	Core porosity, facies
Observation / group	One row / rows that belong together	Depth sample / well
Training, validation, test	Fit / choose / check once	6, 2 and 2 wells
Baseline	The simplest defensible reference	Training mean, persistence
Leakage	Information the model would not have	Tomorrow's pressure as input

The workflow is a loop with the test set locked early

Steps 4–5 repeat inside cross-validation; step 6 happens once per locked pipeline.



Schematic supervised-learning workflow. Colours: navy framing, grey data steps, blue model fitting, orange test data, purple operation.

Supervised problems span the energy value chain

Subsurface	Drilling	Production	Facilities	Power
Facies from logs — per depth sample — C	Rate of penetration — per metre drilled — R	Next-day oil rate — per well-day — R	Corrosion rate — per pipe segment — R	Day-ahead load — per hour — R
Porosity from logs — per depth sample — R	Stuck-pipe warning — per rig hour — C	ESP trip in 7 days — per pump-day — C	Leak-alarm triage — per alarm — C	Wind-farm output — per 10 minutes — R
Missing sonic log — per depth sample — R	Lithology at the bit — per depth sample — C	Gas-lock detection — per time window — C	Compressor fault — per trip event — C	Transformer fault — per gas-analysis sample — C

R regression: numeric target C classification: class label

Schematic overview of typical problem formulations; examples only, no performance implied. R = regression, C = classification.

Baselines make improvement measurable

Regression

- Training-target mean or median
- A physics or domain formula

Time series

- **Persistence**: tomorrow equals today
- Seasonal naive: same hour last week

Classification

- Majority class (`DummyClassifier`)
- A simple logistic regression

A model that cannot beat persistence on the same rows and metric has not earned its complexity.

PRACTICE

5 min · Pairs

Frame five energy problems

Problem	Task type	One row is	Baseline
Oil rate next month	?	?	?
ESP trip within 7 days	?	?	?
Facies from logs	?	?	?
Day-ahead hourly load	?	?	?
Corrosion rate by segment	?	?	?

1. Fill the table, then name one input for each problem that would leak.

PART 2 OF 6

02

Data, EDA and preprocessing

Look at the data before a model does.

- Data shapes and a well-log display
- Distributions, correlation and missingness
- Quality traps: null codes, unit mixes, depth shifts
- Scaling, encoding and imputation

Energy data come in four shapes

Tabular

- One row per well, asset or event
- Completion tables, work orders

Depth-indexed logs

- A row often every 0.15 m
- Adjacent rows are near-duplicates

Time series

- One row per timestamp per asset
- SCADA, ESP and meter streams

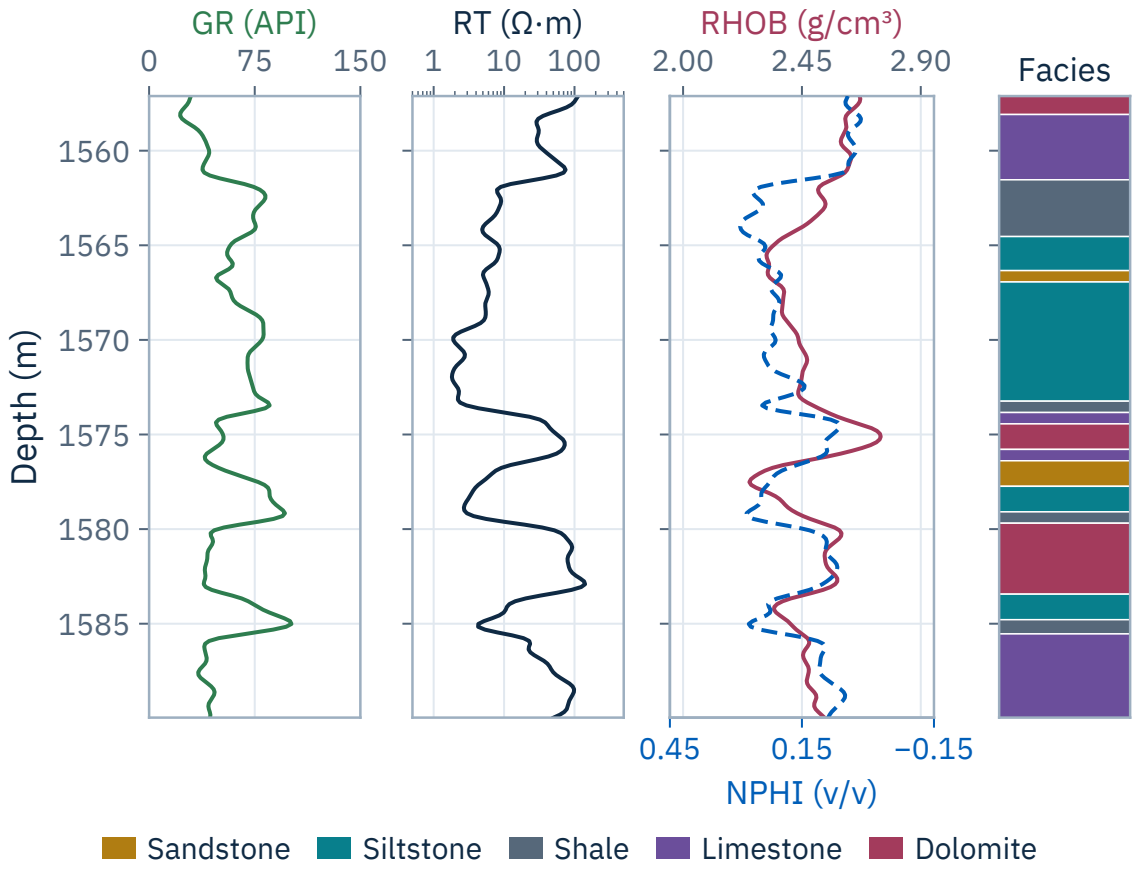
Images and grids

- Pixels with spatial structure
- Seismic sections, core photos (NN Day 4)

◆ The shape decides the split: group rows by well or asset, and respect time order.

A well log is a depth-indexed table

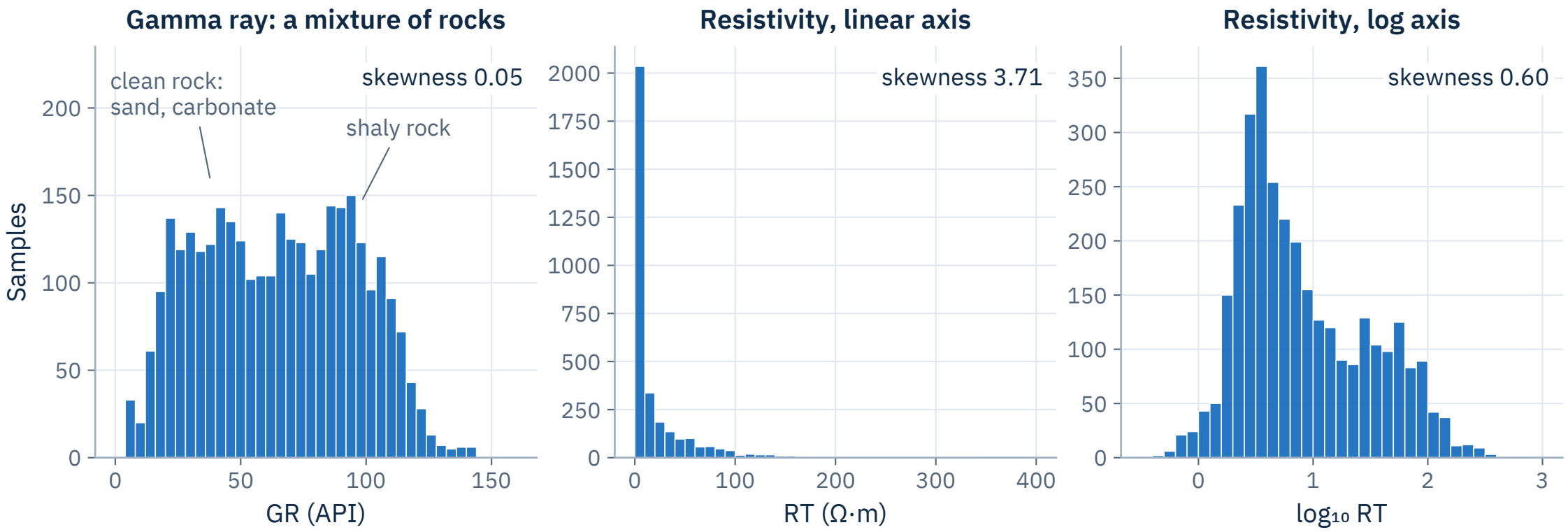
- Each depth sample is one row; each curve is a feature
- Facies from core or interpretation is the target
- Resistivity spans decades, so plot and model $\log_{10}$ RT
- Density and neutron both respond to porosity



Synthetic well W2 (seed 7): 33 m at 0.15 m sampling, generated from facies-dependent distributions. Not a real well.

Distributions reveal mixtures and skew

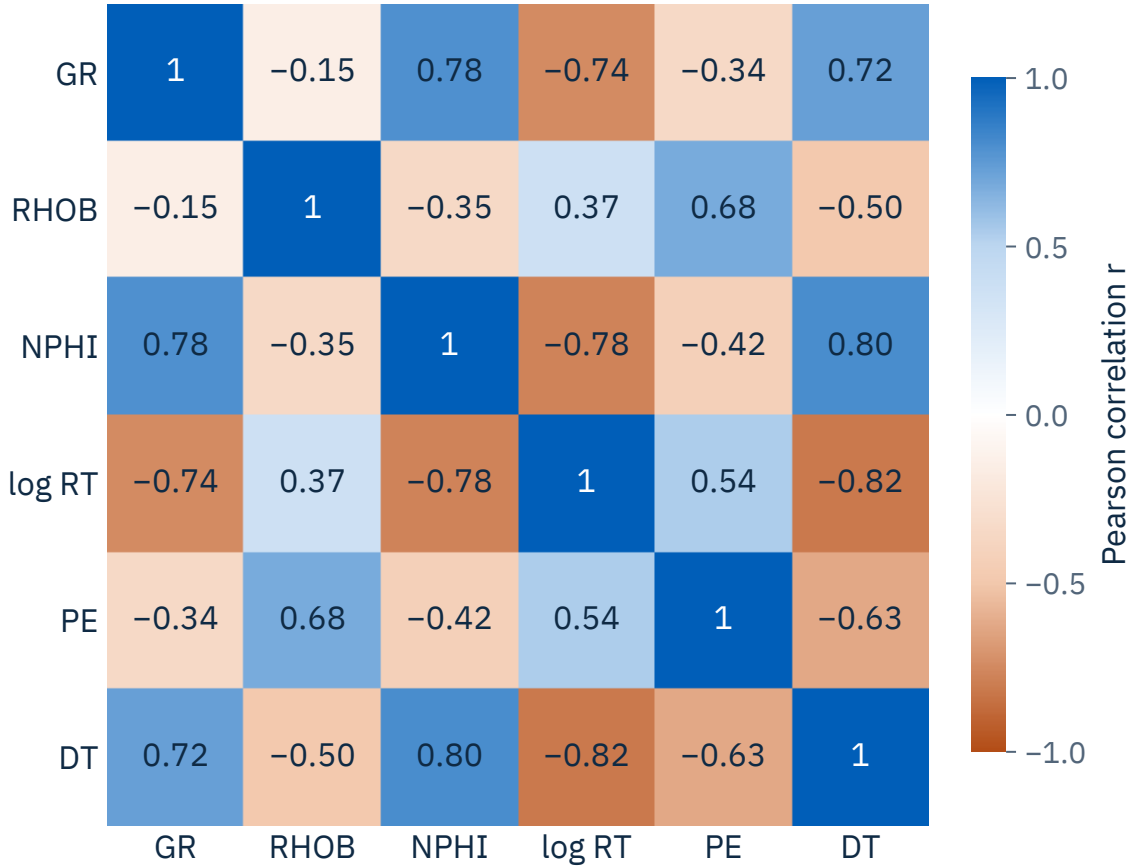
Plot every feature before fitting: mixtures hint at classes, skew hints at a transform.



Computed on the synthetic 8-well dataset (3,200 samples, seed 7); skewness from scipy.stats.skew.

Correlation shows redundancy, not causation

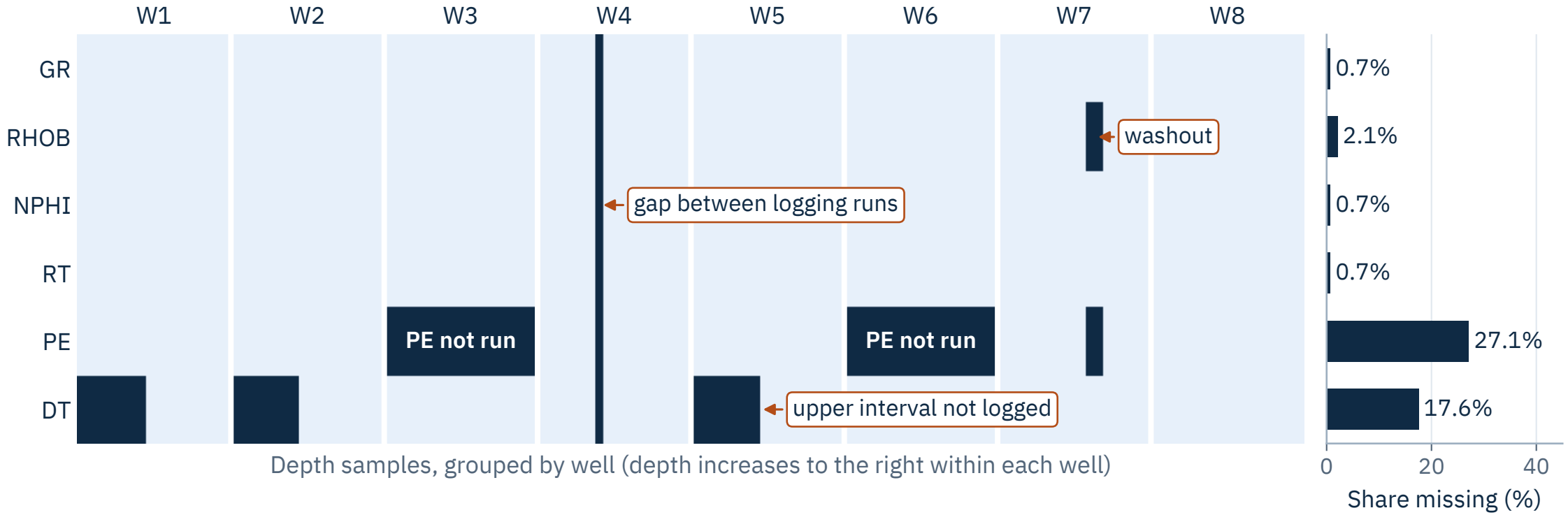
- GR, NPHI and DT rise together (r = 0.72–0.80)
- log RT falls as NPHI and DT rise
- Correlated inputs make linear coefficients unstable
- r near 0 does not rule out a curved relationship



Pearson r over all 3,200 synthetic samples (seed 7). Much of it comes from mixing facies.

Missingness has structure worth recording

Gaps follow tool strings, logging runs and hole conditions: they are patterned, not random.

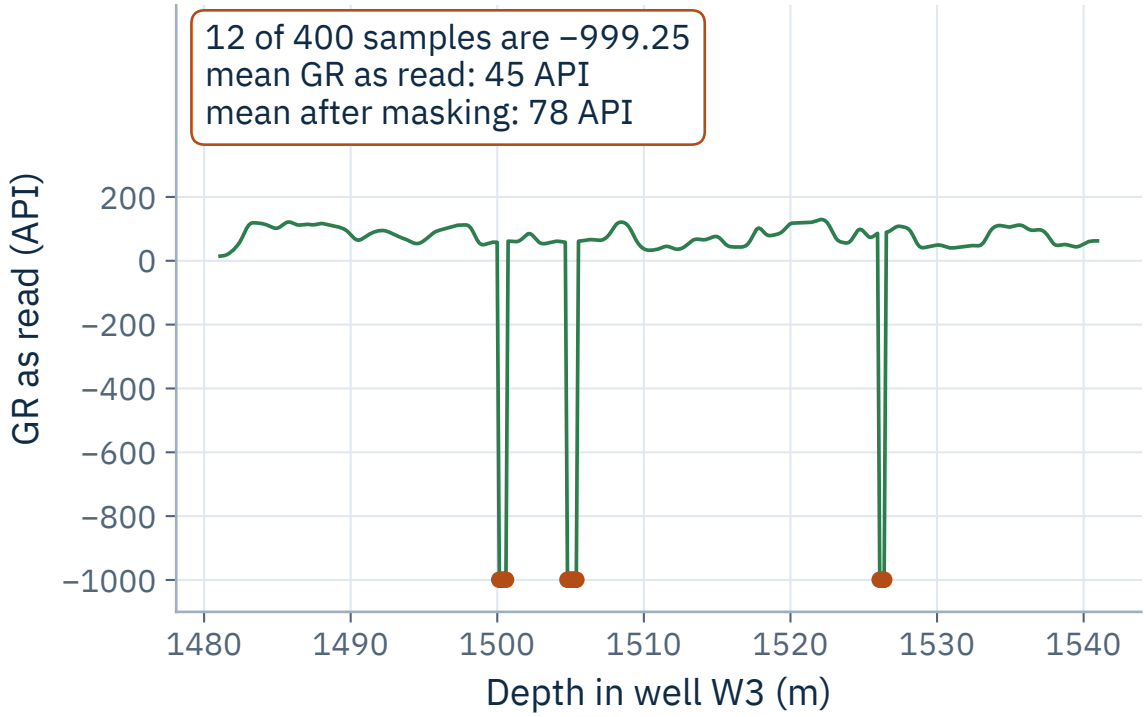


Synthetic dataset (seed 7) with injected, patterned gaps: PE absent in W3 and W6, DT only in the lower interval of W1, W2 and W5, a washout in W7 and a gap between runs in W4.

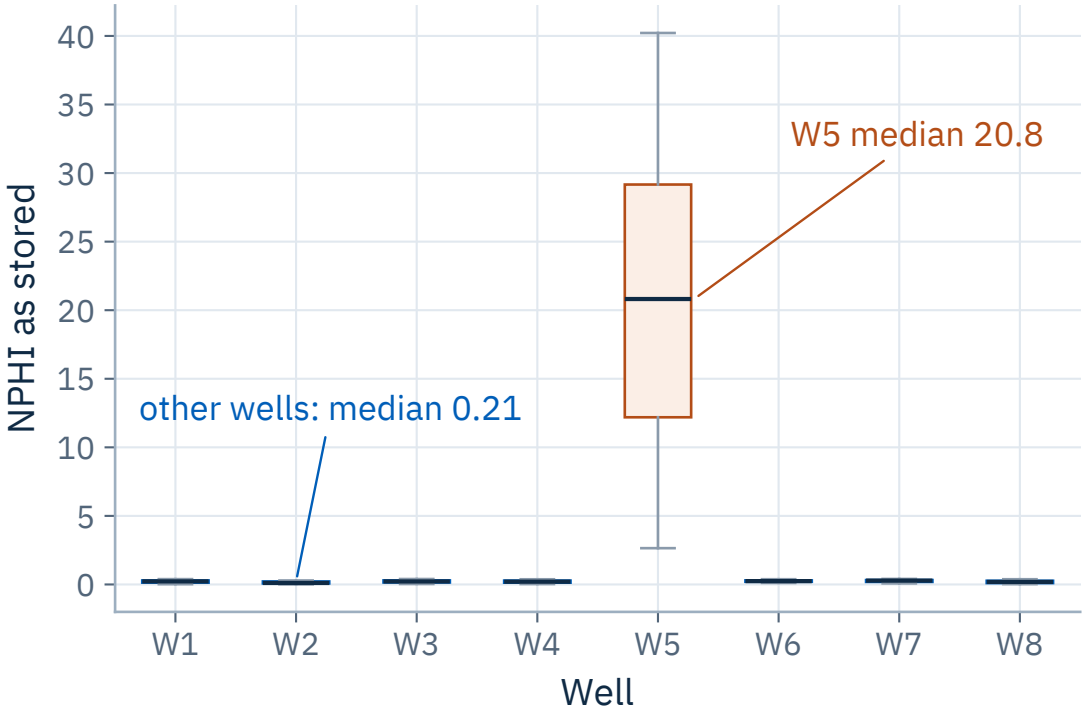
Null codes and unit mixes masquerade as data

Both pass a quick glance at a table; both distort means, scalers and distances.

Null code −999.25 read as a measurement



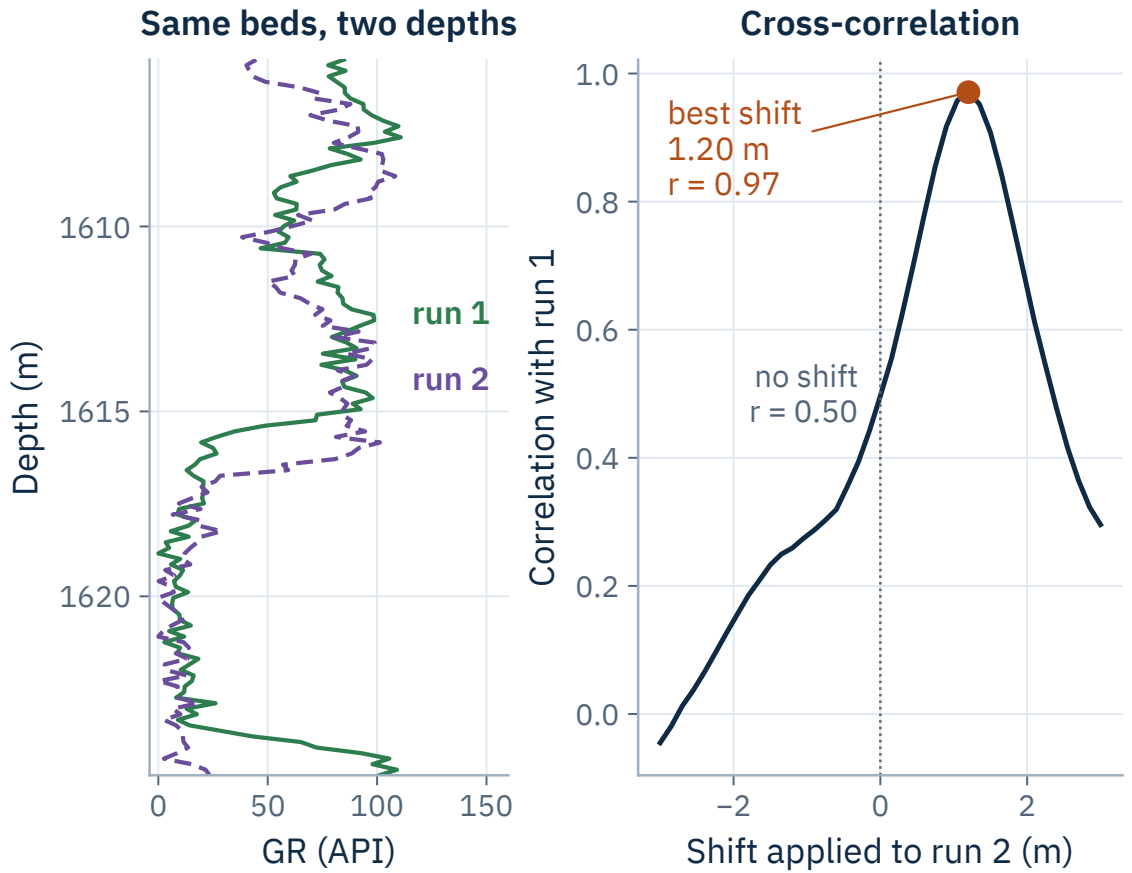
One well stored in percent, the rest in v/v



Synthetic data (seed 7) with injected problems: −999.25 null codes in W3 GR and W5 NPHI stored in percent.

Depth shifts misalign curves from different runs

- Runs can record the same bed at slightly different depths
- **Cross-correlate** a curve logged on both runs, usually GR
- Apply the best shift, then check several beds by eye
- Align before combining curves into features



Synthetic well W4 (seed 7). Run 2 repeats the GR log 1.2 m deeper with independent noise (seed 4); correlation at each trial shift.

PRACTICE

5 min · Pairs

What is wrong with this log summary?

Curve	Rows present of 3,200	Min / median / max
GR (API)	3,178	−999.25 / 65.2 / 142.3
RHOB (g/cm³)	3,133	2.24 / 2.49 / 2.75
NPHI (v/v)	3,178	0.01 / 0.23 / 40.2
RT (Ω·m)	3,178	0.49 / 5.89 / 331.8
PE (b/e)	2,333	1.71 / 3.48 / 6.14
DT (µs/ft)	2,638	45.8 / 90.6 / 128.9

1 Find at least four problems and say what you would check next.

Feature engineering needs physical meaning

✓ Meaningful

- $\log_{10}$ RT instead of raw RT
- Shale volume from GR with per-well clean and shale baselines
- Rolling means within each well, trailing in real time

vs

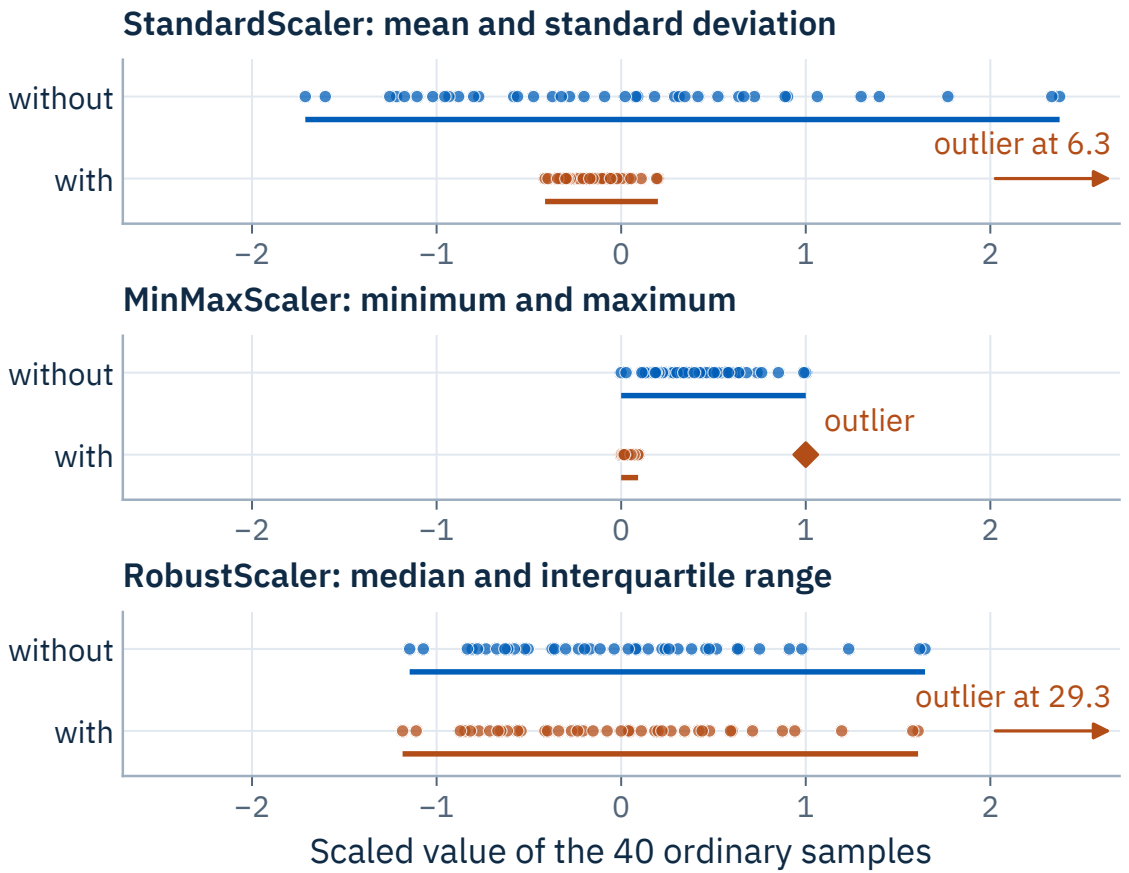
✗ Misleading

- RHOB – NPHI without consistent units or scaling
- Well ID as a predictor for unseen wells
- Centred windows in real-time use: they see the future

◆ Test every engineered feature against a baseline on held-out wells.

Scalers react differently to one outlier

- StandardScaler: mean and SD, both pulled by an outlier
- MinMaxScaler: the range is set by the extremes
- RobustScaler: median and IQR barely move
- Fit any scaler on training rows only



Synthetic feature: 40 samples from N(50, 10), seed 5, plus one outlier at 400. Bars show the range of the 40 ordinary samples.

Encoding and imputation turn columns into numbers

Both steps learn from data, so both belong inside the cross-validated pipeline.

One-hot encoding: one column per category

Formation
A1 SH
A1 LM
B1 SH
A1 LM
B1 SH

→

A1 LM	A1 SH	B1 SH
0	1	0
1	0	0
0	0	1
1	0	0
0	0	1

Integer codes 0, 1, 2 would invent an order that linear models and KNN take literally

Median imputation plus a missing indicator

PE
3.1
NaN
2.8
NaN
4.0

→

PE	PE_missing
3.1	0
3.1	1
2.8	0
3.1	1
4.0	0

median of the training values (2.8, 3.1, 4.0) = 3.1

Both steps are fitted: categories and medians come from training rows only

Schematic worked example; the median 3.1 comes from the three observed training values (2.8, 3.1, 4.0).

PART 3 OF 6

03

Splits, cross-validation and leakage

The split is the experiment.

- Three distinct uses of data
- Cross-validation schemes and what each one estimates
- Parameters versus hyperparameters
- Three forms of leakage, and an exercise

Training, validation and test data do different jobs

Training

- Fits imputation, scaling and model parameters
- Never sees evaluation rows

Validation / CV

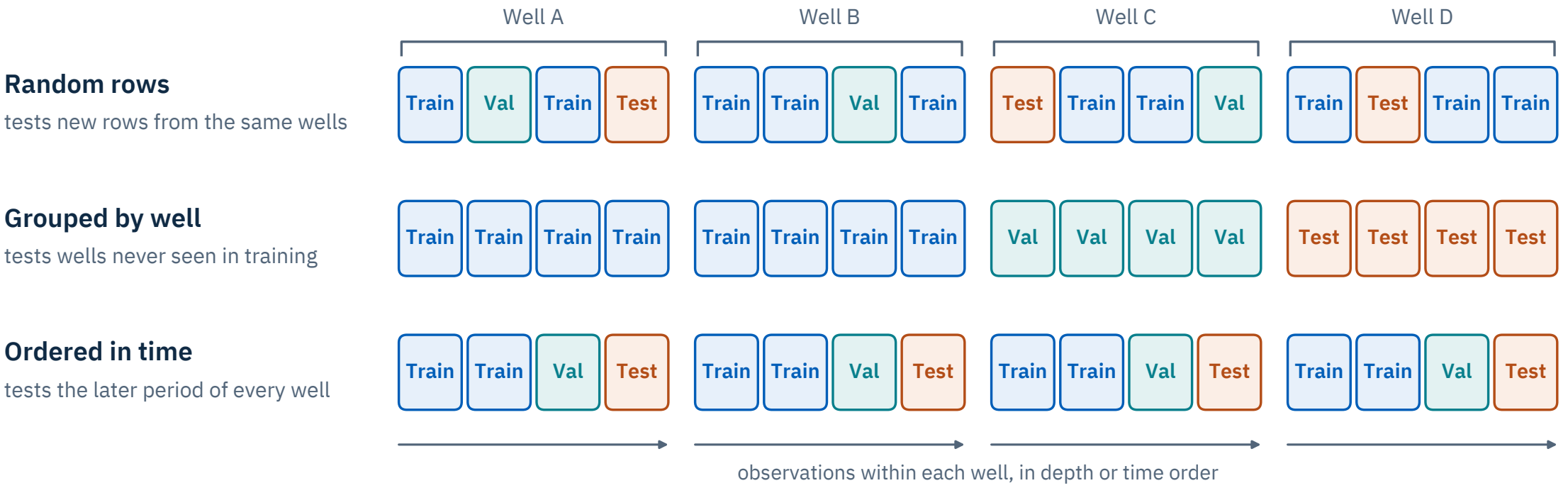
- Selects models, thresholds and settings
- Turns optimistic once you choose on it

Test

- Estimates the locked workflow, once
- Any further tuning turns it into validation

◆ Size the partitions by independent groups and event counts, not by a fixed 60/20/20.

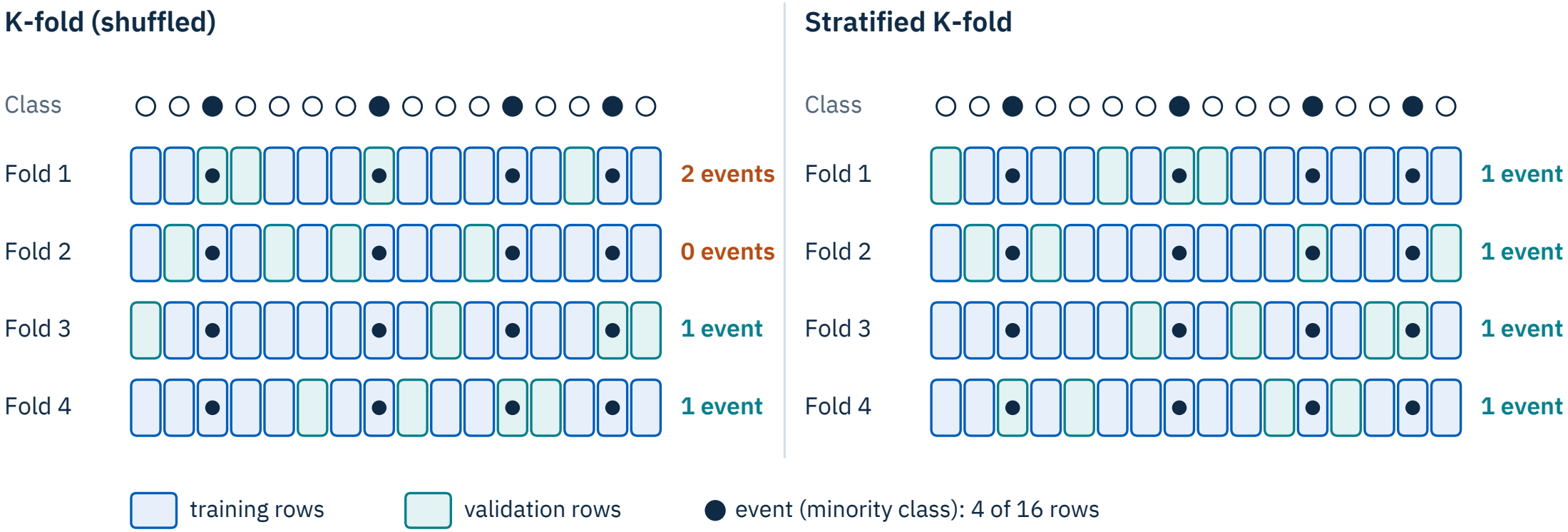
The split follows the deployment question



Schematic split assignments. Each row answers a different generalization question, not a ranking of methods.

K-fold and stratified folds rotate the validation rows

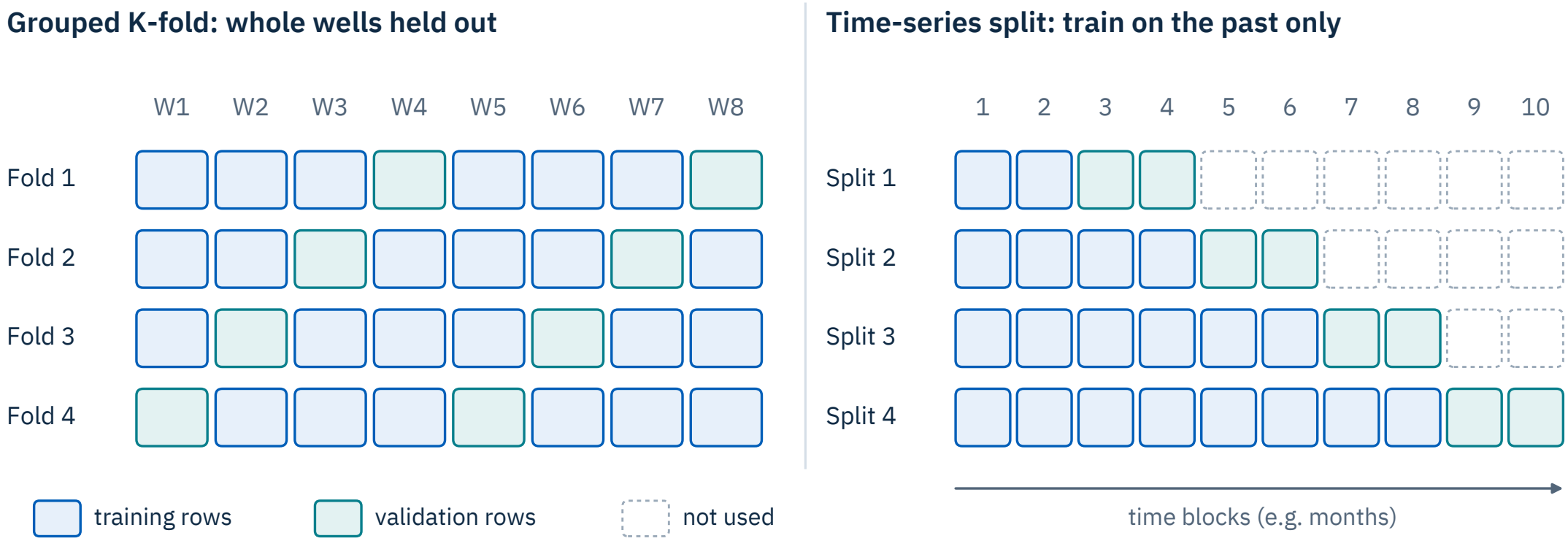
Every row validates exactly once; **stratification** also keeps the class mix in each fold.



Fold assignments computed with scikit-learn KFold and StratifiedKFold (4 folds, shuffle, random_state=1) for 16 rows with 4 events.

Grouped and time-ordered folds mirror deployment

Hold out whole wells for new-well questions; train on the past only for forecasts.



Computed with scikit-learn GroupKFold(4) for 8 equal-size wells and TimeSeriesSplit(4) for 10 time blocks.

Each scheme estimates a different future

Scheme	Estimates performance on	Watch out for
K-fold	New rows like the training rows	Correlated rows inflate scores
Stratified K-fold	The same, with a stable class mix	Still row-level
Grouped K-fold	Wells or assets never seen	Few groups give noisy estimates
Time-series split	The period after training	Needs a gap for the forecast horizon

New wells and future periods at once: group by well and keep time order.

◆ Start from the prediction contract: the deployment question picks the scheme.

Parameters are learned; hyperparameters are chosen

Parameters

- Learned by fit from training rows
- Weights, split thresholds, support vectors
- Also imputer medians and scaler means

vs

Hyperparameters

- Chosen by comparing validation scores
- α, K, C and γ, max_depth
- Also the preprocessing choices themselves

◆ Whatever the validation folds chose, only the test set can judge, and only once.

Leakage takes three common forms

Target leakage

- An input encodes the answer
- A workover flag written after the failure

Train–test contamination

- Evaluation rows shape training
- Scaling all wells; neighbours on both sides

Temporal leakage

- Information from after prediction time
- Centred windows; later-revised allocations

◆ Ask of every input: would this value exist, unchanged, at prediction time?

PRACTICE

3 min · Pairs

Leakage can enter before the model

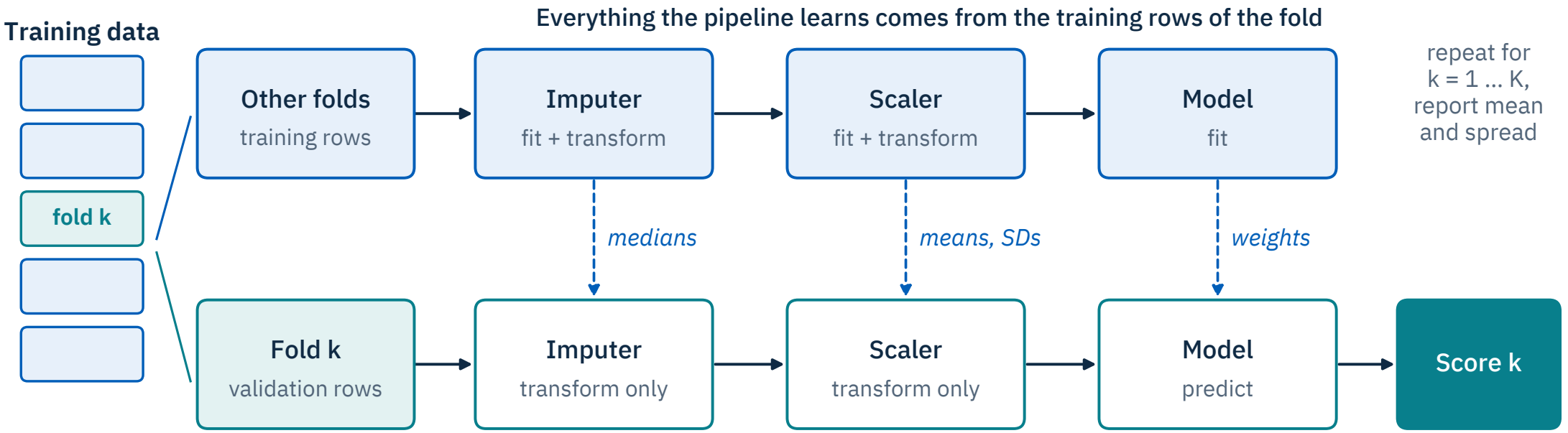
You scale all wells, select features using all labels, then split by well.

1. Which steps have already used the future validation wells?
2. Where should each step run during cross-validation?

DISCUSSION

Fit every step inside the training fold

Answer: refit the imputer, scaler and model on the training rows of each fold; validation rows are only transformed.



Schematic of one cross-validation fold. Module 2 measures how much the wrong order inflates scores.

PART 4 OF 6

04

Linear models and regularization

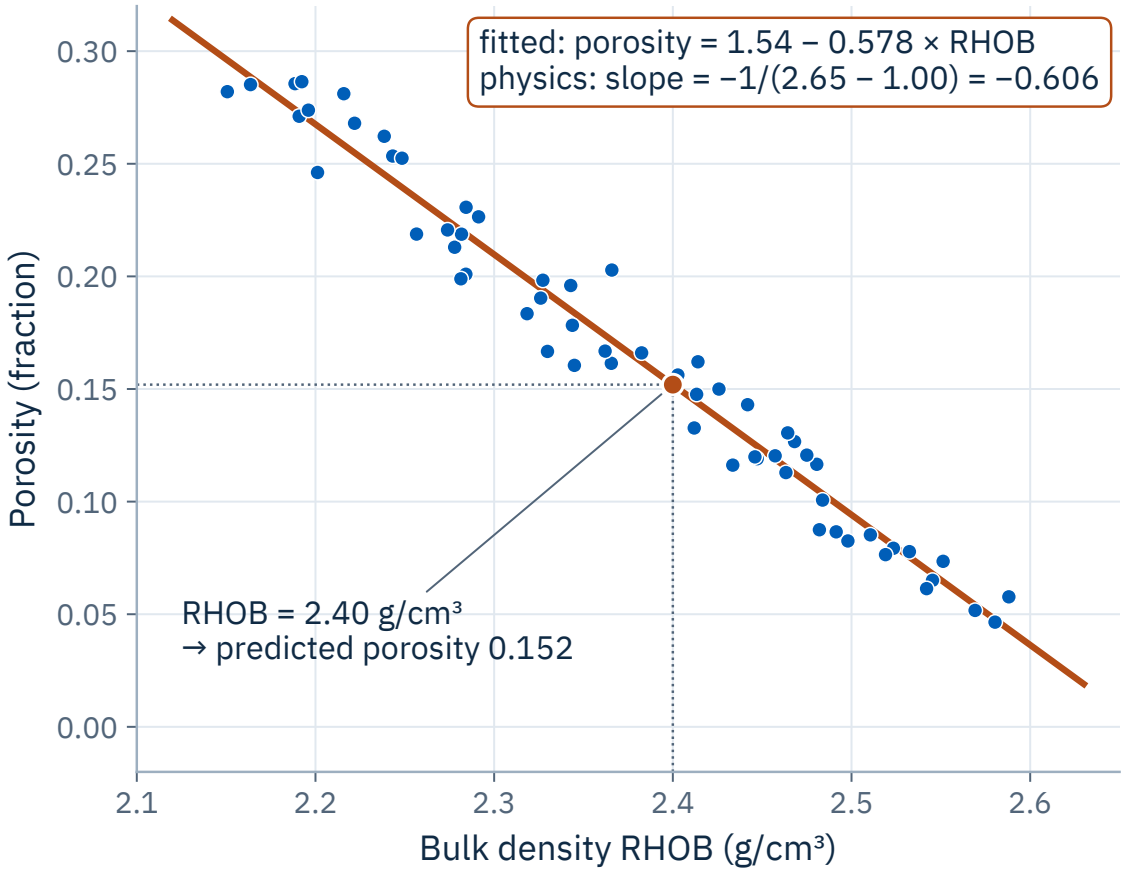
A fitted curve is a hypothesis about unseen observations.

- Linear models, least squares and gradient descent
- Basis expansion, capacity, bias and variance
- Ridge, lasso and robust losses
- Regression metrics

Linear models are linear in their weights

$$\hat{y} = b + \sum_{j=1}^{d} w_j x_j$$

- Each **weight** is the change in prediction per unit of its input, others fixed
- Density–porosity is linear by physics: slope $-1/(\rho_{ma} - \rho_f)$
- Weights depend on units and on correlated inputs



Synthetic data (seed 3) from the density–porosity relation, matrix 2.65 and fluid 1.00 g/cm³.

Least squares chooses the weights

$$L = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

Least squares picks the weights that minimize L, so each residual counts by its square:

1

cost of a residual of 1

4

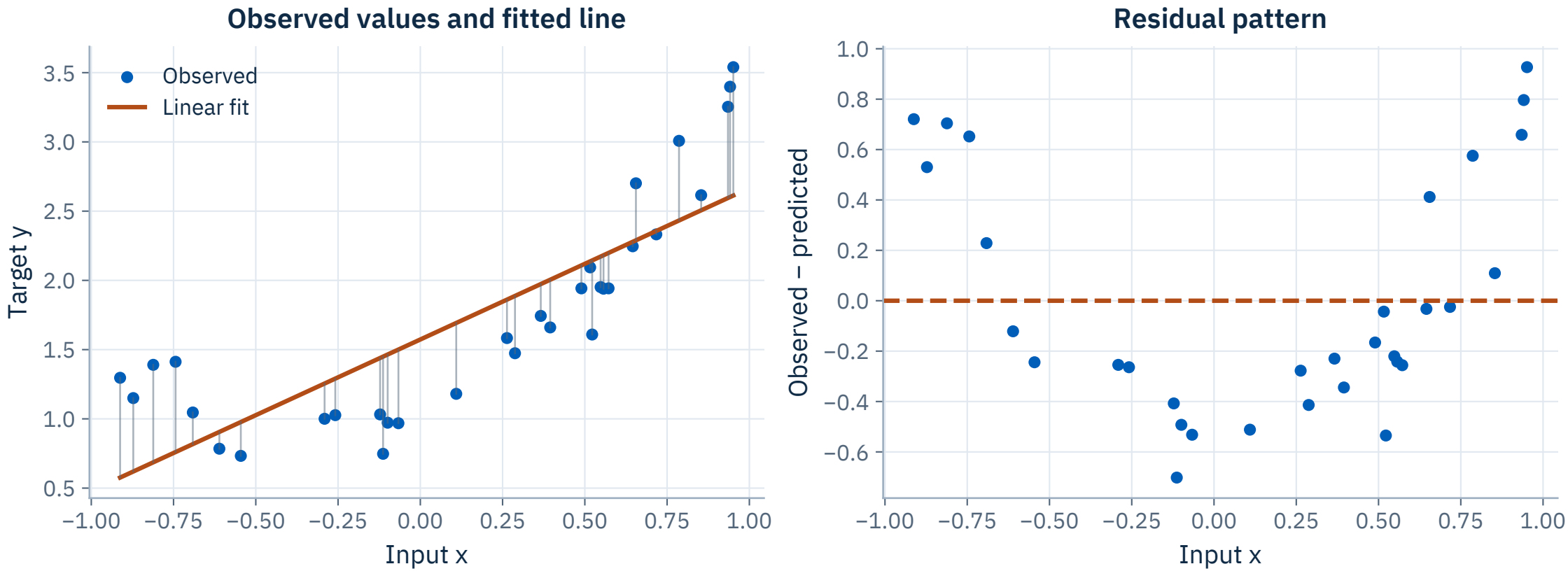
cost of a residual of 2

16

cost of a residual of 4

- Solvers use QR or SVD factorizations, not an explicit matrix inverse.

A residual shows what the model missed



Computed synthetic example. Residual = observed value minus prediction. A curved residual pattern indicates missing structure.

One gradient step, worked by hand

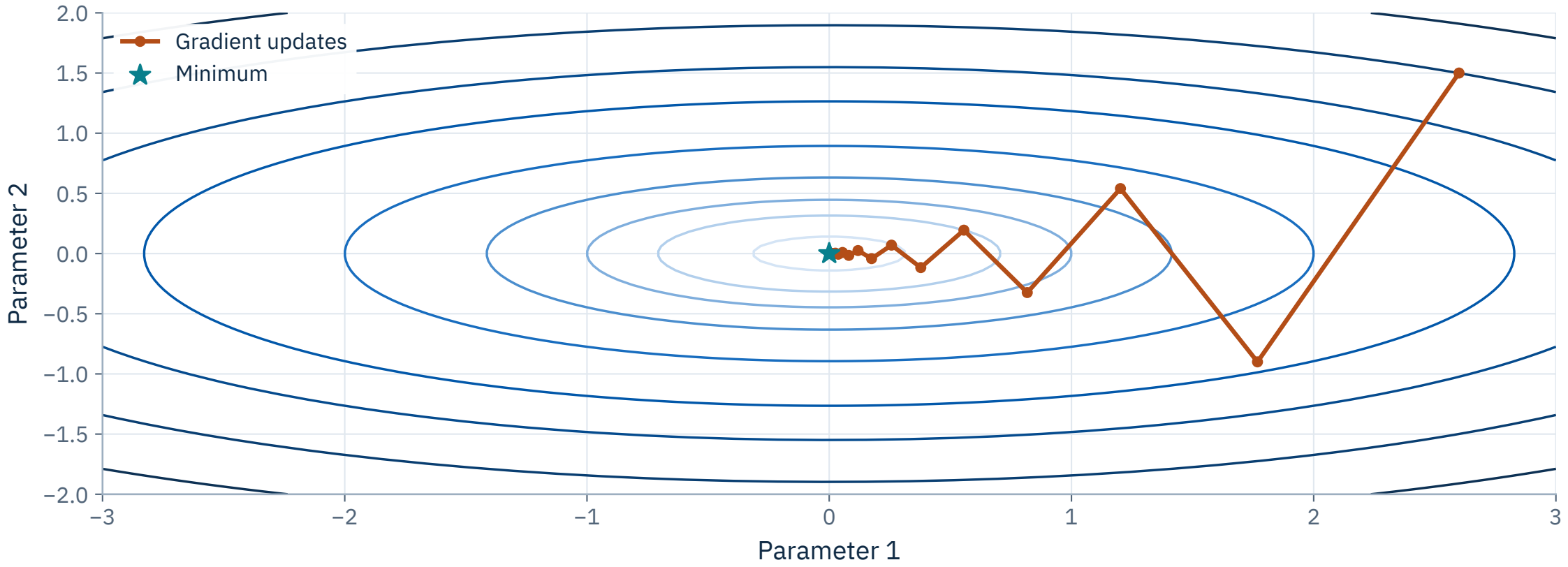
$$w_{new} = w - \alpha \frac{\partial L}{\partial w}$$

One sample x = 2, y = 5; intercept fixed at 0; start at w = 1; learning rate α = 0.1.

1. **Predict** — $\hat{y} = wx = 2$, so the squared error is $(5 - 2)^2 = 9$
2. **Gradient** — $\partial L/\partial w = 2x(wx - y) = 2 \cdot 2 \cdot (2 - 5) = -12$
3. **Step** — $w = 1 - 0.1 \times (-12) = 2.2$
4. **Check** — $\hat{y} = 4.4$, and the squared error falls to $(5 - 4.4)^2 = 0.36$

◆ Too large a learning rate overshoots the minimum and can increase the loss.

Gradient descent zigzags when curvatures differ

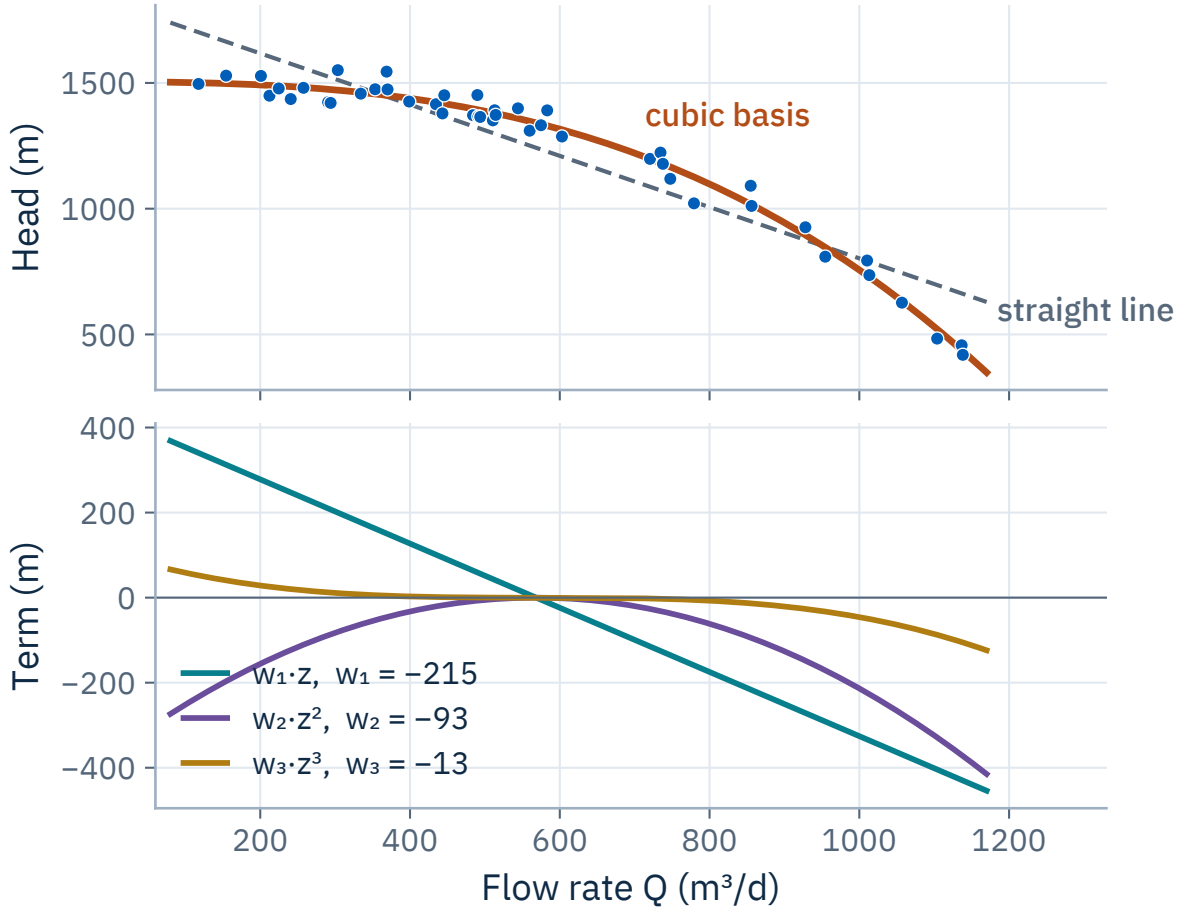


Computed on the exact quadratic loss $L = w_1^2 + 5w_2^2$, learning rate 0.16. The path follows successive updates from one start.

Basis expansion lets a linear model bend

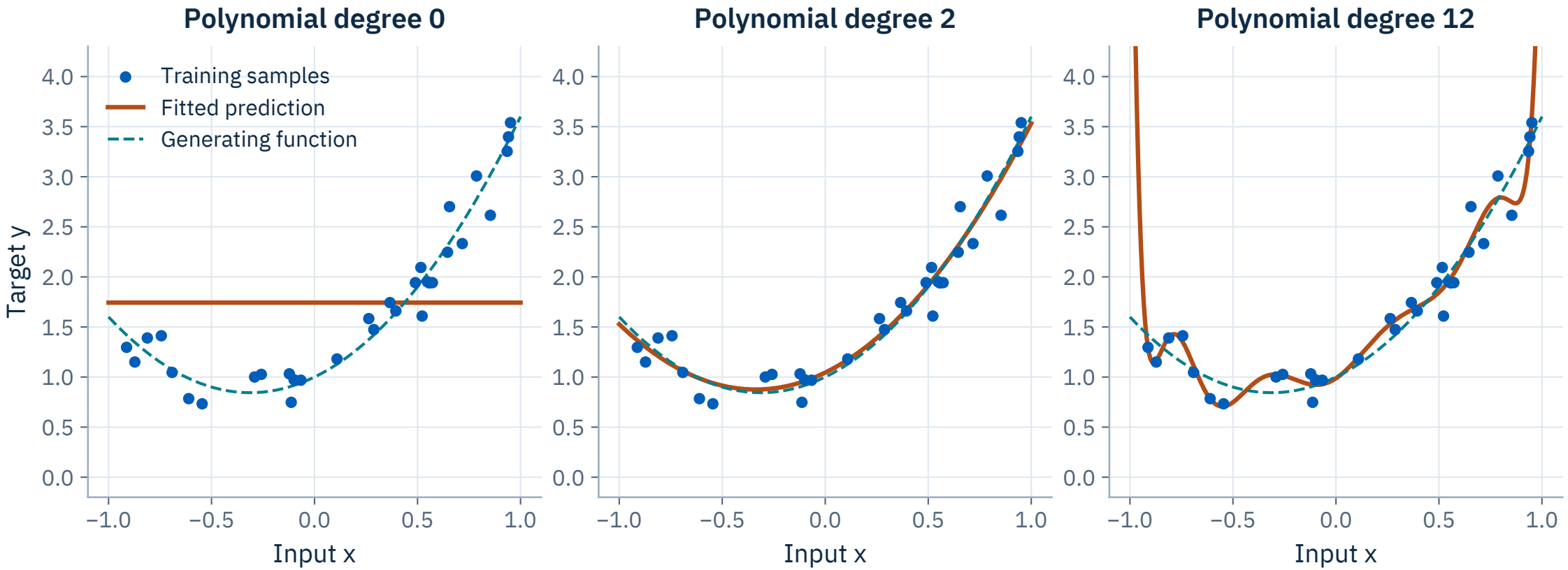
$$\hat{y} = b + w_1 z + w_2 z^2 + w_3 z^3$$

- Still linear in the weights, so least squares still applies
- z is standardized flow: raw powers of Q would be badly conditioned
- The degree is a hyperparameter: choose it by validation



Synthetic head–flow data (seed 8), 45 points; cubic fit on the standardized flow z.

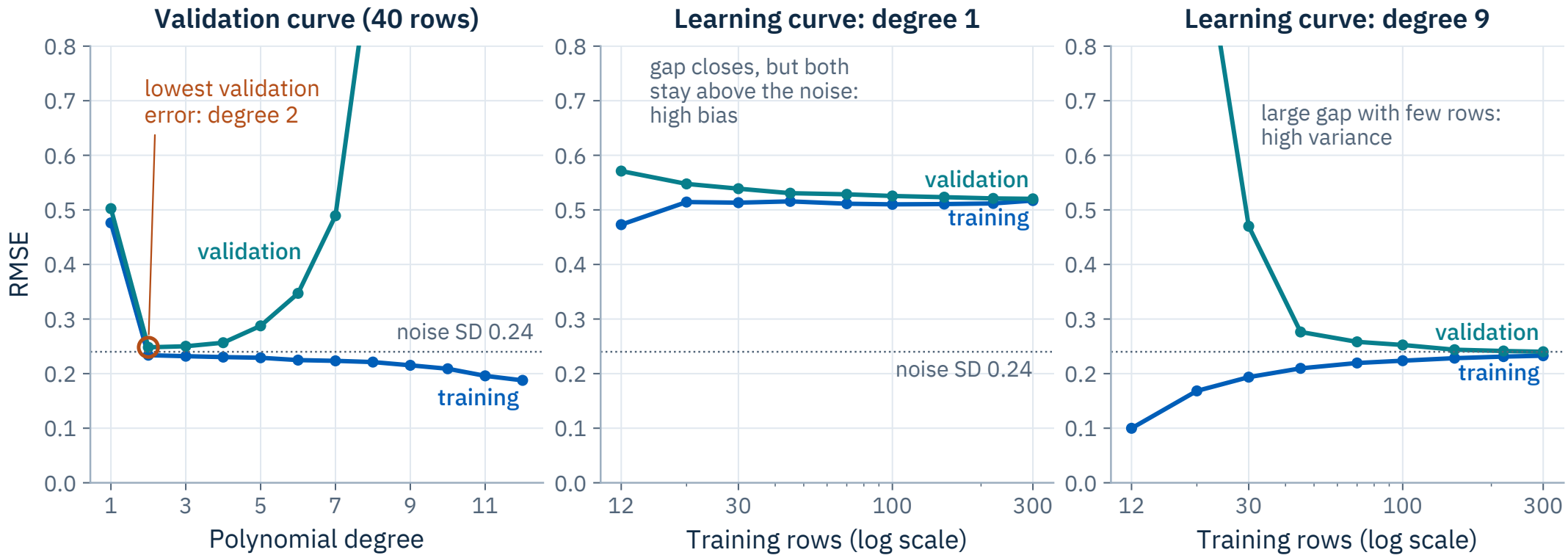
Capacity changes the fitted function



Three models fitted to the same synthetic training observations. The dashed curve is the generating function, not an available training label.

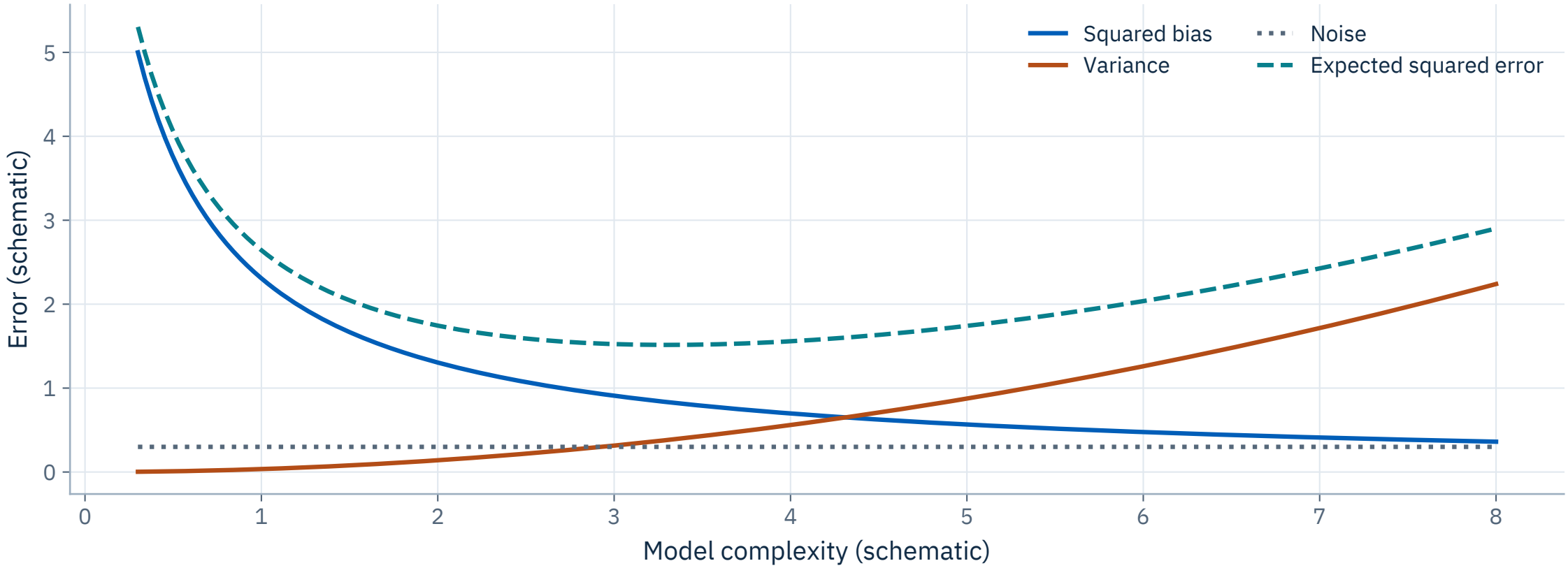
Held-out curves diagnose capacity and data needs

Validation curves choose complexity; learning curves show whether more data would help.



Synthetic $y = 1 + x + 1.6x^2$ + noise (SD 0.24): validation curve on 40 rows (seed 94, 5-fold CV repeated 12 times); learning curves from a 400-row pool (seed 12). Mean RMSE.

Bias and variance explain different failures



Schematic squared-error decomposition. These curves illustrate a concept and are not measured learning curves.

Ridge, lasso and elastic net add a penalty

$$L(w) = \sum_i (y_i - \hat{y}_i)^2 + \alpha\, P(w)$$

Ridge (L2)

- $P(w) = \sum_j w_j^2$
- Shrinks all weights; keeps correlated ones

Lasso (L1)

- $P(w) = \sum_j |w_j|$
- Sets some weights exactly to zero

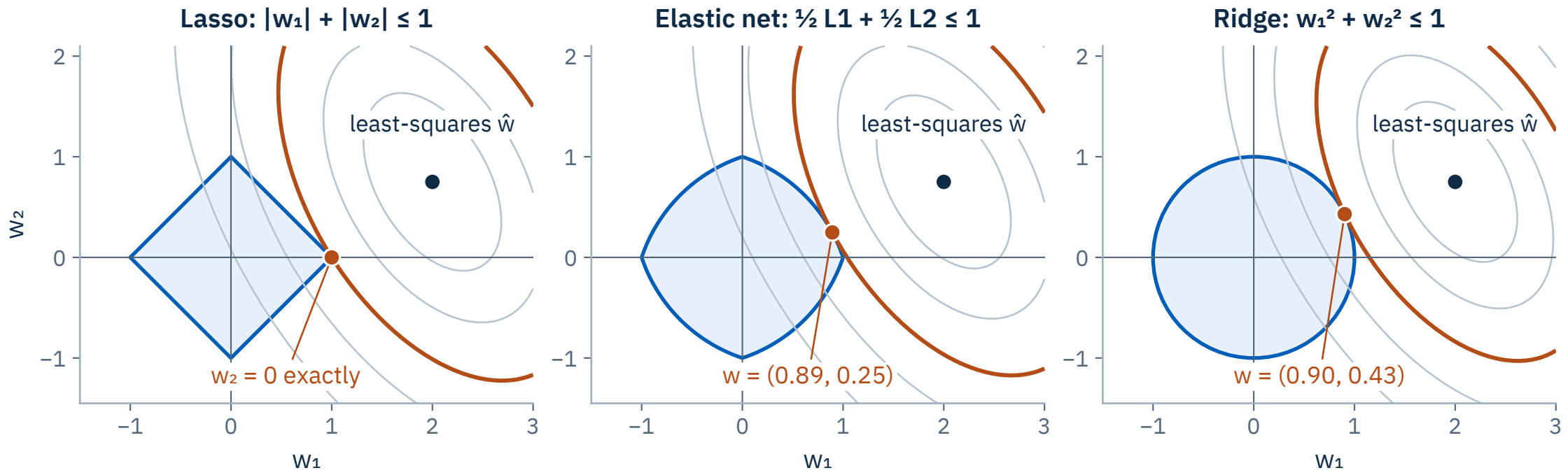
Elastic net

- A weighted mix of both penalties
- Sparse, and steadier with correlated inputs

◆ Standardize predictors first and tune α inside cross-validation; the intercept is not penalized.

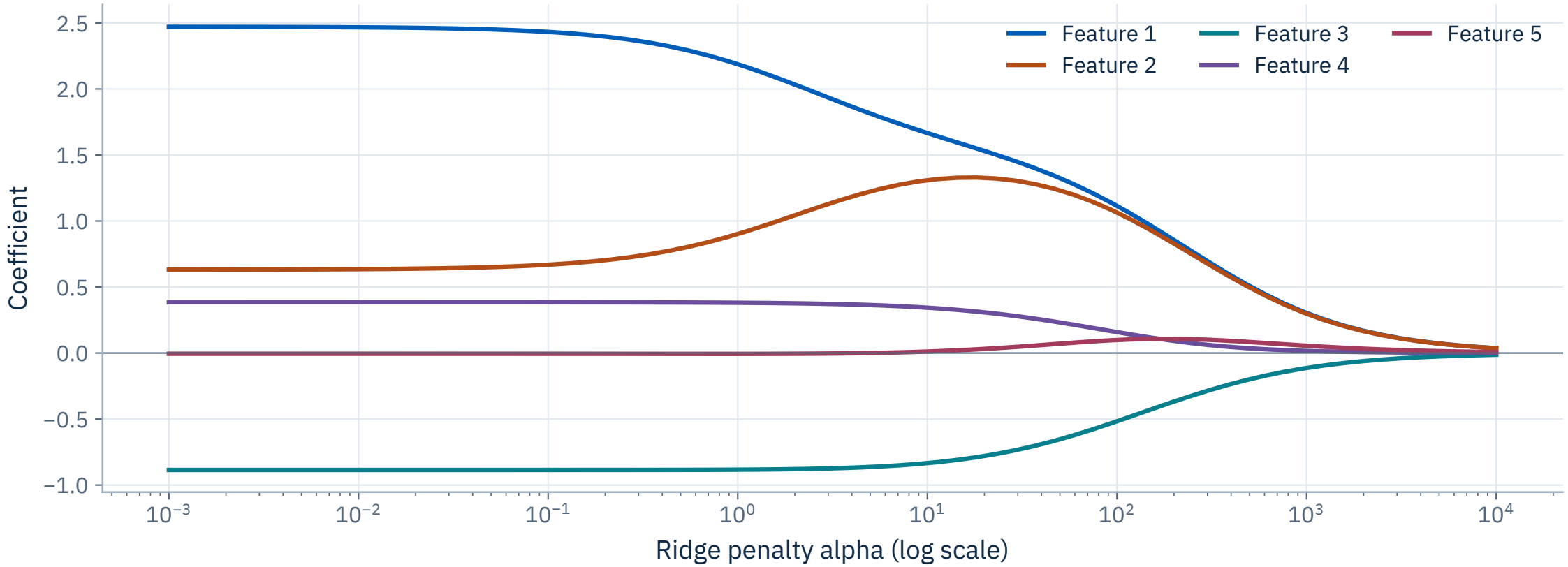
L1 and L2 penalties have different geometry

The fit sits where the smallest loss contour touches the allowed region: corners on the axes give exact zeros.



Exact quadratic loss centred at ŵ = (2.0, 0.75); constrained minima computed numerically. Elastic net uses half L1, half L2.

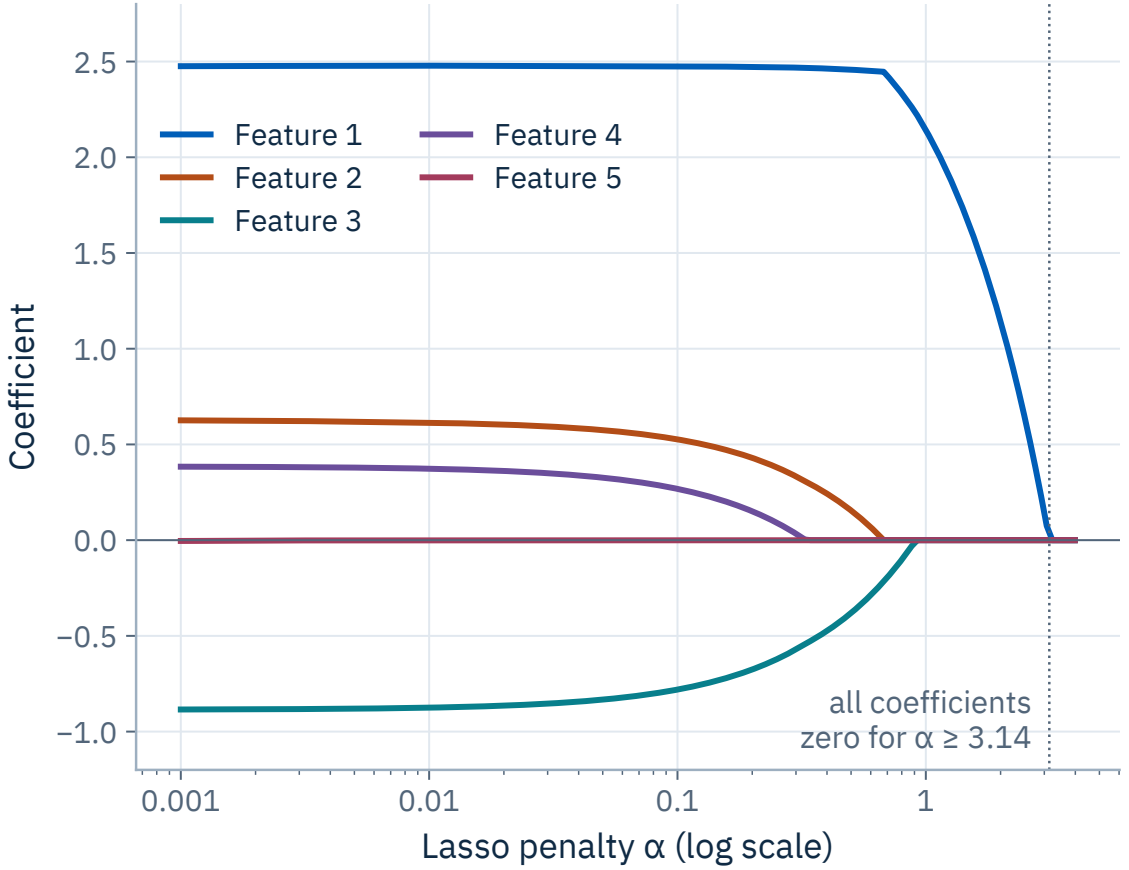
Coefficients shrink as the penalty grows



Ridge fits on one standardized synthetic dataset. The horizontal axis is logarithmic. Lines are coefficients, not importance scores.

Lasso sets coefficients exactly to zero

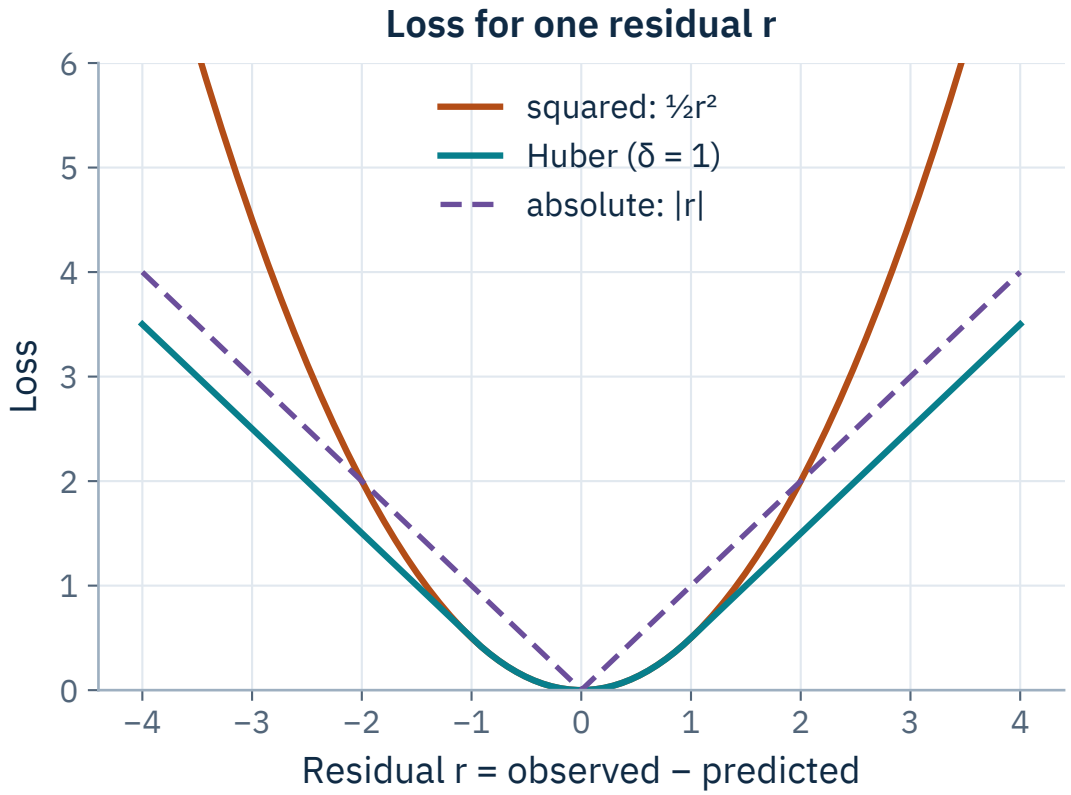
- Same standardized data as the ridge path
- As α decreases, weights leave zero one at a time
- Of the correlated features 1 and 2, lasso drops feature 2 first
- Above α = 3.14 every weight is zero

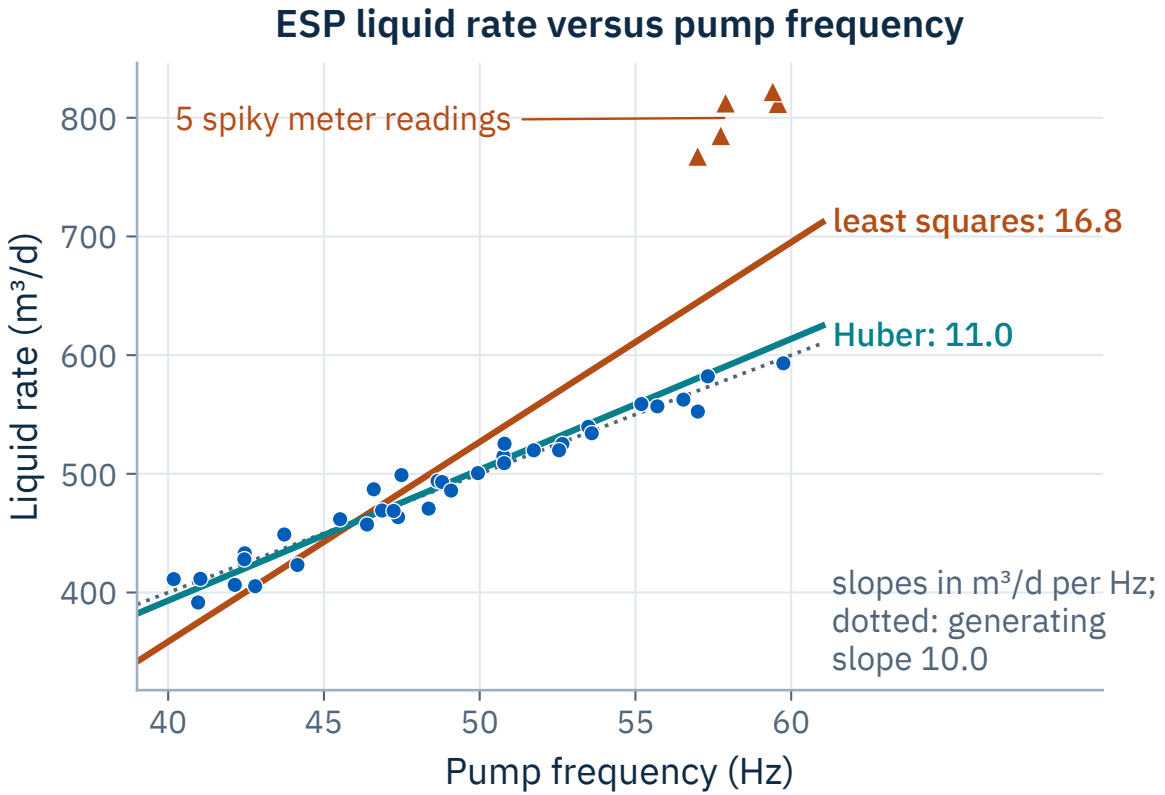


Same synthetic data as the shared ridge path (seed 42); computed with scikit-learn lasso_path.

Squared loss lets a few spikes pull the fit

Huber loss is quadratic for small residuals and linear for large ones, so spikes pull less.





Synthetic ESP data (seed 6): rate = 10 × frequency + noise, with 5 upward spikes. Slopes: least squares 16.8, HuberRegressor 11.0 m³/d per Hz.

Regression metrics answer different questions

$$\mathrm{MAE} = \frac{1}{n}\sum_i |y_i - \hat{y}_i| \qquad \mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_i (y_i - \hat{y}_i)^2} \qquad R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

MAE

- Average miss, in target units
- Treats all error sizes linearly

RMSE

- In target units; large misses count more
- Sensitive to outliers

R²

- Improvement over the evaluation mean
- Can be negative; depends on the spread

◆ Report errors by well or operating regime as well as overall.

PRACTICE

4 min · Individual

Compute MAE, RMSE and R^2 by hand

Observed y = 10, 12, 9, 15; predicted $\hat{y}$ = 9, 13, 9, 11.

1. Compute the residuals, the MAE and the RMSE.
2. Compute R^2, given that the mean of y is 11.5.
3. Which metric reacts most to the one large miss?

LABORATORY

40 min

Lab 1: regression and regularization

- §1.1–1.2: split three ways; fit linear and degree-2 models; record R^2 and MAE
- Put a DummyRegressor mean baseline in the first row of your score table
- §1.3: plot training and validation R^2 against degree, raw and standardized
- §2: compare ridge and lasso over α on validation rows; count nonzero terms
- §2.3: lock one model, then score the test rows once

NOTEBOOK

`exercises/01_supervised_learning_basics/Supervised_Learning_Basics_Student.ipynb`

EVIDENCE TO SUBMIT

- Score table, baseline first
- Degree plots: raw powers and standardized columns
- One sentence: which α, and how it was chosen

DISCUSSION

Lab 1 debrief: what the regression lab shows

Reference run of the solution (16,512 training, 2,064 validation, 2,064 test rows; mean baseline R^2 0.00):

0.59

validation R^2, linear on 8 features

0.66

validation R^2, degree 2 (44 terms)

0.11

training R^2, raw MedInc at degree 15

0.63

test R^2 of the locked ridge, used once

- Training R^2 cannot fall as the degree grows unless the solver fails: standardize the polynomial columns.

PART 5 OF 6

05

Classification models

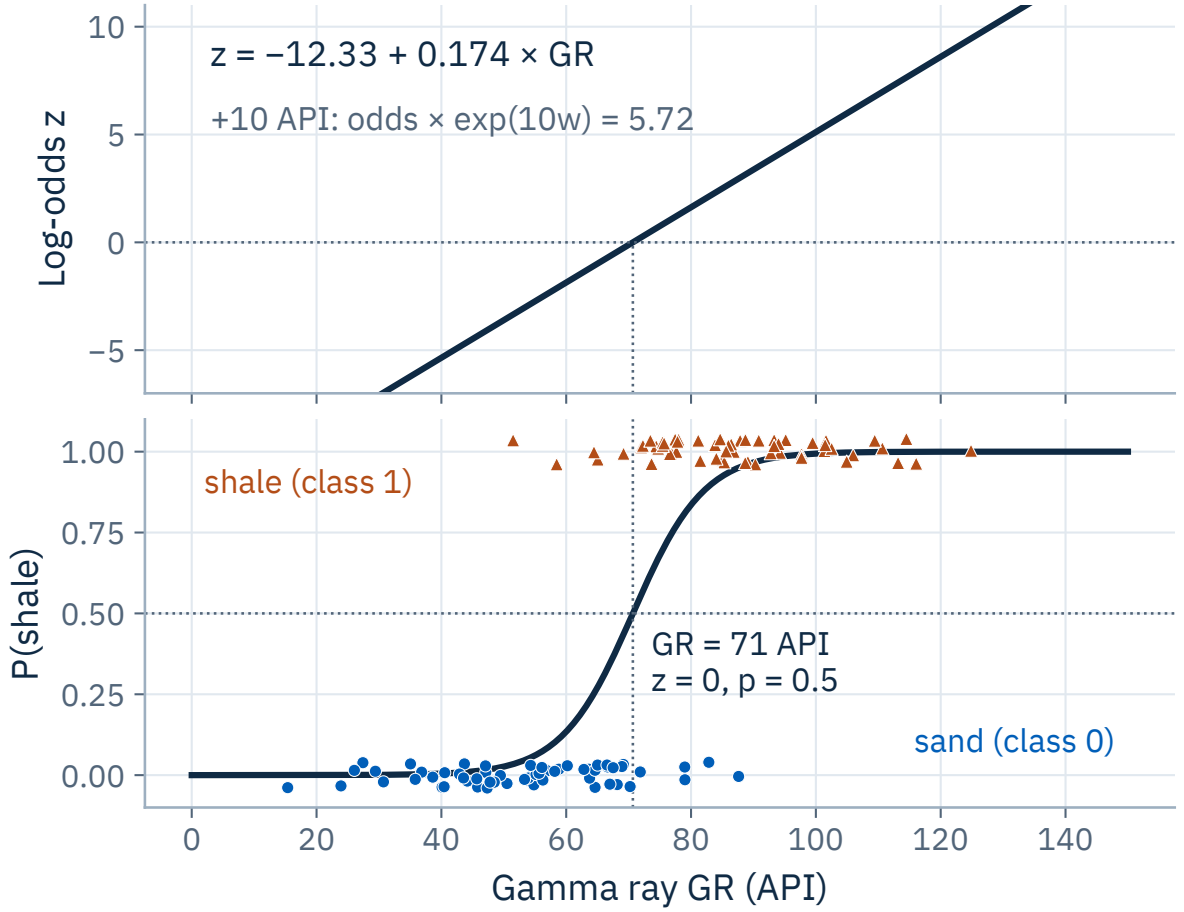
Each model makes a different assumption about the boundary.

- Logistic regression
- Nearest neighbours
- Support vector machines
- Decision trees

Logistic regression maps a score to a probability

$$p = \sigma(z) = \frac{1}{1+e^{-z}}, \qquad z = b + w^{\top} x$$

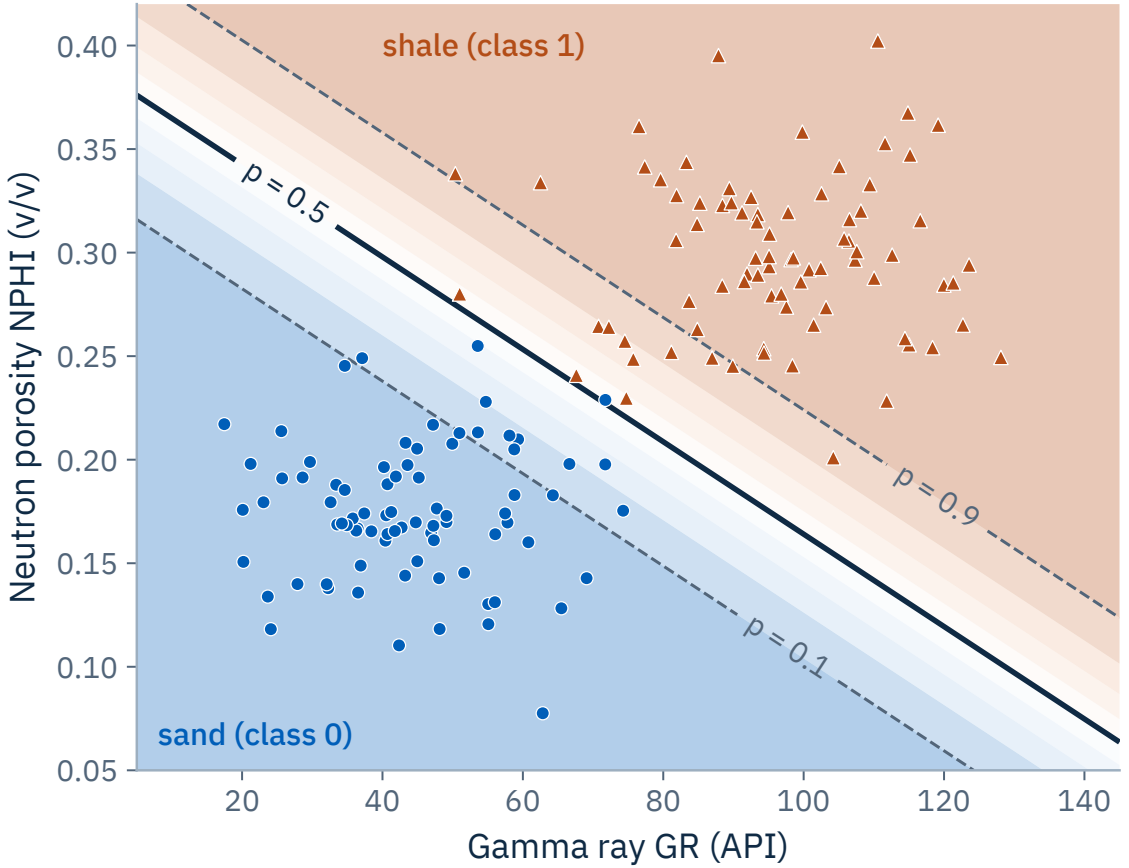
- z is the **log-odds**, ln(p / (1 – p)), linear in the inputs
- Each +1 in an input multiplies the odds by e^{w}
- p = 0.5 exactly where z = 0



Synthetic sand and shale GR (60 each, seed 2); fitted weight 0.174 per API.

The decision boundary is where p equals 0.5

- With two inputs the **decision boundary** z = 0 is a straight line
- Probability contours run parallel to it
- Curved boundaries need basis expansion or another model

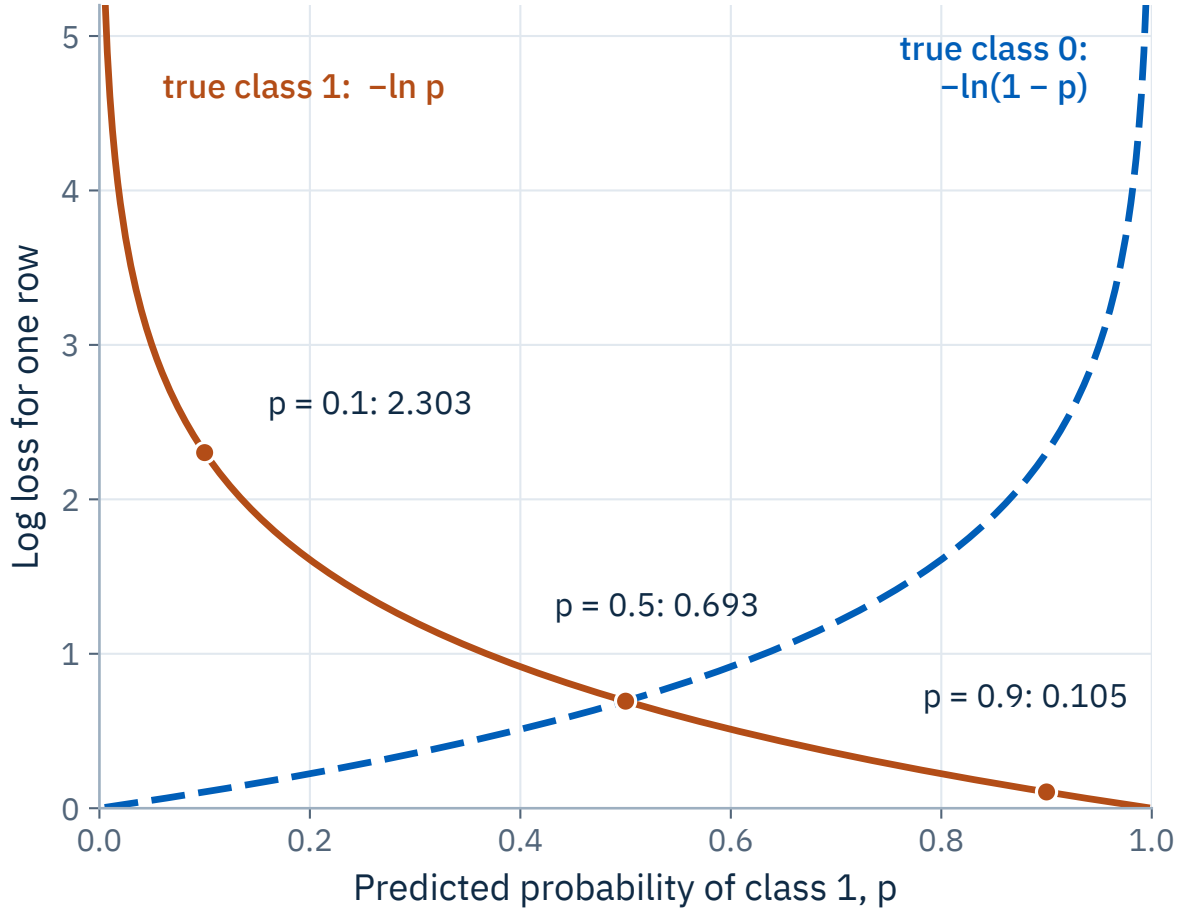


Synthetic sand and shale samples (80 each, seed 4); standardized logistic regression; contours of predicted probability.

Log loss punishes confident mistakes

$$L = -[y \ln p + (1 - y) \ln(1 - p)]$$

- True shale predicted at p = 0.9 costs 0.105
- The same shale at p = 0.1 costs 2.303, about 22 times more
- Fitting minimizes the mean log loss with an iterative solver



Exact functions; marked values −ln 0.9, −ln 0.5 and −ln 0.1.

A worked prediction from two standardized logs

Weights b = −0.4, w_{GR} = 1.5, w_{RHOB} = 0.8; one sample at GR = 1.2 and RHOB = −0.5 (z-scores).

1. **Score** — $z = -0.4 + 1.5 \times 1.2 + 0.8 \times (-0.5) = -0.4 + 1.8 - 0.4 = 1.0$
2. **Probability** — $\sigma(1.0) = 1 / (1 + e^{-1}) \approx 0.73$
3. **Decision** — $0.73 \geq 0.5$, so predict shale
4. **Loss if the sample is shale** — $-\ln 0.73 \approx 0.31$
5. **Loss if the sample is sand** — $-\ln 0.27 \approx 1.31$

◆ GR adds +1.8 and RHOB −0.4 to the score: such comparisons only make sense on standardized inputs.

Scale, fit, then read probabilities with predict_proba

```python
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

clf = make_pipeline(
    StandardScaler(),
    LogisticRegression(C=1.0, max_iter=1000),
)
clf.fit(X_train, y_train)
proba = clf.predict_proba(X_val)[:, 1]
pred = (proba >= 0.5).astype(int)
```

- C is the inverse L2 strength: smaller C means stronger regularization
- More than two classes: a softmax over classes, by default
- Choose thresholds on validation rows, not 0.5 by habit

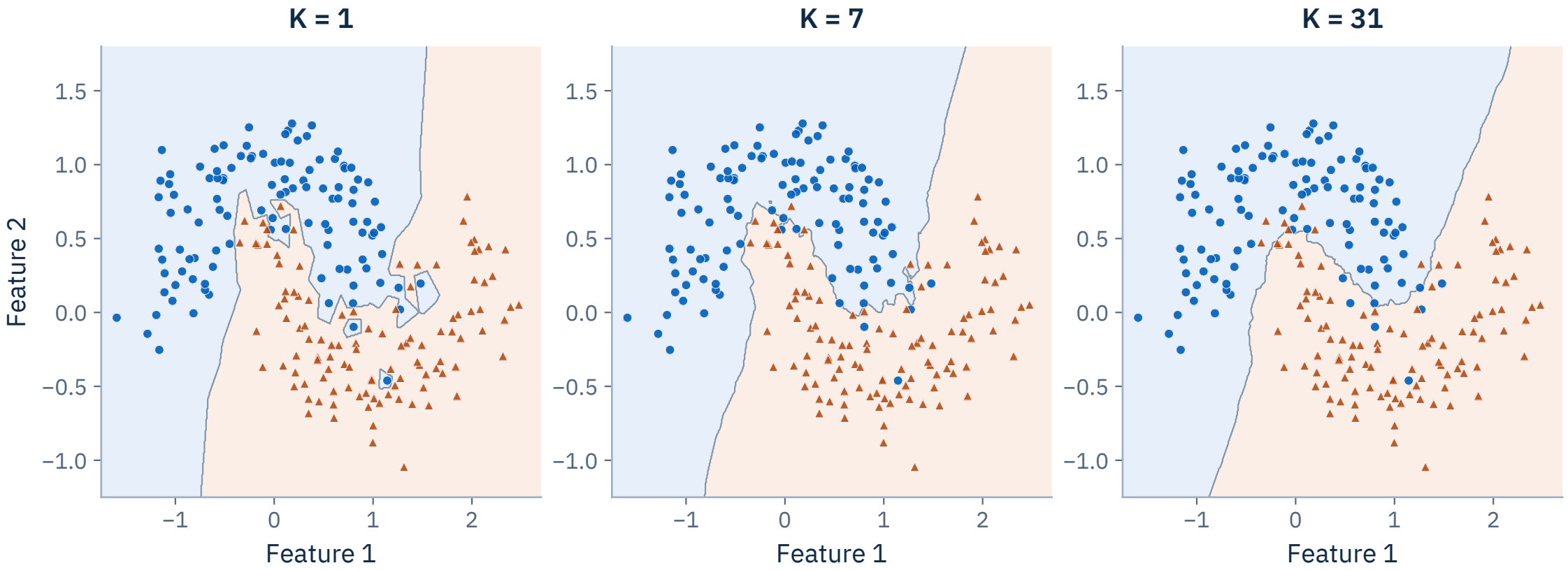
KNN predicts from the nearest labelled examples

Worked vote with K = 5: neighbour labels 1, 0, 1, 1, 0 give class 1, three votes to two.

1. **Measure**
   distance from the query to every training row
2. **Select**
   the K nearest rows
3. **Vote or average**
   majority class, or mean target for regression
4. **Predict**
   break ties by rule or by distance weights

- Standardize with training statistics when units differ; KNN stores rows instead of fitting weights.

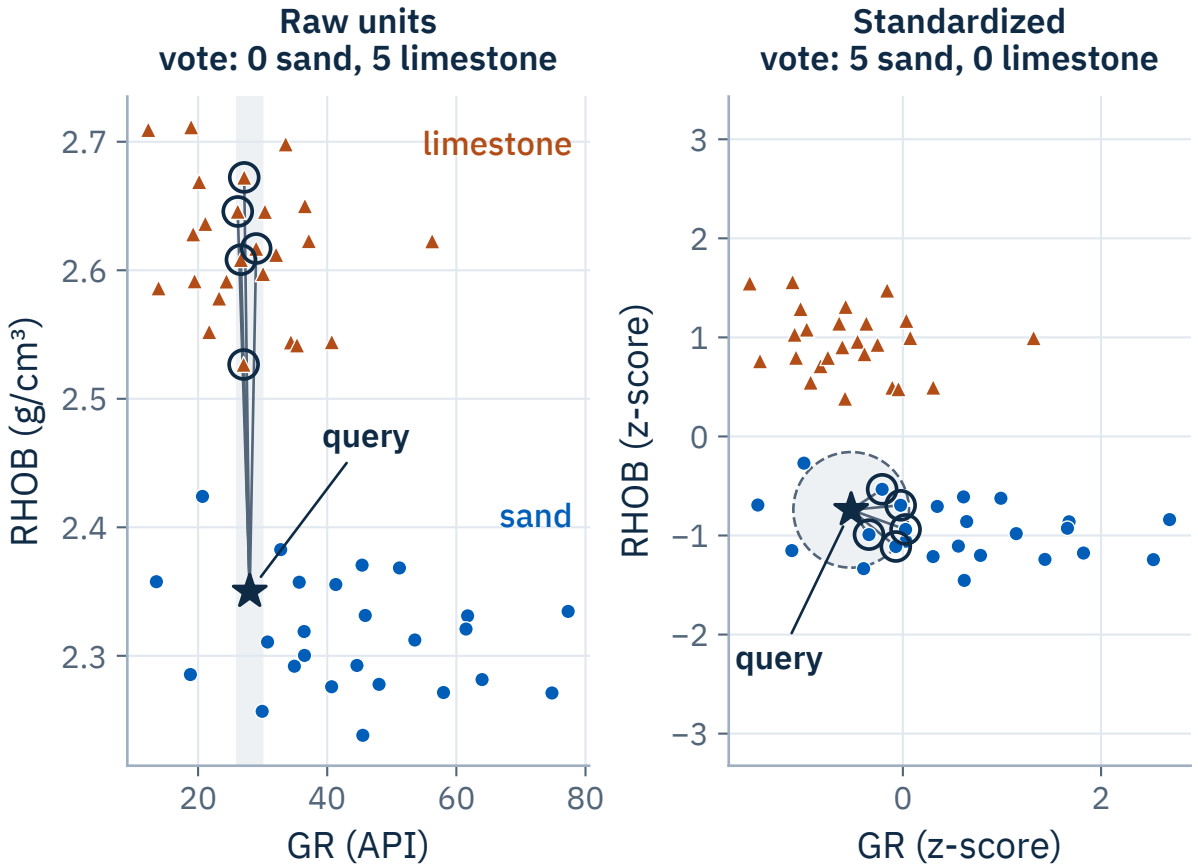
K controls the boundary's detail



Fitted classifiers on the same synthetic data. Backgrounds show predictions; point shape and colour show observed class.

Scaling changes who your neighbours are

- GR spans tens of API units; RHOB spans tenths of a g/cm³
- Raw distance is almost pure GR: all 5 neighbours are limestone
- After standardization RHOB counts: all 5 neighbours are sand



Synthetic sand and limestone (25 each, seed 9); query at GR 28 API, RHOB 2.35 g/cm³ (star); K = 5.

Distance is a modelling choice

Distance between two rows a and b over features j: KNN, RBF kernels and clustering all depend on it.

Euclidean

- $\sqrt{(\sum_j (a_j - b_j)^2)}$
- Large differences dominate; scale matters

Manhattan

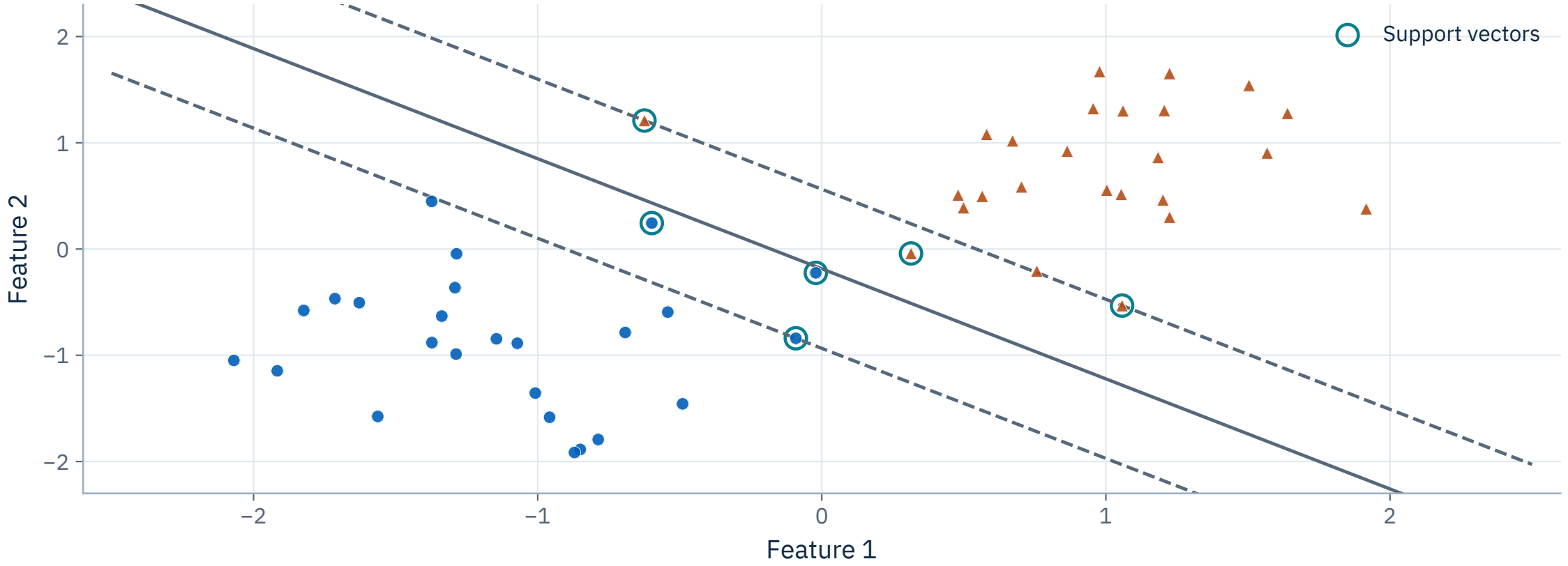
- $\sum_j |a_j - b_j|$
- One large difference counts less

Cosine distance

- $1 - \cos\theta$, θ the angle between the rows
- Compares direction, not magnitude

◆ In many dimensions, nearest and farthest points become harder to tell apart (Module 4).

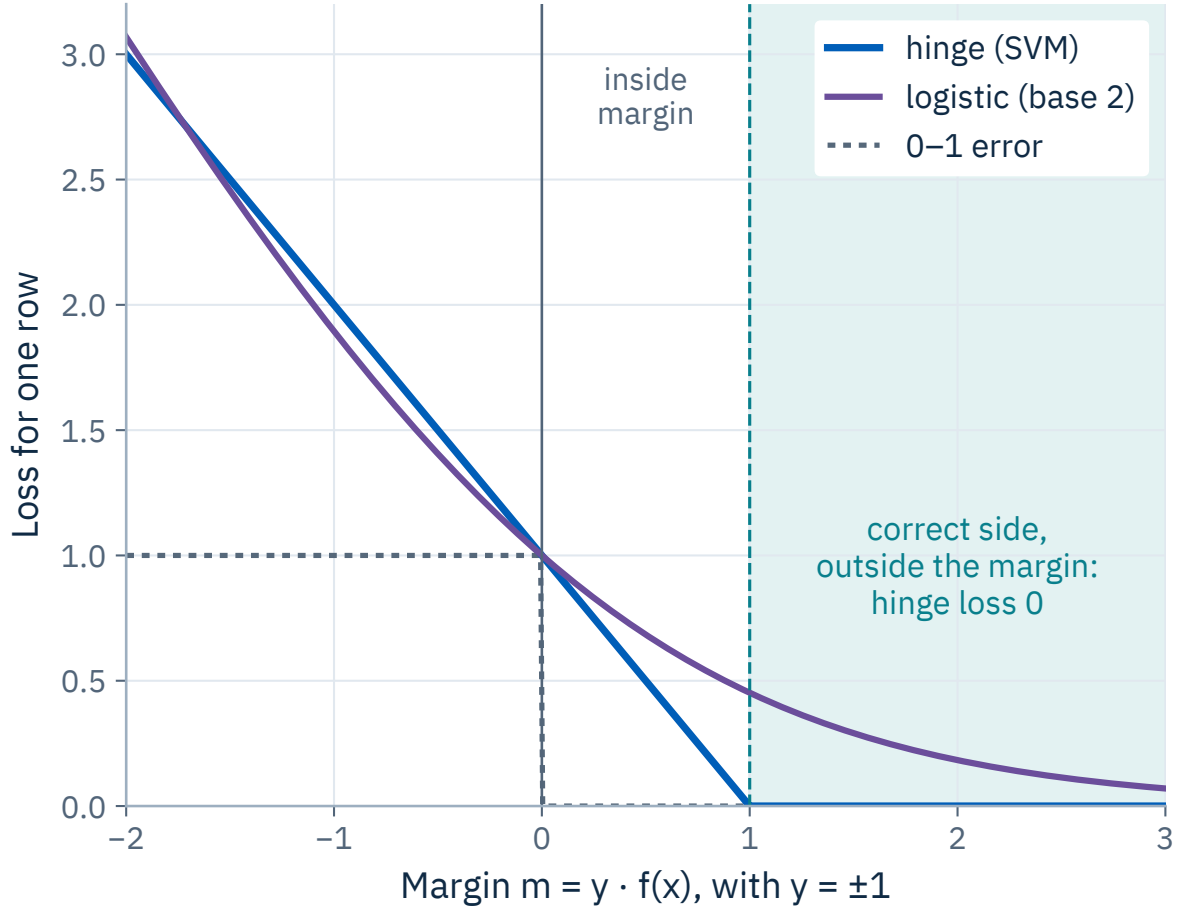
SVM separates classes with a margin



Linear SVM fitted to synthetic data. Dashed lines mark the margins; circled observations are support vectors.

Hinge loss ignores points beyond the margin

- Margin m = y·f(x), with labels y = −1 or +1
- **Hinge loss** max(0, 1 − m) is zero once m ≥ 1
- Only rows on or inside the margin shape the fit: the support vectors
- Logistic loss never reaches zero, so every row contributes



Exact loss functions of the margin m = y·f(x).

The C parameter trades margin for violations

$$\min_{w,\,b} \frac{1}{2}\|w\|^2 + C\sum_i \max(0,\, 1 - y_i f(x_i))$$

Small C

- Violations are cheap: wider margin
- More support vectors, smoother boundary

vs

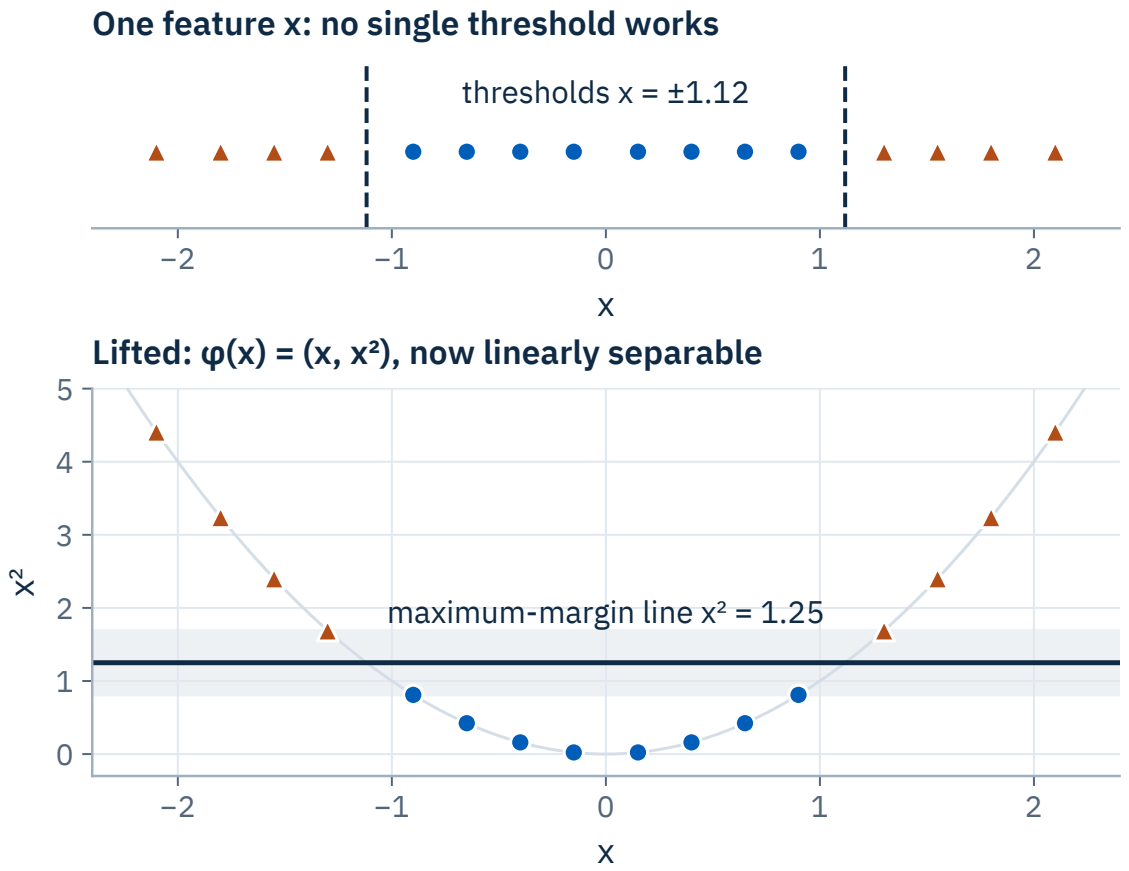
Large C

- Violations are expensive: narrower margin
- Chases hard training rows; can fit noise

◆ Search C and γ together after scaling; SVM scores are not probabilities by default.

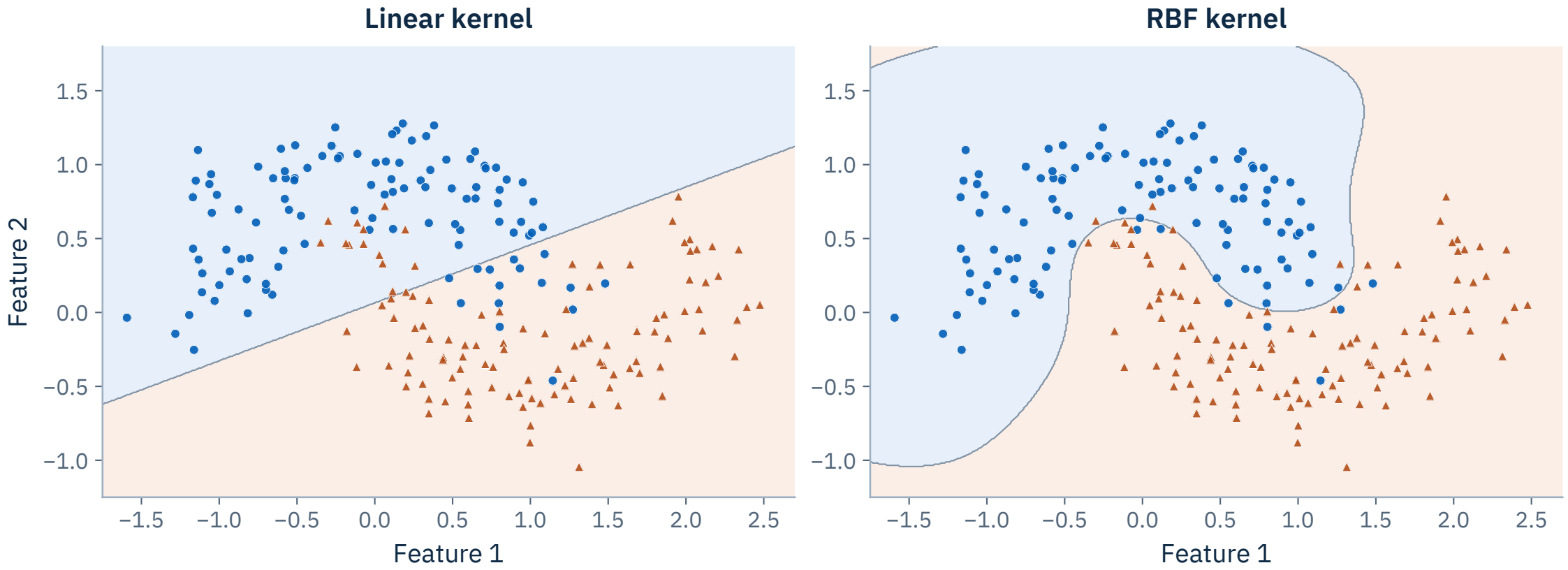
The kernel trick lifts data into a separable space

- One feature, class 0 in the middle: no single threshold works
- Map x to (x, x²): a straight line now separates the classes
- Back in x, that line becomes two thresholds at ±1.12
- The **kernel trick** computes such similarities without building φ(x)



Symmetric toy data, 8 points per class; maximum-margin line x² = 1.25 in the lifted space.

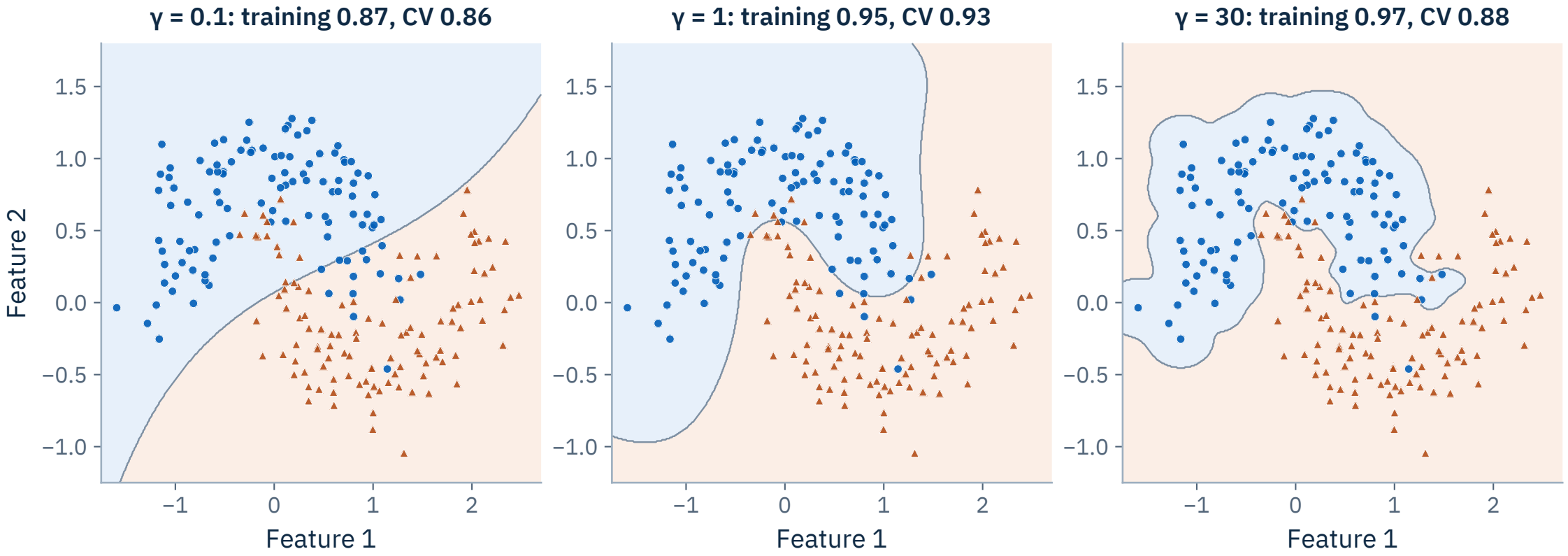
Kernels change the space of boundaries



Linear and RBF SVM fits on the same synthetic observations. The RBF model can represent a curved boundary.

Gamma sets how far each example's influence reaches

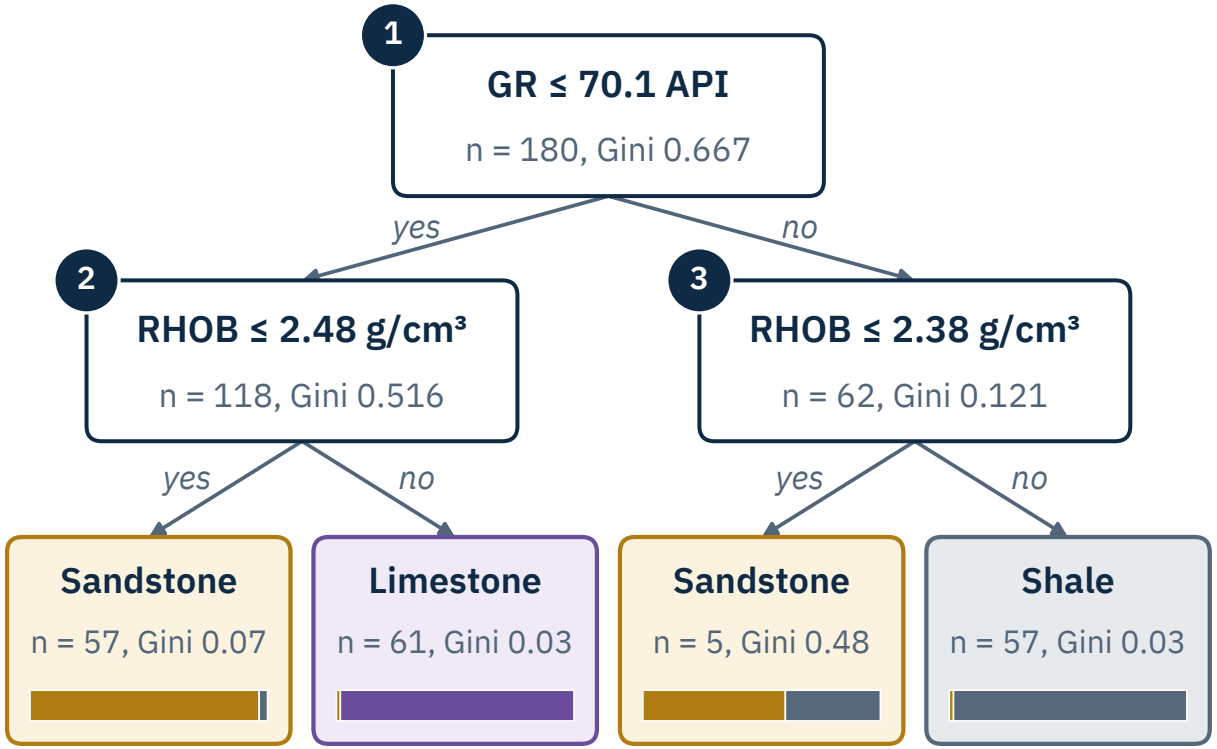
RBF kernel exp(−γ × squared distance): larger γ gives each training row a more local influence.

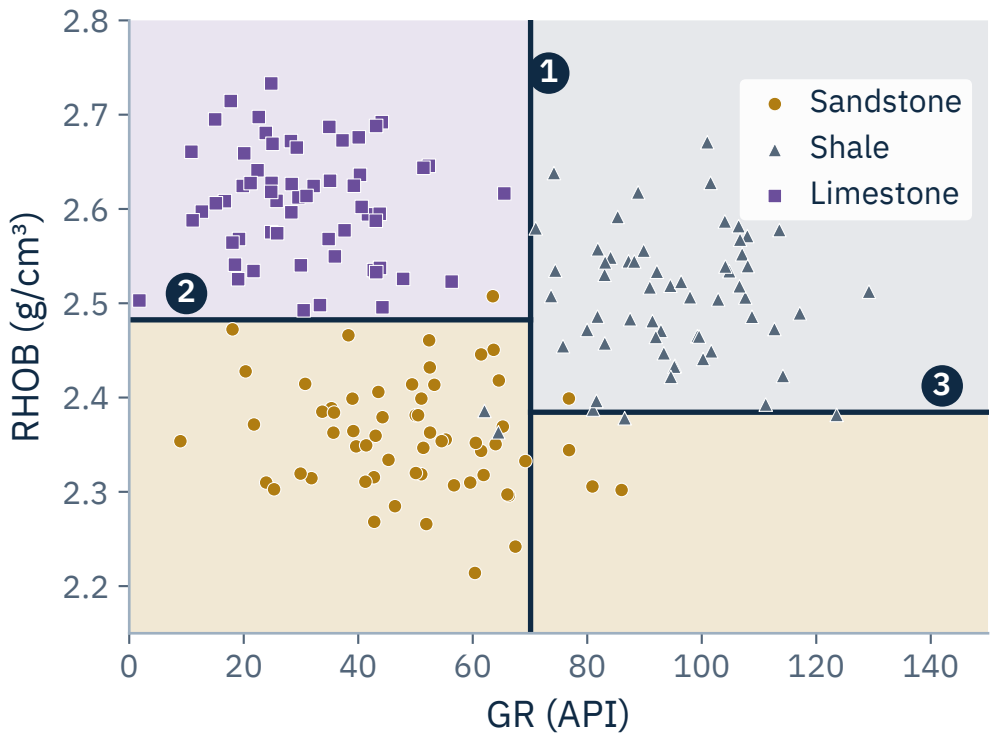


Same synthetic moons as the shared figures (seed 42); standardized RBF SVM with C = 1; accuracy on training rows and 5-fold stratified CV.

Trees ask a sequence of threshold questions

Each internal node tests one feature against a threshold; each leaf predicts its training majority.



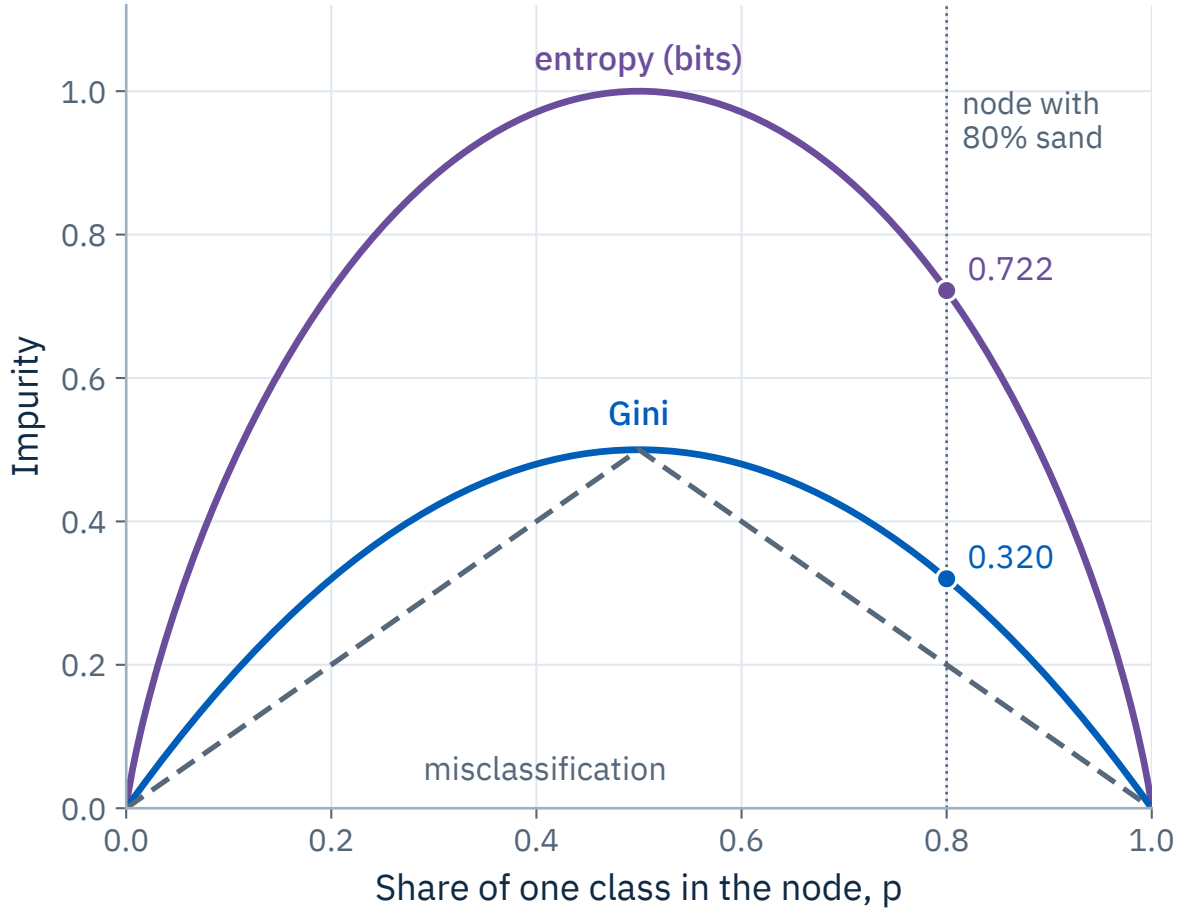


Synthetic facies samples (60 per class, seed 2); a depth-2 tree drawn from scikit-learn's fitted tree_ arrays.

Impurity measures how mixed a node is

$$\text{Gini} = 1 - \sum_k p_k^2, \qquad H = -\sum_k p_k \log_2 p_k$$

- Both are zero for a pure node and largest at 50/50
- A split is scored by the drop in size-weighted child **impurity**
- Gini and entropy usually choose similar splits



Exact two-class impurity functions; marked values at p = 0.8.

PRACTICE

5 min · Pairs

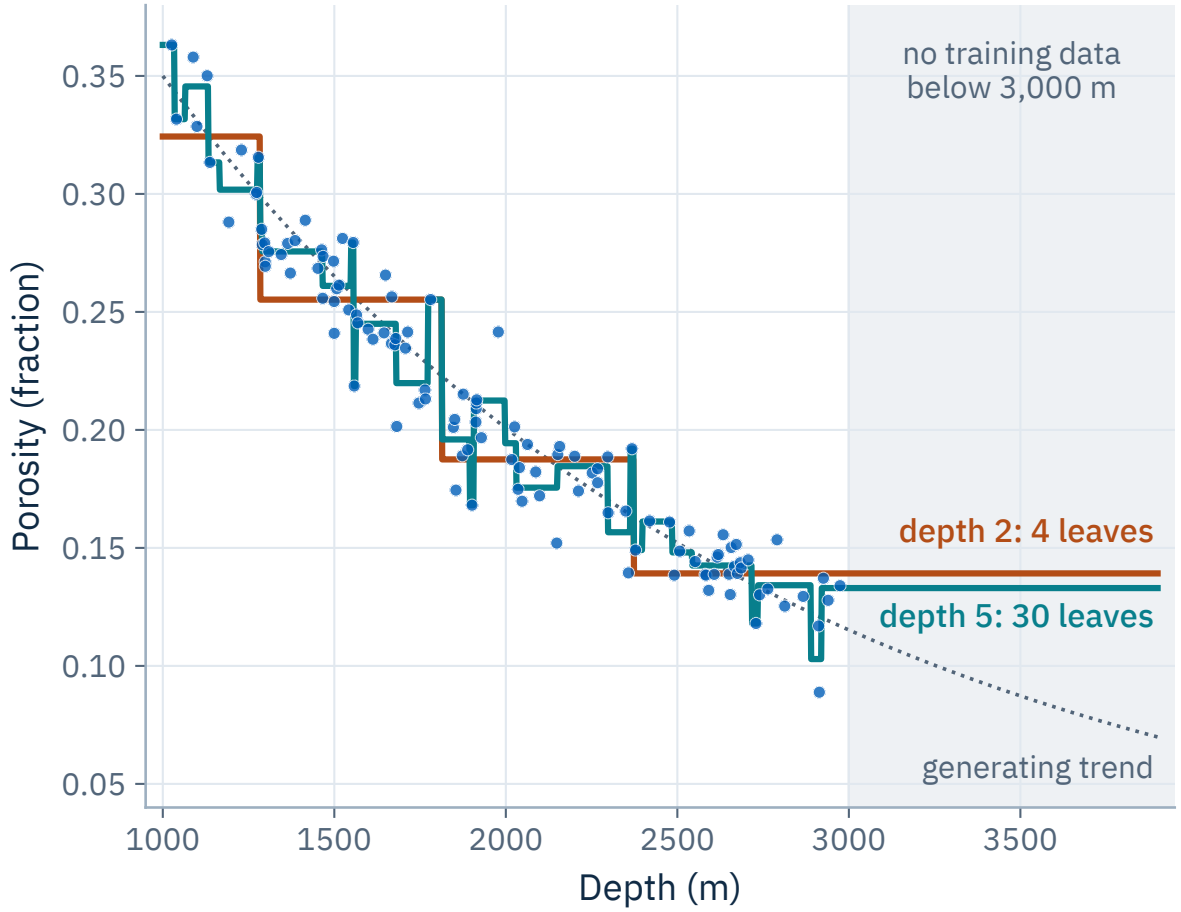
Choose the better first split

Candidate split	Left child: sand / shale	Right child: sand / shale
A: GR ≤ 60 API	8 / 2	2 / 8
B: RHOB ≤ 2.40 g/cm³	9 / 5	1 / 5

1. The parent holds 10 sand and 10 shale: compute its Gini impurity.
2. Compute each child's Gini, weight by child size, and pick the split.

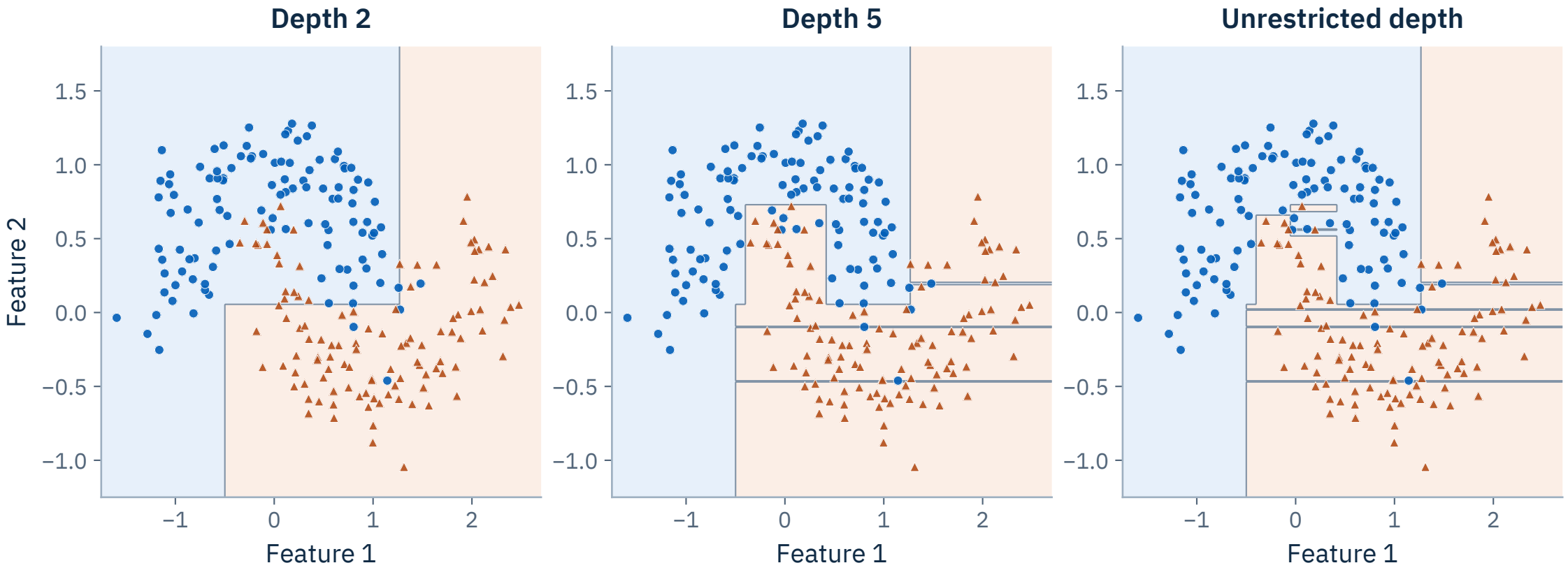
Regression trees predict a constant in each leaf

- Each leaf predicts the mean target of its training rows
- Splits are chosen to reduce the squared error
- The fit is a staircase and cannot extrapolate beyond the data
- Depth and min_samples_leaf set the number of steps



Synthetic compaction trend (seed 10), 120 samples; trees of depth 2 and 5.

Depth controls tree complexity



Trees fitted to the same synthetic observations. More depth permits smaller regions and finer partitions of the training data.

LABORATORY

45 min

Lab 2: classifiers and a facies split design

- ☐ §3: KNN on Iris; plot training and CV accuracy against K
- ☐ §4: trees of depth 1, 2, 3, 5 and unlimited; lock one model, test it once
- ☐ Facies notebook §2–4: audit missing and implausible values by well
- ☐ Facies notebook §6 and §9: write a well-grouped evaluation and tuning plan

NOTEBOOK

`exercises/01_supervised_learning_basics/Supervised_Learning_Basics_Student.ipynb`

EVIDENCE TO SUBMIT

- Accuracy against K and depth, with a majority-class baseline
- A facies data-audit table by well
- A split and tuning plan for unseen wells

DISCUSSION

Lab 2 debrief: the facies file needs a well-aware plan

Computed from facies_dataset.csv: 4,149 rows from 10 wells.

22%

PE missing; 100% in two wells

12

PE values of 326–328.5 (median 3.6)

25

GR values above 200 API

10

wells: the real sample size for CV

◆ Same forest, same nine wells: macro F1 0.76 with shuffled row folds, 0.38 with well folds.

PART 6 OF 6

06

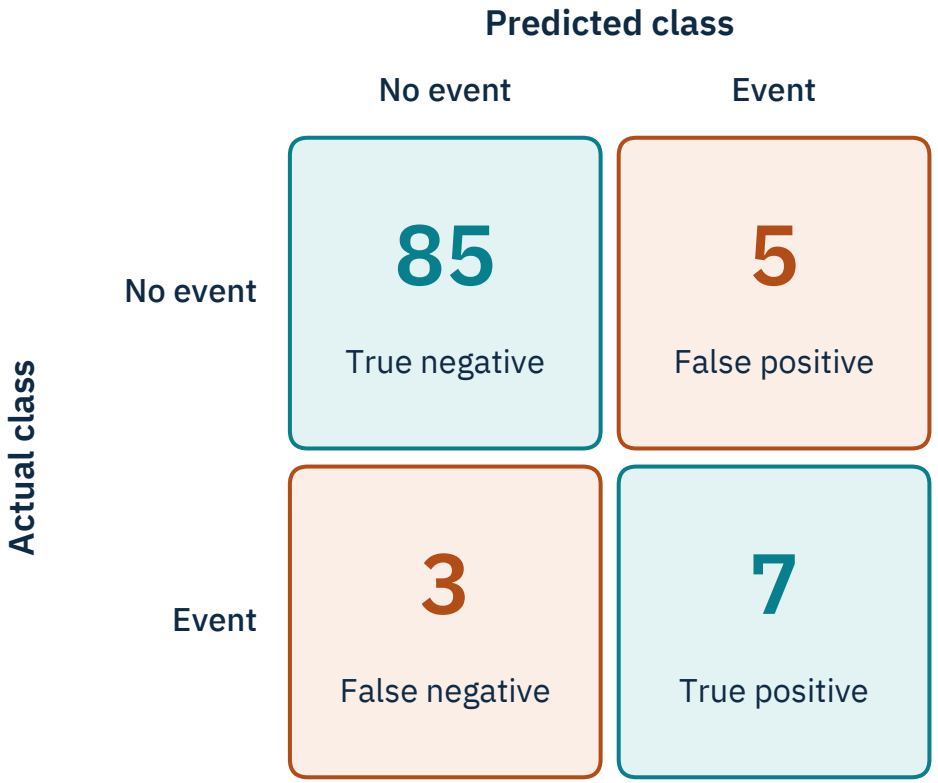
Evaluating and selecting models

A score is only as good as the experiment behind it.

- Confusion matrices and metric choice
- ROC curves and thresholds
- Multi-class averages
- Pipelines, grouped tuning and fair comparison

A confusion matrix preserves the counts

- Rows are actual classes; columns are predictions
- False positives are false alarms; false negatives are missed events
- State which class is the event before quoting any metric
- Some tools transpose this layout: check before reading



Illustrative counts: 100 observations, 10 actual events. Not model performance.

PRACTICE

4 min · Pairs

Accuracy can hide the minority class

Using TN = 85, FP = 5, FN = 3, TP = 7:

1. Calculate accuracy, event precision and event recall.
2. Compare with a model that always predicts no event.
3. Which error would matter most for the proposed use?

DISCUSSION

Never alerting still scores 90% accuracy

Same 100 observations, 10 real events:

92%

accuracy of the model

58%

event precision: 7 of 12 alerts

70%

event recall: 7 of 10 events

90%

accuracy of never alerting

- Report per-class precision and recall whenever events are rare.

Metric choice follows the decision

Precision

- How often an alert is right
- TP / (TP + FP)

Recall

- Share of real events caught
- TP / (TP + FN)

F1

- Harmonic mean of the two
- Encodes no business costs

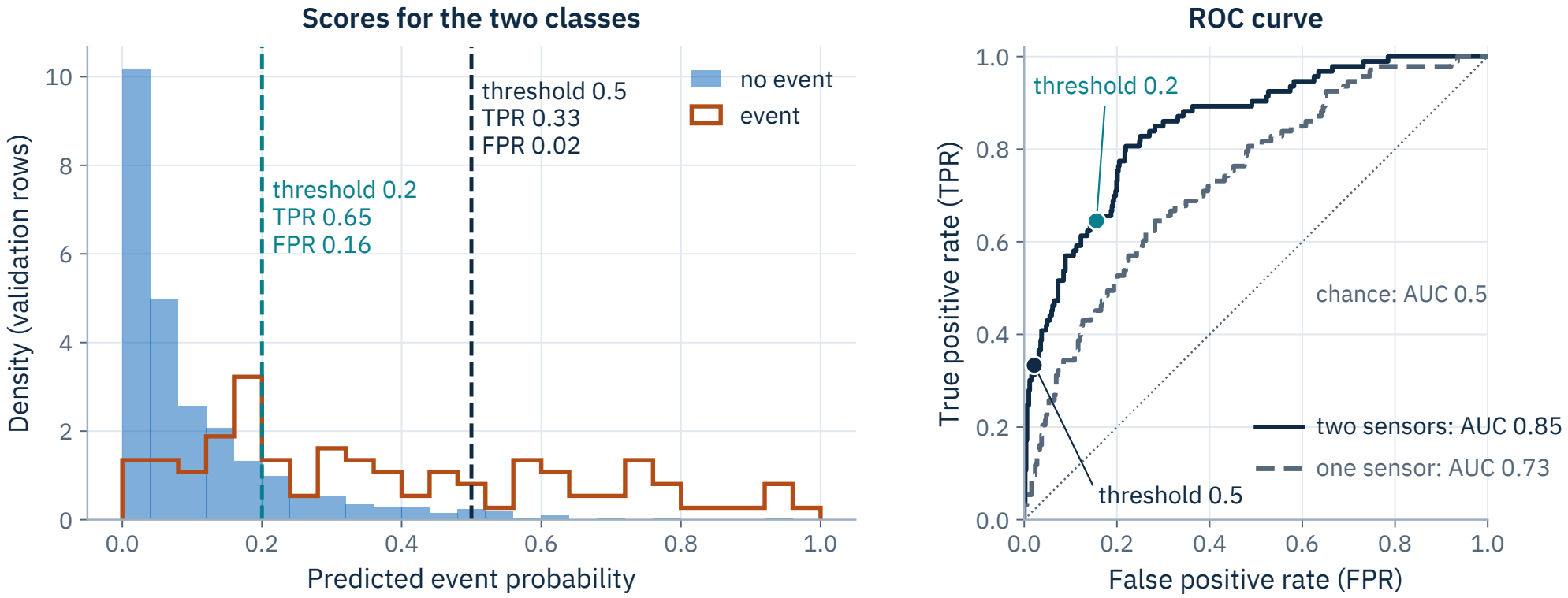
Macro average

- Mean over classes
- Rare facies count as much as common ones

◆ Agree the metric with the decision owner before looking at any scores.

ROC curves summarize every threshold at once

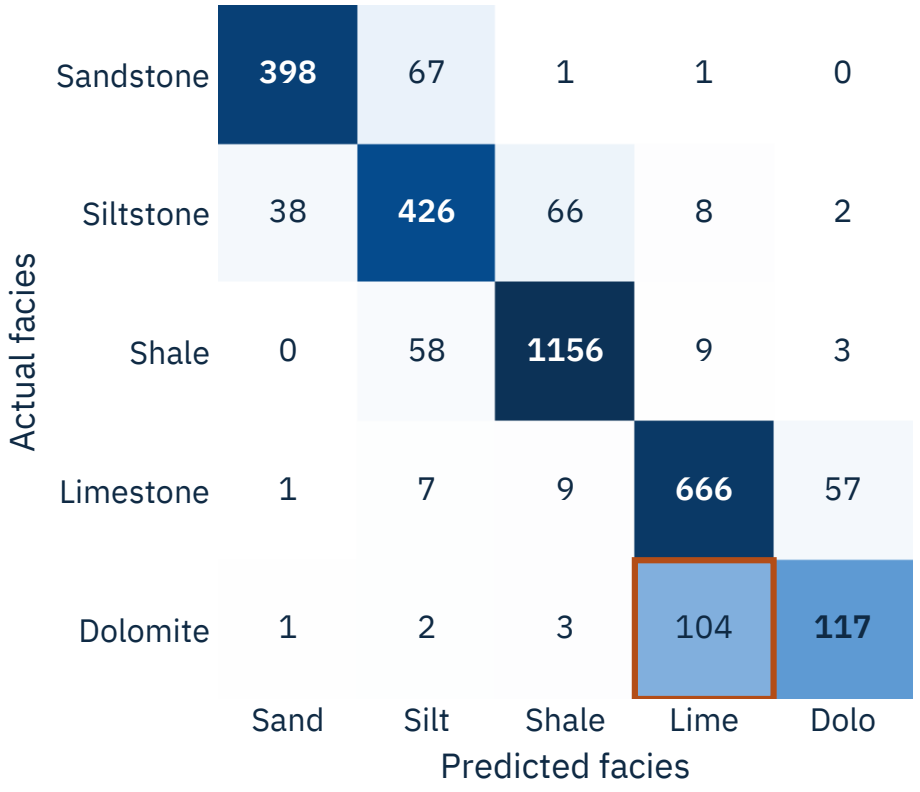
Moving the threshold trades false alarms for missed events; **AUC** summarizes the ranking.

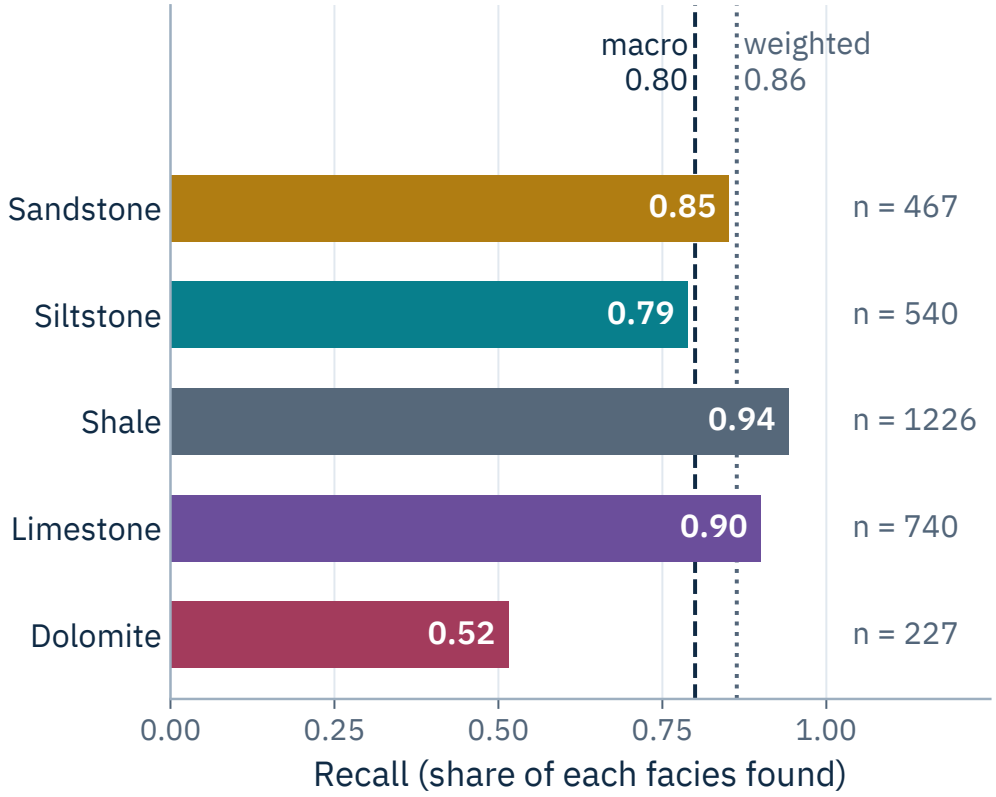


Synthetic ESP-trip data, 15% events (seed 13); logistic regression trained on 1,000 rows, evaluated on 600 validation rows. Precision–recall curves for rare events: Module 5.

Multi-class errors hide off the diagonal

Dolomite is found only about half the time; most of its misses are called limestone.





Synthetic 8-well facies data (seed 7); standardized RBF SVM, leave-one-well-out predictions. Colour = share of each actual facies.

Macro and weighted averages answer different questions

Recall on the synthetic facies example, from the same leave-one-well-out predictions:

0.52–0.94

per facies: one value per class

0.80

macro: every facies counts equally

0.86

weighted by frequency: equals accuracy

◆ A rare facies found poorly drags the macro average down; the weighted average hides it.

A pipeline keeps training steps together

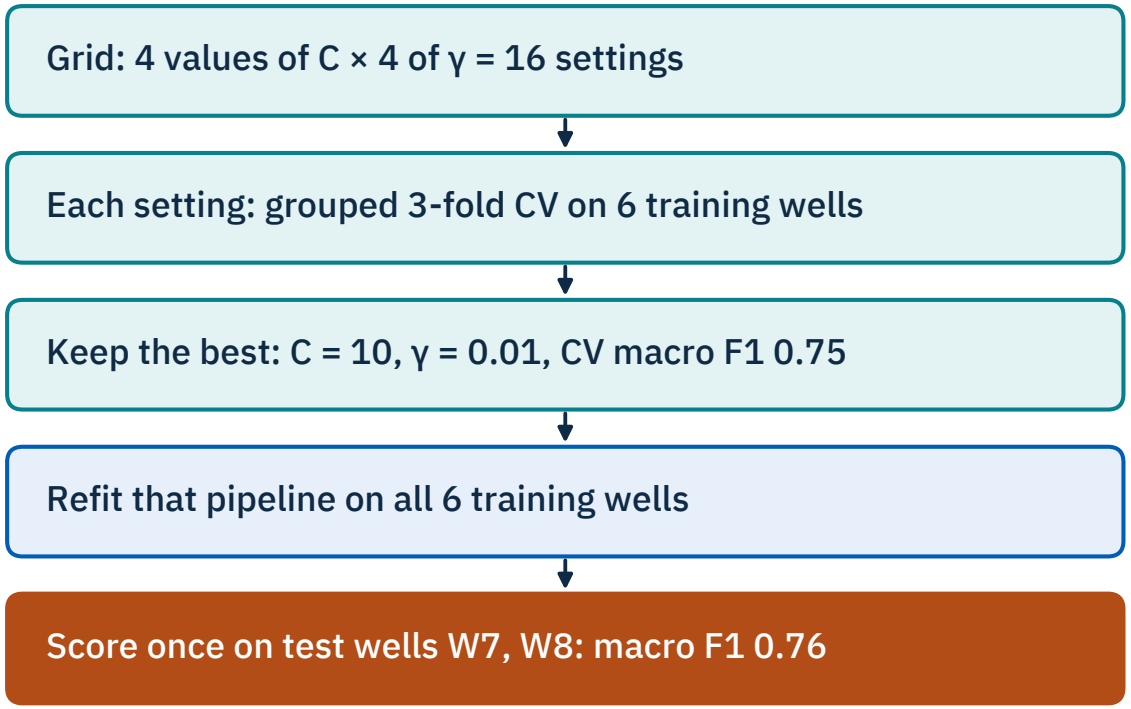
```
from sklearn.pipeline import make_pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

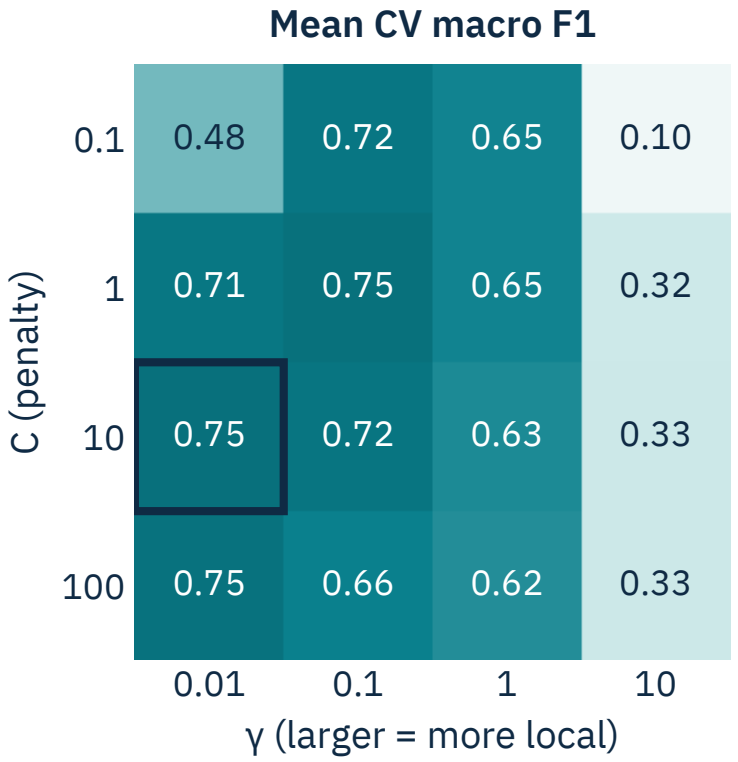
model = make_pipeline(
    SimpleImputer(strategy="median", add_indicator=True),
    StandardScaler(),
    SVC(kernel="rbf"),
)
```

Every cross-validation fit learns fresh imputation and scaling statistics.

Grid search scores every setting with grouped CV

16 settings × 3 grouped folds = 48 fits and one refit; the test wells are opened once.





Synthetic 8-well facies data (seed 7): GridSearchCV with GroupKFold(3) on wells W1–W6; W7–W8 as the test set.

Grouped tuning respects well boundaries

```python
from sklearn.model_selection import GroupKFold, GridSearchCV

search = GridSearchCV(
    model,
    {"svc__C": [0.1, 1, 10, 100], "svc__gamma": [0.01, 0.1, 1, 10]},
    cv=GroupKFold(n_splits=3), scoring="f1_macro",
)
search.fit(X_train, y_train, groups=well_train)
print(search.best_params_, round(search.best_score_, 2))
```

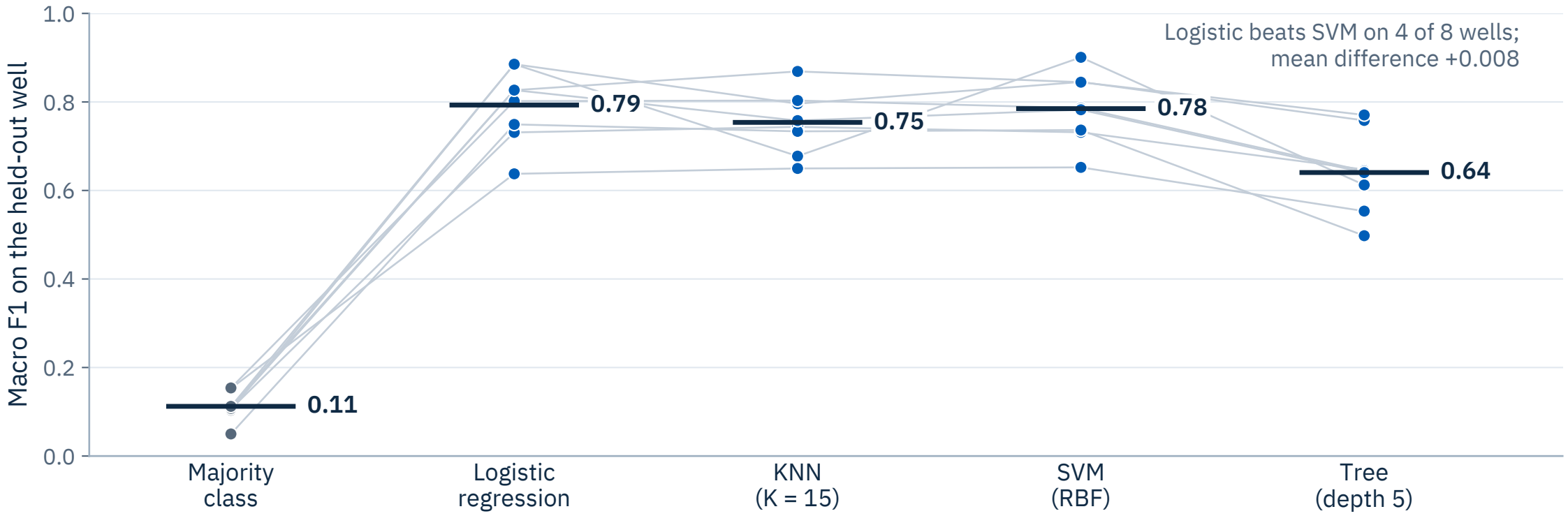
OUTPUT

```
{'svc__C': 10, 'svc__gamma': 0.01} 0.75
```

Run on training wells W1–W6 of the synthetic data; W7 and W8 stay closed.

Compare models by their fold-score distributions

Same eight held-out wells for every pipeline: read the spread and the paired lines.



Synthetic 8-well facies data (seed 7); leave-one-well-out macro F1 for five pipelines on identical folds. Grey lines join scores of one well.

Model comparison is a controlled experiment

Candidate	Strong starting point when	Main check
Linear or logistic	Effects are roughly additive	Residual or probability structure
KNN	Local similarity is meaningful	Scaling and prediction cost
SVM	A margin or kernel helps	C, γ and calibration
Tree	Threshold interactions are plausible	Leaf size and stability

Hold the split, folds and metric fixed; the baseline row comes first.

◆ Pair every score with an error review and the runtime at the point of use.

LABORATORY

25 min

Lab 3: a tuned pipeline and a locked test

- §1: explain why PCA fitted on all rows leaks information
- §2–3: fit the 2-D teaching models and record the retained variance
- §4: run the CV grid search; record the kernel, C and γ it selects
- §5: add PCA inside the pipeline; compare 2, 10 and 30 components
- §6: set RUN_FINAL = True once, after choosing, and report that score

NOTEBOOK

`exercises/03_svm/SVM_Exercise_Student.ipynb`

EVIDENCE TO SUBMIT

- What each partition was used for
- A matched comparison table
- One final score with its caveat

DISCUSSION

Lab 3 debrief: small holdouts give noisy verdicts

Reference run of the SVM notebook on the local Sonar file (208 rows):

124/42/42

training, validation and final rows

39%

variance kept by 2 principal components

0.79

CV balanced accuracy of the chosen RBF SVM

2.4

accuracy points per row in a 42-row holdout

- The same model scored 0.92 on the 42 validation rows: one procedure, two noisy numbers.

Key takeaways

- ✓ Write the prediction contract before touching a model.
- ✓ Audit the data: null codes, units, depth shifts and patterned gaps.
- ✓ Make the split answer the deployment question: group by well, respect time.
- ✓ Fit every preprocessing step inside the training fold of a pipeline.
- ✓ Compare regularized candidates with a baseline on identical grouped folds.
- ✓ Report per-class metrics and all fold scores; open the test set once.

PRACTICE

5 min | Individual

Exit check: four questions before you leave

1. Why must scaling run inside a cross-validation pipeline?
2. When is a random row split misleading for well logs?
3. A shale sample was predicted at p = 0.1. What is its log loss, and why is it large?
4. What evidence would justify a more complex model?

References and further study

- scikit-learn User Guide: linear models, neighbours, SVM, trees, cross-validation and common pitfalls.
- James, Witten, Hastie, Tibshirani: *An Introduction to Statistical Learning*; Hastie, Tibshirani, Friedman: *The Elements of Statistical Learning*.
- Kaufman, Rosset, Perlich, Stitelman (2012). Leakage in data mining: formulation, detection, and avoidance. *ACM TKDD* 6(4).
- Roberts et al. (2017). Cross-validation strategies for data with temporal, spatial, hierarchical, or phylogenetic structure. *Ecography* 40.
- Hall (2016). Facies classification using machine learning. *The Leading Edge* 35(10).

MACHINE LEARNING / MODULE 2

Ensembles, interpretation, and pipelines

Voting, bagging, boosting, honest explanations, and a predictor you can deploy

Full day · 09:00–17:00 | 6 teaching parts | 2 laboratories | Builds on Module 1

Today's plan

09:00	Welcome, warm-up, and a drilling case	Lecture
09:15	Part 1: Why combining models works	Lecture
09:50	Part 2: Bagging and random forests	Lecture
10:35	Break	Break
10:50	Part 3: From AdaBoost to gradient boosting	Lecture
11:45	Lab 1: tree, forest, and boosting compared	Lab
12:45	Lunch	Lunch
13:45	Part 4: Boosting libraries and stacking	Lecture
14:15	Part 5: Interpreting ensemble models	Lecture
15:00	Break	Break
15:15	Part 6: Pipelines, leakage, and deployment	Lecture
15:50	Lab 2: importance, pipelines, and tuning	Lab
16:45	Evidence review and exit check	Discussion

What you will be able to do

By the end of the day you will be able to:

1. Explain when voting or averaging reduces error, using member accuracy and correlation.
2. Fit and tune random forests, and weigh out-of-bag scores against grouped validation.
3. Trace AdaBoost and gradient-boosting updates and control them with depth, learning rate, and early stopping.
4. Choose between a forest, boosting, and stacking under a validation and deployment budget.
5. Interpret an ensemble with permutation importance, PDP/ICE, and Shapley values, and state their limits.
6. Package preprocessing and model in one pipeline, avoid leakage, and save it with its context.

PRACTICE

4 min · Pairs

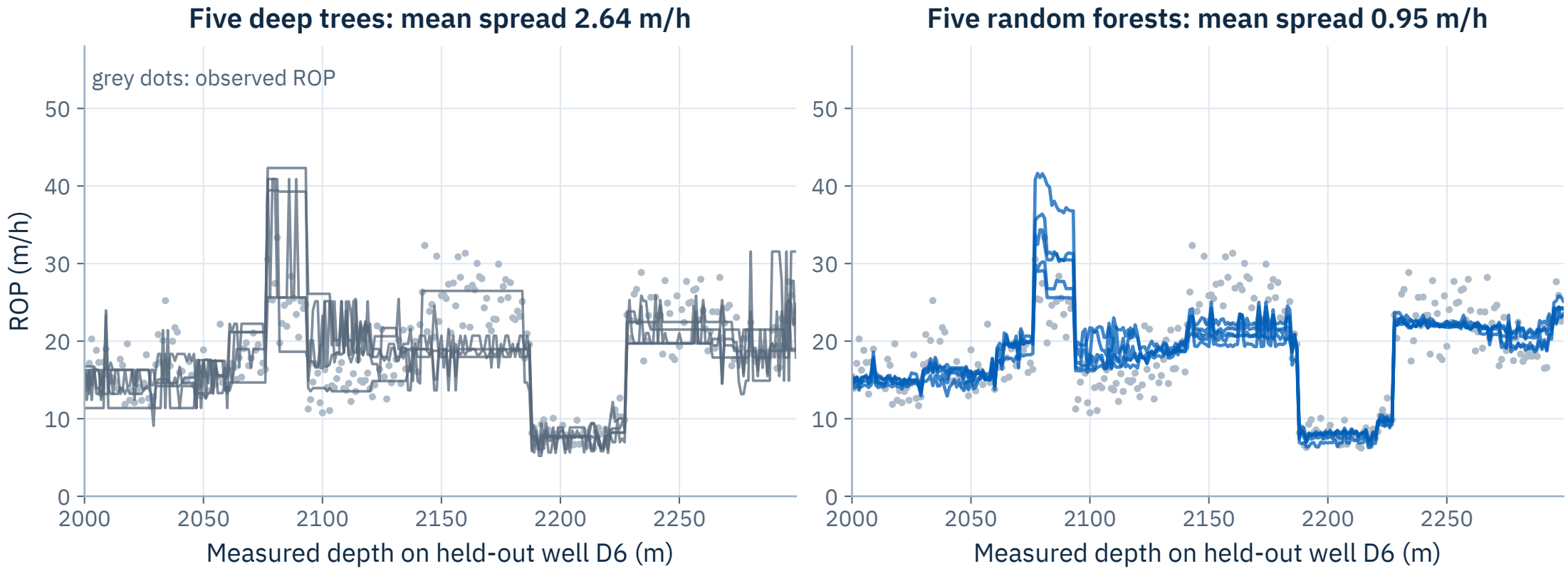
A fragile model can become a useful component

Three classifiers produce event probabilities 0.2, 0.7, and 0.6.

1. What is their mean probability?
2. What prediction results from a 0.5 threshold?
3. What if all three learned the same systematic error?

Retrained trees disagree; retrained forests agree

Five resamples of wells D1–D5; every model predicts rate of penetration (ROP) on well D6.



Synthetic drilling data (seed 11). Spread = standard deviation across the five models, averaged over depth: trees 2.64 m/h, forests 0.95 m/h.

PART 1 OF 6

01

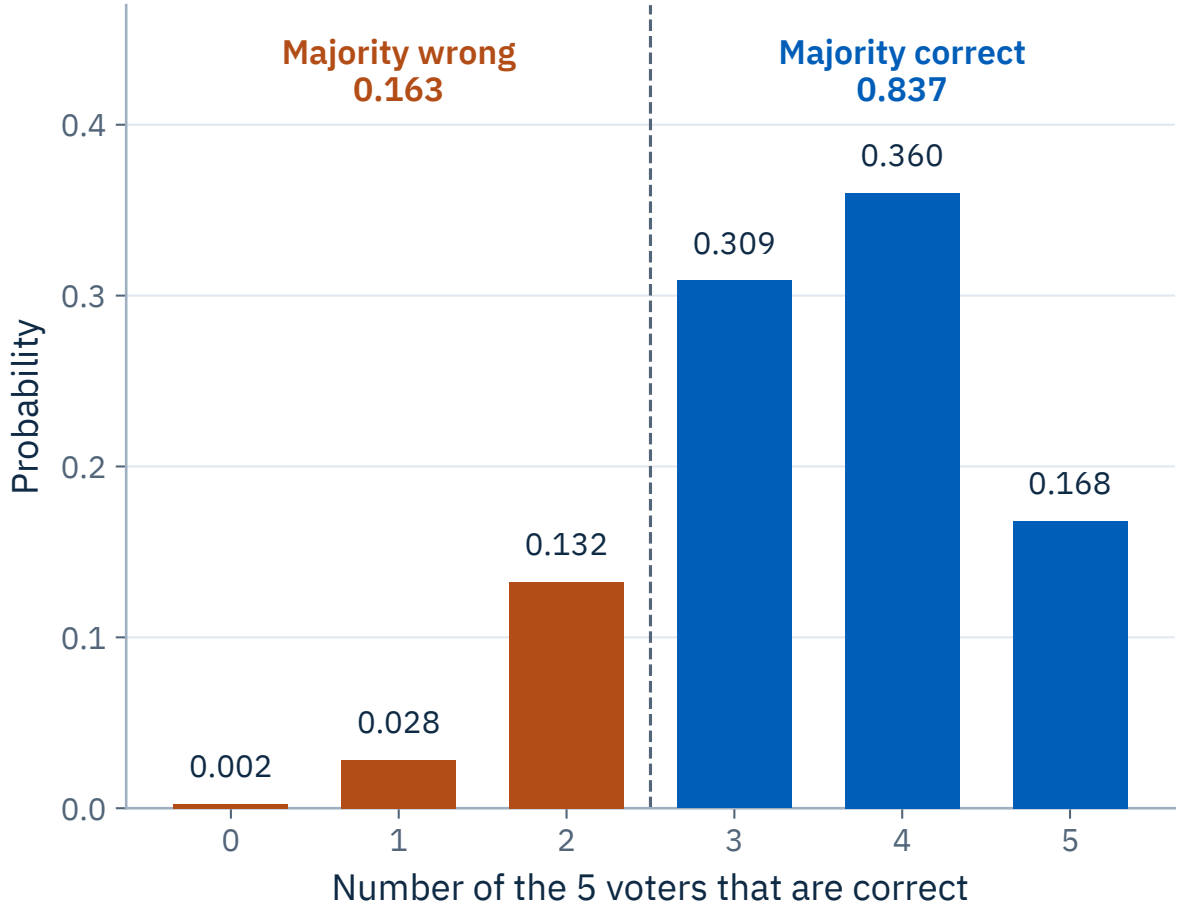
Why combining models works

Many imperfect models can beat one, if their errors differ.

- Majority votes and the binomial argument
- Correlation and the variance floor
- Hard and soft voting
- Diversity by design

Independent errors rarely line up

- Five voters, each right 70% of the time, with independent errors.
- The vote fails only when three or more are wrong: probability 0.163.
- Majority accuracy is 0.837, better than any single member.

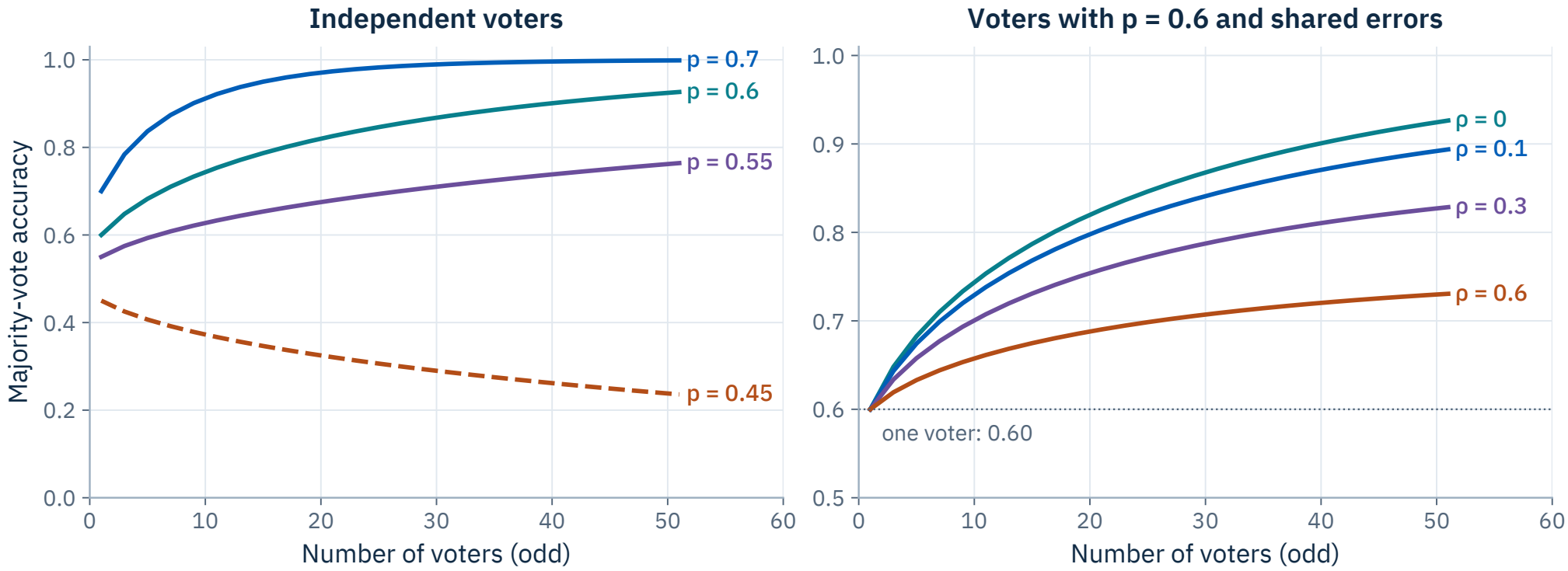


Exact binomial probabilities for n = 5 voters, each correct with p = 0.7.

◆ Independence is the assumption doing the work.

Voting helps only accurate, independent members

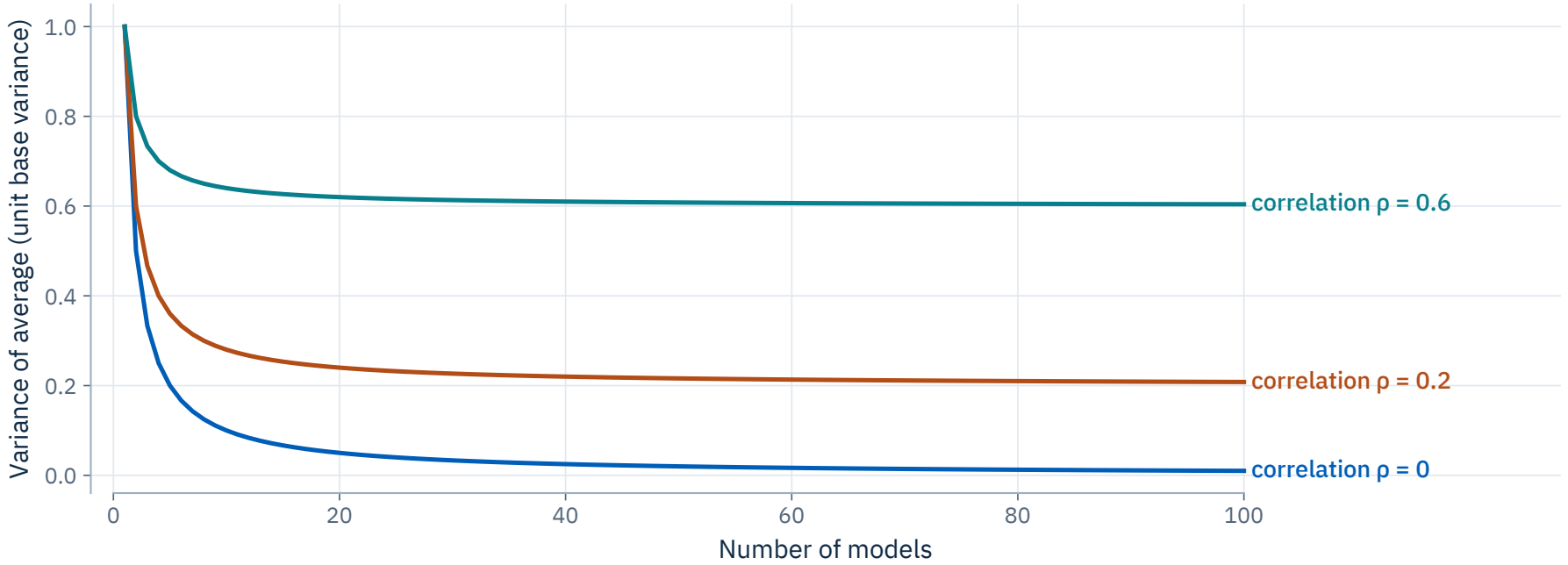
Left: members above 50% improve with numbers, members below 50% get worse. Right: shared errors cap the gain.



Exact binomial majority probabilities. Right: with probability ρ all voters copy one shared vote; otherwise they vote independently.

Correlation leaves a variance floor

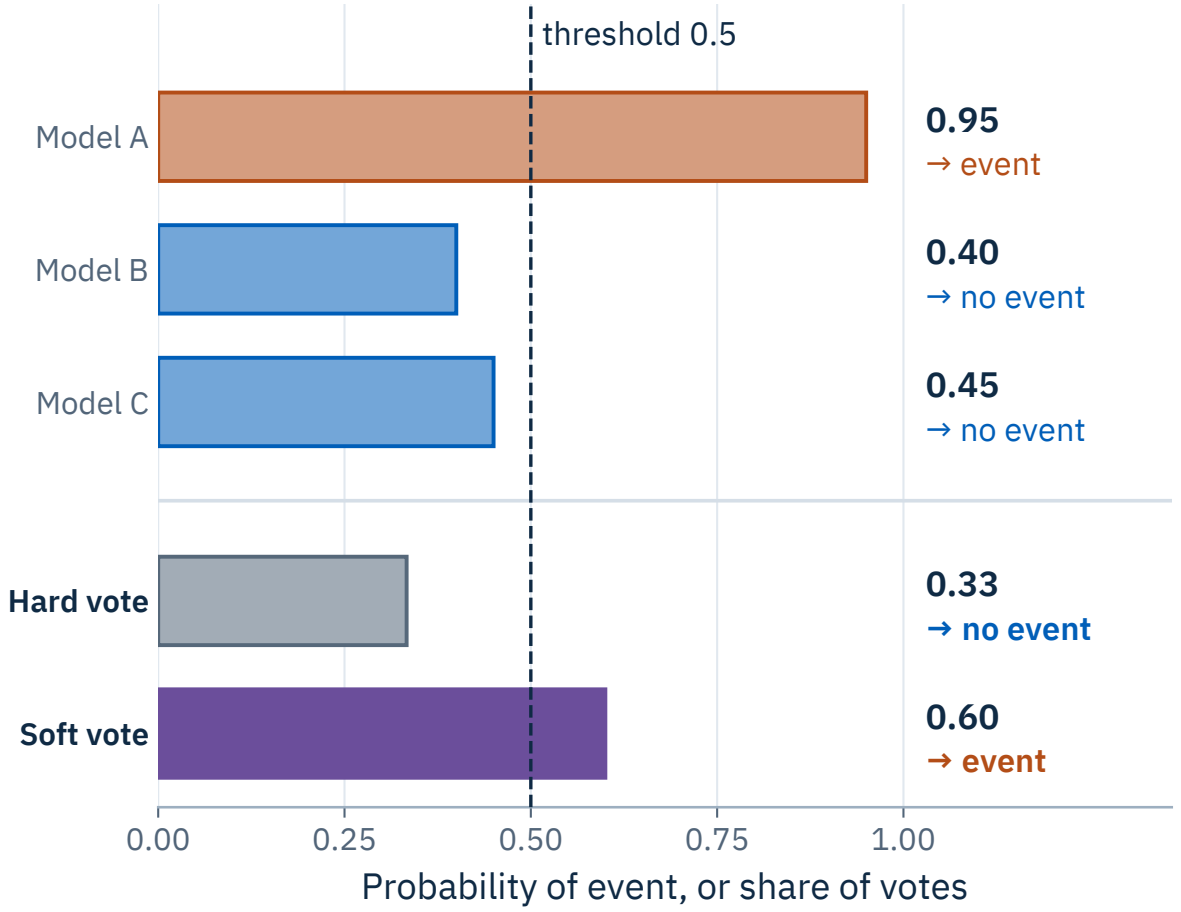
$$\mathrm{Var}(\bar{f}) = \sigma^2\left(\rho + \frac{1-\rho}{M}\right)$$



Analytical curves for unit model variance and several assumed error correlations; not a measured benchmark. Example: ρ = 0.2, M = 10 gives 0.28; the floor is 0.20.

Hard and soft voting can disagree

- Hard vote: each model casts one vote for its predicted class.
- Soft vote: average the predicted probabilities, then apply the threshold.
- Soft voting uses confidence, so members need sensible probabilities.



Exact arithmetic for three illustrative probabilities.

◆ Same three models, opposite decisions.

VotingClassifier mixes model families

```
from sklearn.ensemble import VotingClassifier
members = [
    ("logit", make_pipeline(StandardScaler(), LogisticRegression())),
    ("knn", make_pipeline(StandardScaler(), KNeighborsClassifier(15))),
    ("forest", RandomForestClassifier(300, random_state=0)),
]
vote = VotingClassifier(members, voting="soft")

cv = GroupKFold(n_splits=4)                    # new wells in every fold
for name, model in members + [("soft vote", vote)]:
    s = cross_val_score(model, X, y, groups=wells, cv=cv)
    print(f"{name:10s} {s.mean():.3f}")
```

OUTPUT

```
logit      0.857
knn        0.813
forest     0.776
soft vote  0.841
```

- The vote trails its best member here: combining is a hypothesis to test.

Diversity comes from rows, features, and algorithms

Different rows

- Bootstrap samples (bagging, random forests)
- Row subsampling in stochastic boosting

Different features

- Random subset of features at each split
- Random feature subsets per model

Different algorithms

- Linear, neighbours, and trees in one vote
- A learned combination (stacking)

Different randomization

- Random split thresholds (extra-trees)
- Different seeds and initializations

◆ Diversity helps only while each member stays better than chance.

PRACTICE

5 min · Pairs

How much can three voters gain?

Assume independent errors unless stated otherwise.

1. Three classifiers are each correct 80% of the time. What is the majority-vote accuracy?
2. Repeat for three classifiers that are each correct 40% of the time.
3. What is the accuracy if the three are identical copies of one 80% model?

PART 2 OF 6

02

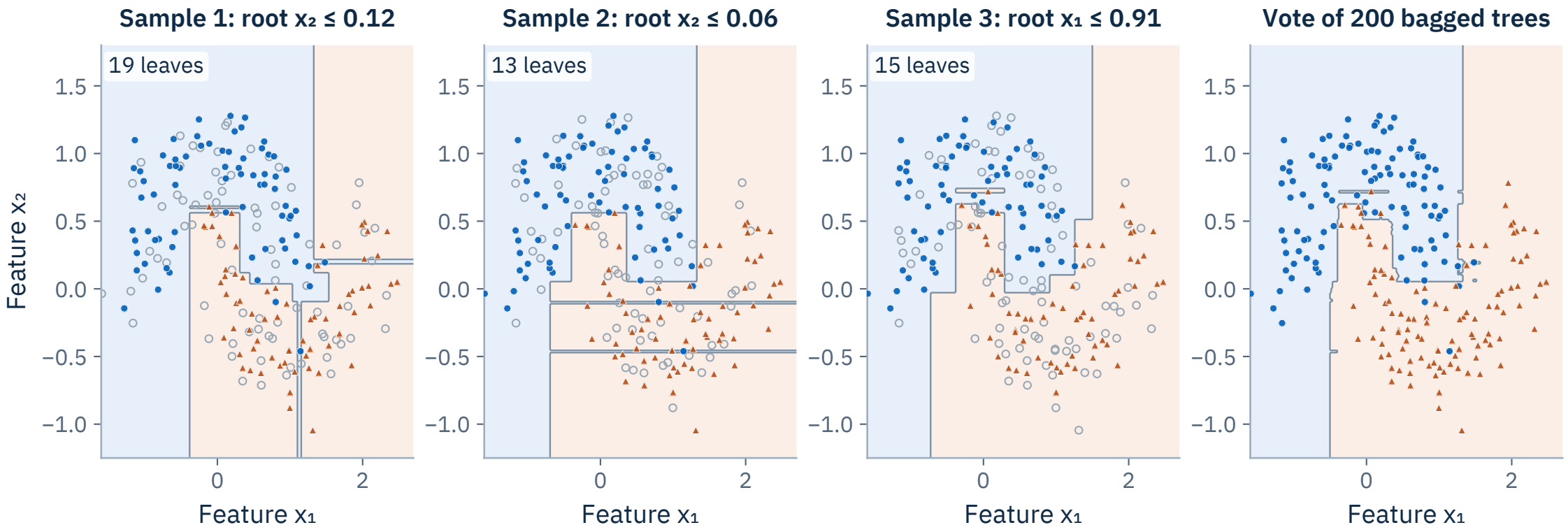
Bagging and random forests

Average many deep trees trained on perturbed data.

- Why single trees are unstable
- Bootstrap aggregation and feature subsampling
- Out-of-bag estimates and their limits
- A facies case on a new well

A deep tree changes with every bootstrap sample

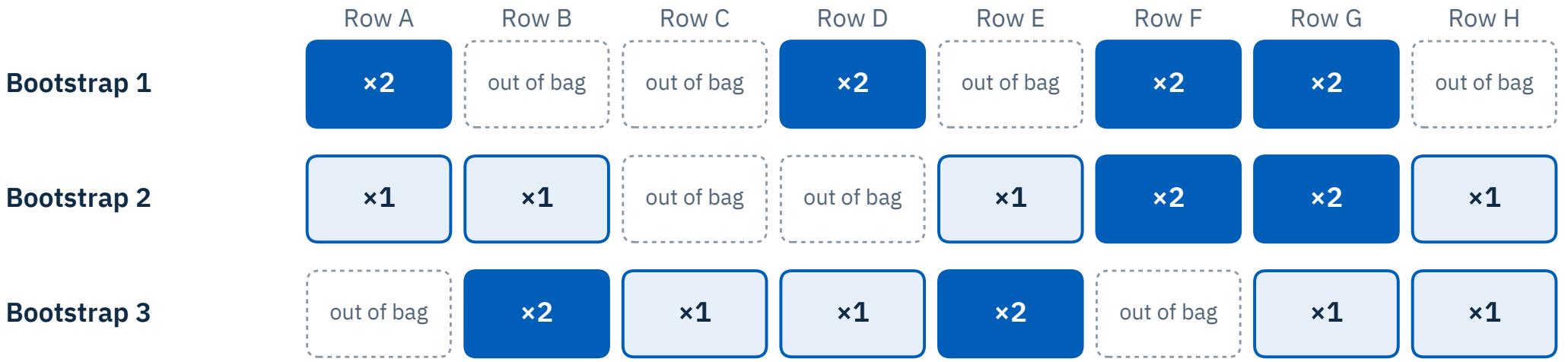
Same 220 rows, three bootstrap resamples: different root splits, leaf counts, and boundaries.



Synthetic two-moons data (seed 42); bootstrap seed 18; unpruned scikit-learn trees. Hollow markers: rows left out of that sample.

Bootstrap samples contain duplicates

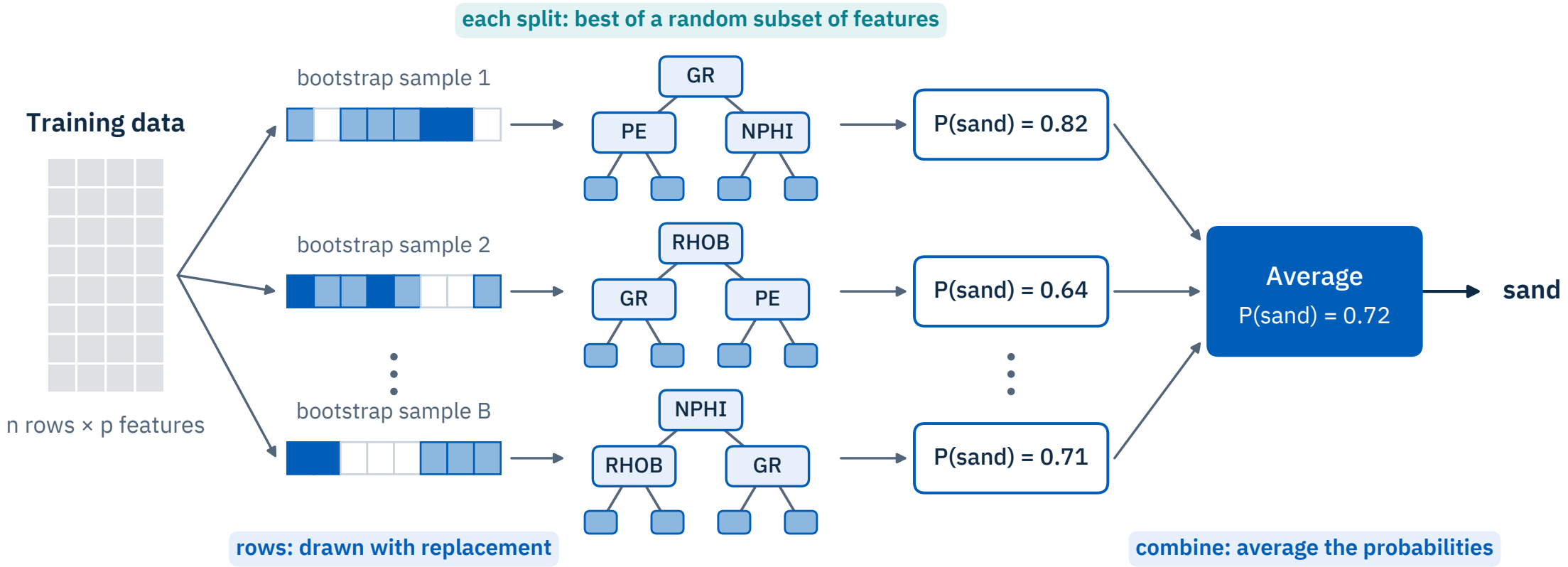
A row drawn twice counts twice in that tree's fit; rows never drawn are its out-of-bag rows.



Each bootstrap draws 8 rows with replacement: some rows repeat, others are left out (out of bag).

One small synthetic bootstrap example (seed 42), n = 8 rows.

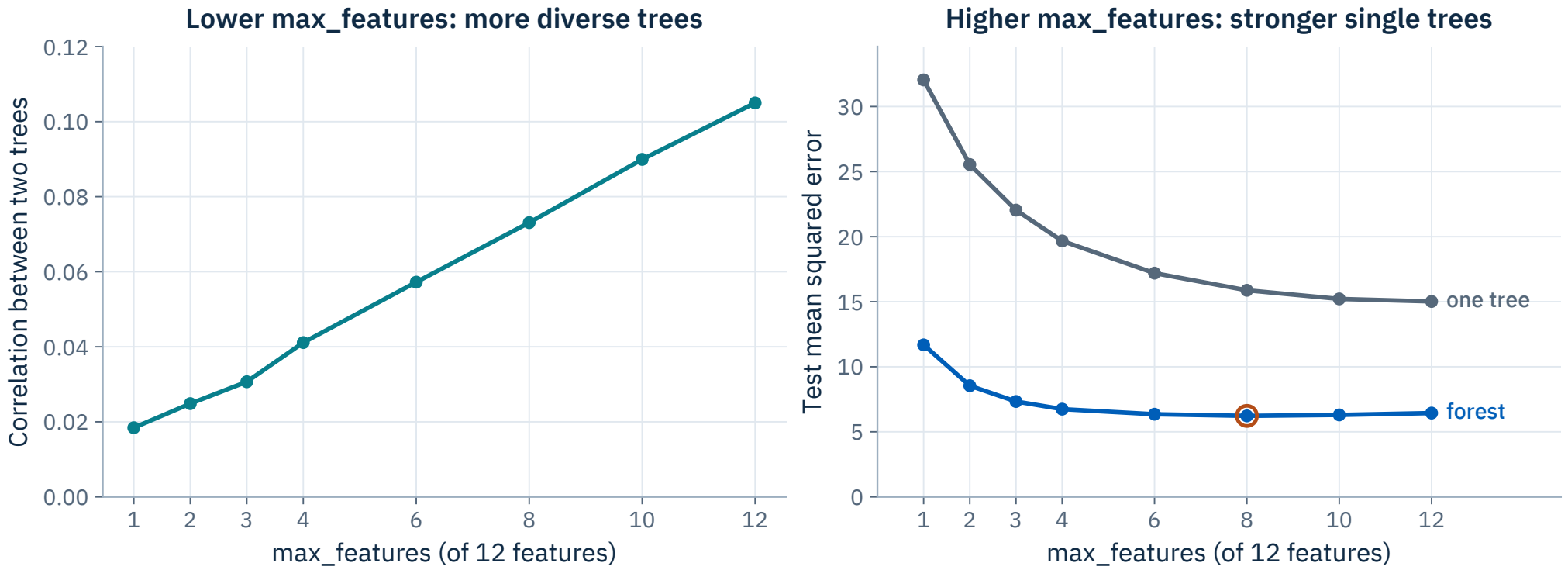
A random forest decorrelates trees before voting



Schematic. Feature names and probabilities are illustrative; the average of 0.82, 0.64, and 0.71 is 0.72.

Feature subsampling trades strength for diversity

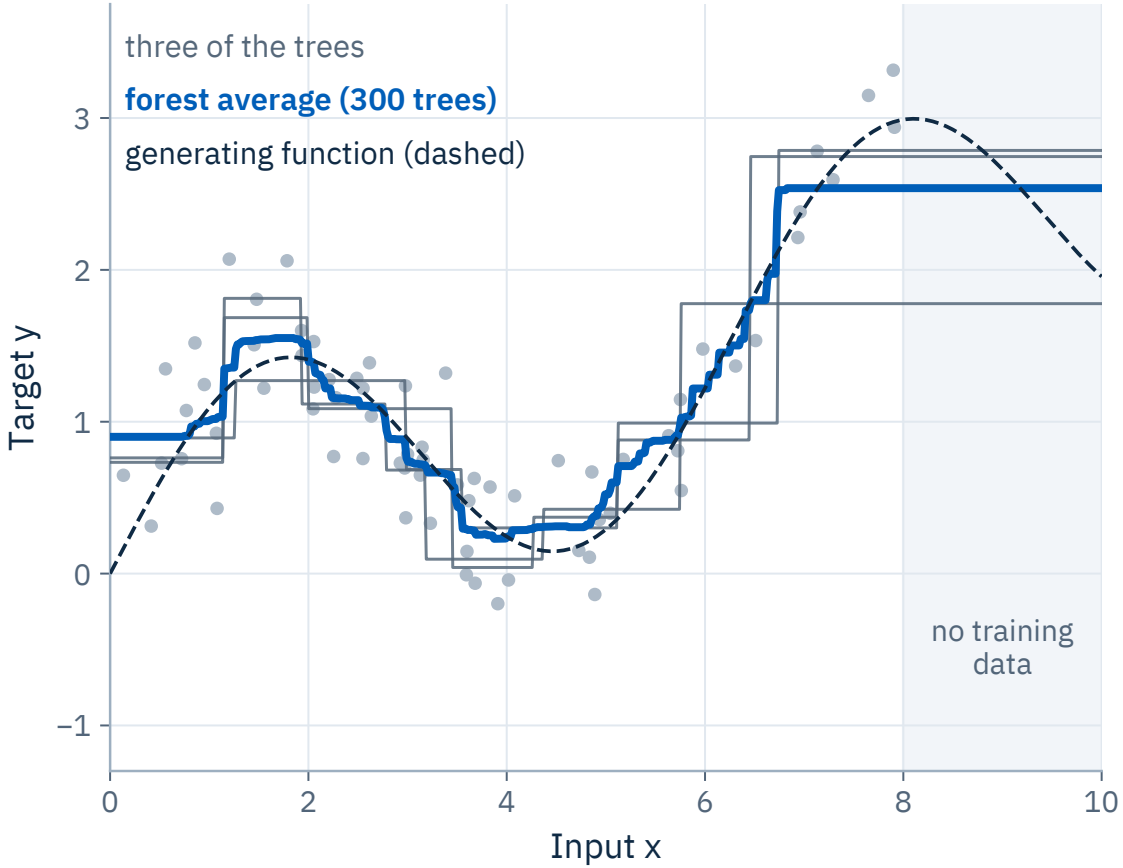
Fewer features per split: less correlated trees but weaker ones. The forest does best in between.



Friedman #1 synthetic regression (12 features, 5 informative): 16 independent training sets of 300 rows, forests of 60 trees. Circle: lowest forest error.

Averaging turns jagged trees into a smoother fit

- Each tree is a step function fitted to its own bootstrap sample.
- The forest averages 300 step functions: steps blur and noise cancels.
- Beyond the data (shaded) the average stays flat: trees cannot extrapolate.



Synthetic data y = sin(x) + 0.25x + noise, n = 70 on 0 ≤ x ≤ 8 (seed 8); 300 trees, min_samples_leaf = 5.

◆ Forests average predictions; classifiers average class probabilities.

Forest controls have practical meanings

Parameter	What it changes	Diagnostic
n_estimators	Number of trees	Score stability and runtime
max_features	Features considered at each split	Tree diversity and fit
min_samples_leaf	Smallest terminal group	Smoothing and rare patterns
max_depth	Longest decision path	Complexity and memory

Tune leaf size and feature subsampling before expanding the search.

◆ Defaults are starting points: max_features is "sqrt" for classifiers and 1.0 for regressors.

Each row is scored only by trees that never saw it

- About a third of the trees leave out any given row.
- A row's out-of-bag prediction uses only those trees.
- The OOB score needs no separate validation fold.

Trees (each fitted to one bootstrap sample)

	T1	T2	T3	T4	T5	T6	OOB vote
W1 1500.0 m	×1	×1	out	×1	out	out	T3, T5, T6
W1 1500.5 m	×1	×1	×1	×1	out	×2	T5
W1 1501.0 m	×2	×2	×1	×1	out	×1	T5
W1 1501.5 m	×1	out	×1	×2	×3	out	T2, T6
W1 1502.0 m	×2	out	×2	out	out	×2	T2, T4, T5
W1 1502.5 m	out	×1	×1	out	×2	×1	T1, T4
W1 1503.0 m	×1	×1	×2	×2	×1	×1	none
W1 1503.5 m	out	×1	×1	out	×1	×1	T1, T4
W1 1504.0 m	×2	×2	out	×1	×2	out	T3, T6
W1 1504.5 m	out	×1	×1	×2	×1	×2	T1

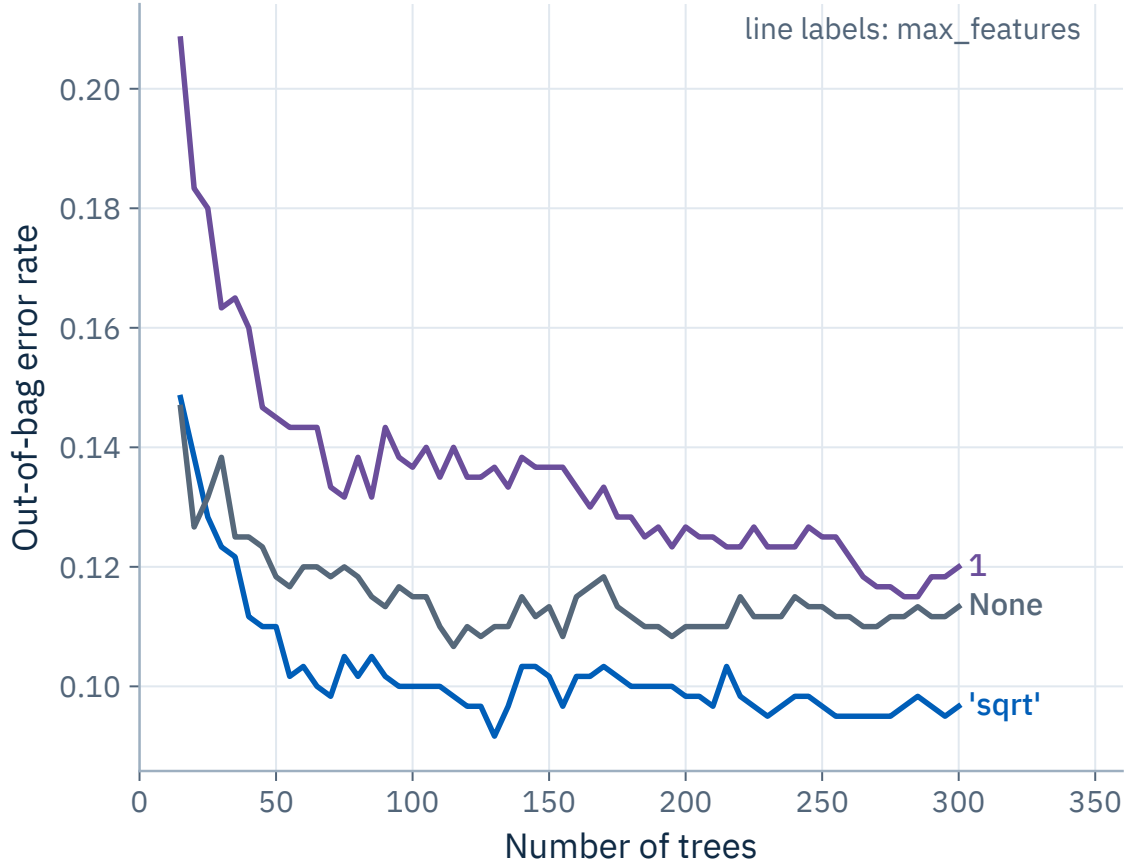
17 of 60 cells out of bag (28%); expected $(1 - 1/n)^n = 35\%$

◆ Neighbouring depth rows are often in-bag when a row is out of bag.

Bootstrap draws for 10 adjacent depth rows of one synthetic well and 6 trees (seed 2).

Out-of-bag error settles as trees are added

- Error falls fast, then flattens: extra trees stabilize the average.
- Pick n_estimators on the flat part, for cost rather than accuracy.
- max_features shifts the level of the curve, not its shape.



Synthetic classification data (600 rows, 10 features, 5% label noise, seed 3); forests grown with warm_start, seed 0.

A forest reports its out-of-bag score for free

```python
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GroupKFold, cross_val_score

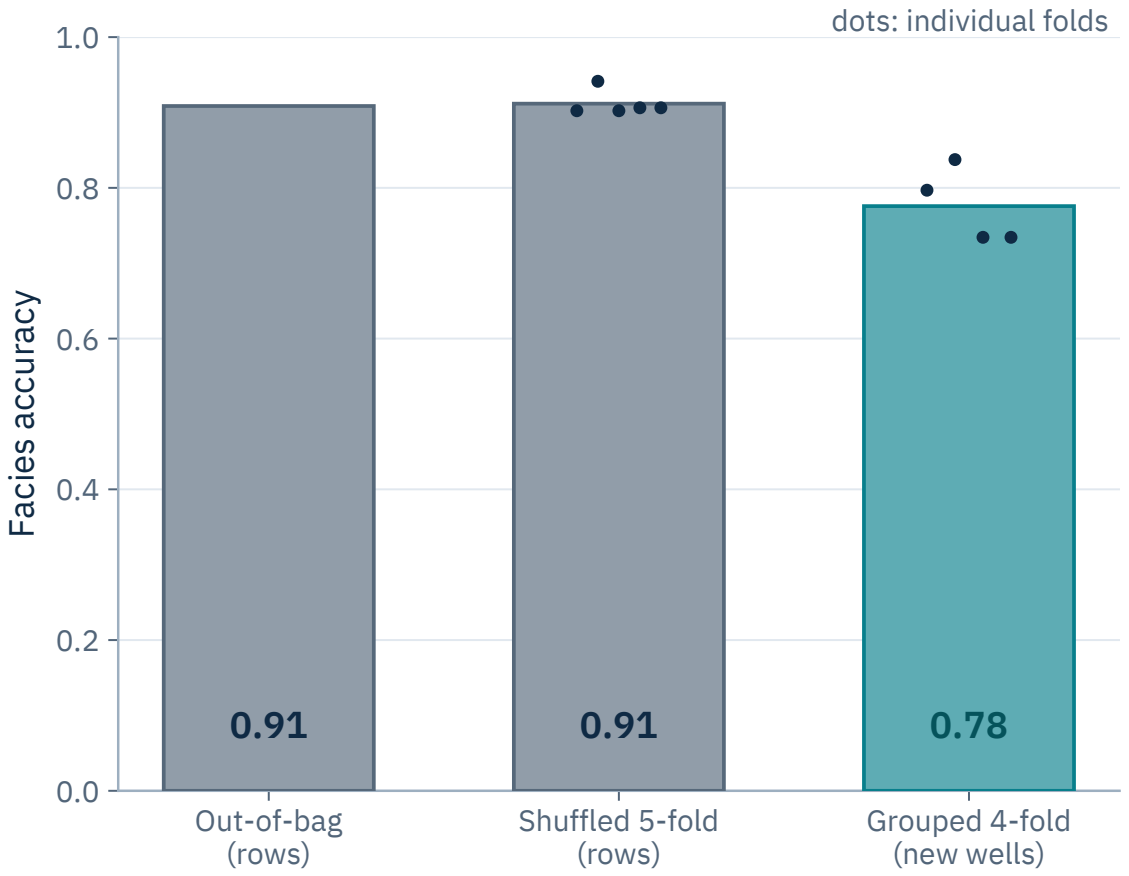
forest = RandomForestClassifier(
    n_estimators=300,        # enough for a stable average
    max_features="sqrt",     # features tried at each split
    min_samples_leaf=3,      # smallest leaf allowed
    oob_score=True, n_jobs=-1, random_state=0,
)
forest.fit(X, y)
print(f"OOB accuracy     {forest.oob_score_:.3f}")
s = cross_val_score(forest, X, y, groups=wells, cv=GroupKFold(4))
print(f"grouped accuracy {s.mean():.3f}")
```

OUTPUT

```
OOB accuracy     0.911
grouped accuracy 0.785
```

Out-of-bag evaluation has assumptions

- Each training row receives predictions from trees that did not sample it.
- This is an internal estimate for approximately independent rows.
- It does not protect against leakage across related well samples.
- Keep grouped or chronological evaluation for those deployment questions.



Synthetic facies logs, 8 wells with well offsets and depth-correlated noise (seed 7); forest of 300 trees, seed 0.

Extra-trees also randomize the split thresholds

Random forest

- Bootstrap sample of rows for each tree
- Best threshold for each candidate feature
- Out-of-bag score possible (bootstrap=True)

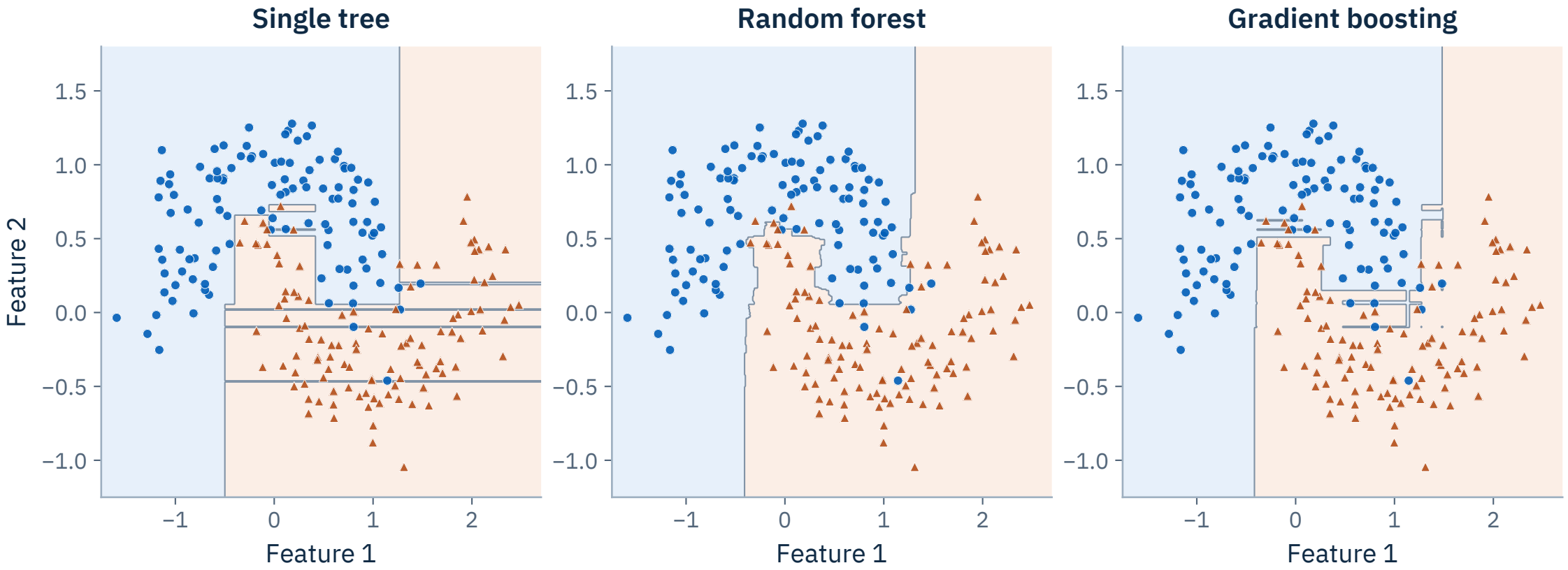
vs

Extra-trees

- All training rows by default (bootstrap=False)
- One random threshold per candidate feature
- Faster to train; often smoother boundaries

◆ Try both with the same folds; neither wins everywhere.

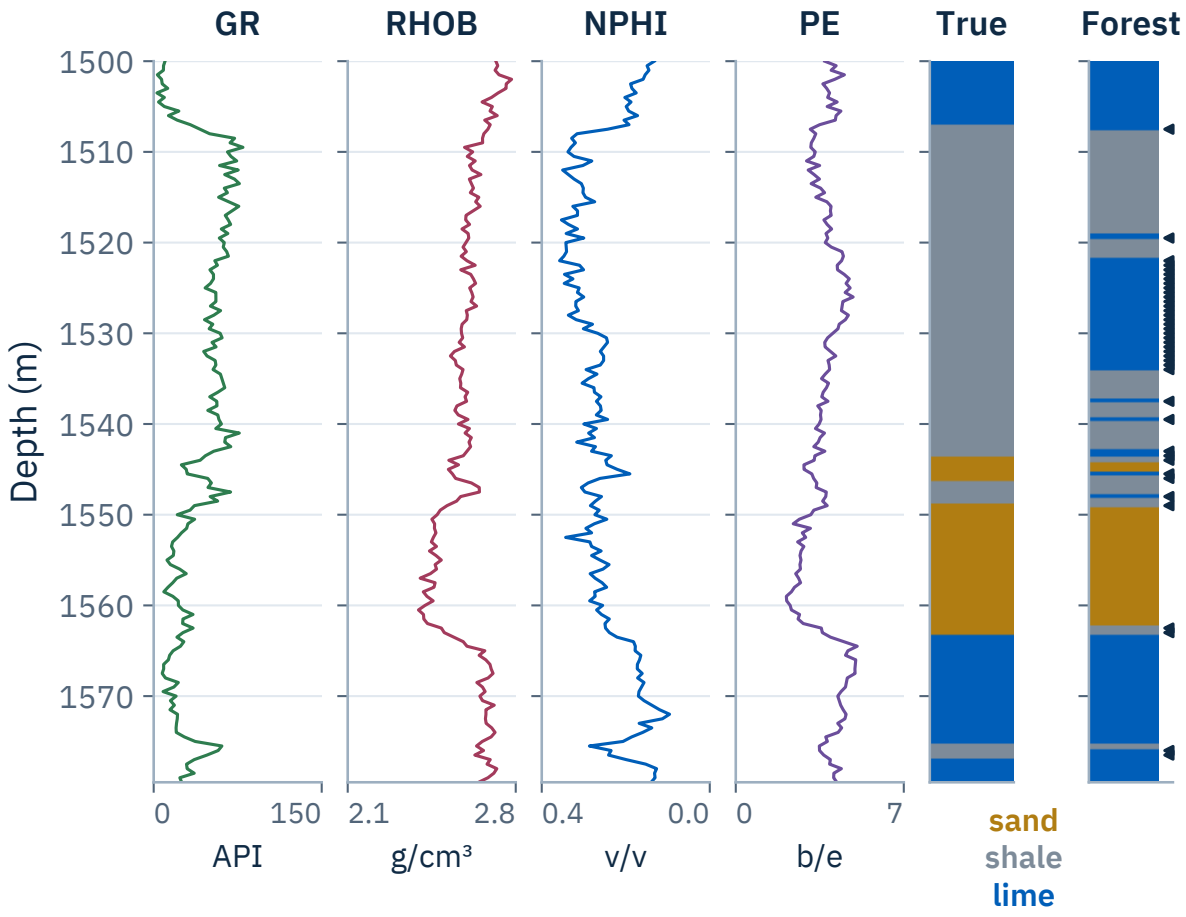
Ensembles change decision boundaries



Tree, forest, and boosting fits on the same synthetic data. Boundary appearance alone does not establish generalization quality.

A forest classifies facies on a held-out well

- Trained on wells W1–W7, scored on W8 only.
- 75% of W8's rows are correct.
- One 12 m shale interval is called limestone; most other errors sit at bed boundaries.



Synthetic logs; forest of 300 trees (min_samples_leaf 3, seed 0).

◆ Report errors per well and per facies, not only overall.

PRACTICE

5 min · Pairs

Which forest change would you try first?

Scenario: a forest predicts ESP trips from sensor summaries of 12 wells. Training accuracy 1.00, OOB 0.93, grouped CV by well 0.79.

1. Grow 2,000 trees instead of 300: what changes?
2. Raise min_samples_leaf from 1 to 20: what might improve, and how do you check?
3. Switch max_features from "sqrt" to all features: likely effect?
4. Add the well identifier as a feature: acceptable?

PART 3 OF 6

03

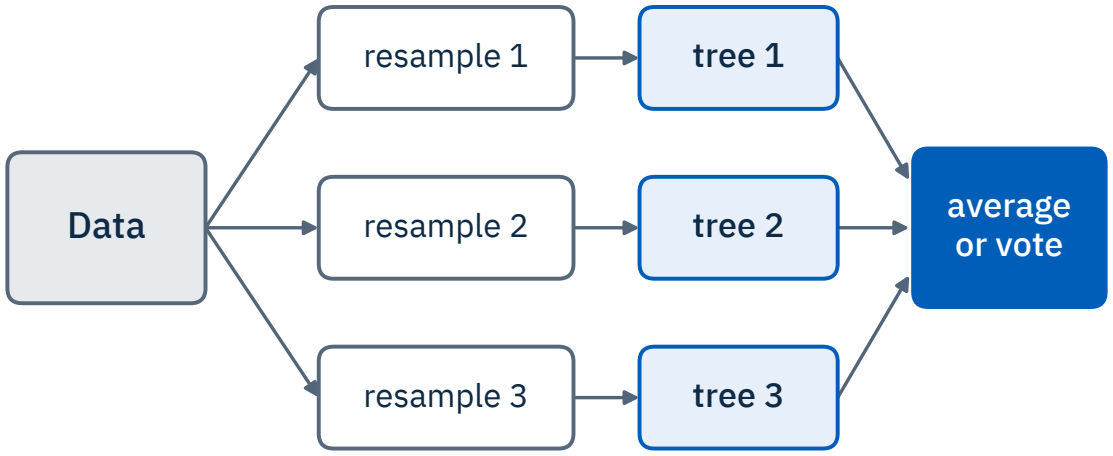
From AdaBoost to gradient boosting

Each new learner fixes what the ensemble still gets wrong.

- AdaBoost and sample weights
- Gradient boosting as gradient descent on predictions
- Losses, learning rate, and early stopping
- ROP on a new well

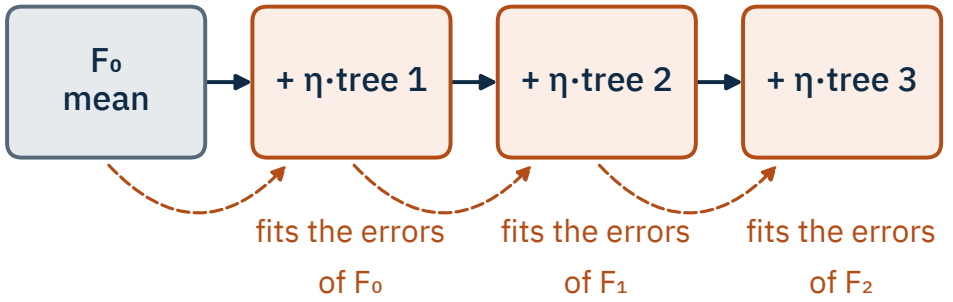
Bagging runs in parallel; boosting runs in sequence

Bagging: independent models, trained in parallel



Trees never see each other's errors.
Deep trees: low bias, averaging cuts variance.

Boosting: each model corrects the ensemble so far



$$F_3(x) = F_0 + \eta \cdot h_1(x) + \eta \cdot h_2(x) + \eta \cdot h_3(x)$$

Shallow trees: each step is small; the sum becomes flexible.

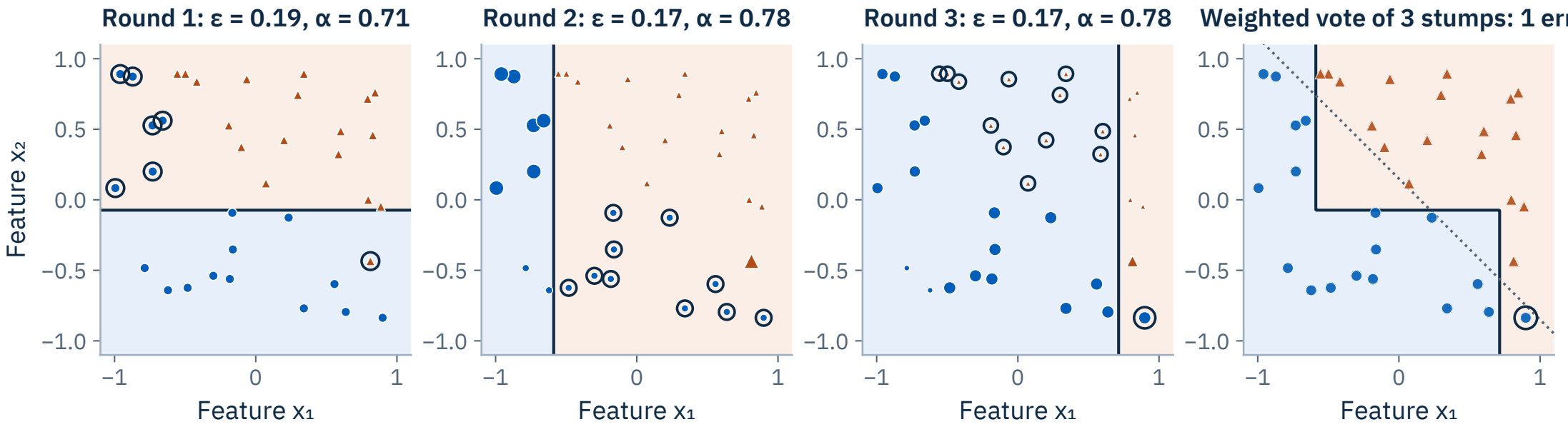
Schematic comparison of bagging and boosting.

Bagging and boosting use different training logic

Question	Bagging	Gradient boosting
How are models trained?	Separately on perturbed samples	Sequentially against current loss
What combines them?	Average or vote	Additive weighted updates
Typical base learner	Relatively deep tree	Shallow tree
Main sensitivity	Correlated trees, leaf size	Learning rate, stages, noisy targets

AdaBoost up-weights the rows it gets wrong

Marker area = row weight; rings mark rows the current stump gets wrong, which gain weight next round.



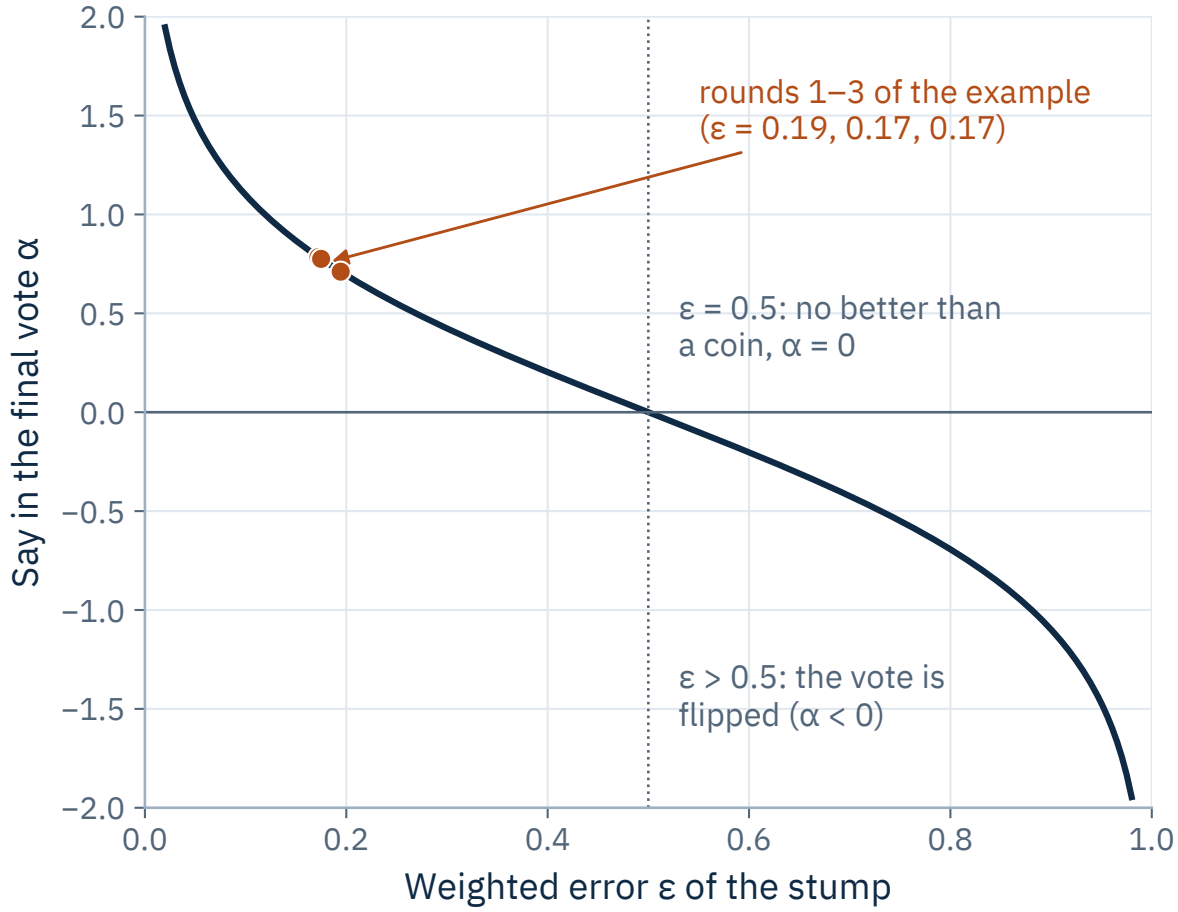
Discrete AdaBoost with decision stumps on 36 synthetic points (seed 12); dotted line: true boundary $x_1 + x_2 = 0.15$.

AdaBoost gives accurate stumps a larger say

$$\alpha_m = \frac{1}{2}\ln\frac{1-\varepsilon_m}{\varepsilon_m}$$

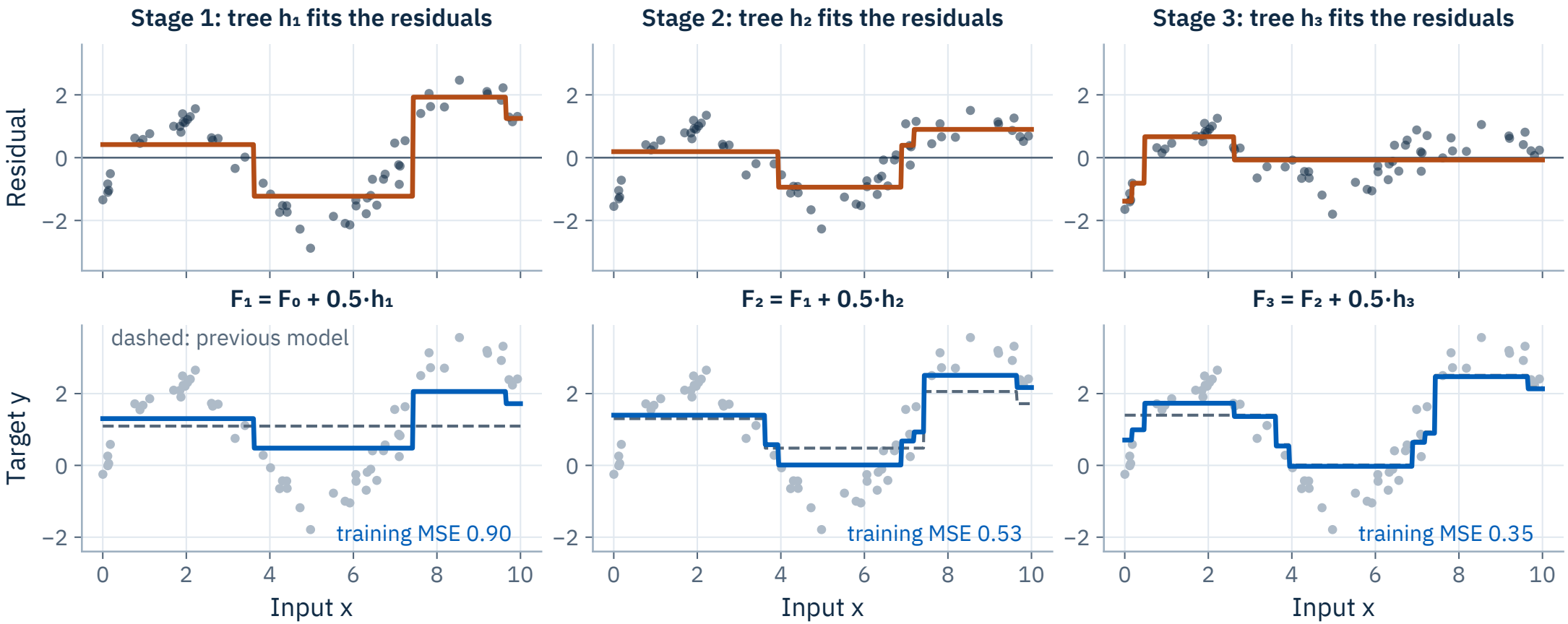
$$F(x) = \operatorname{sign}\left(\sum_m \alpha_m\, h_m(x)\right)$$

- ε_m: weighted error of stump m; weights sum to 1.
- Misclassified rows are multiplied by e^{α}, correct rows by $e^{-\alpha}$; then renormalize.
- After each update, the misclassified rows hold exactly half the weight.



Exact formula; points are the three rounds of the example.

Gradient boosting fits the residuals of the current model



Synthetic data (n = 60, seed 21); F_0 = mean of y; depth-2 regression trees; learning rate 0.5. Top: residuals and the tree fitted to them. Bottom: updated model.

Boosting is gradient descent on the predictions

$$F_m(x) = F_{m-1}(x) + \eta\, h_m(x)$$

$$r_{im} = -\left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)}\right] F = F_{m-1}$$

- Start with a constant prediction that minimizes the loss.
- Fit the next tree h_m to the negative gradients r_{im} (pseudo-residuals).
- Shrink each update by the learning rate η.
- Squared loss: r_{im} is the ordinary residual $y_i - F_{m-1}(x_i)$.

PRACTICE

5 min · Individual

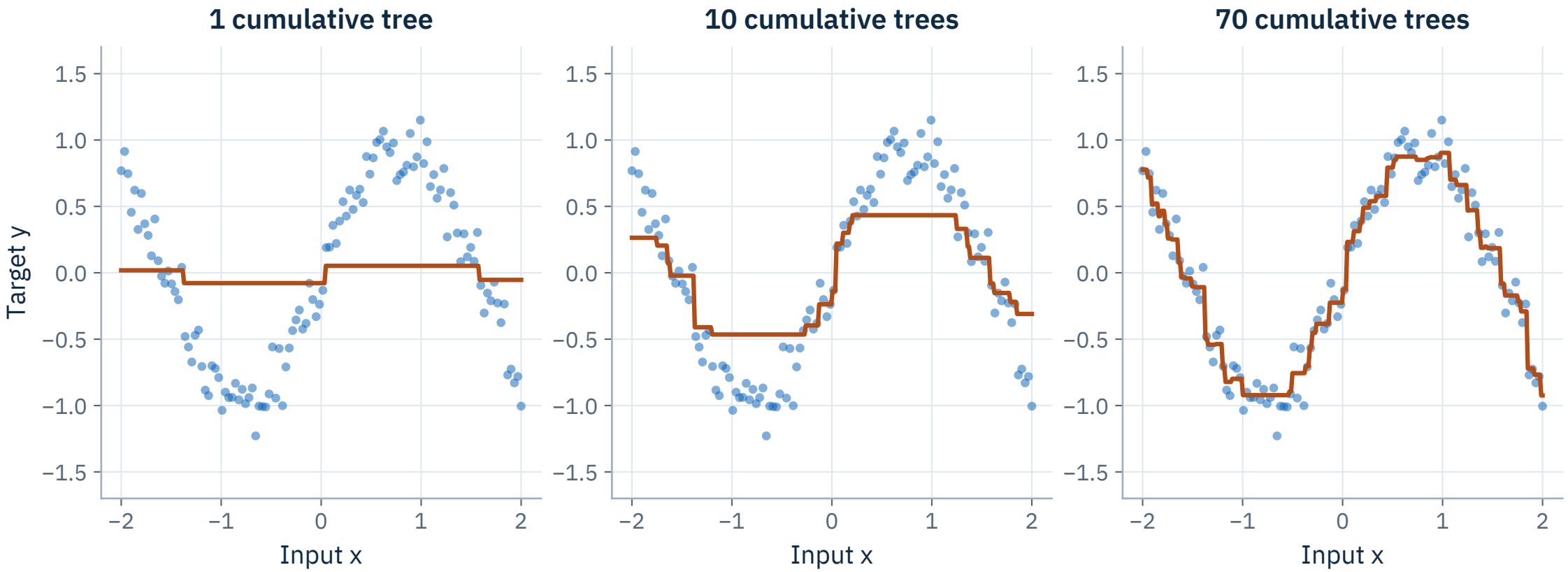
A boosting update by hand

Observed targets: [2, 4, 6]. Current predictions: [4, 4, 4].

The new tree predicts residuals [−2, 0, 2]. Learning rate = 0.1.

1. Calculate the updated predictions.
2. Compare mean squared error before and after the update.

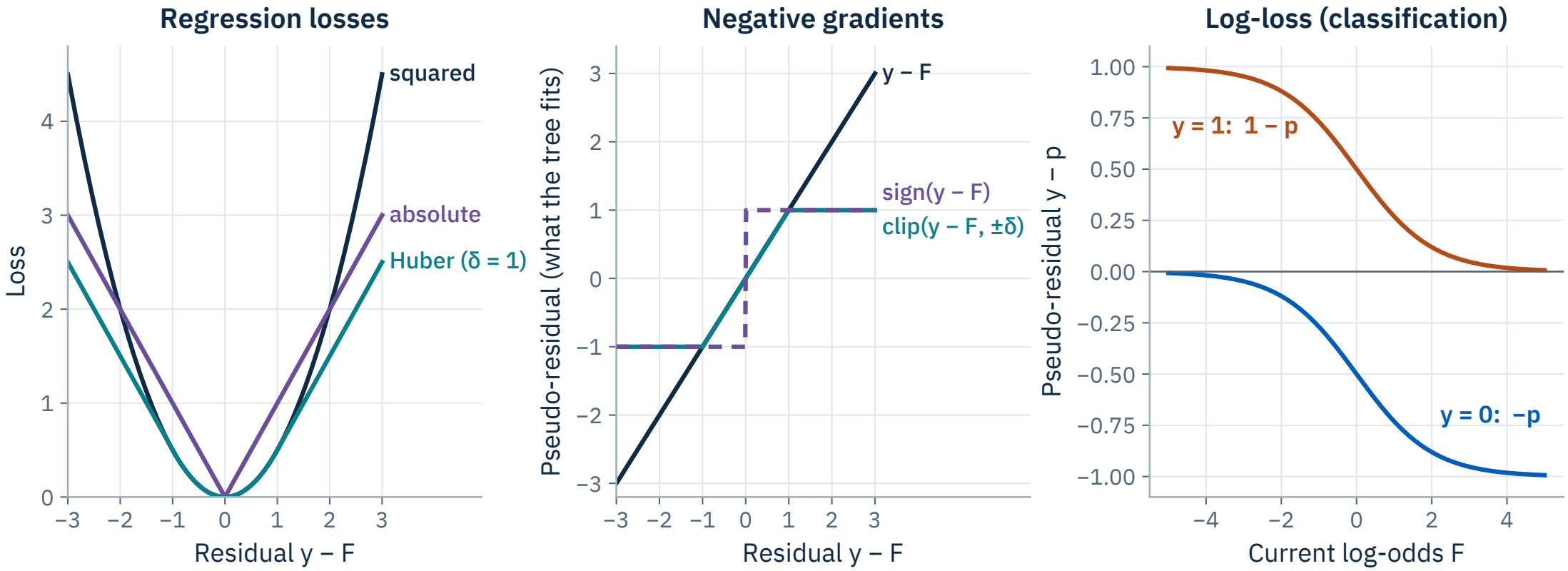
Successive stages reduce residual structure



Gradient boosting fitted to synthetic regression observations. Each panel shows the cumulative prediction after the stated number of trees.

The loss decides what each tree fits

Each tree fits the negative gradient of the loss at the current predictions: the pseudo-residual.



Exact functions. Huber threshold δ = 1; p = 1 / (1 + e^(−F)).

Each loss has its own pseudo-residual

Loss	L(y, F)	Pseudo-residual	Use when
Squared	$\frac{1}{2}(y - F)^2$	$y - F$	Default for regression
Absolute	$\lvert y - F \rvert$	$\operatorname{sign}(y - F)$	Heavy-tailed target errors
Huber (δ)	Quadratic within ±δ, linear beyond	$y - F$ clipped to ±δ	Mostly clean, some bad readings
Log-loss	$-y \ln p - (1 - y) \ln(1 - p)$	$y - p$	Binary classification (F = log-odds)

In scikit-learn: loss="squared_error", "absolute_error", "huber", or "log_loss".

PRACTICE

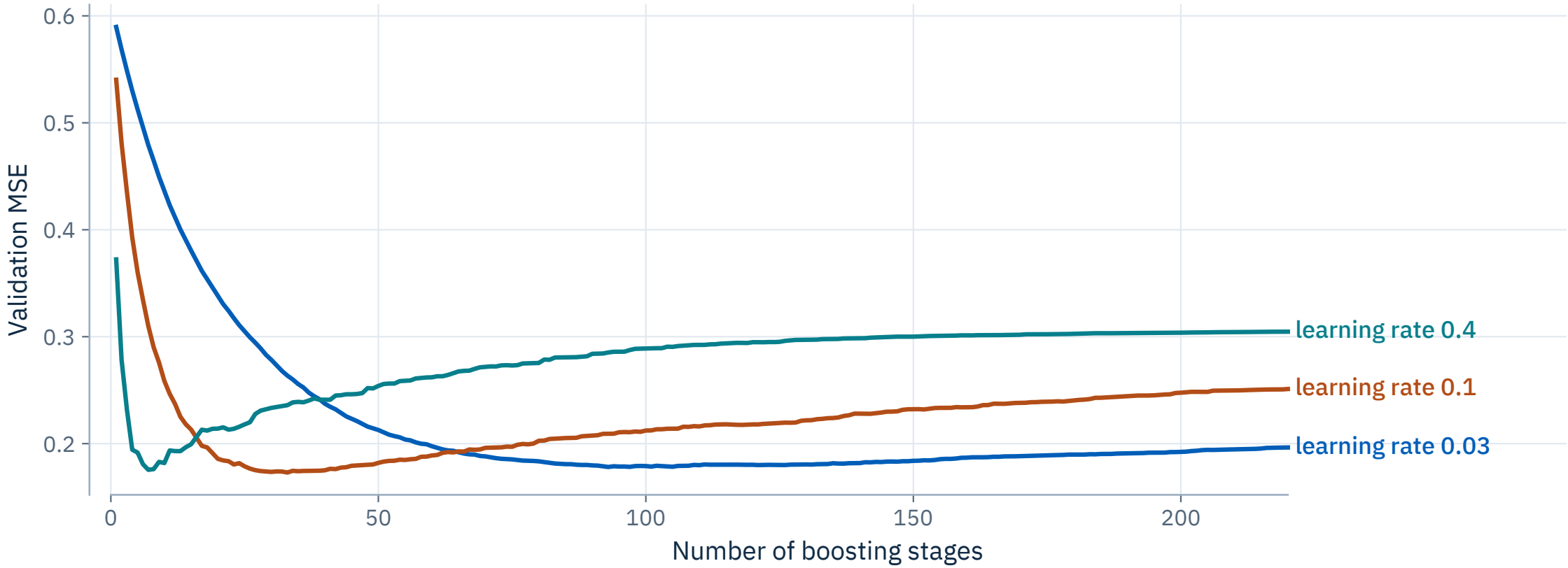
5 min · Pairs

Pseudo-residuals by hand

Current residuals y – F for four rows: −3, −0.5, 0.2, and 4.

1. Write the pseudo-residuals the next tree fits under squared, absolute, and Huber (δ = 1) loss.
2. Which loss lets the row with residual 4 dominate the next tree?
3. Log-loss: y = 1 with p = 0.8, and y = 0 with p = 0.3. What are the pseudo-residuals?

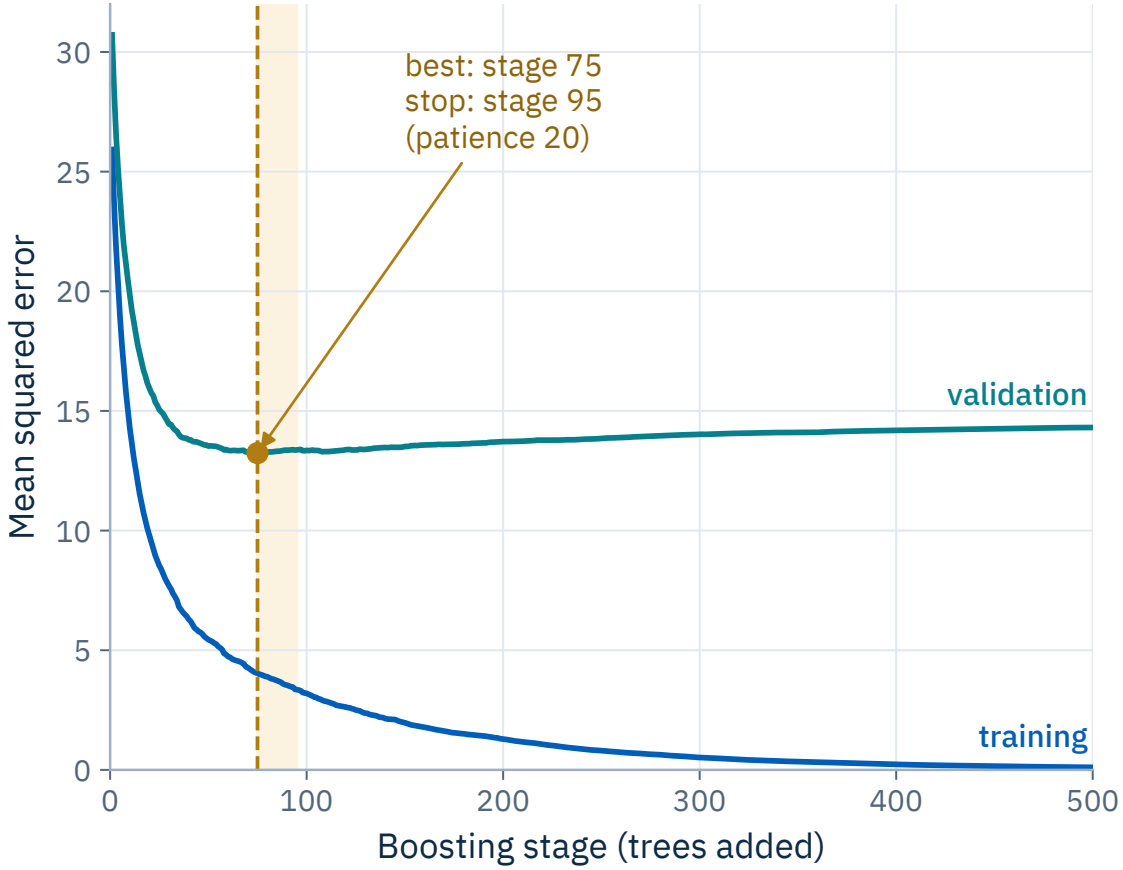
Learning rate and tree count interact



Measured validation error on a fixed synthetic holdout. Curves compare learning rates using the same data and tree depth.

Early stopping chooses the number of trees

- Hold out validation rows: another well or a later period.
- Stop after a patience of N stages without improvement.
- Keep the final test rows out of this loop.

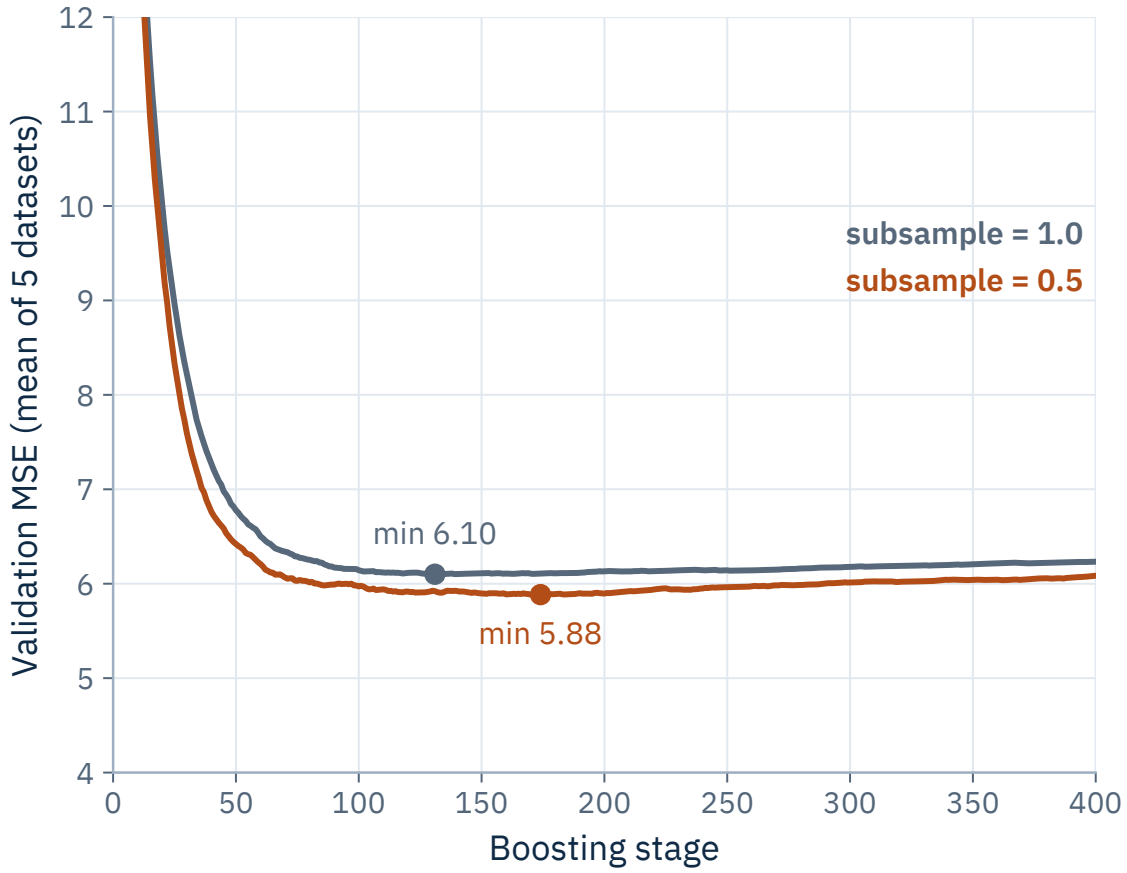


Friedman #1 synthetic regression: 400 training and 2,000 validation rows; learning rate 0.1, depth 3. Stop rule: 20 stages without improvement.

◆ Training error always falls; only validation error can say stop.

Regularization in boosting

- Limit tree depth or the number of leaves.
- Use a minimum leaf size to reduce fragile local updates.
- Row or feature subsampling reduces dependence on individual samples.
- Tune learning rate and iteration count together.



Friedman #1 synthetic regression (noise sd 2), mean of five datasets; learning rate 0.1, depth 3.

Three controls set a gradient-boosting run

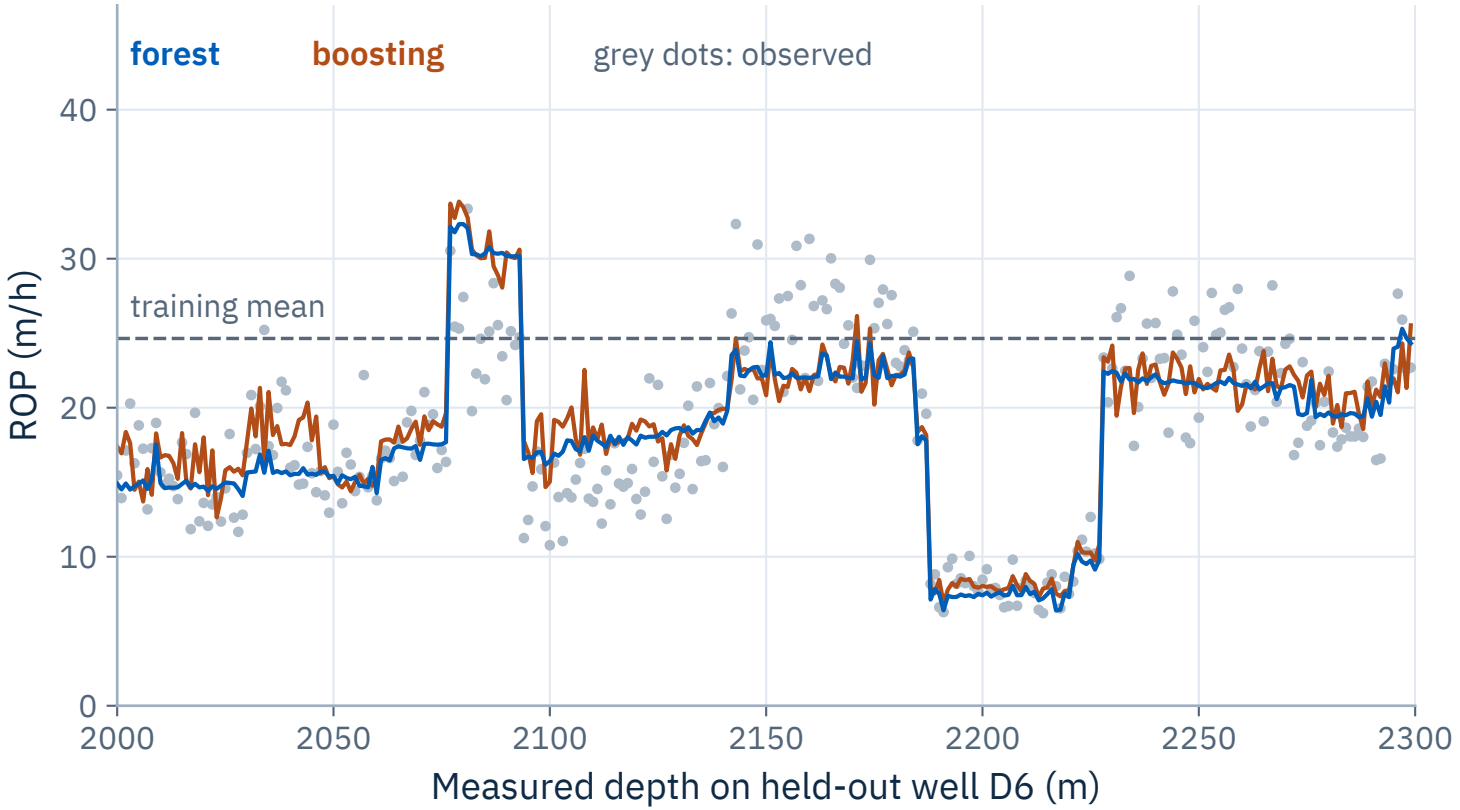
```
gbr = GradientBoostingRegressor(
    loss="huber",            # robust to bad readings
    learning_rate=0.05,      # shrinkage eta
    n_estimators=2000,       # upper limit only
    max_depth=3,             # shallow trees
    subsample=0.7,           # stochastic boosting
    validation_fraction=0.15,
    n_iter_no_change=30,
    random_state=0,
)
gbr.fit(X_train, y_train)
print("stages kept:", gbr.n_estimators_)
```

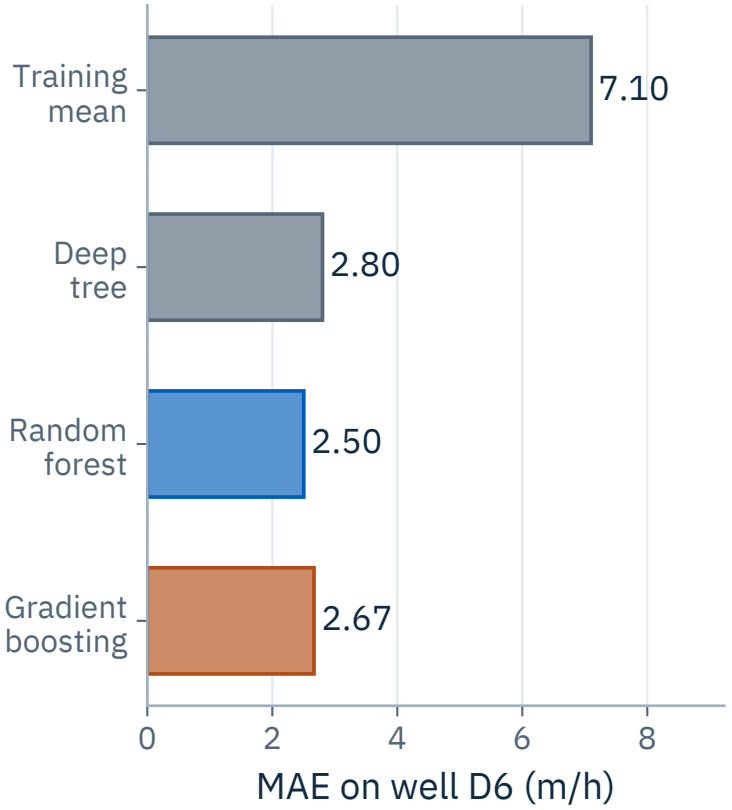
OUTPUT

```
stages kept: 173
```

- Learning rate, tree size, and stages are the core trio.
- n_estimators is a ceiling; early stopping picks the count.
- The internal validation split is random rows.

Forest and boosting score alike on a new well





Synthetic drilling wells (seed 11): trained on D1–D5, evaluated on held-out well D6. Forest: 300 trees, min_samples_leaf 3; boosting: HistGradientBoostingRegressor defaults.

KEY TAKEAWAYS

Morning takeaways before the laboratory

- ✓ Combining helps when members are useful and their errors differ.
- ✓ Bagging averages deep trees; random forests also decorrelate them with max_features.
- ✓ Out-of-bag scores assume unrelated rows; use grouped validation for new wells.
- ✓ Boosting adds shallow trees fitted to pseudo-residuals, shrunk by the learning rate.
- ✓ Early stopping on deployment-like validation data sets the number of stages.

LABORATORY

60 min

Laboratory 1: tree, forest, and boosting compared

- Parts 1–2: stratified 80/20 split (random_state 42), then a single decision tree.
- Parts 3–4: a random forest and gradient boosting on the same split.
- Part 5: 5-fold CV of all three on the training rows; then open the test rows once.
- Extension: set oob_score=True and plot a staged_predict validation curve.

NOTEBOOK

`exercises/Ensemble_Methods_Student.ipynb`

EVIDENCE TO SUBMIT

- A score table: CV mean ± sd per model
- One OOB or validation-curve plot
- One sentence: is any difference larger than fold noise?

DISCUSSION

Lab 1 debrief: the ranking is within the noise

Model	Test accuracy (30 rows)	5-fold CV on training rows
Single tree	0.933	0.942 ± 0.020
Random forest	0.900	0.950 ± 0.017
Gradient boosting	0.967	0.967 ± 0.017

- One test row is worth 3.3 points: the test ranking rests on one or two rows.
- The forest's OOB accuracy on the training rows is 0.942.

PART 4 OF 6

04

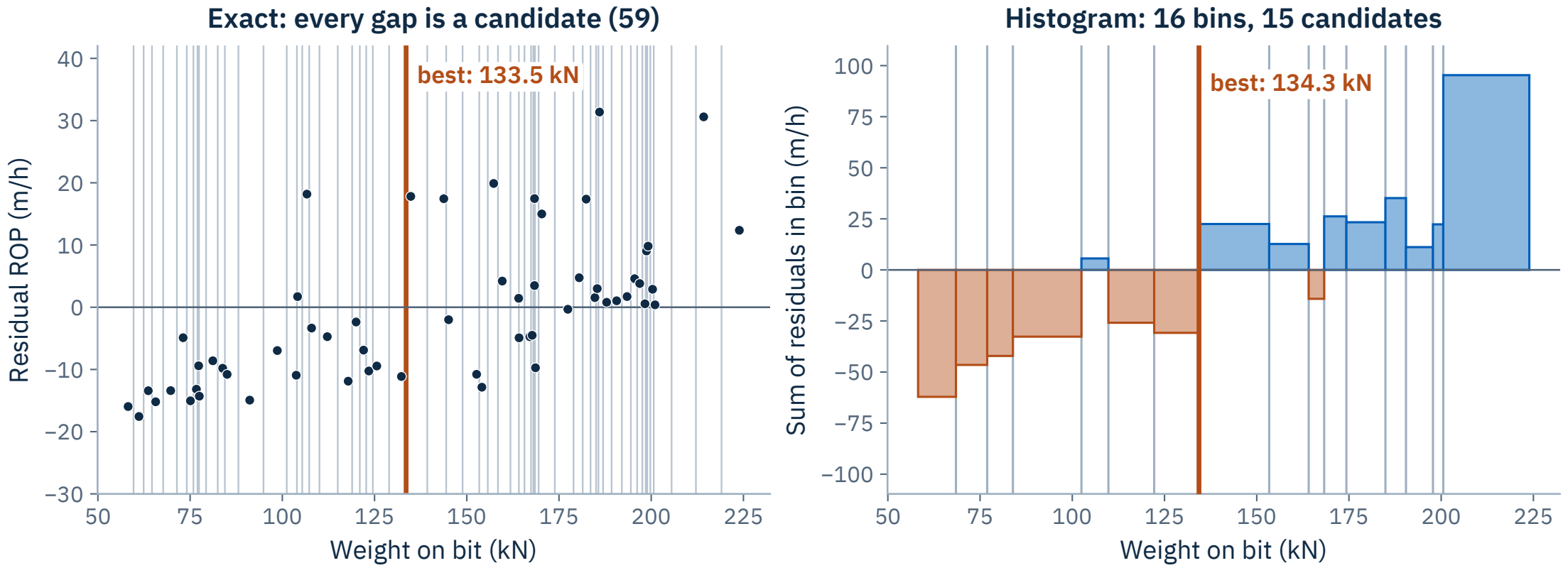
Boosting libraries and stacking

Faster trees, native gaps and categories, and learned combinations.

- Histogram binning and tree-growth policies
- Missing values and categories inside the model
- XGBoost, LightGBM, CatBoost, and scikit-learn
- Stacking with out-of-fold predictions

Histogram binning makes split search cheap

Bin each feature once; every split then needs only per-bin sums of gradients and counts.



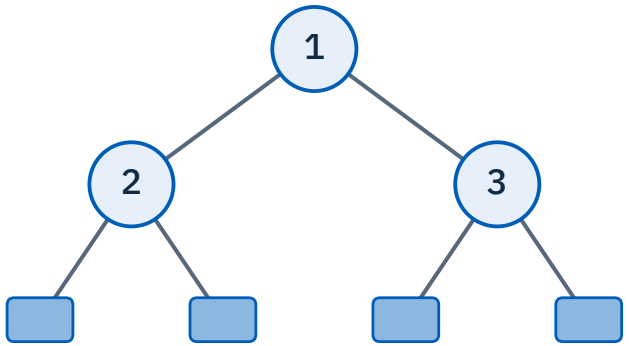
Sixty rows of synthetic drilling data (seed 4); residuals after the first-stage mean prediction; 16 quantile bins. Both searches choose the same partition.

Libraries grow trees in different orders

Same three splits, different shapes: leaf-wise trees grow deep where the gain is.

Level-wise (depth-wise)

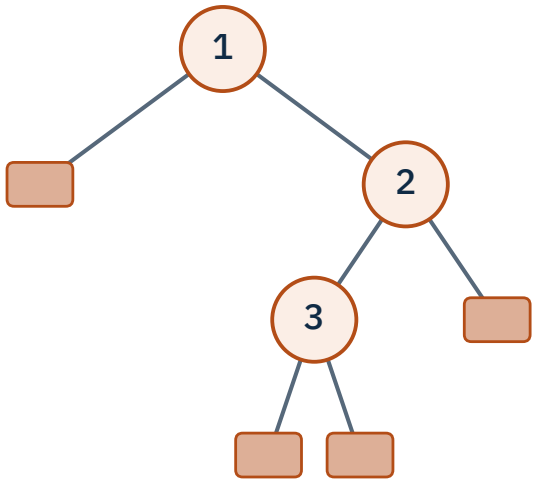
fills each level before going deeper



XGBoost default · limit: max_depth

Leaf-wise (best-first)

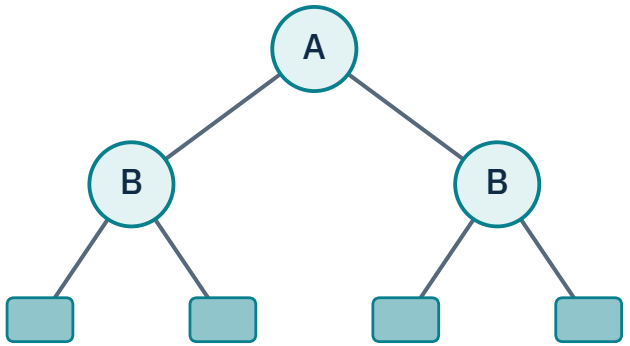
splits the leaf with the largest gain



LightGBM, sklearn HistGB · limit: leaves

Symmetric (oblivious)

one condition per level (A, then B)

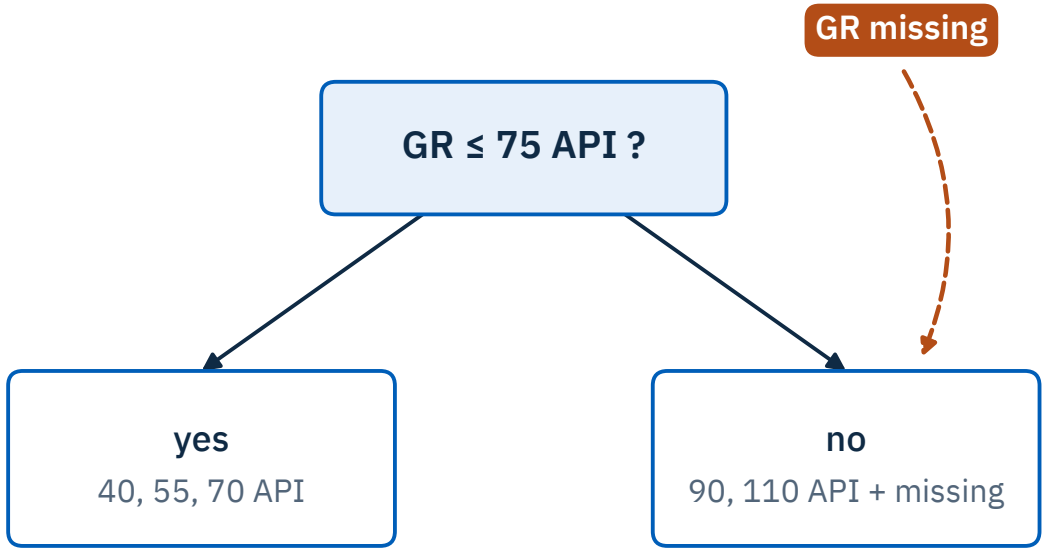


CatBoost default · limit: depth

Schematic. Numbers give the order in which splits are made; A and B mark one condition shared across a level.

Missing values get a learned direction at each split

- No imputation needed: rows with a missing value go left or right.
- The direction is the one with the larger training gain.
- Categories split as groups, not as one-hot columns.



During training both directions are scored;

the one with the larger gain is stored.

No missing GR in training? sklearn sends them to the larger child.

◆ A gap is informative only if it is missing for the same reason in production.

Schematic of native missing-value handling in histogram gradient boosting.

HistGradientBoosting handles gaps and categories natively

```
cols = ["WOB", "RPM", "FLOW", "BIT_HOURS", "FORMATION"]
names = ["Claystone", "Sandstone", "Limestone", "Dolomite"]
kinds = pd.CategoricalDtype(names)                    # fixed category list
X_tr = train[cols].astype({"FORMATION": kinds})
X_va = valid[cols].astype({"FORMATION": kinds})       # a validation well
print(X_tr.isna().mean().round(3).to_dict())

hgb = HistGradientBoostingRegressor(
    max_iter=1000, learning_rate=0.1, early_stopping=True,
    n_iter_no_change=20, random_state=0)
hgb.fit(X_tr, train["ROP"], X_val=X_va, y_val=valid["ROP"])   # sklearn 1.7
print("iterations used:", hgb.n_iter_)
```

OUTPUT

```
{'WOB': 0.0, 'RPM': 0.014, 'FLOW': 0.062, 'BIT_HOURS': 0.0, 'FORMATION': 0.0}
iterations used: 160
```

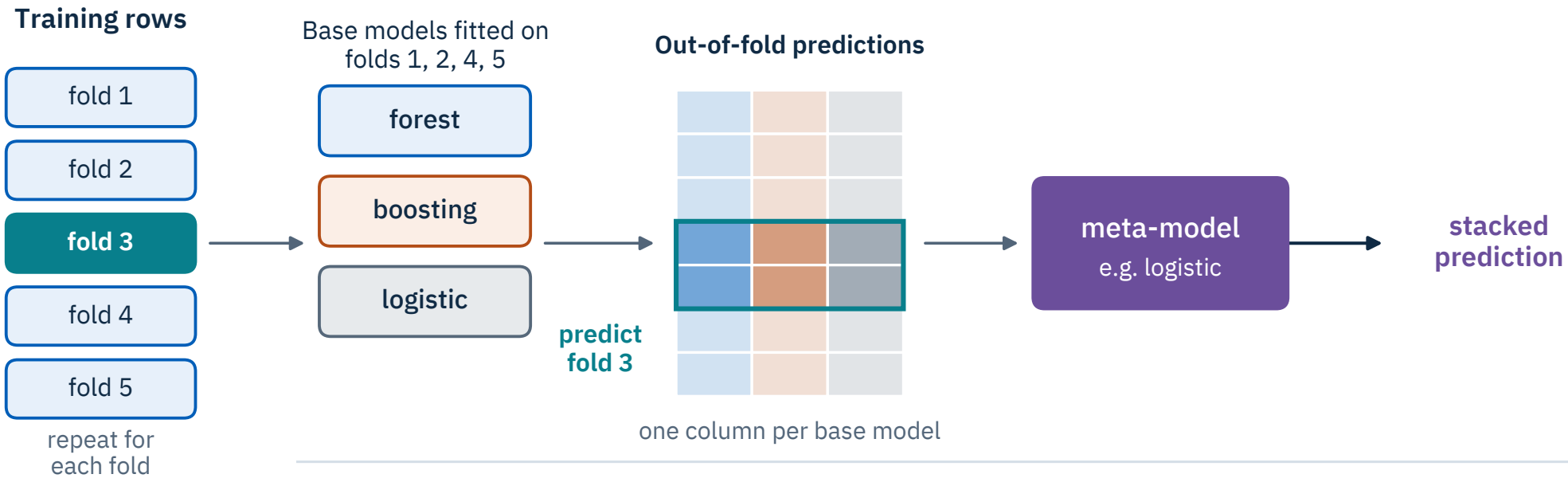
Four libraries, one idea, different defaults

Library	Default tree growth	Categorical inputs	Missing values
XGBoost	Level-wise (max_depth)	enable_categorical=True	Learned default direction
LightGBM	Leaf-wise (num_leaves)	categorical_feature	Learned direction
CatBoost	Symmetric (oblivious)	Ordered target statistics	Numeric NaN below minimum
scikit-learn HistGB	Leaf-wise (max_leaf_nodes)	categorical_features	Learned direction

No universal ranking: compare on the same split, metric, hardware, and tuning budget.

Stacking needs out-of-fold predictions

The meta-model must learn from predictions on rows each base model did not fit.



Schematic of stacking with K = 5 folds and three base models.

StackingClassifier builds the out-of-fold features

```
members = [("forest", RandomForestClassifier(300, random_state=0)),
           ("hgb", HistGradientBoostingClassifier(random_state=0)),
           ("logit", make_pipeline(StandardScaler(), LogisticRegression()))]
stack = StackingClassifier(members, final_estimator=LogisticRegression(),
                           cv=5, stack_method="predict_proba")

for name, model in members + [("stack", stack)]:
    s = cross_val_score(model, X, y, groups=wells, cv=GroupKFold(4))
    print(f"{name:7s} {s.mean():.3f}")
```

OUTPUT

```
forest  0.776
hgb     0.820
logit   0.857
stack   0.852
```

⚠ The internal cv=5 splits rows, not wells: its out-of-fold columns still leak within wells.

PRACTICE

5 min · Groups of 3

Choosing forest or boosting

A tabular model will run at a remote wellsite. You have limited tuning time.

1. Which baseline would you try first, and why?
2. What comparison would make you change your choice?
3. What must fit within the deployment budget?

PART 5 OF 6

05

Interpreting ensemble models

What did the model learn, and how far can we trust the explanation?

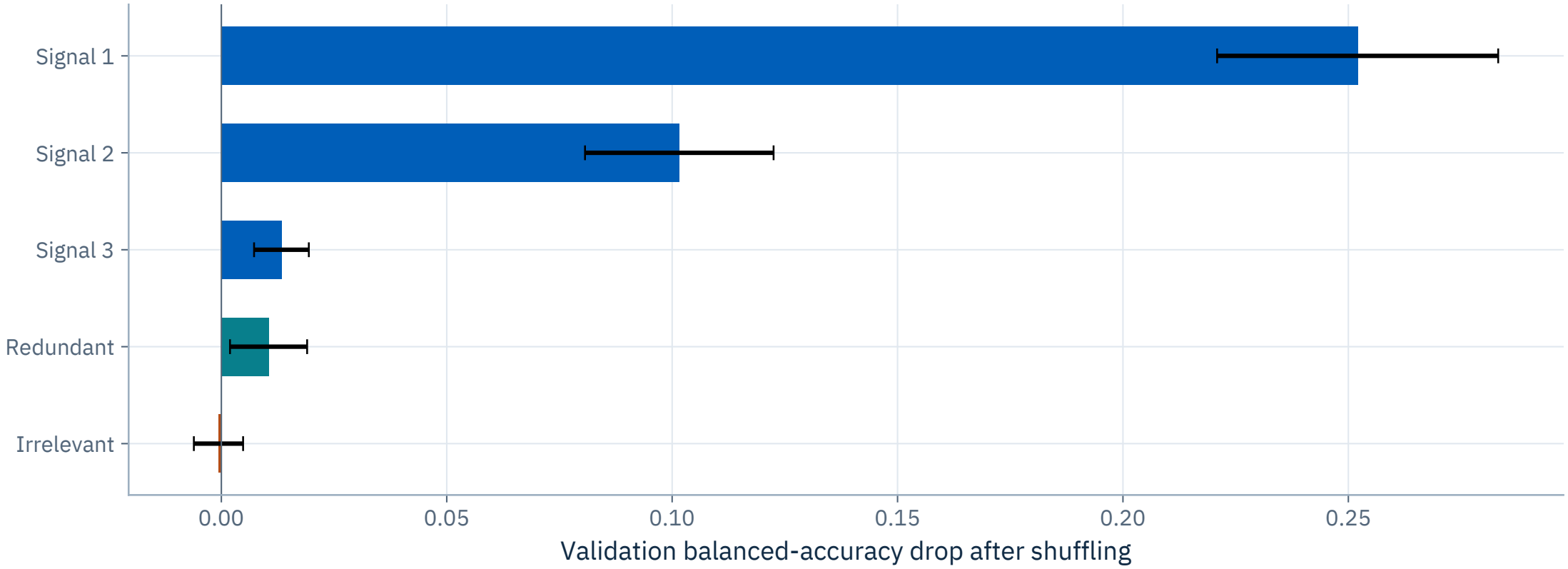
- Impurity and permutation importance
- Partial dependence and ICE curves
- Shapley values, computed exactly
- Cautions: association, extrapolation, stability

Feature importance depends on the question

Method	Measures	Main limitation
Split impurity importance	Training impurity reduction credited to a feature	Can favour high-cardinality predictors
Permutation importance	Evaluation score drop after shuffling a feature	Correlated substitutes can hide reliance
Domain sensitivity review	Plausibility of model response to input changes	Requires careful constraints

Importance can expose an irrelevant feature

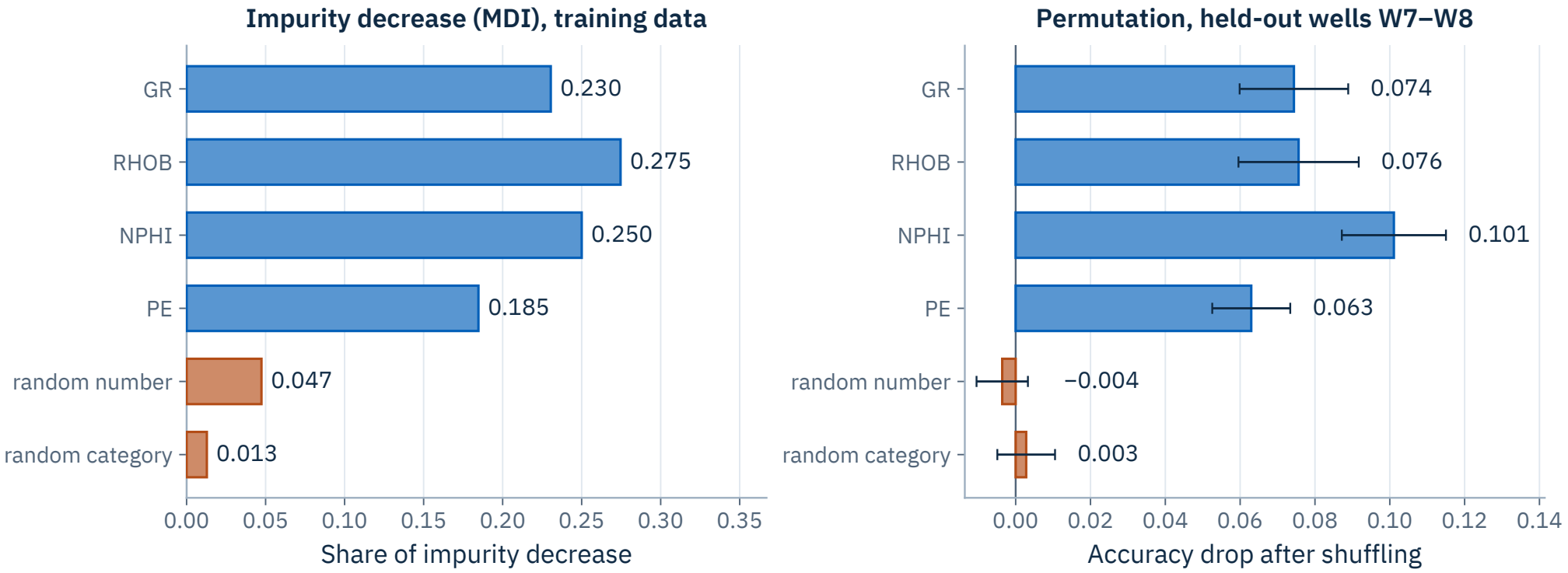
Shuffle one column in the validation rows; the score drop measures the model's reliance on it.



Computed synthetic validation experiment with a deliberately irrelevant feature. Error bars show variation over repeated shuffles.

Impurity importance rewards high-cardinality noise

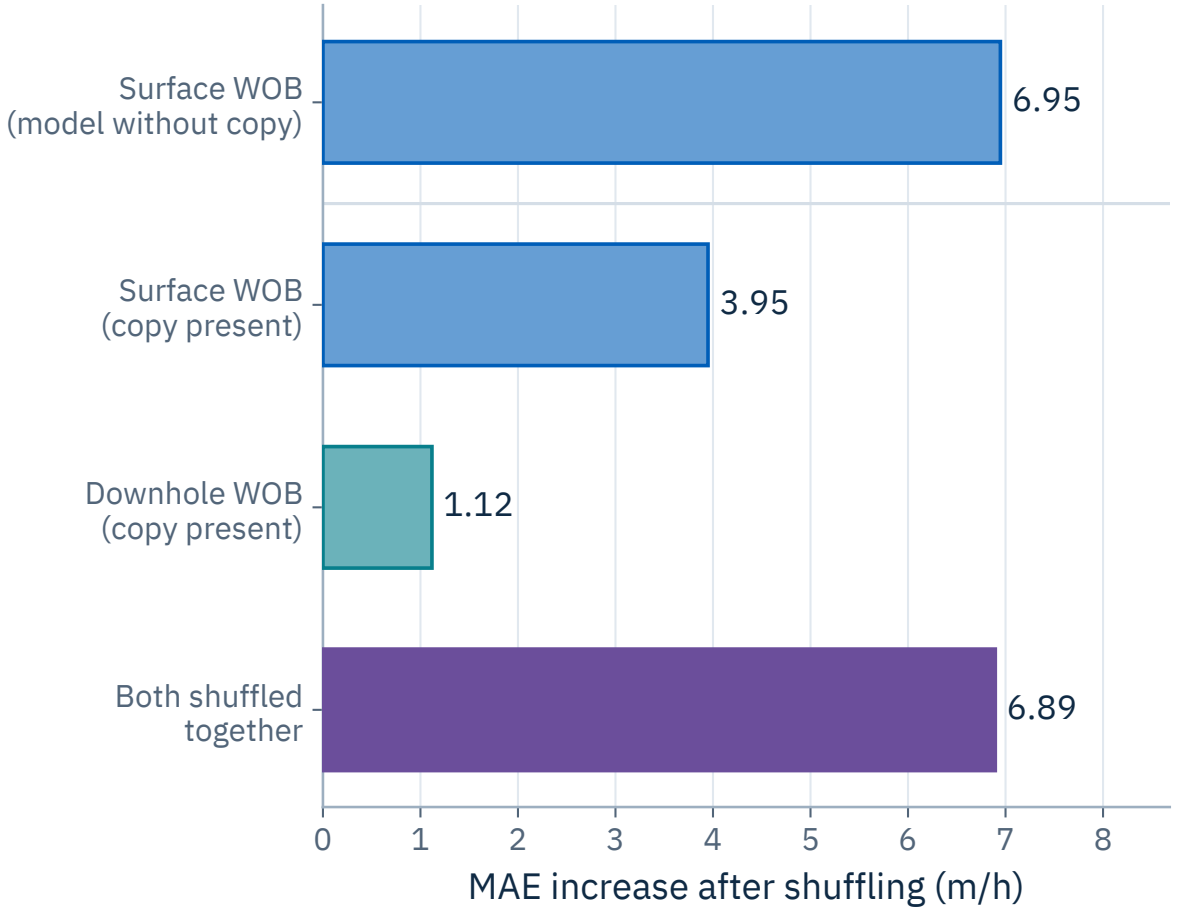
Random numbers get a quarter of PE's impurity credit, yet shuffling them changes nothing on new wells.



Synthetic facies logs plus two pure-noise columns (seed 0); forest of 300 unpruned trees trained on W1–W6; permutation on W7–W8, 20 repeats.

Correlated features share and hide importance

- Downhole WOB is a near copy of surface WOB (r = 0.997).
- With both present, shuffling one leaves the other: 3.95 + 1.12.
- Shuffling them together recovers the joint effect: 6.89.



◆ Group correlated inputs before ranking them.

Synthetic drilling data; forest trained on D1–D4, permutation on D5–D6, 30 repeats.

Importance and dependence plots take a few lines

```python
from sklearn.inspection import permutation_importance
from sklearn.inspection import PartialDependenceDisplay

r = permutation_importance(model, X_val, y_val, n_repeats=20, random_state=0,
                           scoring="neg_mean_absolute_error")
for name, m, s in zip(X_val.columns, r.importances_mean, r.importances_std):
    print(f"{name:10s} {m:5.2f} ± {s:.2f}")

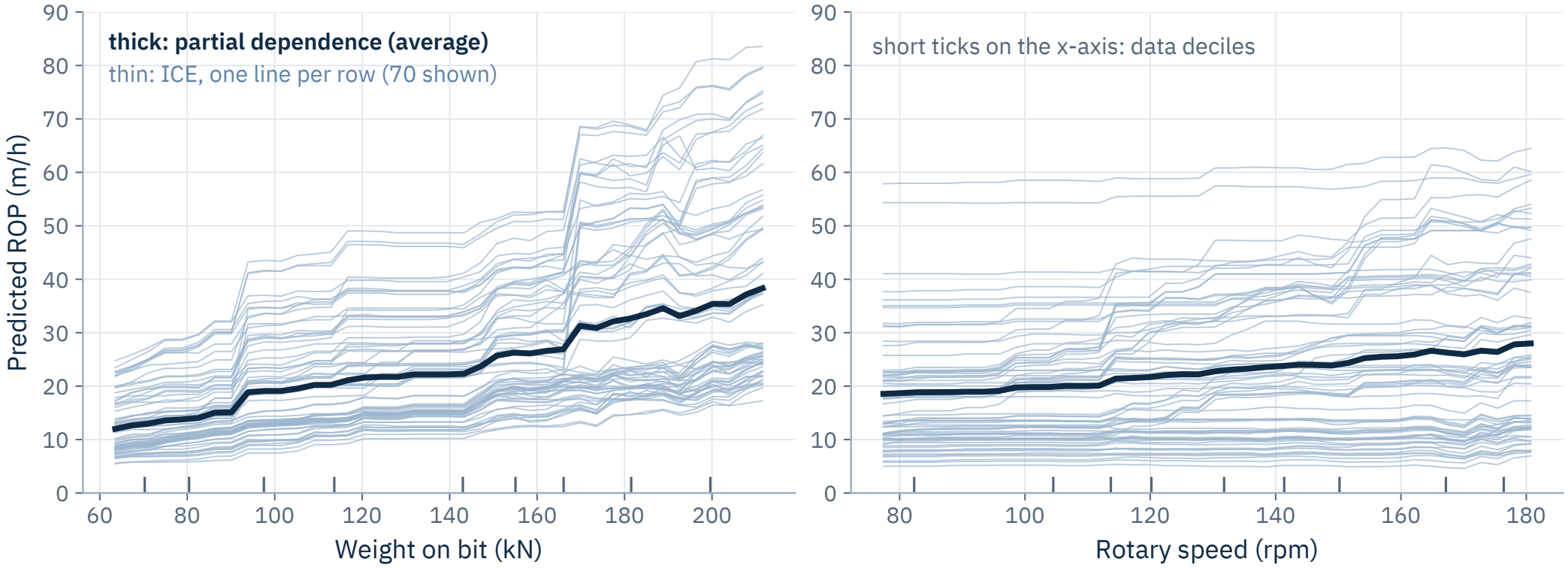
PartialDependenceDisplay.from_estimator(
    model, X_val, features=["WOB", "RPM"], kind="both",
    subsample=70, random_state=0)
```

OUTPUT

```
WOB         6.87 ± 0.33
RPM         1.83 ± 0.13
FLOW        0.48 ± 0.04
BIT_HOURS   0.13 ± 0.03
FORMATION   8.99 ± 0.39
```

Partial dependence averages the model's response

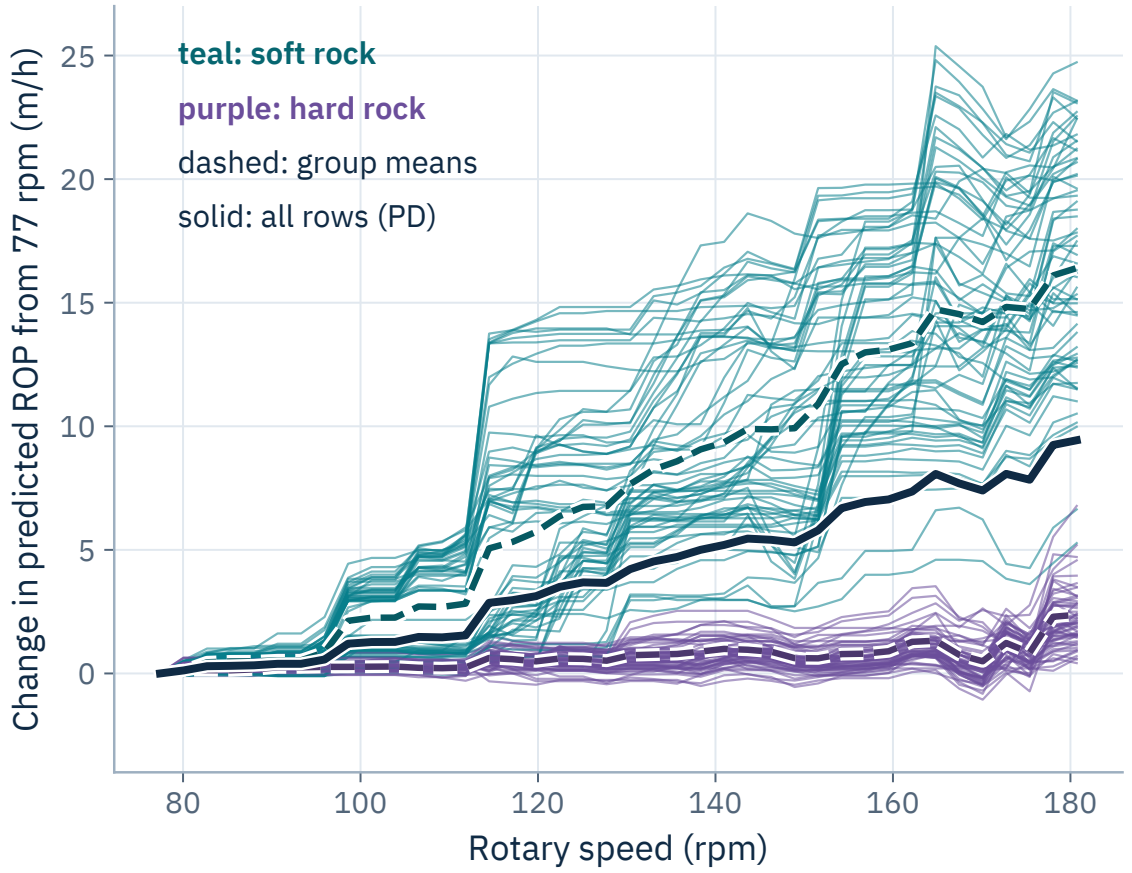
PD: set WOB to each grid value for every row, predict, and average. ICE: keep one line per row.



HistGradientBoostingRegressor trained on synthetic wells D1–D5; sklearn.inspection.partial_dependence over 300 sampled rows, 40 grid points between the 5th and 95th percentiles.

ICE curves reveal what the average hides

- Centre each curve at the lowest rpm to compare slopes.
- Soft rock: predicted ROP rises with rpm; hard rock: almost flat.
- The average curve describes neither group.

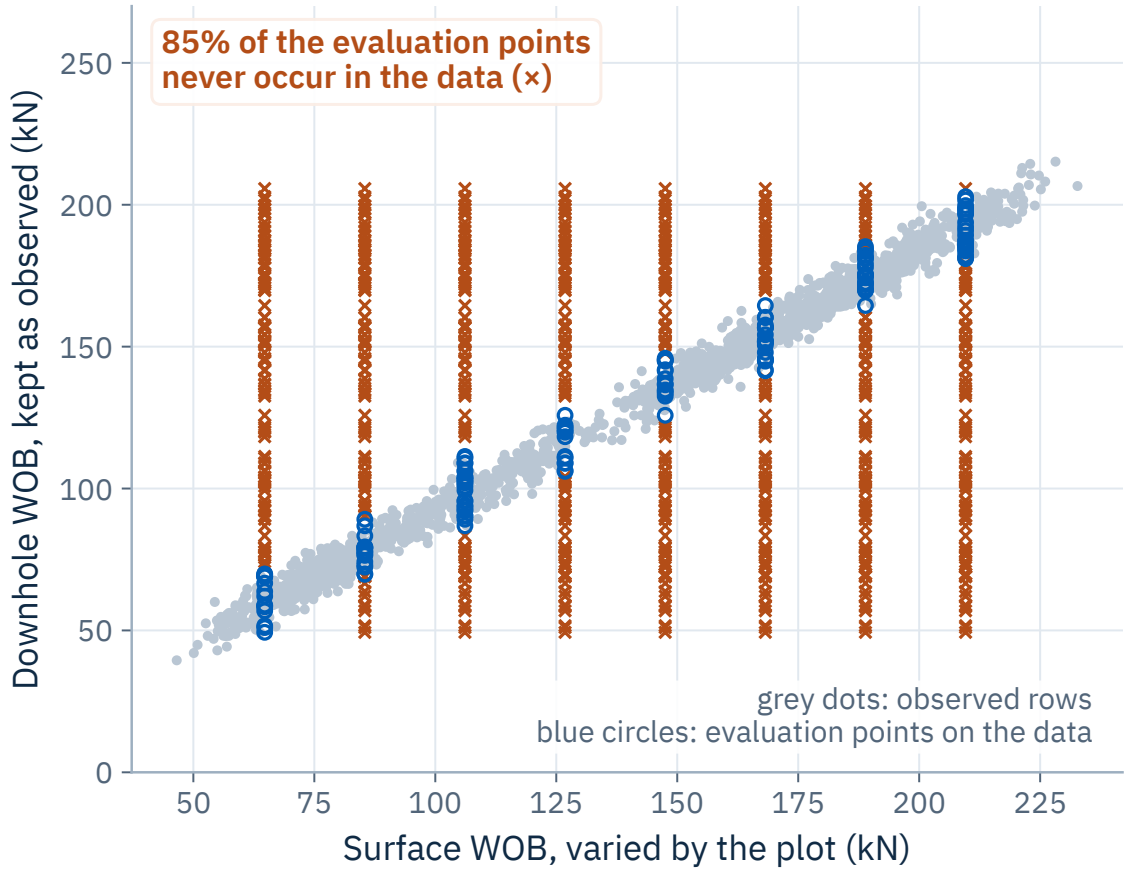


Centred ICE curves from the synthetic-drilling boosting model; 120 of 300 rows shown, coloured by formation.

◆ Diverging ICE curves signal an interaction: plot them by group.

Partial dependence can query impossible inputs

- PD pairs each grid value with every observed value of the other inputs.
- With correlated inputs most pairs never occur: 85% here.
- The model's answers there are extrapolation, not evidence.

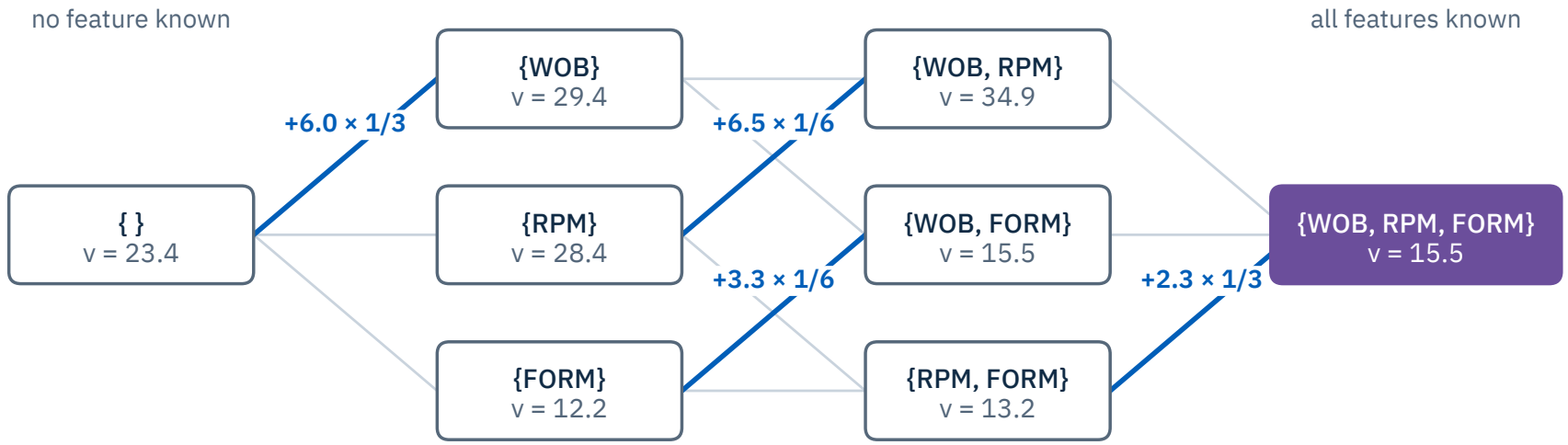


Synthetic drilling data with a downhole WOB column; 8 grid values × 120 observed downhole values.

◆ Restrict to observed combinations, or use accumulated local effects.

Shapley values average a feature's marginal steps

$$\phi_j = \sum_{S \subseteq F \setminus \{j\}} \frac{|S|!\ (p - |S| - 1)!}{p!} \left[v(S \cup \{j\}) - v(S) \right]$$



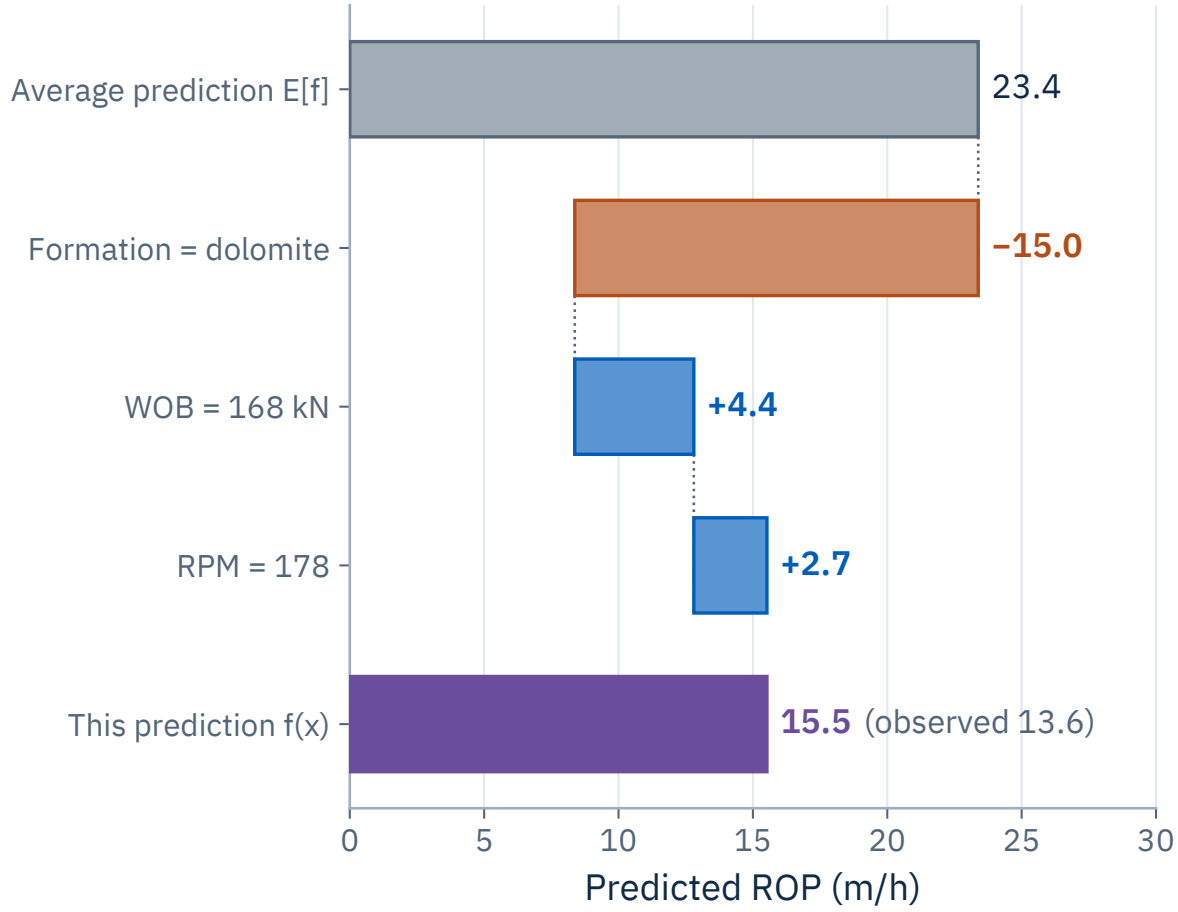
φ(WOB) = weighted sum of the four blue steps = +4.43 m/h · v(S): mean prediction with the features in S fixed to this row

Exact computation for one row of well D6; 3-feature boosting model; v(S) in m/h over 200 background rows.

Shapley values add up to the prediction

$$f(x) = \mathrm{E}[f(X)] + \sum_{j=1}^{p} \phi_j$$

- Base: the average prediction over the background rows, 23.4 m/h.
- Dolomite –15.0, WOB +4.4, RPM +2.7: this row predicts 15.5 m/h.
- Features outside S are filled from background rows (interventional).



Exact interventional Shapley values for one row of synthetic well D6.

PRACTICE

5 min · Pairs

Shapley values by hand

A two-feature ROP model gives average predictions (m/h): no feature known 20; WOB known 26; RPM known 22; both known 30.

1. Compute the Shapley value of WOB and of RPM.
2. Check that they add up to the prediction minus the base.
3. Why is WOB's step larger once RPM is known?

Choose the explanation that answers your question

Method	Question it answers	Scope	Main pitfall
Impurity (MDI)	Which splits cut training impurity?	Global, training	Rewards high-cardinality noise
Permutation	How much does the score rely on a feature?	Global, held-out	Correlated inputs share credit
PD and ICE	How do predictions change with one input?	Average and per row	Extrapolates with correlated inputs
Shapley values	How is one prediction split among inputs?	One row	Depends on the background data

Explanations describe the model, not the rock

Association, not causation

- Importance shows what the model uses
- Controlled tests show what the rock does

Extrapolation

- PD and Shapley query unseen combinations
- Check the data support first

Correlated inputs

- Credit is split or hidden
- Group copies before ranking

Stability

- Recompute on other folds and wells
- Unstable rankings are not findings

Background choice

- Shapley values move with the reference rows
- State the background in every report

Leakage shows up here too

- A leaky feature often ranks first
- Treat a surprise winner as a bug report

◆ Explanations are evidence about the fitted model; decisions need domain review.

PART 6 OF 6

06

Pipelines, leakage, and deployment

The deployed predictor is the whole pipeline, with its evidence.

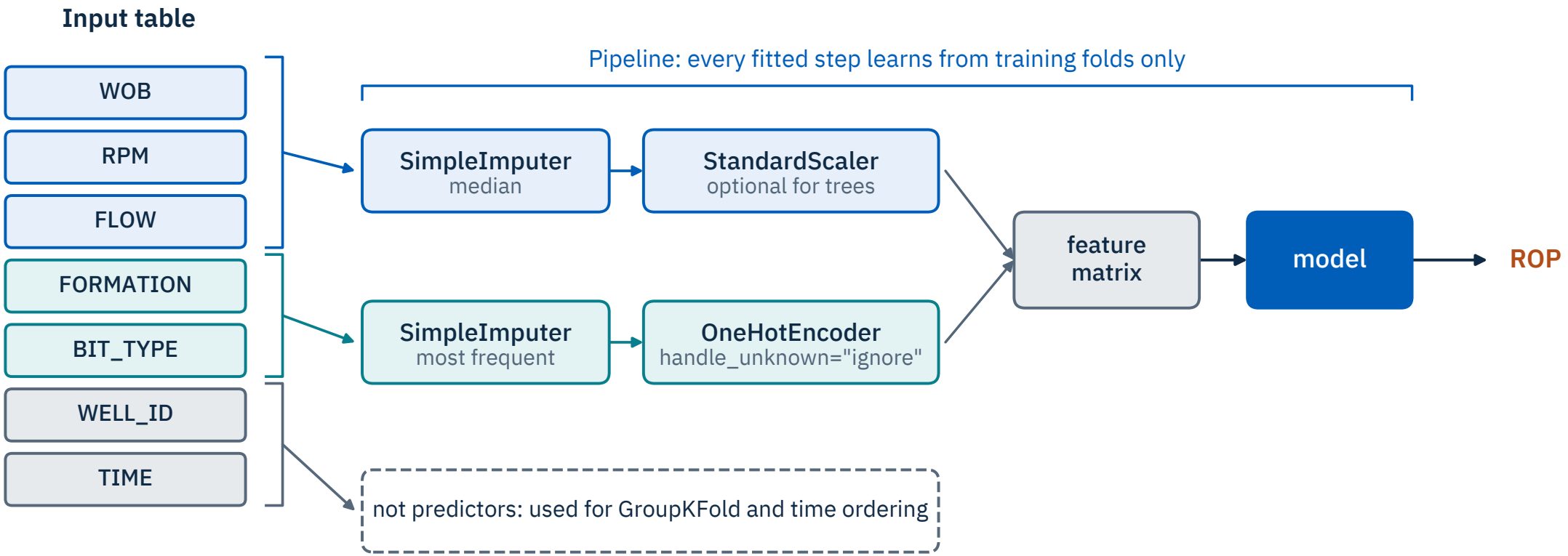
- ColumnTransformer for mixed inputs
- A leakage demonstration
- Saving an artifact with its context
- Model cards and monitoring

Mixed data needs separate transformations

Input family	Example	Training transformation
Numeric	Pressure, flow, gamma ray	Imputation and optional scaling
Categorical	Formation or equipment type	Imputation and category encoding
Metadata	Well ID, timestamp, record ID	Split, join, or ordering key

New categories and missing sensors should have defined behaviour.

A ColumnTransformer routes each column family



Schematic of a ColumnTransformer inside a Pipeline for the synthetic drilling table; column names are illustrative.

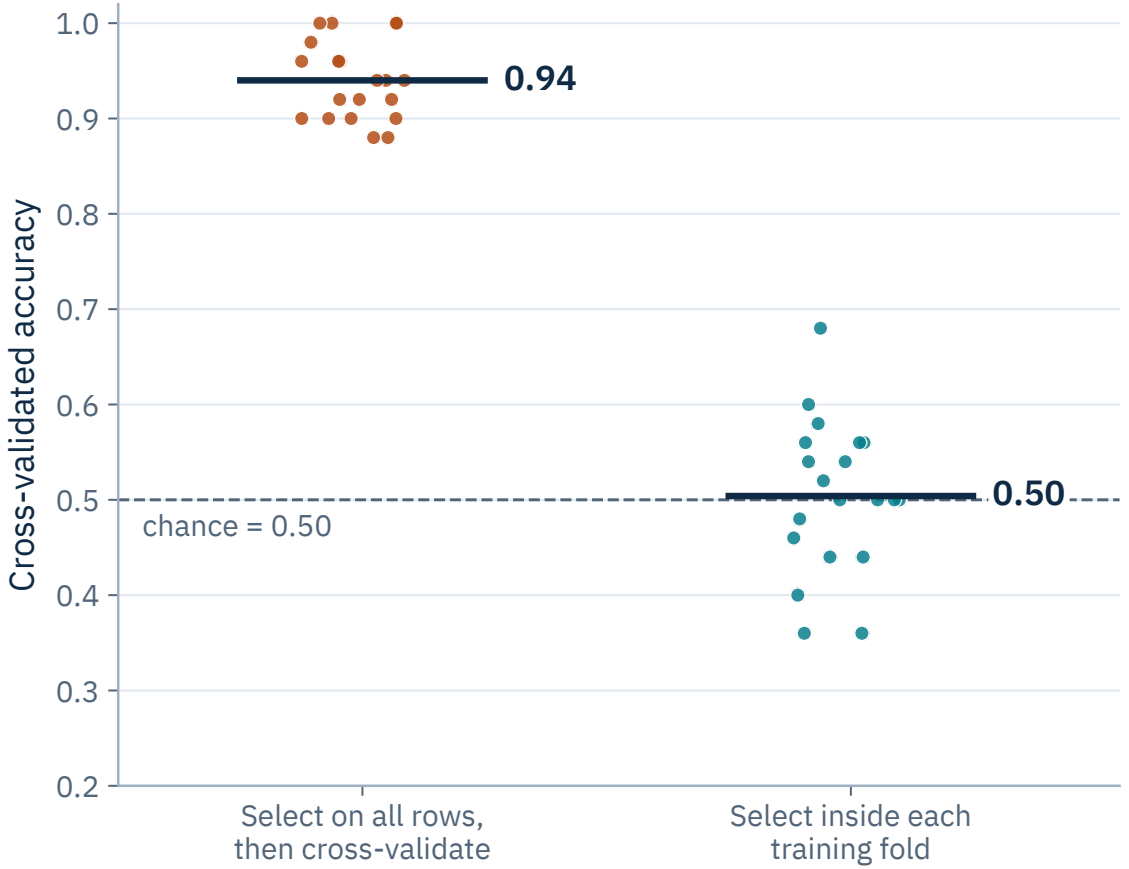
A ColumnTransformer preserves the schema

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.pipeline import make_pipeline

numeric = make_pipeline(SimpleImputer(strategy="median"))
categorical = make_pipeline(
    SimpleImputer(strategy="most_frequent"),
    OneHotEncoder(handle_unknown="ignore")
)
prep = ColumnTransformer([
    ("num", numeric, numeric_columns),
    ("cat", categorical, categorical_columns)
])
```

Selecting features on all rows fakes a signal

- 50 rows, 2,000 noise features, random labels: nothing to learn.
- Select the 20 "best" features on all rows, then cross-validate: 0.94.
- Select inside each training fold: 0.50, which is chance.



◆ Any step that looks at labels must sit inside the pipeline.

20 synthetic datasets (seed 0), SelectKBest(f_classif, k = 20) and logistic regression; 5-fold stratified CV.

Feature selection belongs inside the pipeline

```
rng = np.random.default_rng(0)
X = rng.normal(size=(50, 2000))                # 2,000 pure-noise features
y = rng.permutation(np.repeat([0, 1], 25))     # random labels
cv = StratifiedKFold(5, shuffle=True, random_state=0)

# Wrong: the selector sees every row, including each validation fold
X_sel = SelectKBest(f_classif, k=20).fit_transform(X, y)
wrong = cross_val_score(LogisticRegression(), X_sel, y, cv=cv)

# Right: selection is refitted inside every training fold
pipe = make_pipeline(SelectKBest(f_classif, k=20), LogisticRegression())
right = cross_val_score(pipe, X, y, cv=cv)
print(f"wrong {wrong.mean():.2f}   right {right.mean():.2f}")
```

OUTPUT

```
wrong 0.90   right 0.36
```

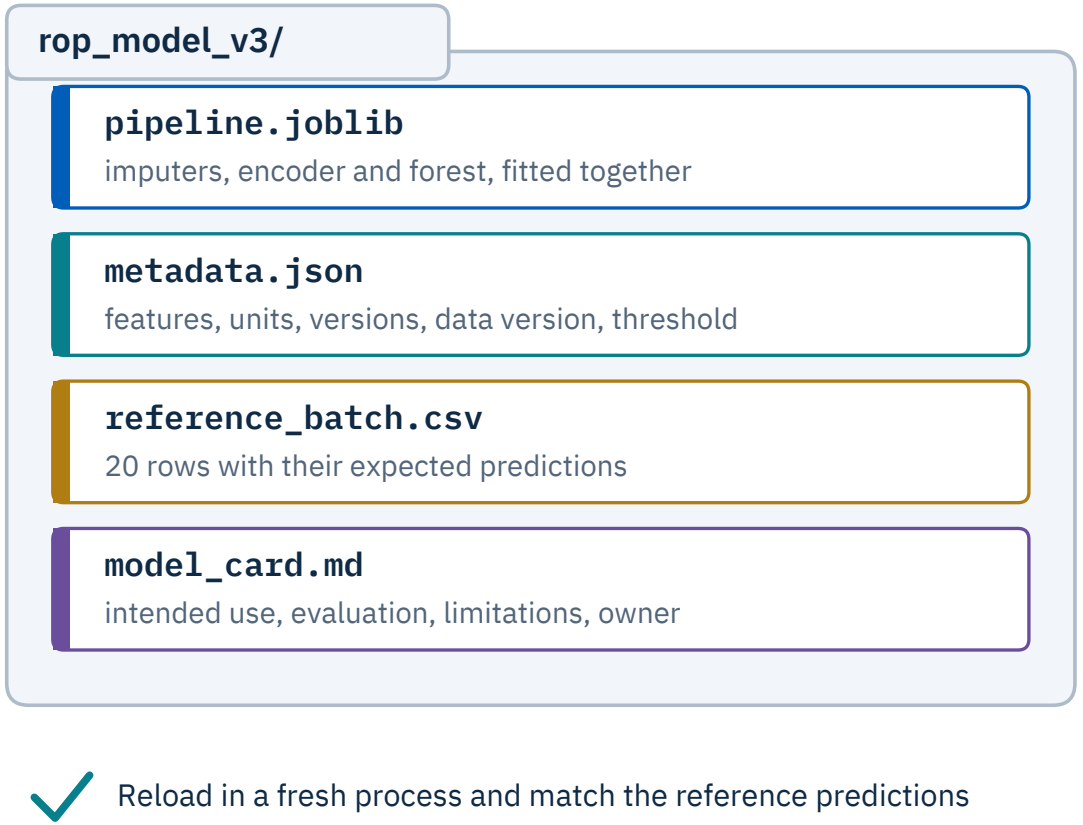
Tuning operates on the complete pipeline

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.pipeline import Pipeline

model = Pipeline([
    ("prep", prep),
    ("forest", RandomForestClassifier(
        n_estimators=200, random_state=42, n_jobs=-1))
])
param_grid = {
    "forest__min_samples_leaf": [1, 5, 15],
    "forest__max_features": ["sqrt", 0.7]
}
```

The saved artifact needs context

- Save fitted preprocessing and estimator together.
- Record feature names, units, versions, data version, and threshold.
- Reload in a fresh process and compare predictions on a reference batch.
- Load serialized Python objects only from trusted sources.



Schematic of a versioned model artifact; file names are illustrative.

Persist the pipeline and pin the versions

```python
import json, sys, joblib, sklearn, numpy as np

joblib.dump(model, "rop_model_v3/pipeline.joblib")
meta = {
    "features": numeric_columns + categorical_columns,
    "target": "ROP (m/h)",
    "sklearn": sklearn.__version__,
    "numpy": np.__version__,
    "python": sys.version.split()[0],
    "training_data": "rop_wells_2026-09 (wells D1-D5)",
}
with open("rop_model_v3/metadata.json", "w") as f:
    json.dump(meta, f, indent=2)

reloaded = joblib.load("rop_model_v3/pipeline.joblib")   # trusted files only
assert np.allclose(reloaded.predict(X_ref), expected_ref)
```

A model card states what the model is for

Intended use

- Predict ROP on new wells in field X
- Planning aid, not an autodriller input

Training data

- Wells D1–D5, 2026 campaign, 1 m rows
- Formations covered; known gaps

Evaluation

- Grouped CV by well; MAE per formation
- Baseline: training mean and offset wells

Limitations

- No extrapolation beyond seen WOB and RPM
- Untested in formations absent from training

Owner and review

- Named owner, review every 3 months
- Retraining and rollback triggers

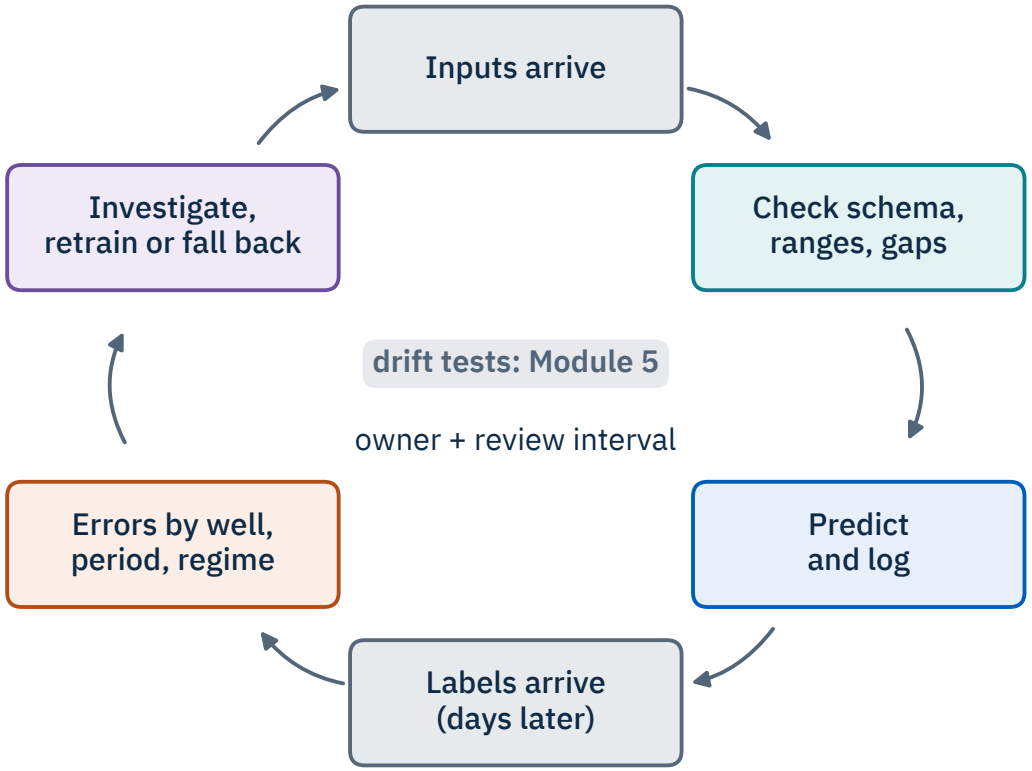
Version

- Artifact hash, library versions, date
- Link to metadata and reference batch

◆ If you cannot fill a card, the model is not ready to hand over.

Production evaluation continues after release

- Compare incoming sensor distributions with the training population.
- Track missingness, new categories, latency, and rejected inputs.
- When labels arrive, inspect errors by well, period, and operating regime.
- Define when to investigate, retrain, or fall back to a simpler method.



Schematic of a monitoring loop; drift tests are covered in Module 5.

PRACTICE

5 min · Pairs

Spot the leaks in this workflow

```
logs = pd.read_csv("facies_logs.csv")          # 8 wells, depth-ordered rows
logs["GR_z"] = (logs.GR - logs.GR.mean()) / logs.GR.std()
logs["SAND_SHARE"] = logs.groupby("WELL").FACIES.transform(
    lambda f: (f == "sandstone").mean())
sel = SelectKBest(f_classif, k=4).fit(logs[cols], logs.FACIES)
X = logs[cols].loc[:, sel.get_support()]
X_tr, X_te, y_tr, y_te = train_test_split(X, logs.FACIES, test_size=0.2)
search = GridSearchCV(RandomForestClassifier(), grid, cv=5).fit(X_tr, y_tr)
print("test accuracy:", search.score(X_te, y_te))
```

1. Find at least four problems and say how each inflates the score.
2. Rewrite the workflow as one pipeline with a grouped split.

LABORATORY

55 min

Laboratory 2: importance, pipelines, and tuning

- Part 6: compare the forest's MDI with permutation importance on the test rows.
- Part 7: build the pipeline, save it with joblib, reload it, and match predictions.
- Part 8: grid-search the pipeline and inspect every row of cv_results_.
- Part 9 (optional): XGBoost if installed, otherwise HistGradientBoostingClassifier.

NOTEBOOK

`exercises/Ensemble_Methods_Student.ipynb`

EVIDENCE TO SUBMIT

- MDI versus permutation table with one sentence of interpretation
- A reload check passing on five reference rows
- The grid-search table and your reading of the best setting

DISCUSSION

Lab 2 debrief: impurity and permutation disagree on Iris

Feature	MDI (training)	Permutation (test, 20 repeats)
Petal width	0.437	0.137
Petal length	0.431	0.128
Sepal length	0.116	−0.052
Sepal width	0.015	−0.033

- Grid search: all 18 settings tie at 0.950, so best_params_ is simply the first in the grid.
- The reloaded pipeline must reproduce predictions exactly; the scaler did not matter.

PRACTICE

5 min · Groups of 3

Evidence review

The best validation score improves by 0.002 after testing 150 settings.

1. What evidence would support calling this an improvement?
2. What uncertainty comes from the search itself?
3. How would you choose if the larger model costs ten times more to serve?

Key takeaways

- ✓ Ensembles help when members are useful and their errors differ; correlation sets the floor.
- ✓ Forests average decorrelated deep trees; boosting adds shallow trees fitted to pseudo-residuals.
- ✓ Out-of-bag scores and random validation fractions assume unrelated rows: validate by well or time.
- ✓ Explanations describe a fitted model on given data: use held-out permutation, check support, group copies.
- ✓ Ship the whole pipeline with its metadata, a reference batch, a model card, and an owner.

PRACTICE

7 min · Individual

Exit check

1. Explain the role of diversity in a forest, using the variance formula.
2. Explain what the next boosting learner fits under squared loss.
3. Explain why saving only the estimator may be insufficient.
4. Name one reason an out-of-bag score can be optimistic for a new well.
5. Give one reason impurity importance can rank a noise feature highly.

References and further study

- Breiman (1996) Bagging predictors; Breiman (2001) Random forests. *Machine Learning* 24 and 45.
- Freund and Schapire (1997) A decision-theoretic generalization of on-line learning (AdaBoost). *J. Computer and System Sciences* 55.
- Friedman (2001) Greedy function approximation: a gradient boosting machine. *Annals of Statistics* 29(5); Friedman (2002) Stochastic gradient boosting.
- Chen and Guestrin (2016) XGBoost; Ke et al. (2017) LightGBM; Prokhorenkova et al. (2018) CatBoost.
- Wolpert (1992) Stacked generalization; Geurts, Ernst and Wehenkel (2006) Extremely randomized trees.
- Strobl et al. (2007) Bias in random forest variable importance; Goldstein et al. (2015) ICE plots; Lundberg and Lee (2017) SHAP.
- Mitchell et al. (2019) Model cards for model reporting; Hastie, Tibshirani and Friedman, *The Elements of Statistical Learning*, chapters 7, 10, 15, 16.
- scikit-learn User Guide: ensembles, permutation importance, partial dependence, pipelines and composite estimators, model persistence.

MACHINE LEARNING / MODULE 3

Clustering and mixture models

K-means, density, hierarchy, and probabilistic membership for energy data

Full day · 09:00–17:00 | 6 teaching parts | 3 laboratories | Python · scikit-learn

Machine learning for energy applications

Today's plan

09:00	Why cluster energy data	Lecture
09:15	Part 1: Similarity and scaling	Lecture
09:45	Part 2: K-means	Lecture
10:30	Break	Break
10:45	Lab 1: K-means on well W5	Lab
11:35	Part 3: Density clustering	Lecture
12:15	Lunch	Lunch
13:15	Lab 2: DBSCAN on logs	Lab
14:00	Part 4: Hierarchy, zonation	Lecture
14:40	Break	Break
14:55	Part 5: Gaussian mixtures	Lecture
15:35	Part 6: Validation	Lecture
16:05	Lab 3: Mixtures on rocks	Lab
16:45	Recap and exit check	Discussion

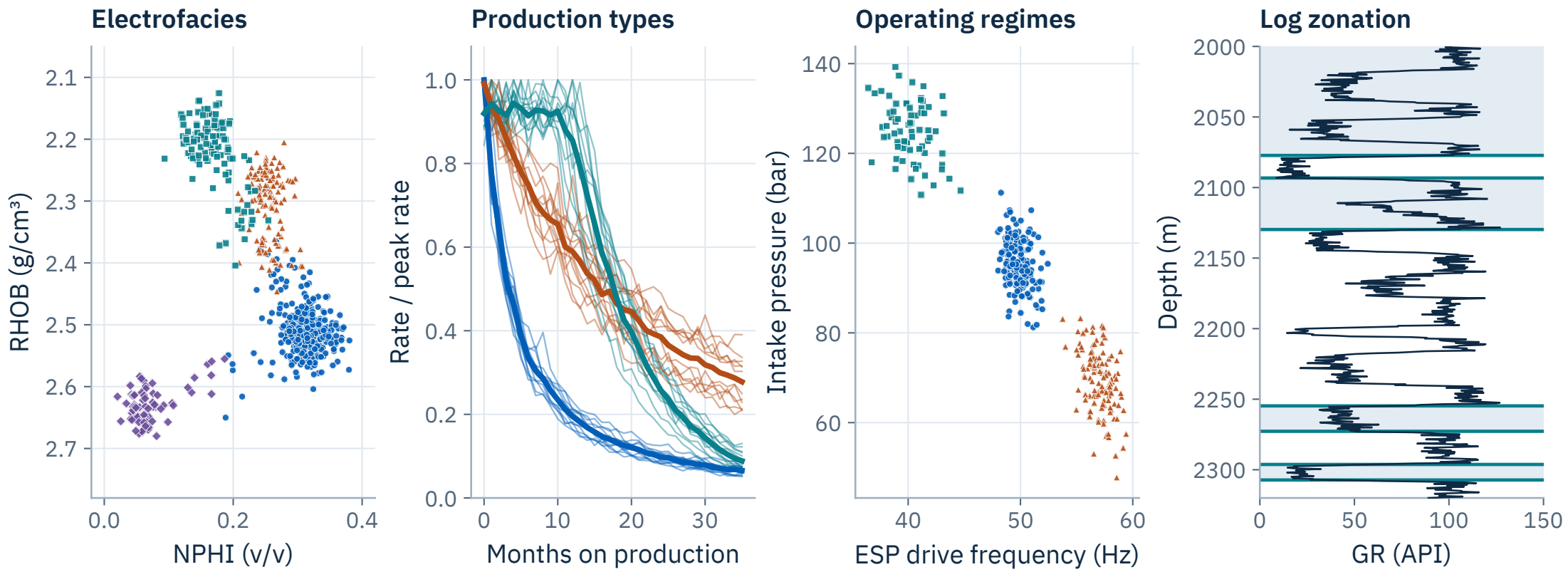
What you will be able to do

By the end of the day you will be able to:

1. Choose features, transforms and scaling that encode the similarity you intend.
2. Run and diagnose K-means: seeding, restarts and the choice of K.
3. Tune DBSCAN or HDBSCAN with a k-distance plot and interpret noise.
4. Build, cut and compare dendrograms, including depth-constrained zonation.
5. Fit Gaussian mixtures and choose K and covariance type with BIC.
6. Validate with stability and external labels; profile clusters in original units.

Clustering finds structure nobody has labelled yet

Four recurring energy tasks where the groups are not known in advance.



Synthetic: well S-1 logs (K-means, K = 4; constrained Ward), decline curves (K-means, K = 3), ESP data (mixture, K = 3).

Every grouping should serve a named decision

Electrofacies

- Rock-type candidates from logs
- Guides core sampling and petrophysical models

Production types

- Wells with similar decline shapes
- Supports type curves and analogue selection

Operating regimes

- Normal, ramped and restart states
- Regime-specific baselines and alarms

Zonation

- Contiguous depth intervals
- Layering for models and completions

◆ Write the decision down before choosing features: it defines what "similar" should mean.

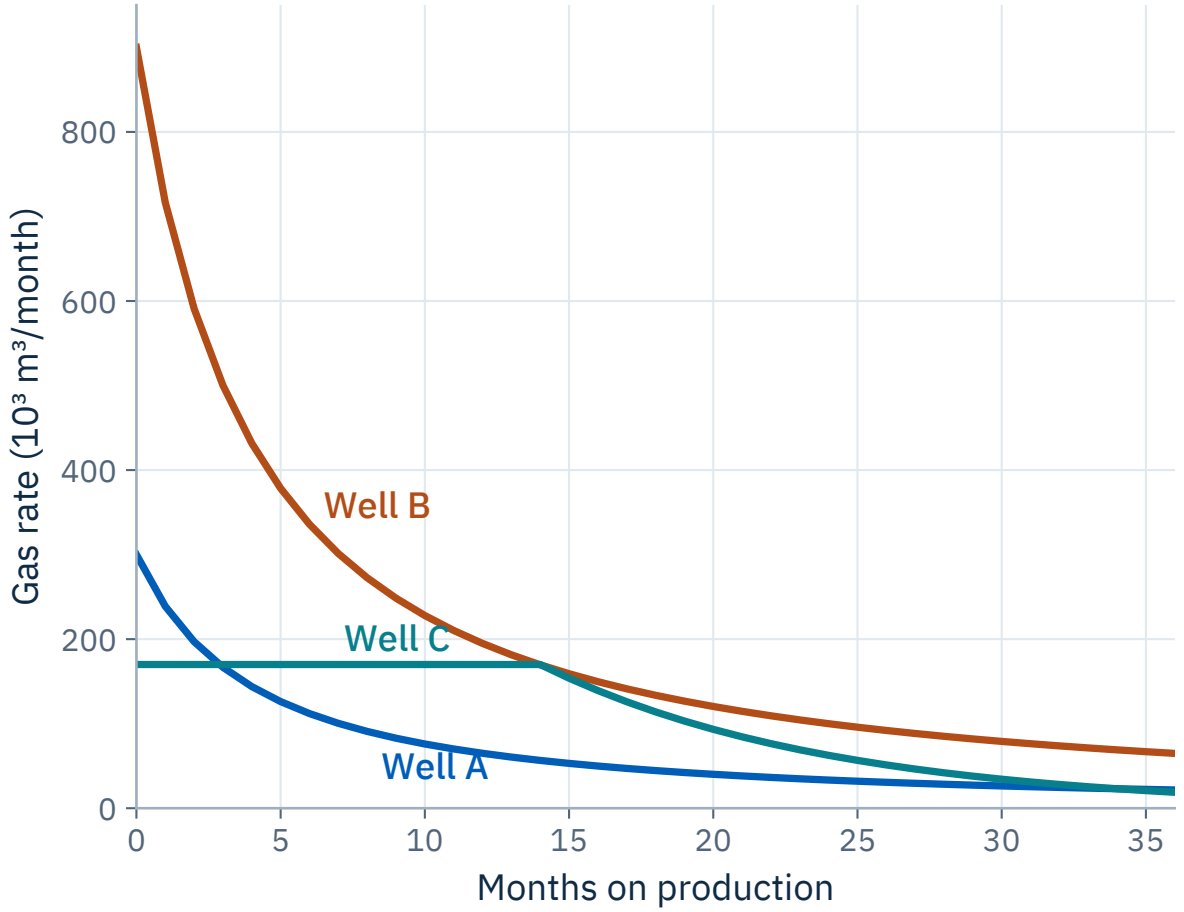
PRACTICE

5 min · Pairs

Similarity is part of the problem definition

You want to group wells with similar production behaviour.

1. Which well is most similar to well A, and why?
2. Should total production level affect similarity?
3. Which variables might simply encode well identity or age?



Synthetic decline curves: B is exactly 3 × A; C has a 14-month plateau.

PART 1 OF 6

01

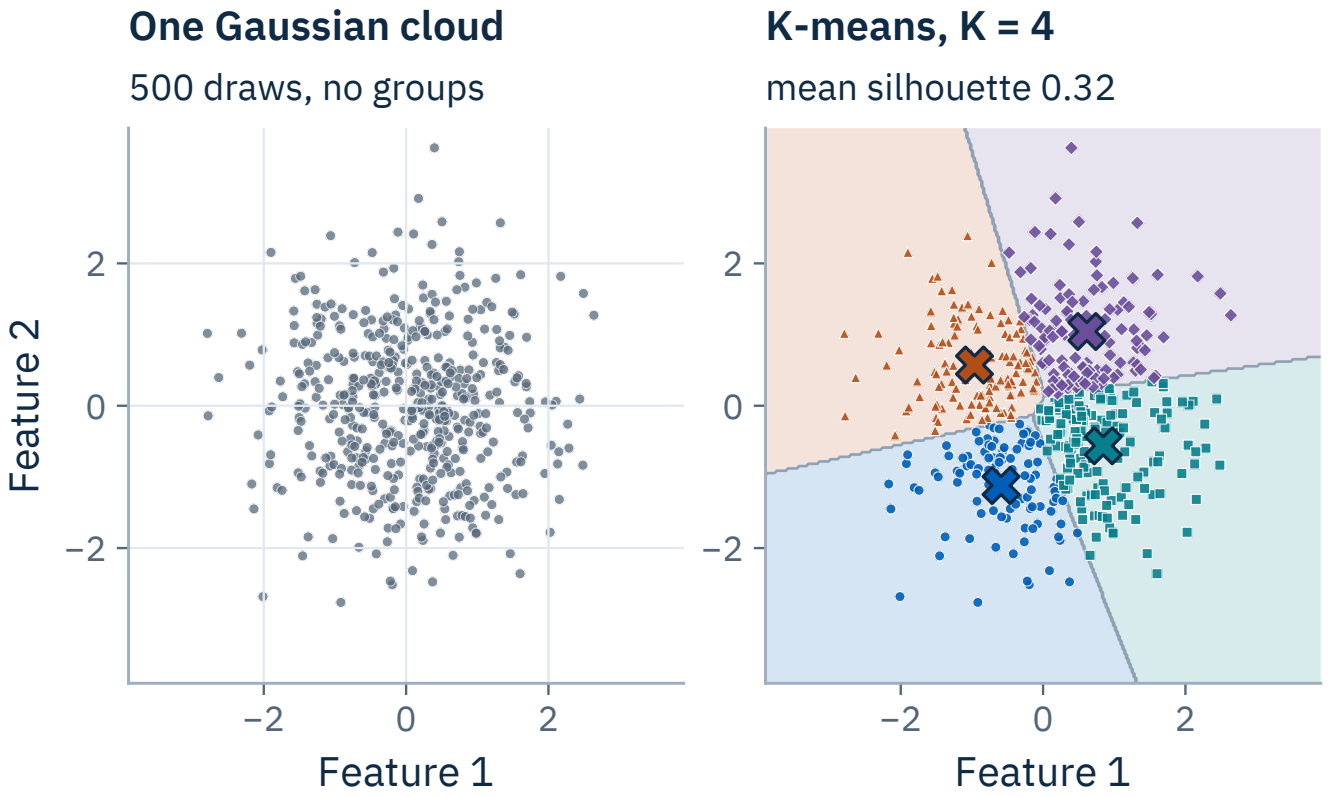
Similarity, features and scaling

A clustering is only as meaningful as its distance.

- Clusters as hypotheses
- The clustering workflow
- Distances, scaling and transforms
- Feature choice and weights

A cluster label is a hypothesis

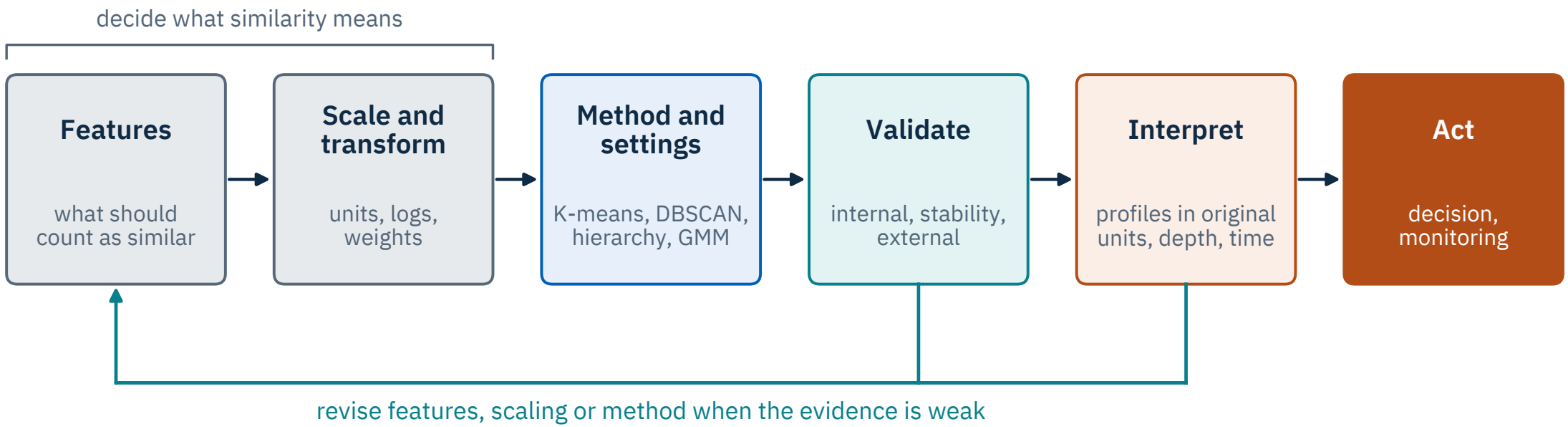
- Algorithms group by their objective and distance, not by physical truth
- Scaling, features and resolution change the groups
- Cluster numbers have no order or physical meaning
- Naming a cluster as a facies needs external evidence



Synthetic: 500 draws from one bivariate normal (seed 11); K-means, K = 4.

Clustering is a loop, not a single fit

Most of the scientific decisions happen before and after the algorithm runs.



Schematic. The feedback loop returns to features, scaling or method whenever validation or interpretation is weak.

The distance decides what counts as similar

Euclidean

- Straight-line distance; K-means uses its square
- Large gaps on one feature dominate

Manhattan

- Sum of absolute differences
- Less dominated by a single large gap

Correlation or cosine

- Compares shape or direction
- Magnitude drops out, e.g. rate level

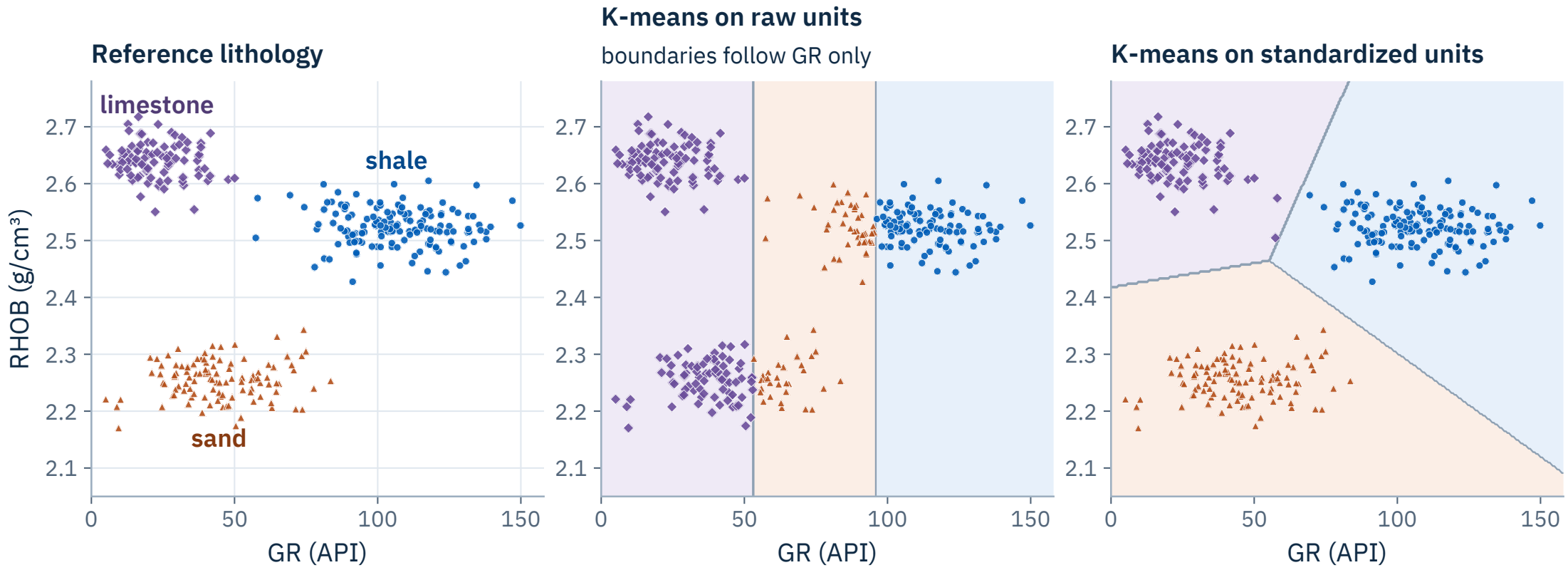
Mahalanobis

- Accounts for spread and correlation
- The distance inside each GMM component

◆ K-means is tied to squared Euclidean distance; DBSCAN and most hierarchical linkages accept other metrics.

Feature scaling changes the clusters

Same data and K: raw GR (SD 39 API) swamps RHOB (SD 0.16 g/cm³) until we rescale.



Synthetic GR and RHOB samples of three lithologies (seed 4); K-means, K = 3, n_init = 10; cluster colours matched to the reference.

Transforms and scalers are modelling choices

Standardize

- Mean 0 and SD 1 per feature
- Equal variance, not equal importance

Robust scale

- Median and interquartile range
- Outliers no longer set the scale

Log first

- Resistivity, permeability, rates
- Skewed, multiplicative variables

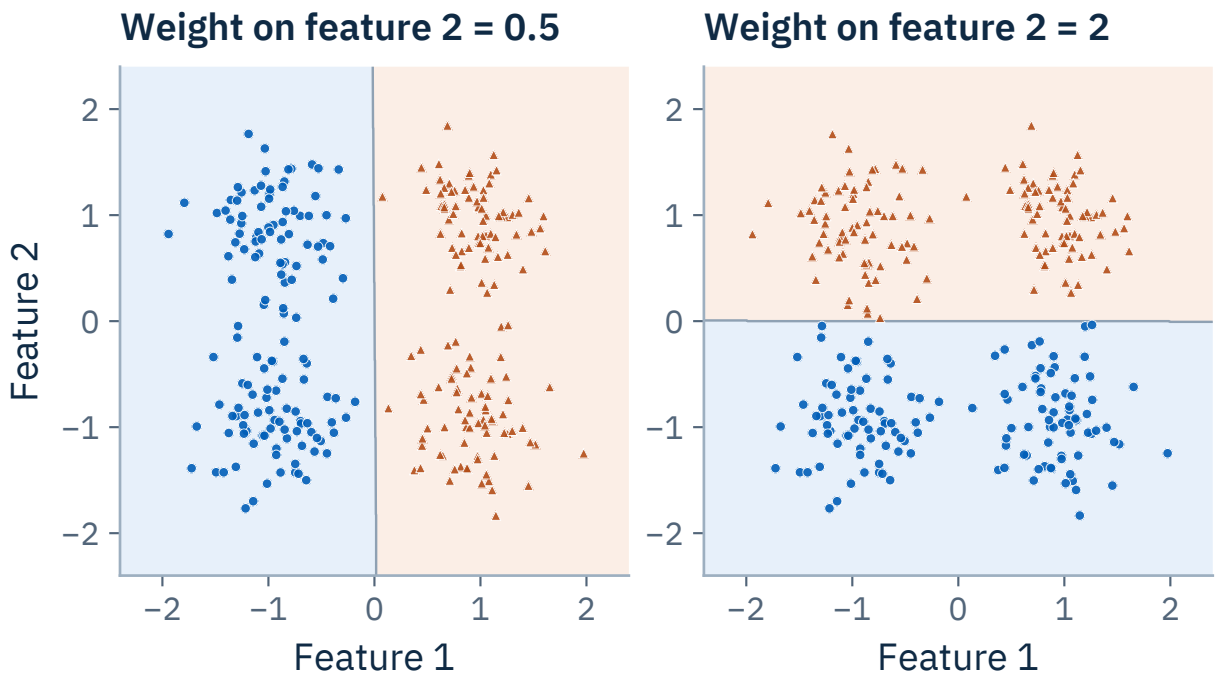
Keep units

- When features share one unit and scale
- e.g. normalized production profiles

◆ Fit the scaler once on the clustering data and reuse the fitted object for every new well.

Feature choice and weights set the answer

- Each feature's influence grows with its spread after scaling
- A duplicated or strongly correlated log counts twice
- Depth, well ID and dates encode location, not rock or behaviour
- Weight deliberately: multiply a standardized column by w



Synthetic four-group data (seed 8), standardized; K-means, K = 2, with feature 2 multiplied by the stated weight.

PRACTICE

5 min · Pairs

Choose the features for an electrofacies study

Available for 12 wells: GR, RHOB, NPHI, RT, DT, CALI, DEPTH and well name.

1. Which logs would you cluster on, and which would you leave out?
2. Which need a transform before scaling?
3. What would you check before trusting the clusters?

PART 2 OF 6

02

K-means

A compact-cluster model based on squared Euclidean distance

- The objective and Lloyd iterations
- Initialization: k-means++ and restarts
- Choosing K with elbow and silhouette
- Failure modes and variants

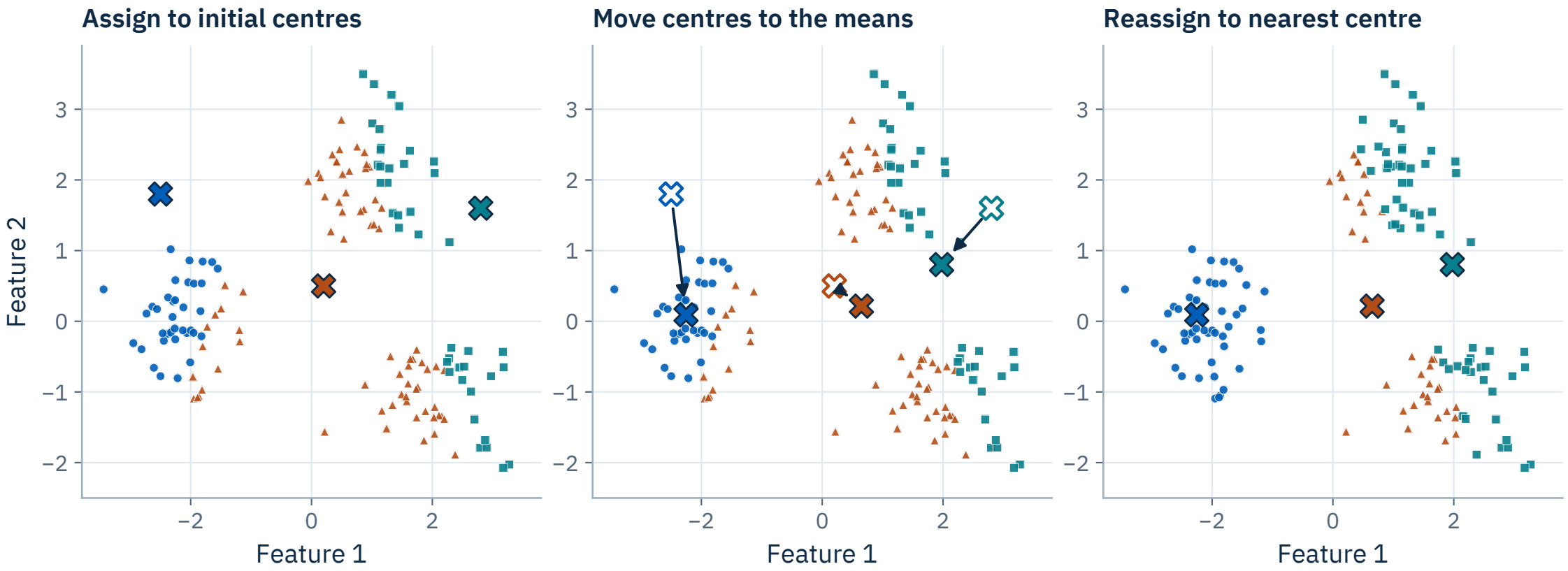
K-means minimizes within-cluster squared distance

$$J = \sum_{i=1}^{n} \| x_i - \mu_{c_i} \|^2$$

1. Choose K initial centroids
2. Assign each observation to its nearest centroid
3. Move each centroid to the mean of its assigned observations
4. Repeat until the assignments stop changing

- Neither step can increase J, so the loop converges, but only to a local minimum.

Assignment and update are separate steps



Synthetic three blobs (random_state = 42); one Lloyd iteration from fixed starting centres. Crosses mark centroids (hollow: starting position); colours show current assignments.

PRACTICE

4 min · Individual

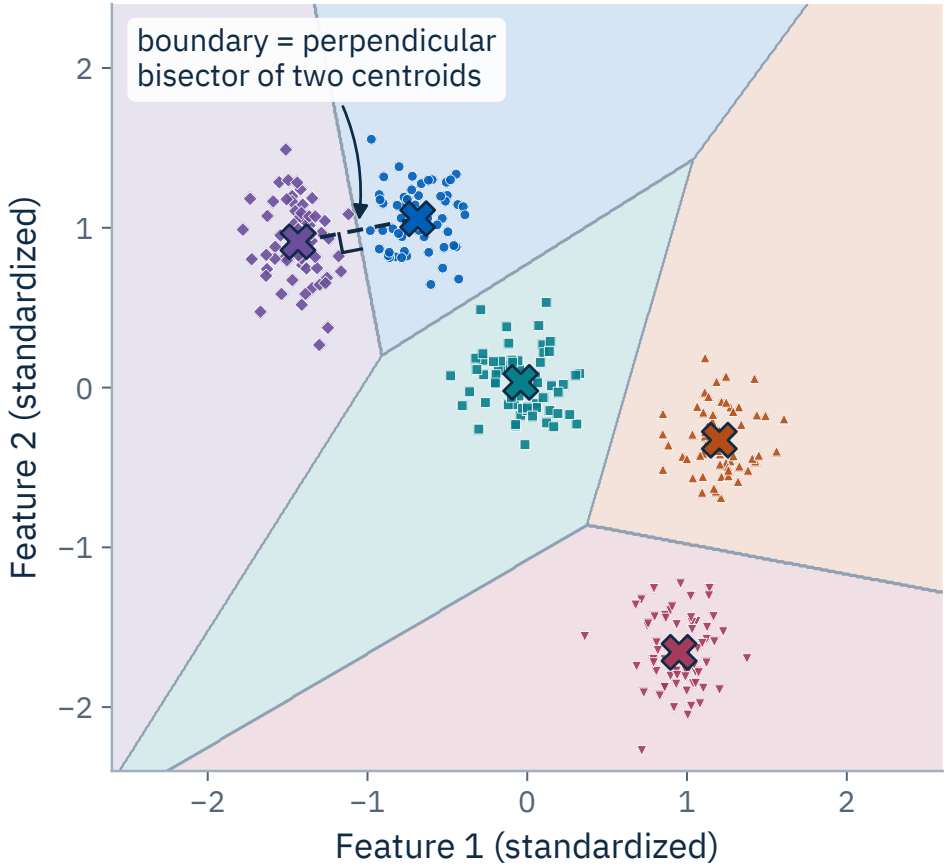
One update by hand

Observations on a line: 1, 2, 3, 8, 9, 10. Initial centroids: 2 and 8.

1. Assign each observation to a centroid.
2. Calculate the updated centroids.
3. Calculate the within-cluster sum of squares before and after the update.

Assignments carve the space into Voronoi cells

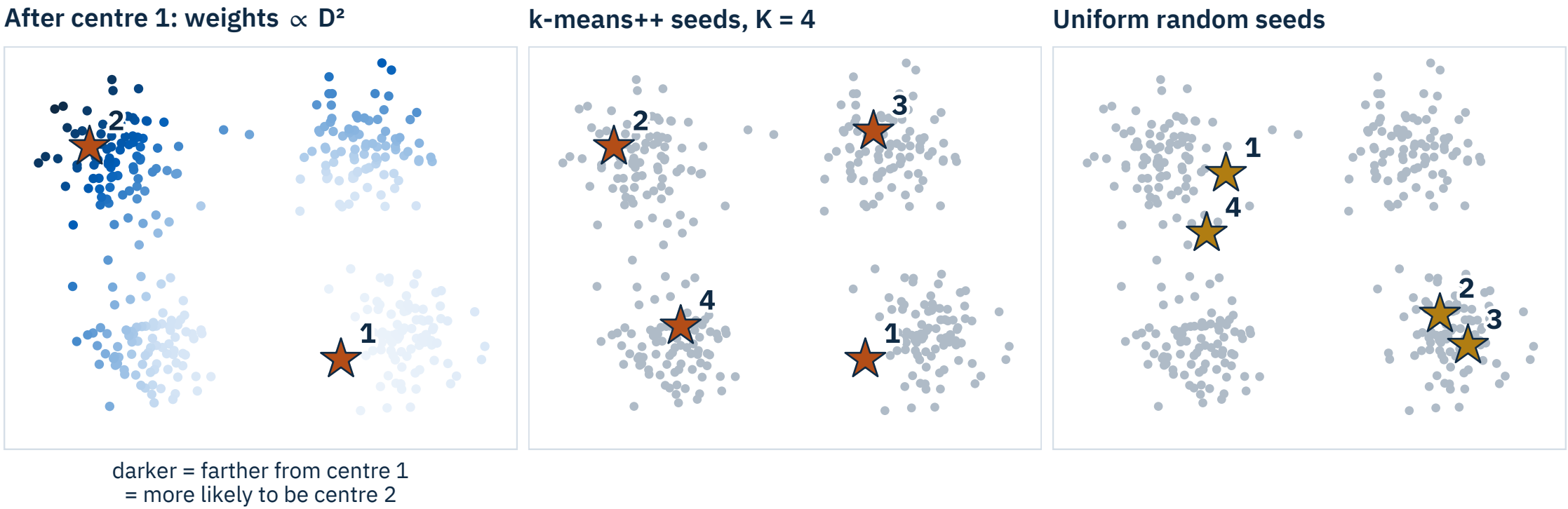
- Each point joins its nearest centroid
- Boundaries are perpendicular bisectors between centroids
- Every cell is convex: one cluster cannot curve around another
- New samples are assigned by the same rule with `predict`



Synthetic blobs (random_state = 6), standardized; K-means, K = 5. Shading marks each centroid's cell.

k-means++ spreads the initial centroids

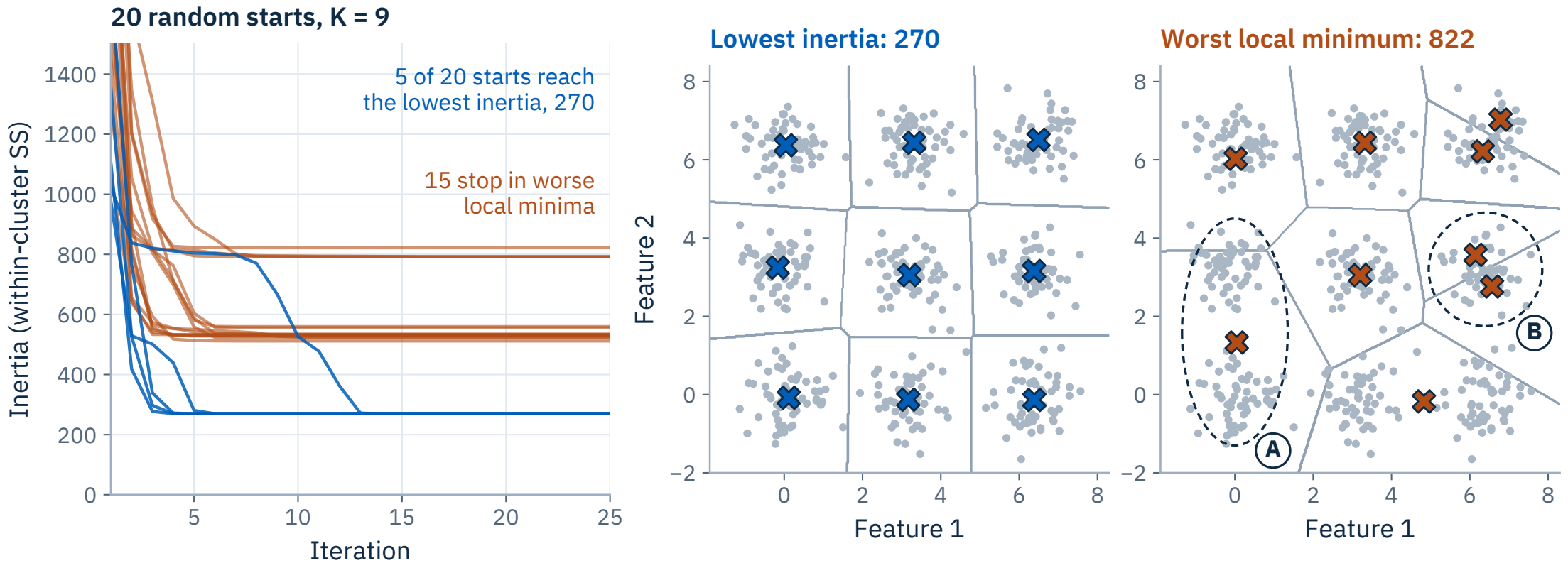
Each new centre is sampled with weight D^2: squared distance to the nearest centre.



Synthetic four blobs (random_state = 2); one draw each. One seed per blob in 66% of 2,000 textbook k-means++ draws, 95% with scikit-learn's greedy variant, 9.5% for uniform sampling (exact).

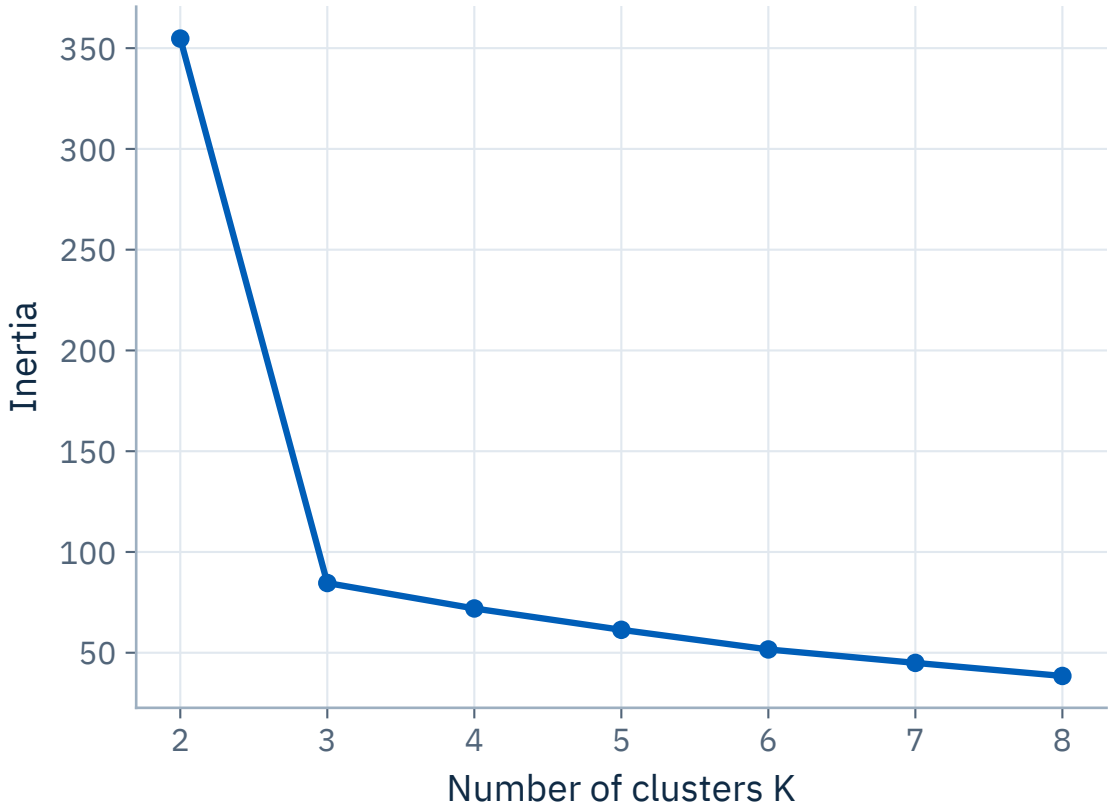
Random starts end in different local minima

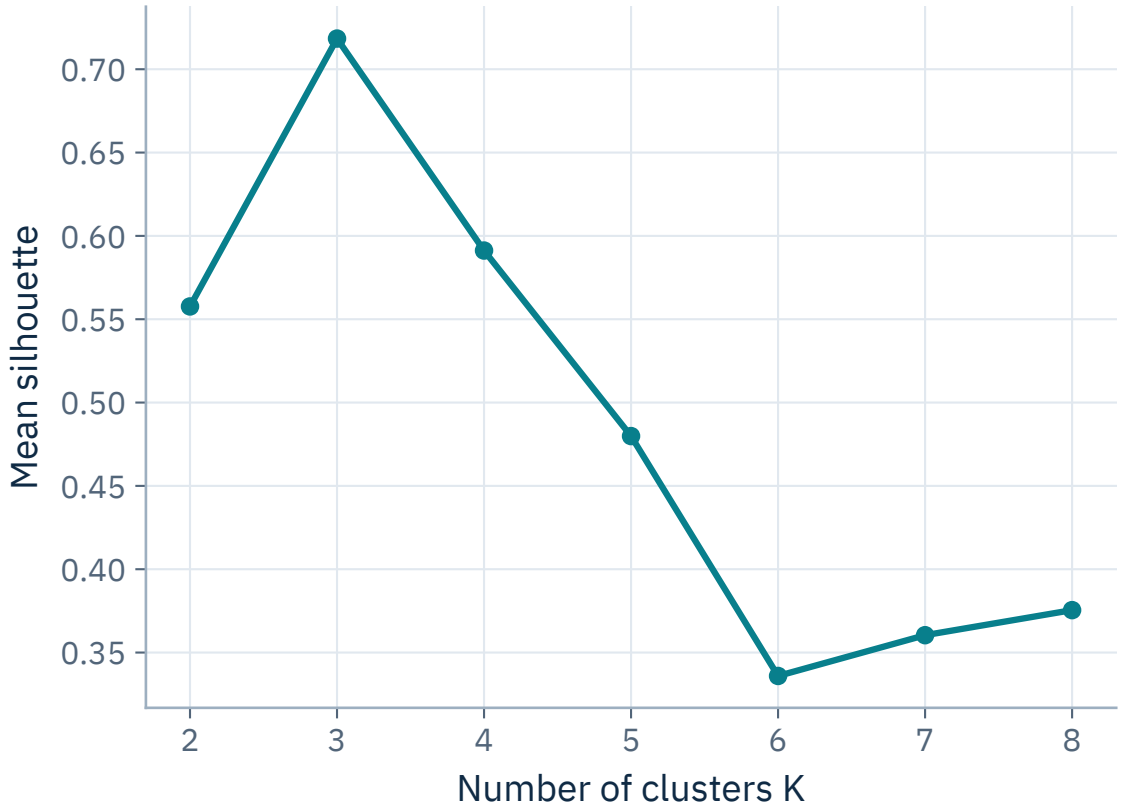
Lloyd iterations only go downhill; where they stop depends on where they start.



Synthetic 3 × 3 blob grid; 20 uniform random starts, K = 9. A: one centroid covers two blobs. B: two centroids split one blob.

Choosing K needs more than an elbow





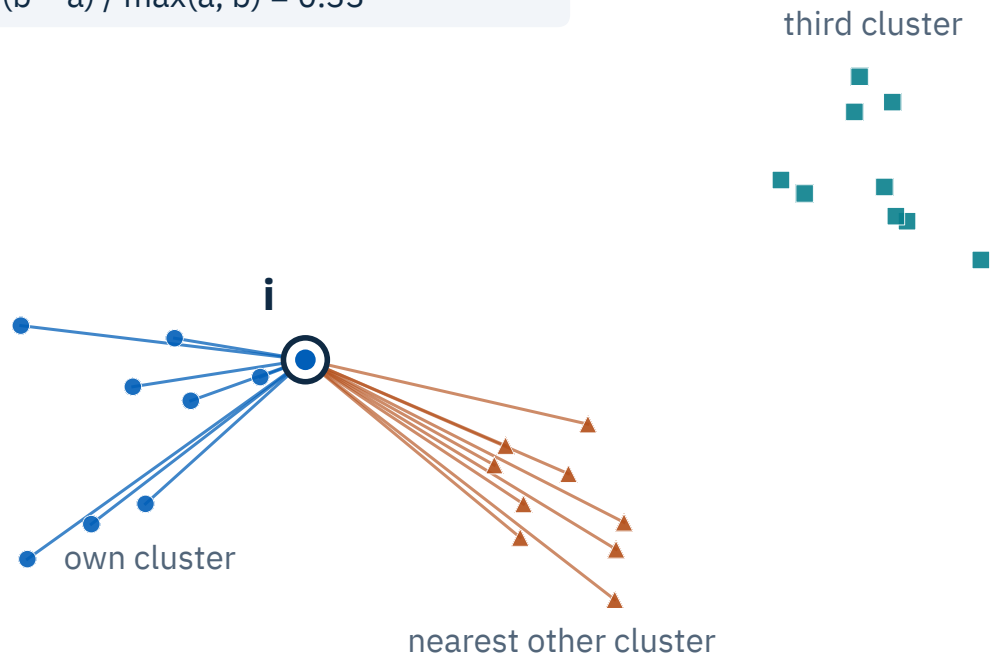
Inertia and mean silhouette measured on one synthetic dataset. Both depend on the feature space and cluster assumptions.

Silhouette compares cohesion with separation

$$s_i = \frac{b_i - a_i}{\max(a_i, b_i)}$$

- a_i: mean distance to the other members of its own cluster
- b_i: smallest mean distance to the members of another cluster
- Near 1: well placed; near 0: on a boundary; negative: closer to another cluster

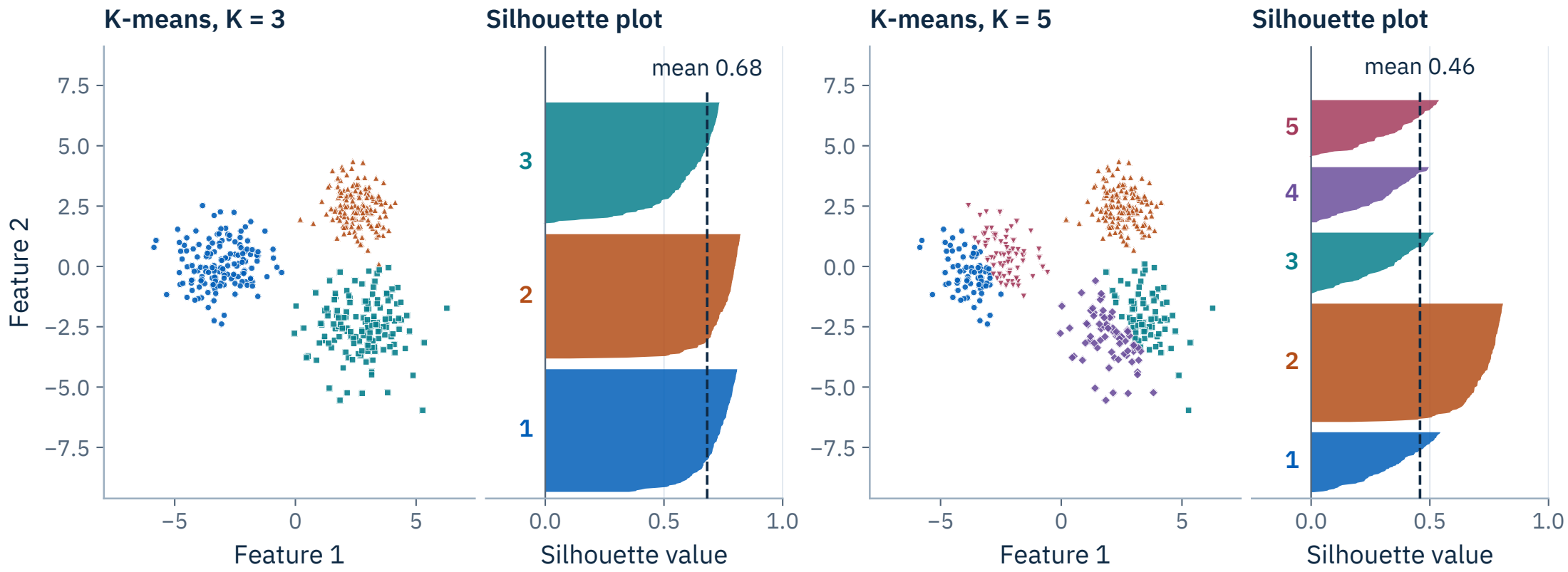
a = 1.53 mean distance to own cluster
b = 2.27 mean distance to nearest other
s = (b − a) / max(a, b) = 0.33



Synthetic three-cluster example (seed 21); values checked against scikit-learn silhouette_samples.

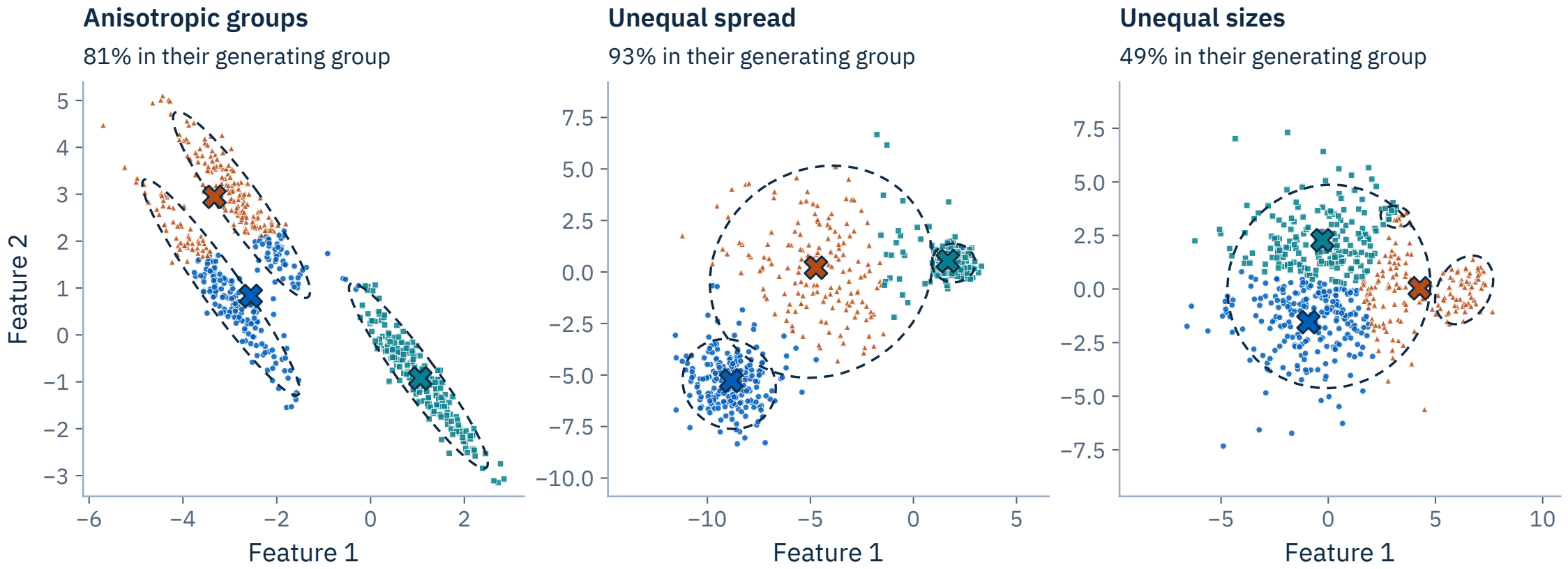
A silhouette plot shows every observation

Read each knife's width (size), length (fit) and tail (low or negative values).



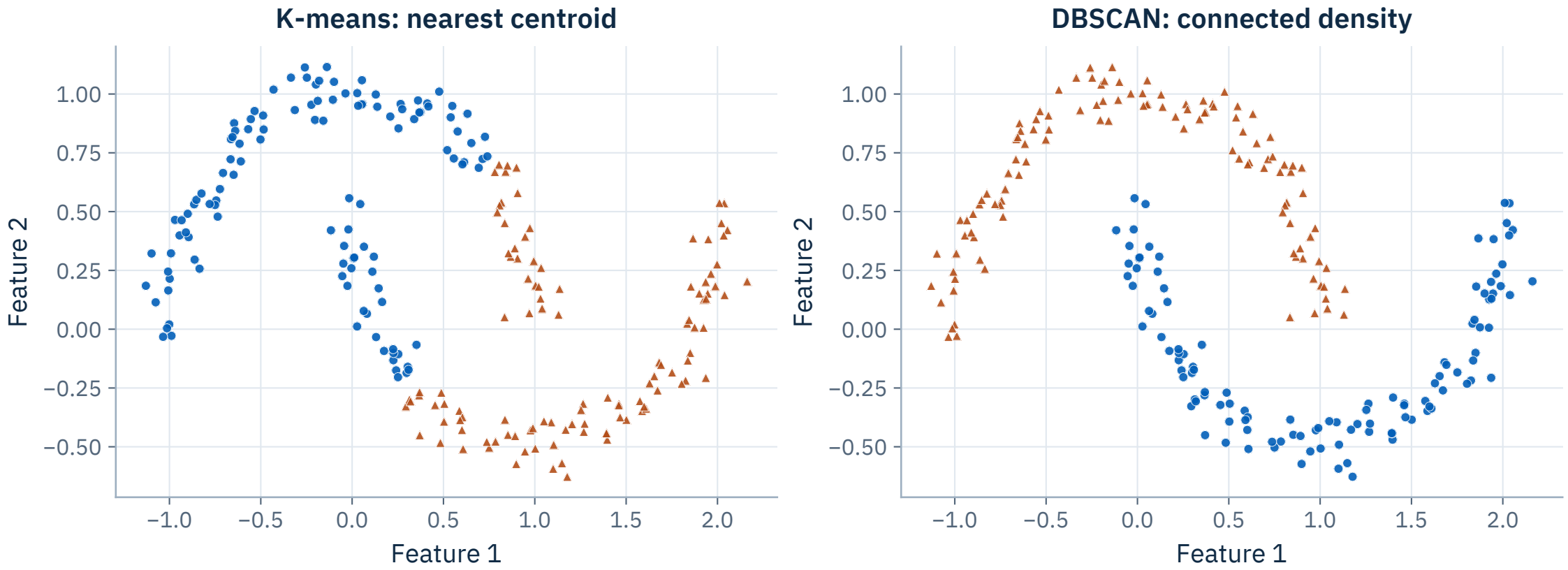
Synthetic three-blob data (random_state = 5); K-means with K = 3 and K = 5; per-sample values from silhouette_samples.

K-means fails in predictable ways



Synthetic data after the scikit-learn k-means assumptions example; K-means, K = 3, n_init = 10. Dashed ellipses: generating groups (90%).

The same data, different assumptions



K-means and DBSCAN fitted to the same synthetic two-moon data. Grey crosses mark density noise when present.

K-means works best with compatible geometry

✓ Works well when

- Groups are compact and similar in spread
- Features are scaled to comparable influence
- K can be checked with several kinds of evidence

vs

✗ Struggles when

- Groups are elongated, curved or nested
- Densities or sizes differ strongly
- Outliers pull centroids away

◆ Standardization is a choice: it can raise a noisy low-variance log to full weight.

Variants trade exactness for speed or robustness

MiniBatchKMeans

- Updates centroids from small random batches
- Much faster on large n; quality slightly reduced

k-medoids

- Centres are actual observations
- Any distance; less pulled by outliers
- Not in scikit-learn itself (scikit-learn-extra)

BisectingKMeans

- Splits one cluster at a time
- Produces a hierarchy of K-means splits

◆ Use MiniBatchKMeans when a full fit is too costly, and compare it with KMeans on a subsample.

A reproducible K-means fit

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

model = make_pipeline(
    StandardScaler(),
    KMeans(n_clusters=4, init="k-means++", n_init=10, random_state=0),
)
labels = model.fit_predict(logs[["GR", "RHOB", "NPHI", "LOG_RT"]])
km = model[-1]
centres = model[0].inverse_transform(km.cluster_centers_)  # original units
print(km.inertia_, km.n_iter_)
```

Since scikit-learn 1.4, n_init="auto" runs k-means++ once: set n_init yourself.

PRACTICE

5 min · Pairs

How many restarts are enough?

In the random-start experiment, 5 of 20 starts reached the lowest inertia.

1. Estimate p, the chance that one random start succeeds.
2. What is the chance that n_init = 10 finds the best solution at least once?
3. How many starts give 99%, and what are you assuming?

LABORATORY

50 min

Lab 1: K-means electrofacies on well W5

- ☐ Load w5.csv, inspect the six logs and standardize them.
- ☐ Fit K-means with K = 3 and n_init = 10; plot crossplots by cluster.
- ☐ Compare K = 2–6 with inertia, mean silhouette and silhouette plots.
- ☐ Profile the chosen clusters in original units and plot them against depth.

NOTEBOOK

`exercises/01_kmeans/KMeans_Clustering_Student.ipynb`

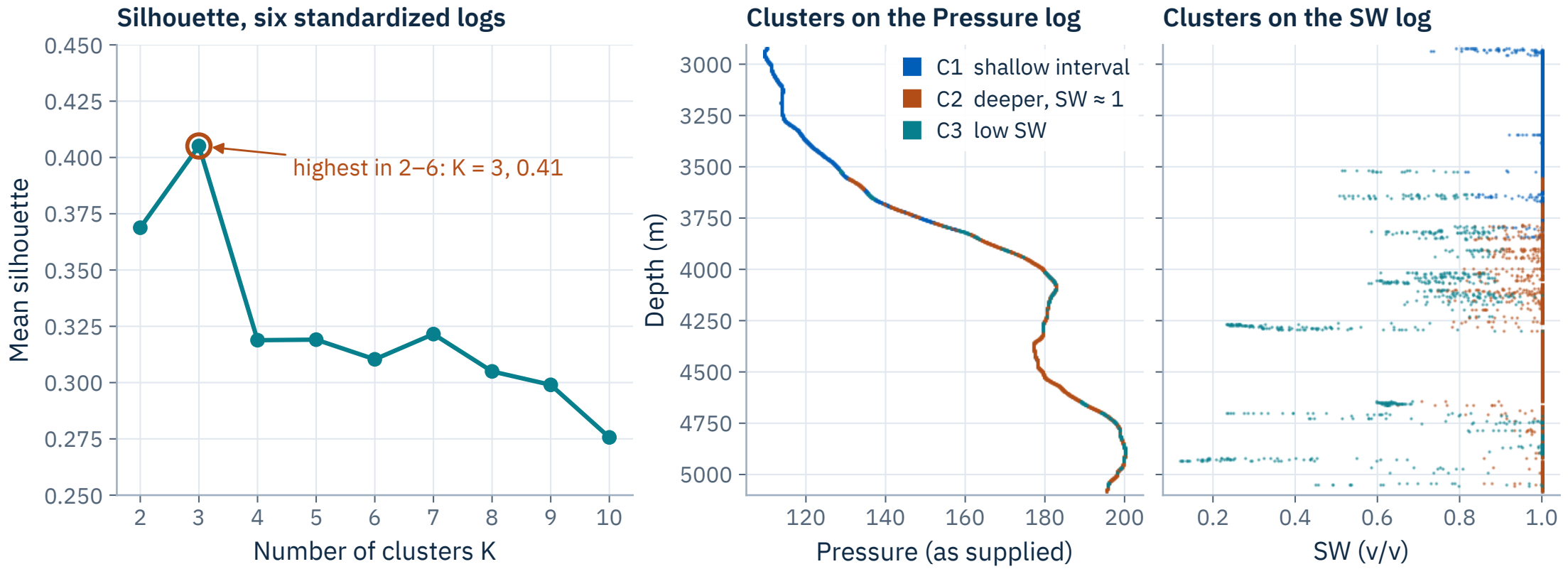
EVIDENCE TO SUBMIT

- Elbow and silhouette plots with your chosen K
- A cluster profile table in original units
- A depth plot and one hypothesis per cluster

DISCUSSION

Lab 1 debrief: W5 clusters follow depth and SW

The best silhouette (0.41, K = 3) mostly splits depth and low SW, not lithology.



Computed from w5.csv (course data): six standardized logs, KMeans(n_init = 10, random_state = 42), as in the notebook.

PART 3 OF 6

03

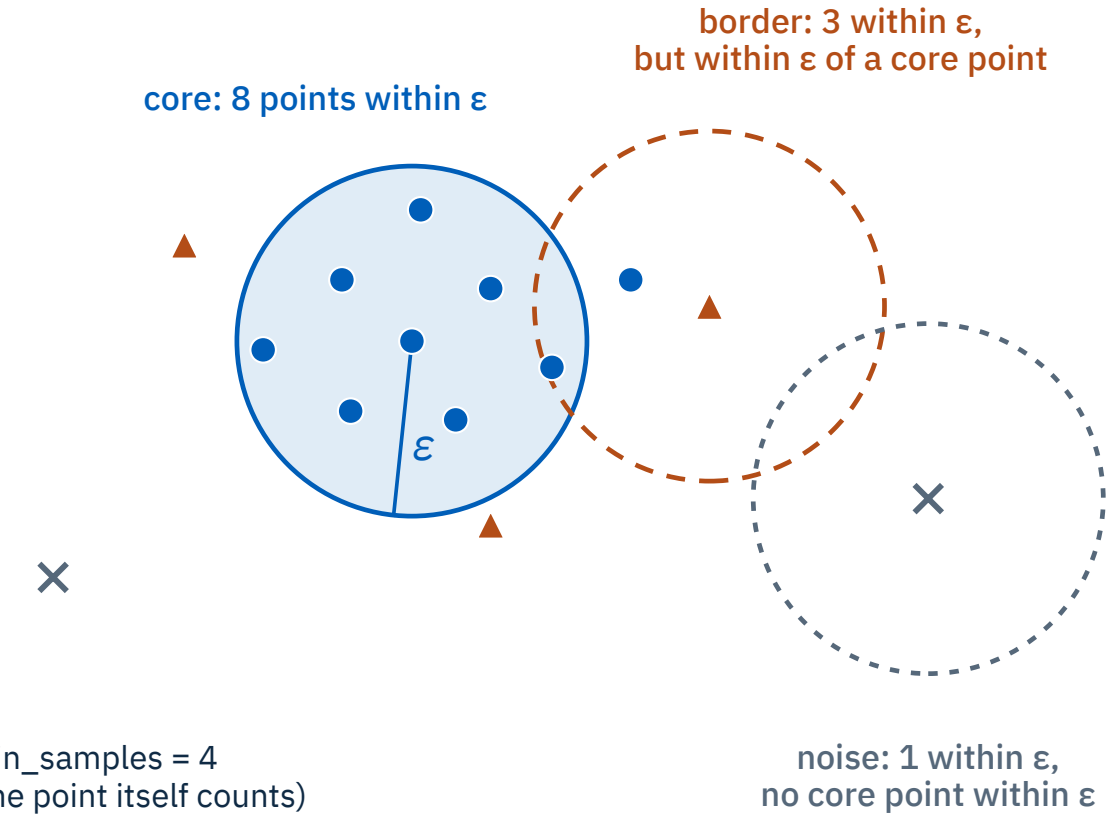
Density-based clustering

Clusters are dense regions separated by sparse ones.

- Core, border and noise points
- Choosing eps with a k-distance plot
- HDBSCAN for varying densities
- OPTICS reachability plots

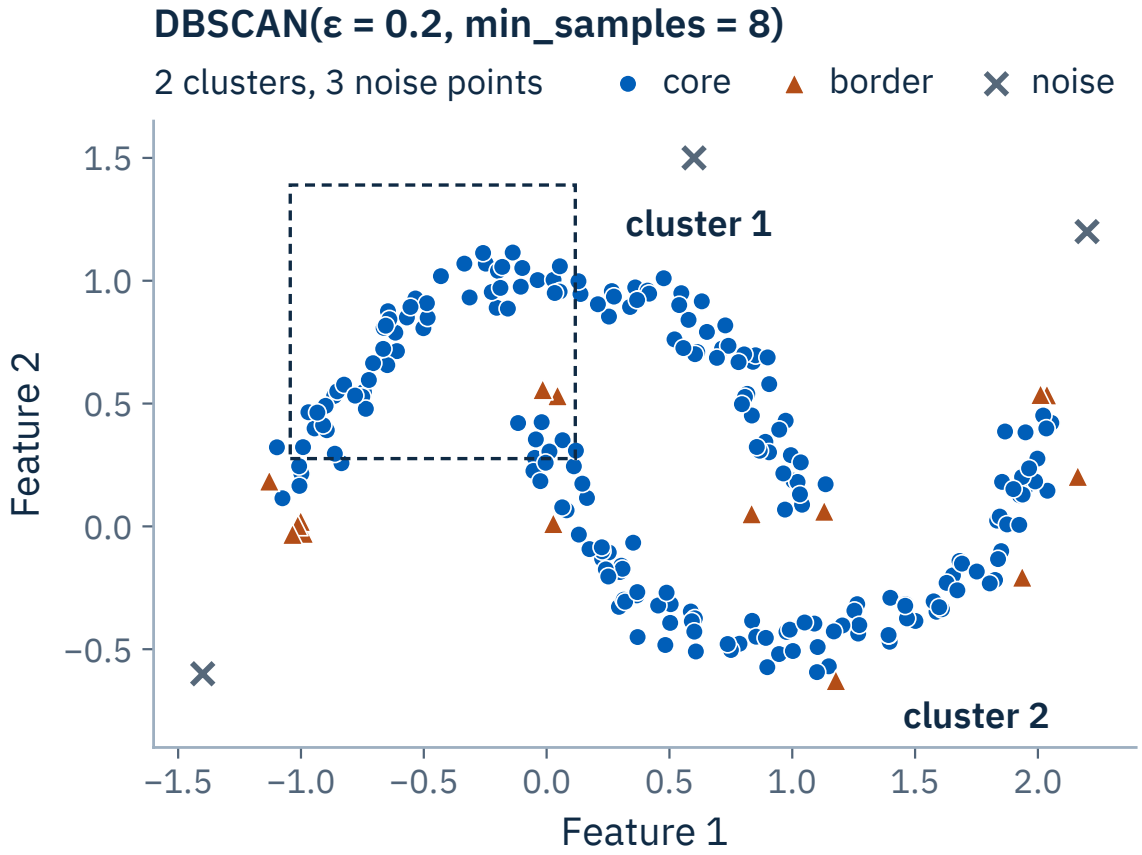
DBSCAN labels points as core, border or noise

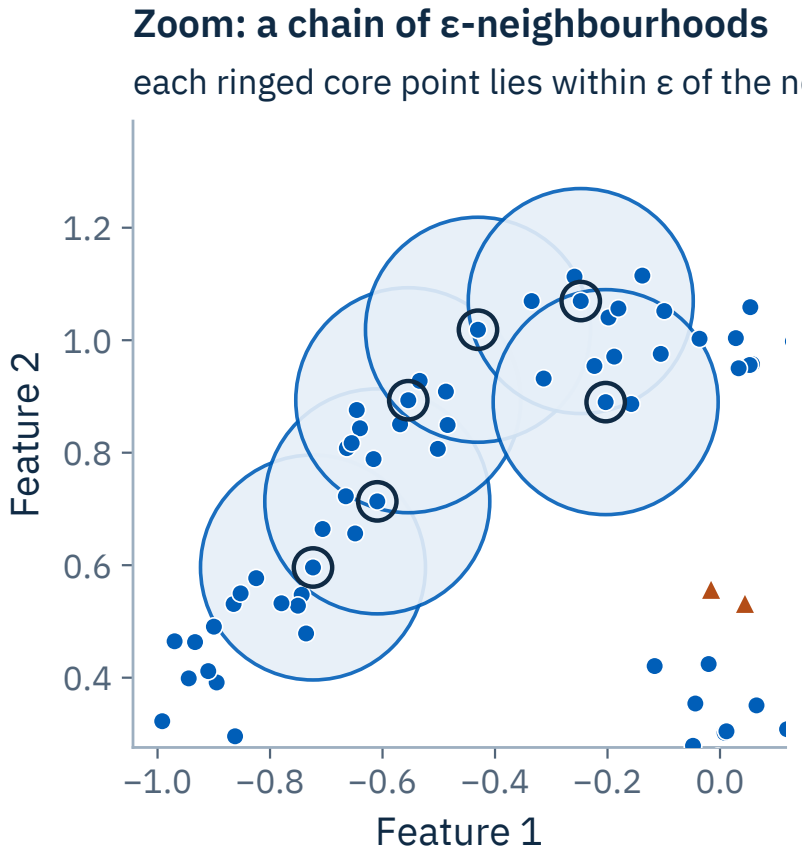
- **Core**: at least min_samples points within ε, counting itself
- Border: not core, but within ε of a core point
- Noise: neither; label −1
- Clusters are core points linked through overlapping ε-neighbourhoods



Hand-placed points; types computed with DBSCAN(eps = 1.0, min_samples = 4).

Clusters grow by chaining core neighbourhoods





Synthetic two moons (random_state = 42) plus three outliers; DBSCAN(eps = 0.2, min_samples = 8). Right: ε-discs along a chain of core points.

eps and min_samples define a density scale

eps

- Neighbourhood radius in the scaled feature space
- Too small: most points become noise
- Too large: distinct groups merge

min_samples

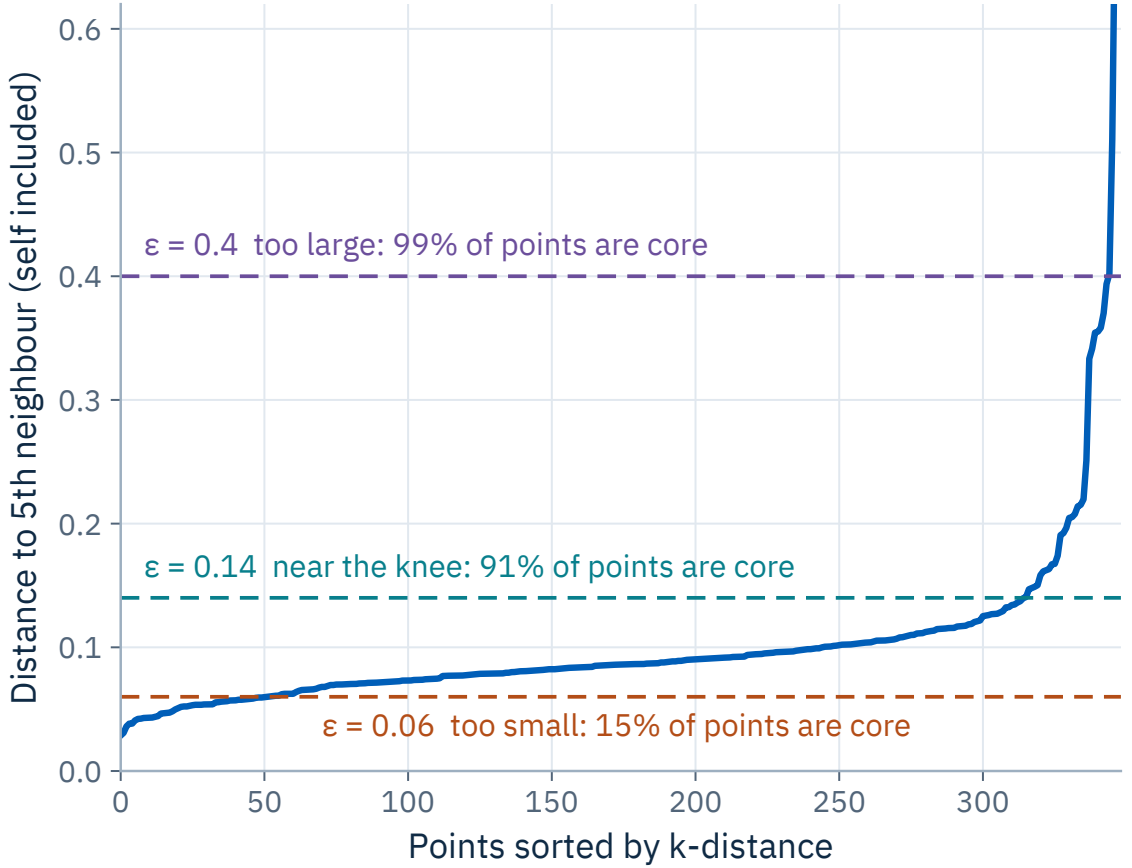
- Points required within eps, including itself
- Larger values: smoother clusters, more noise
- Heuristic start: about 2 × number of features

Together

- Set a minimum density for a core point
- Re-tune after any change of features or scaling
- Report the noise fraction with every result

A k-distance plot suggests a range for eps

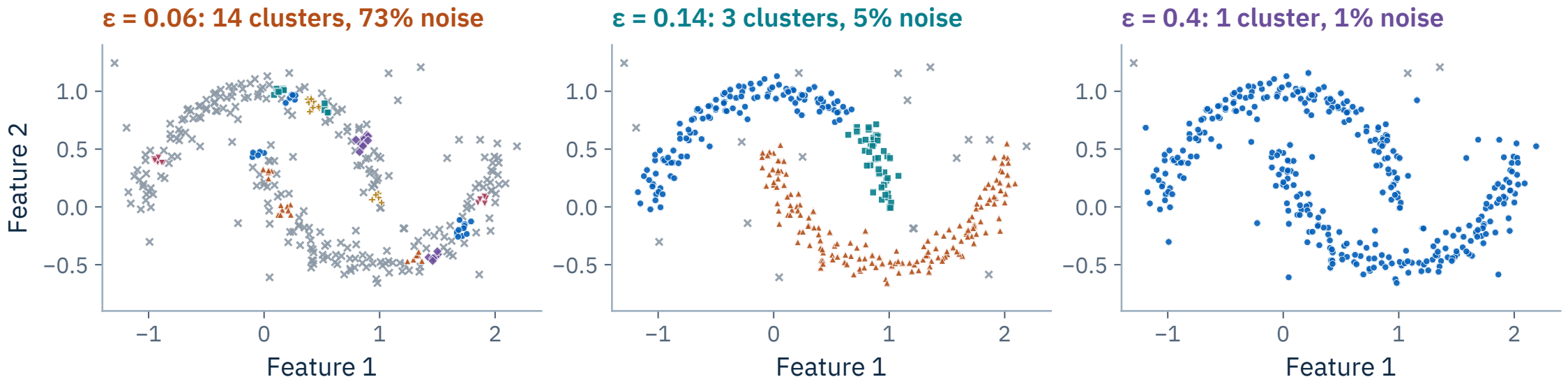
- For each point, find the distance to its min_samples-th neighbour
- Sort and plot these k-distances
- Points below the chosen eps are core points
- Pick eps near the knee, then check a range around it



Synthetic moons (random_state = 3) plus 28 uniform noise points; distances to the 5th neighbour, counting the point itself.

Small changes in eps change the answer

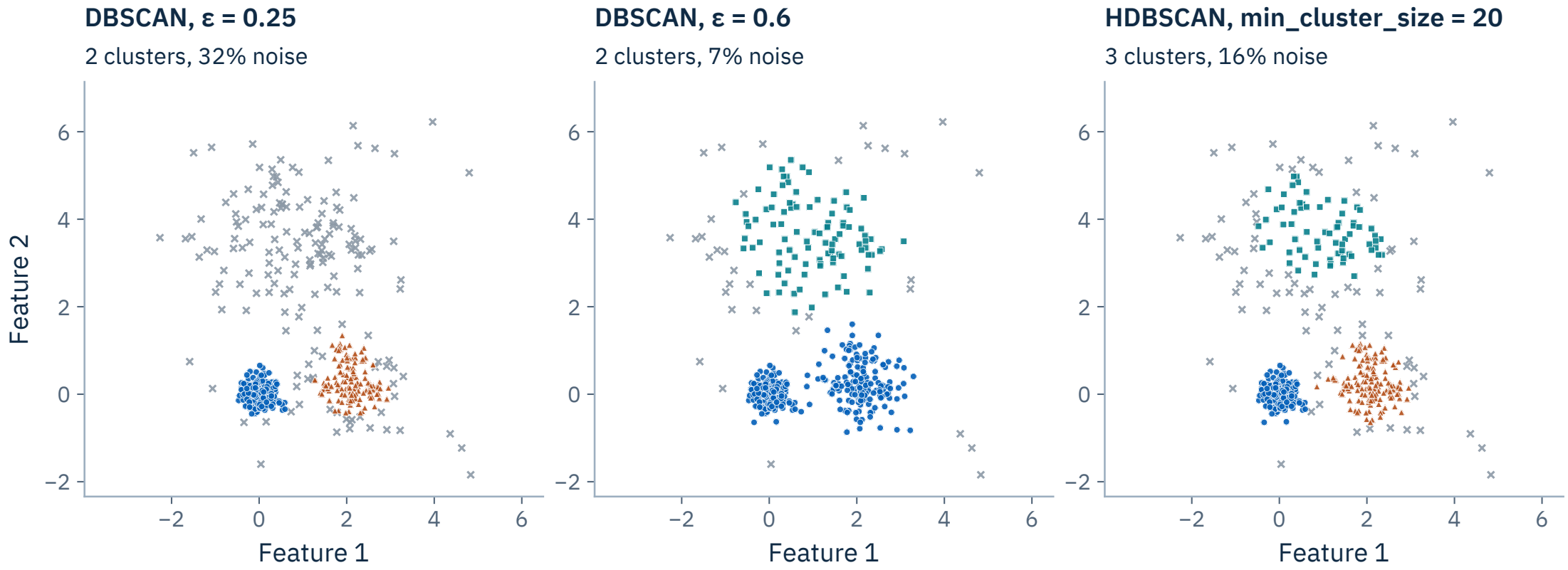
The knee value recovers both moons; either side of it DBSCAN fragments or merges them.



Same synthetic data as the k-distance plot; DBSCAN(min_samples = 5) at three eps values. Grey crosses are noise.

One eps cannot fit clusters of different density

Small eps loses the sparse group; large eps merges the dense ones. HDBSCAN adapts.



Synthetic dense, medium and sparse groups plus uniform background (seed 14); DBSCAN(min_samples = 10) and HDBSCAN(min_cluster_size = 20).

HDBSCAN keeps the clusters that persist across densities

HDBSCAN runs DBSCAN at every density level at once and keeps the clusters that persist.

1

Core distance

distance to the min_samples-th neighbour

2

Mutual reachability

max of both core distances and their distance

3

Spanning tree

connects all points at lowest total distance

4

Hierarchy

cut tree edges from longest to shortest

5

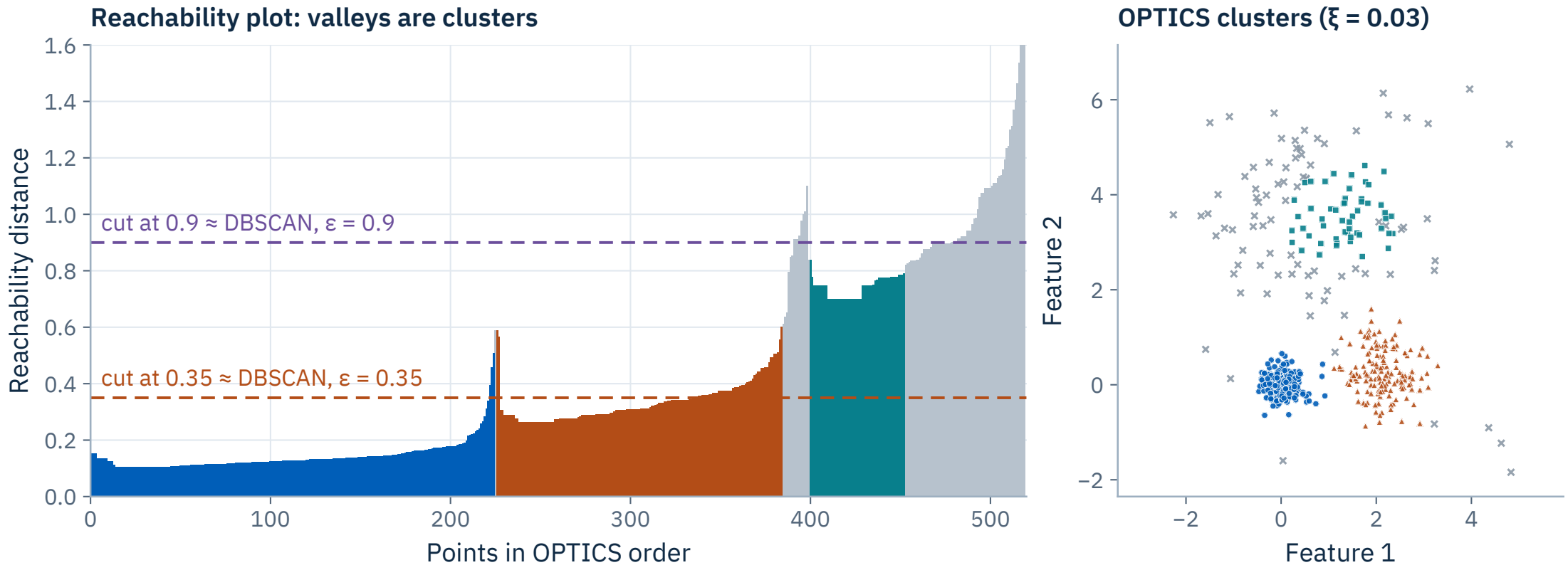
Stable clusters

keep clusters that persist; smaller than min_cluster_size is noise

◆ HDBSCAN needs no eps; min_cluster_size is the main setting.

OPTICS orders points by reachability

Valleys are dense regions; a cut at height ε reproduces DBSCAN with that ε.



Same synthetic data as the HDBSCAN comparison; OPTICS(min_samples = 30, xi = 0.03). Bars are coloured by extracted cluster; grey = noise.

HDBSCAN and OPTICS adapt where DBSCAN cannot

Method	Main settings	Varying density	New samples
DBSCAN	eps, min_samples	Poor: one global eps	No predict: refit
HDBSCAN	min_cluster_size	Good	No predict in scikit-learn
OPTICS	min_samples, xi	Good	No predict: refit
K-means (reference)	K	Poor	predict: nearest centroid

All three density methods label noise as −1 and need a scaled, meaningful distance.

Tune eps from k-distances, then report the noise

```
from sklearn.neighbors import NearestNeighbors
from sklearn.cluster import DBSCAN

ms = 5                                          # counts the point itself
nn = NearestNeighbors(n_neighbors=ms).fit(Z)    # Z: scaled features
kdist = np.sort(nn.kneighbors(Z)[0][:, -1])     # plot; look for a knee

db = DBSCAN(eps=0.5, min_samples=ms).fit(Z)
labels = db.labels_                             # -1 marks noise
n_clusters = len(set(labels) - {-1})
noise_frac = np.mean(labels == -1)
print(n_clusters, round(noise_frac, 3))
```

Report the number of clusters, the noise fraction and the settings together.

PRACTICE

5 min · Pairs

Density clustering diagnosis

DBSCAN assigns 85% of observations to noise.

1. List two possible explanations other than bad data.
2. What would you plot or inspect before changing eps?
3. What happens if you simply raise eps until noise is below 5%?

LABORATORY

45 min

Lab 2: density clustering on log data

- Standardize SP and GR from log.csv; run DBSCAN with eps = 0.5, min_samples = 5.
- Draw the k-distance plot and mark the default eps on it.
- Sweep eps and min_samples; tabulate clusters, noise and largest cluster.
- Repeat on the six W5 logs and inspect what the noise points have in common.

NOTEBOOK

`exercises/02_dbscan/DBSCAN_Clustering_Student.ipynb`

EVIDENCE TO SUBMIT

- A k-distance plot with your chosen eps
- A sensitivity table for eps and min_samples
- Two sentences on what the noise represents

DISCUSSION

Lab 2 debrief: noise flags rare responses, not errors

eps (standardized)	log.csv: clusters, noise	W5: clusters, noise	W5 largest cluster
0.3	5, 57 of 110	76, 30%	1,699
0.5	1, 8 of 110	15, 6.6%	6,342
0.8	1, 4 of 110	3, 1.2%	6,969
1.0	1, 1 of 110	3, 0.3%	7,037

With min_samples = 5, both datasets are essentially one dense region plus sparse outliers.

PART 4 OF 6

04

Hierarchy and zonation

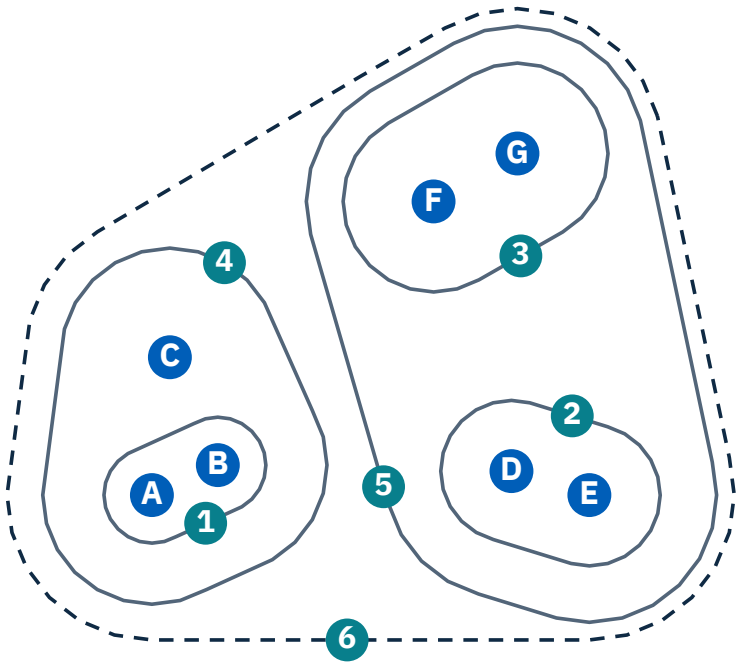
Nested merges give every level of detail at once.

- Agglomerative clustering and linkage
- Reading and cutting dendrograms
- Depth-constrained zonation of a well
- Cost and scaling limits

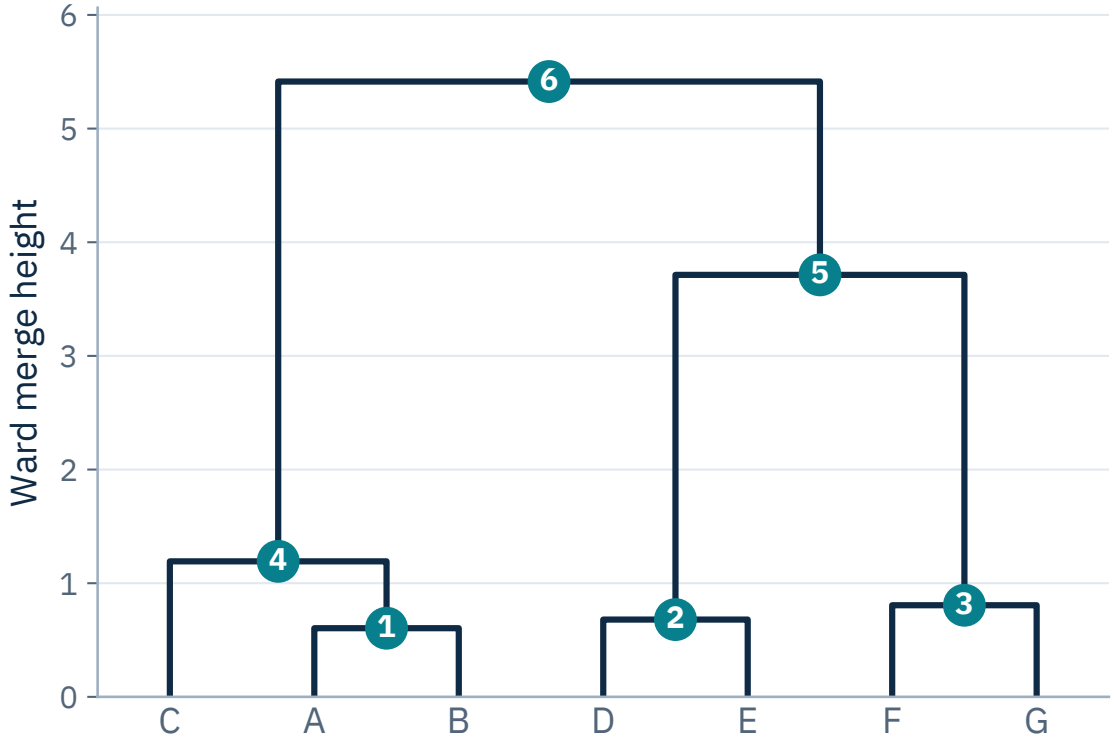
Agglomerative clustering builds a merge history

Start with singletons, merge the closest pair of clusters, and record each height.

Nested merges (numbers = merge order)



Dendrogram of the same merges



Seven hand-placed points; scipy Ward linkage. Merge numbers follow the rows of the linkage matrix.

Linkage changes the meaning of close

Linkage	Distance between groups	Common behaviour
Single	Closest pair of observations	Can chain through bridges
Complete	Farthest pair	Favours compact groups
Average	Mean of all pairwise distances	Intermediate behaviour
Ward	Increase in within-cluster squared error	Compact, similar-size groups

Ward is defined for Euclidean distance; the other three accept any distance.

Four linkages measure the same two groups differently

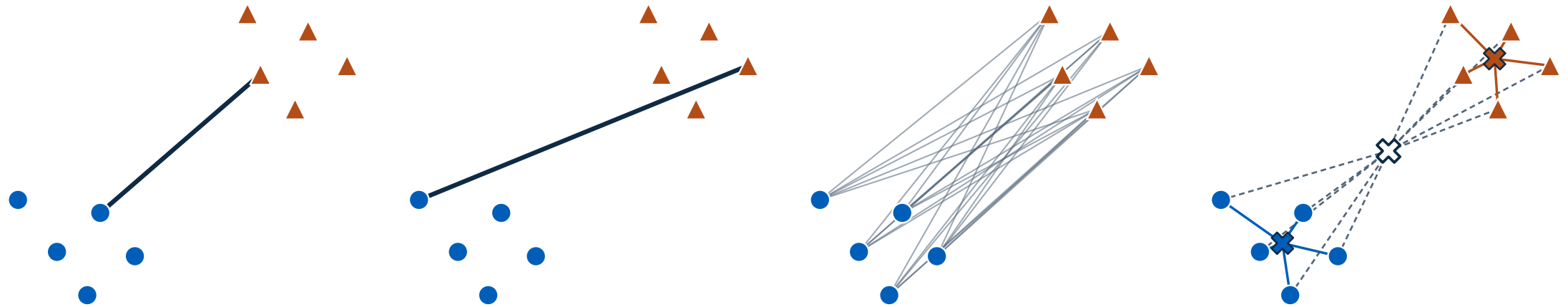
Same two groups, four definitions of the distance between them.

Single: closest pair
d = 2.45

Complete: farthest pair
d = 4.10

Average: mean of all 25 pairs
d = 3.31

Ward: growth in within-SS
ΔSS = 26.35



Two hand-placed five-point groups. Ward: increase in within-cluster sum of squares if the groups merge (dashed = distances to the merged centroid).

PRACTICE

5 min · Individual

Merge heights by hand

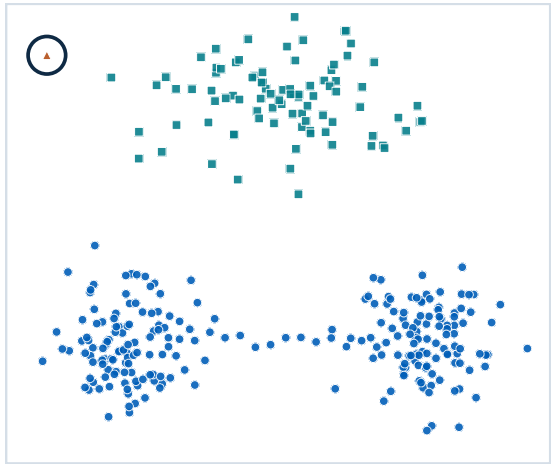
Five observations on a line: 0, 1, 3, 7, 8.

1. List the merges and their heights with single linkage.
2. Repeat with complete linkage.
3. Where would you cut each tree to get two clusters?

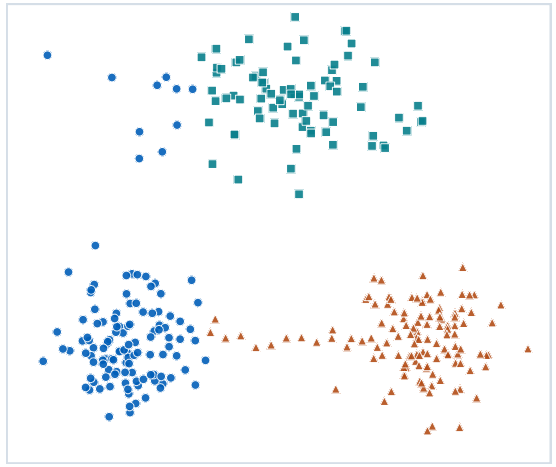
Single linkage chains through bridges

Single linkage merges two blobs through a 14-point bridge and isolates one point.

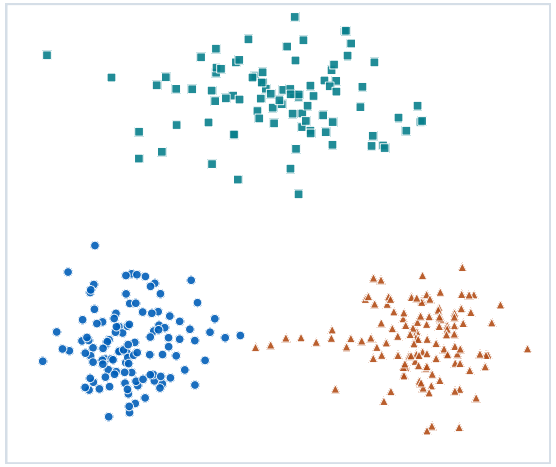
Single
sizes 234 · 89 · 1 (ringed)



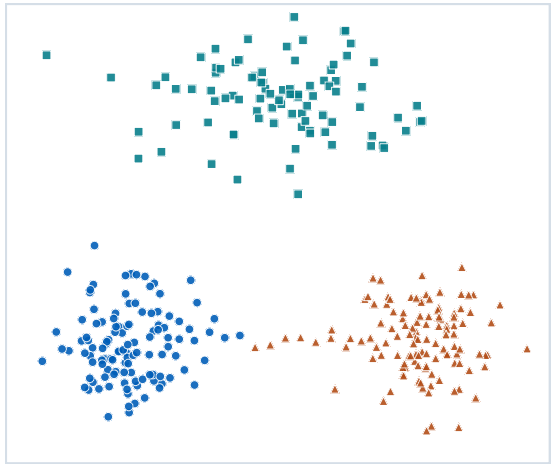
Complete
sizes 123 · 121 · 80



Average
sizes 119 · 115 · 90

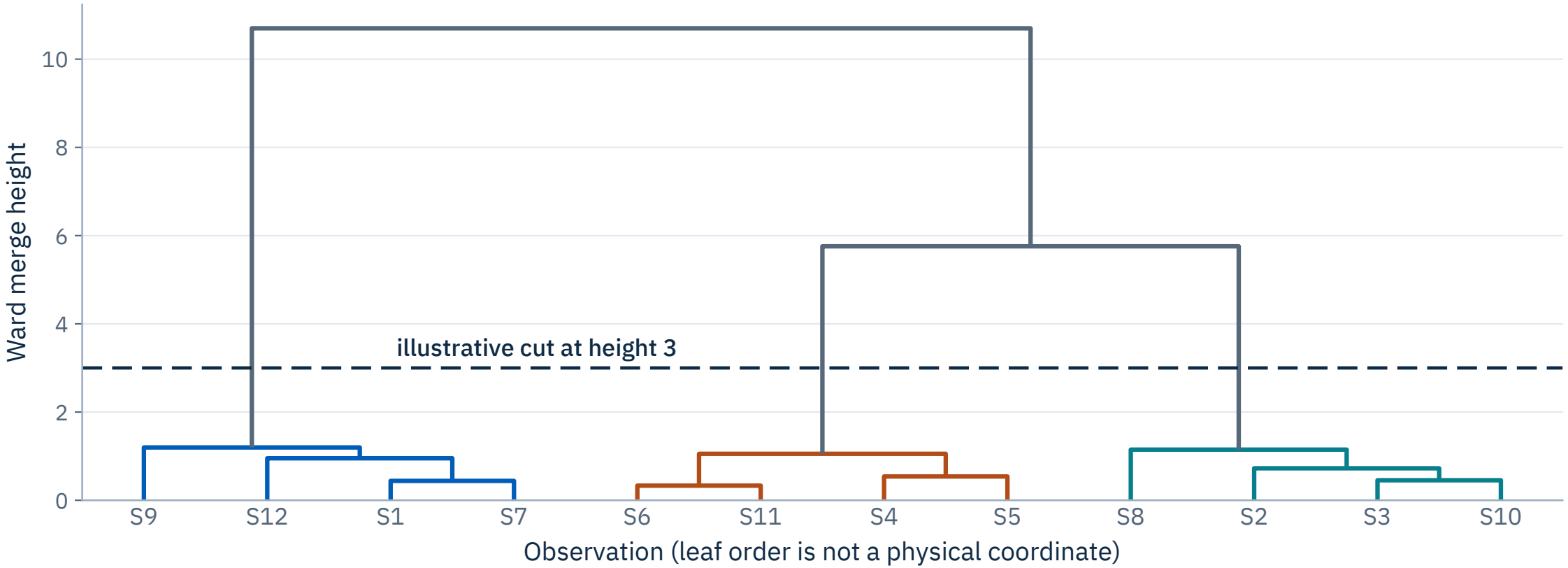


Ward
sizes 119 · 115 · 90



Synthetic: two dense blobs joined by a thin bridge, plus an elongated third group (seed 5); AgglomerativeClustering(n_clusters = 3).

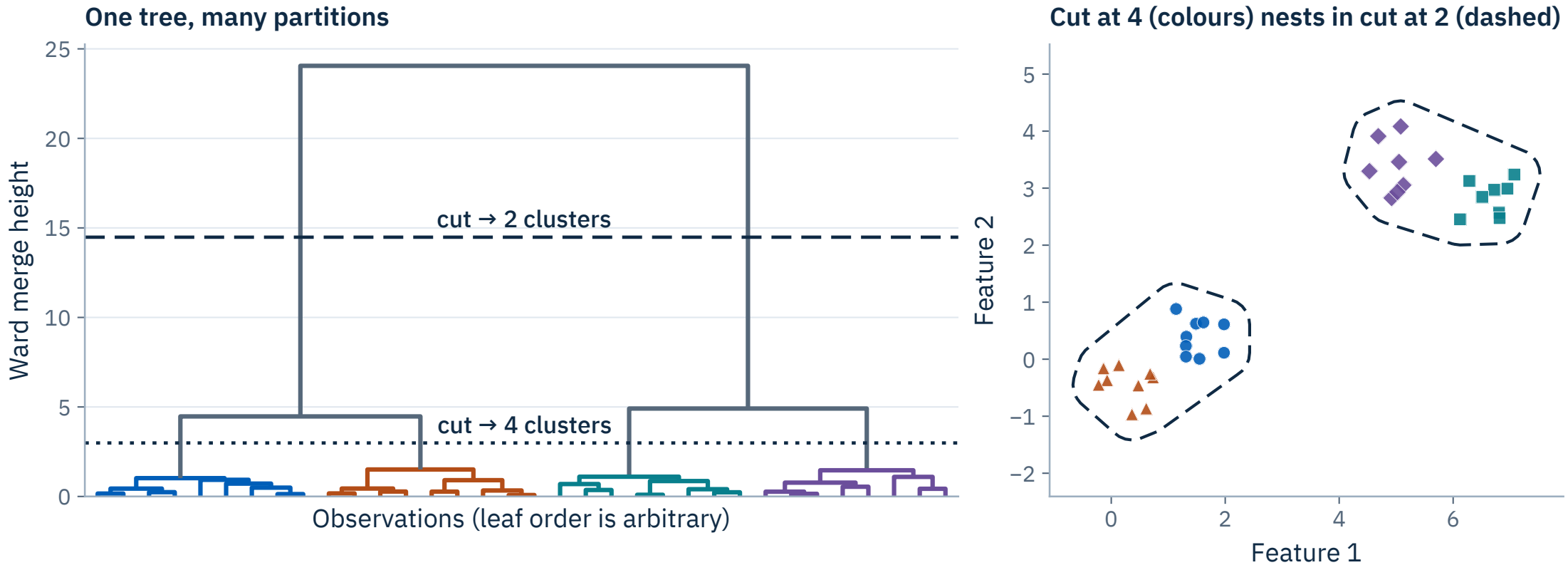
A dendrogram records merge heights



Ward hierarchy of twelve synthetic observations (random_state = 42). Colours mark the three clusters below the dashed cut at height 3.

Cutting the tree at a height gives a partition

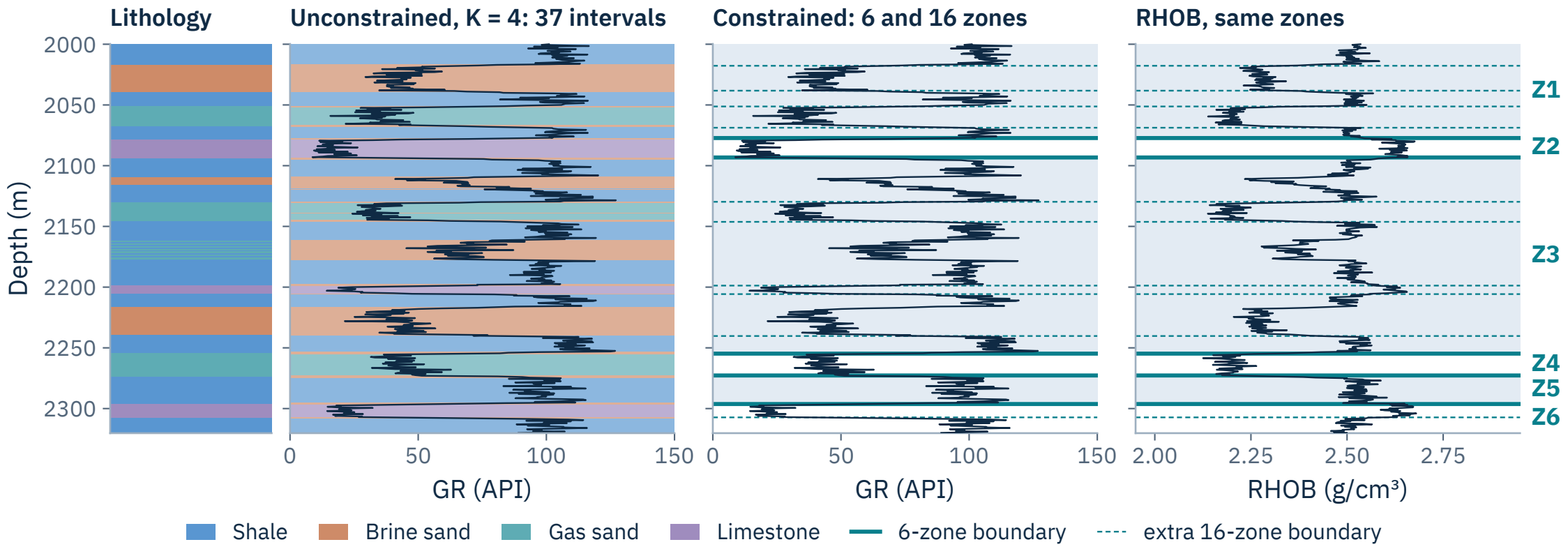
Cuts at different heights give nested partitions of the same tree.



Synthetic 34 points from four blobs (random_state = 1); scipy Ward linkage; cuts halfway between merge heights; fcluster(criterion = "maxclust").

Depth constraints turn clustering into zonation

Merging only adjacent samples gives contiguous zones that nest across scales.



Synthetic well S-1 (seed 7): Ward on four standardized logs, unconstrained (K = 4) and depth-constrained.

Zonation needs only a connectivity matrix

```
from scipy.sparse import diags
from sklearn.cluster import AgglomerativeClustering
from sklearn.preprocessing import StandardScaler

logs = well.sort_values("DEPTH")
Z = StandardScaler().fit_transform(logs[["GR", "RHOB", "NPHI", "LOG_RT"]])
n = len(Z)
A = diags([np.ones(n - 1), np.ones(n - 1)], [-1, 1])  # adjacent samples
zones = AgglomerativeClustering(
    n_clusters=6, linkage="ward", connectivity=A
).fit_predict(Z)
```

Sort by depth first; remove links across logging gaps or well boundaries.

Unconstrained hierarchy grows with n²

Standard hierarchical clustering needs all pairwise distances between samples.

25.0 M

pairwise distances for the 7,071 W5 samples

200 MB

to store them as 8-byte numbers

40 GB

for 100,000 samples

- Subsample, cluster summaries (for example many K-means centroids), or add connectivity before running a full hierarchy.

PRACTICE

4 min · Pairs

Which linkage for which task?

Choose a linkage or constraint for each task and justify it.

1. Separate two dense sand bodies joined by a thin streak of transitional samples.
2. Follow a long, narrow channel feature traced in a seismic attribute map.
3. Build contiguous layers from logs for a reservoir model.

PART 5 OF 6

05

Gaussian mixture models

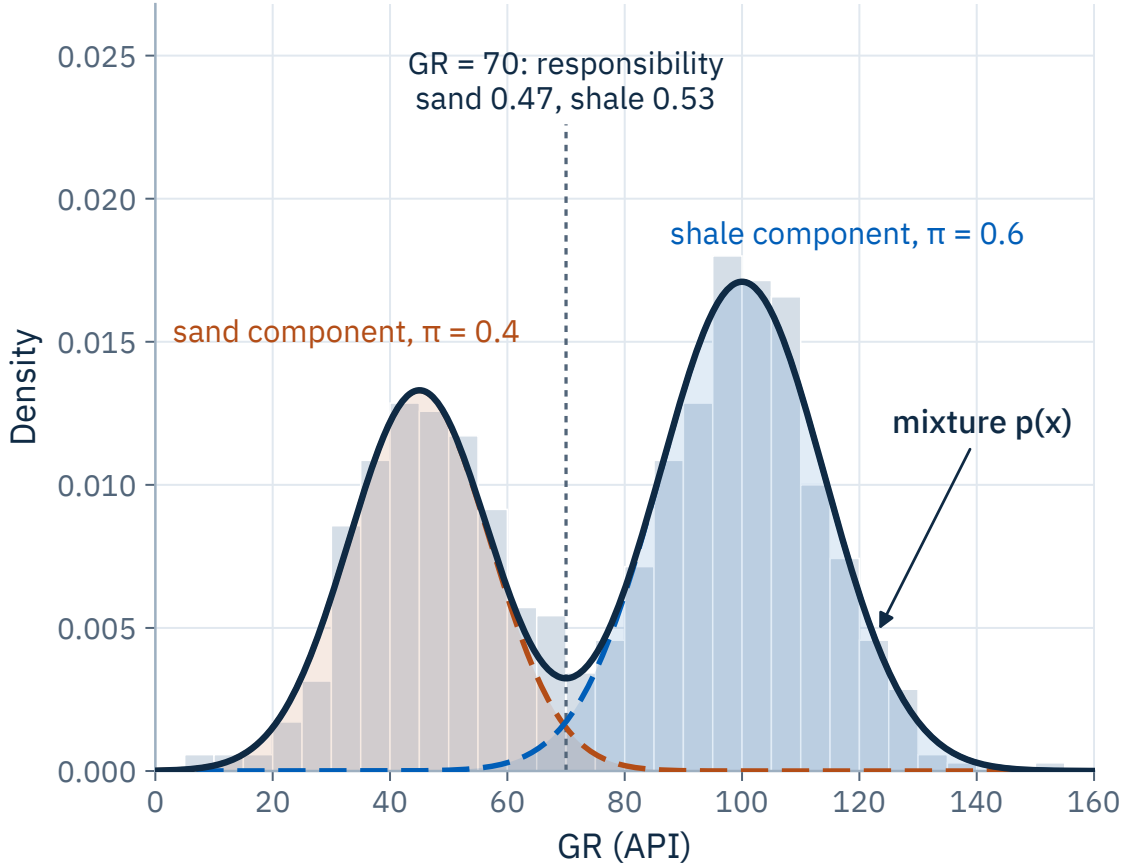
Soft membership allows uncertainty near overlapping groups.

- The mixture density and responsibilities
- Expectation-maximization
- Covariance types and BIC
- Uncertainty maps

The mixture density adds weighted Gaussians

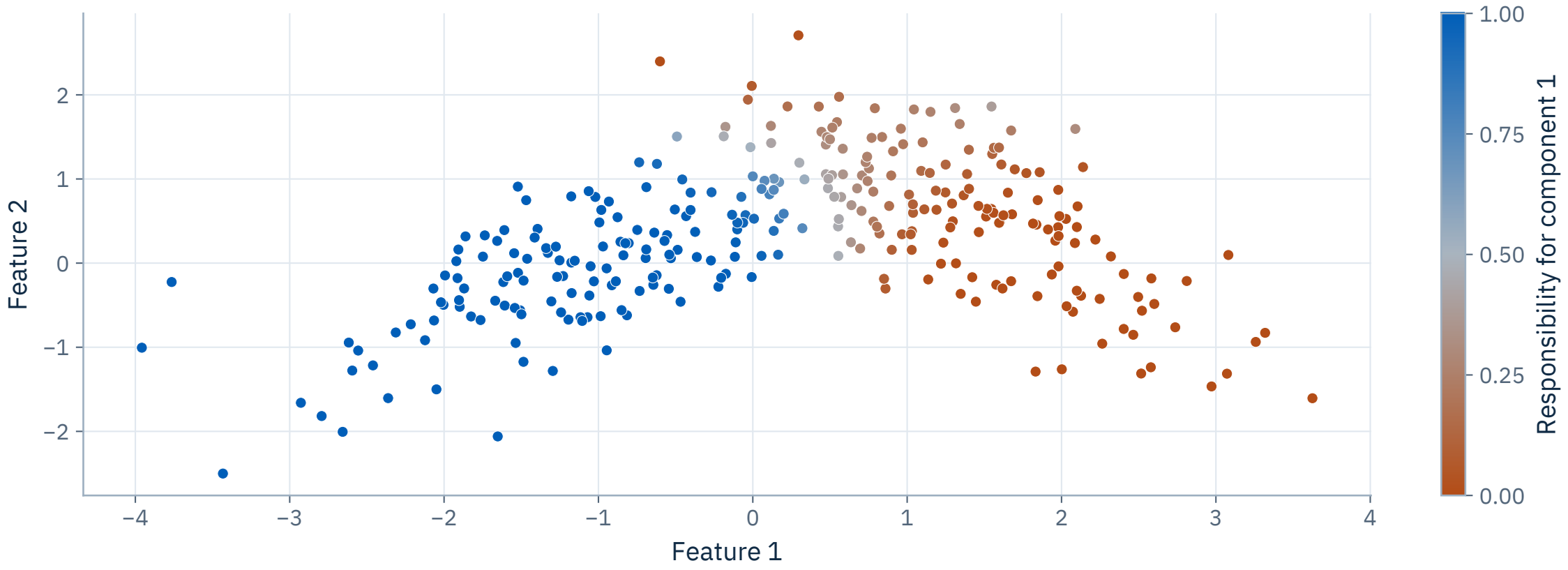
$$p(x) = \sum_{k=1}^{K} \pi_k \mathcal{N}(x \mid \mu_k, \Sigma_k)$$

- π_k: mixing weights, non-negative, summing to one
- μ_k and Σ_k: mean and covariance of component k
- **Responsibility** r_{ik}: component k's share of the density at x_i



Exact two-component normal mixture (weights 0.4 and 0.6; means 45 and 100 API; SD 12 and 14 API) with a synthetic histogram (seed 9).

Soft assignments expose overlap



Synthetic overlapping groups (seed 42); two-component Gaussian mixture. Colour: responsibility for component 1; grey points are near 0.5, the ambiguous ones.

Expectation-maximization alternates two steps

Step	Quantity updated	Intuition
E step	Responsibilities	How much does each component explain each observation?
M step	Weights, means, covariances	Refit components using fractional memberships

Repeat until the likelihood improvement is small, with safeguards such as reg_covar.

EM moves the components until the likelihood levels off

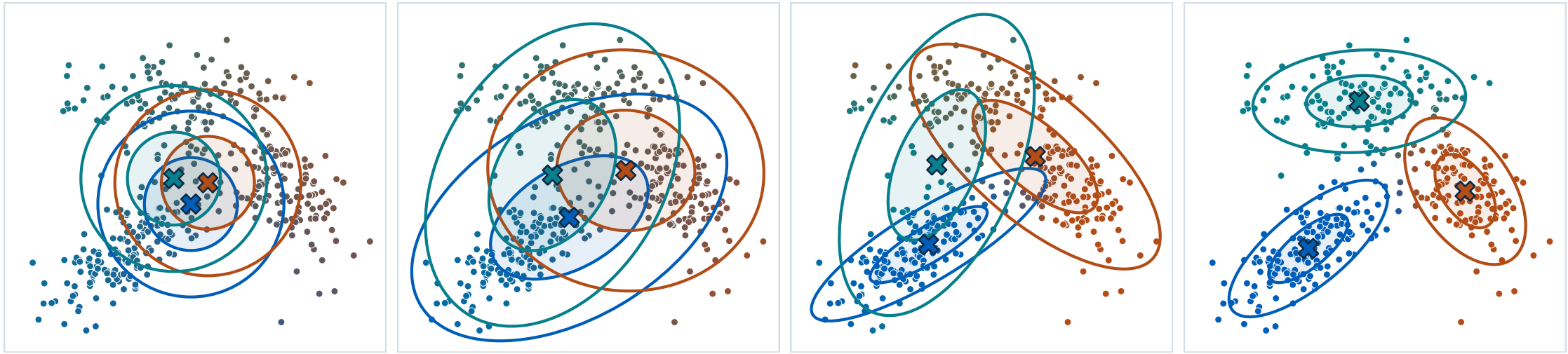
E step: recompute responsibilities. M step: refit weights, means and covariances.

Start
mean log-likelihood −4.433

Iteration 1
mean log-likelihood −3.829

Iteration 5
mean log-likelihood −3.503

Iteration 40
mean log-likelihood −3.311



Synthetic three-component data (seed 31); EM for a full-covariance mixture written in numpy from a deliberately poor start. Point colour blends component colours by responsibility.

PRACTICE

4 min · Individual

A weighted mean update

$$\mu_k^{\text{new}} = \frac{\sum_i r_{ik}\, x_i}{\sum_i r_{ik}}$$

Two observations, x = 0 and x = 10, have responsibilities 0.8 and 0.2 for component 1.

1. Compute the new mean of component 1.
2. Compute the new mean of component 2, whose responsibilities are 0.2 and 0.8.
3. What are the new mixing weights, and the new variance of component 1?

Covariance structure controls flexibility

Type	Assumption	Covariance parameters	Trade-off
Spherical	One variance per component	K	Few parameters, round groups
Diagonal	No within-component correlation	K·d	Axis-aligned ellipses
Tied	One shared full covariance	d(d+1)/2	Same shape and tilt for all
Full	Own full covariance each	K·d(d+1)/2	Flexible, needs more data

Add K·d mean parameters and K – 1 weights to get the total that BIC penalizes.

Covariance type sets the shape of each component

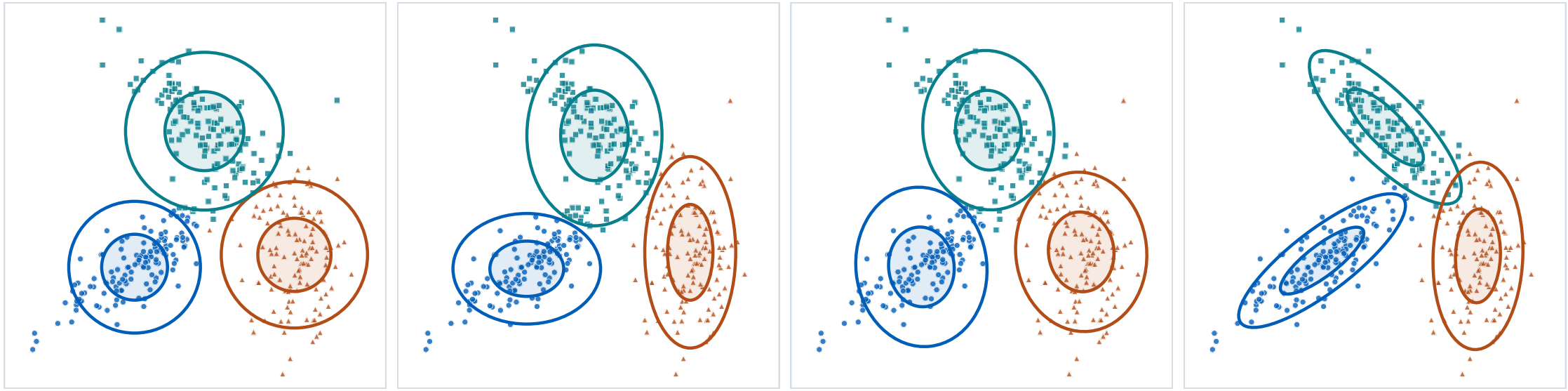
Full covariance costs 3 more parameters than diagonal, yet BIC prefers it clearly.

spherical: 11 parameters
BIC 3086

diag: 14 parameters
BIC 3039

tied: 11 parameters
BIC 3083

full: 17 parameters
BIC 2825

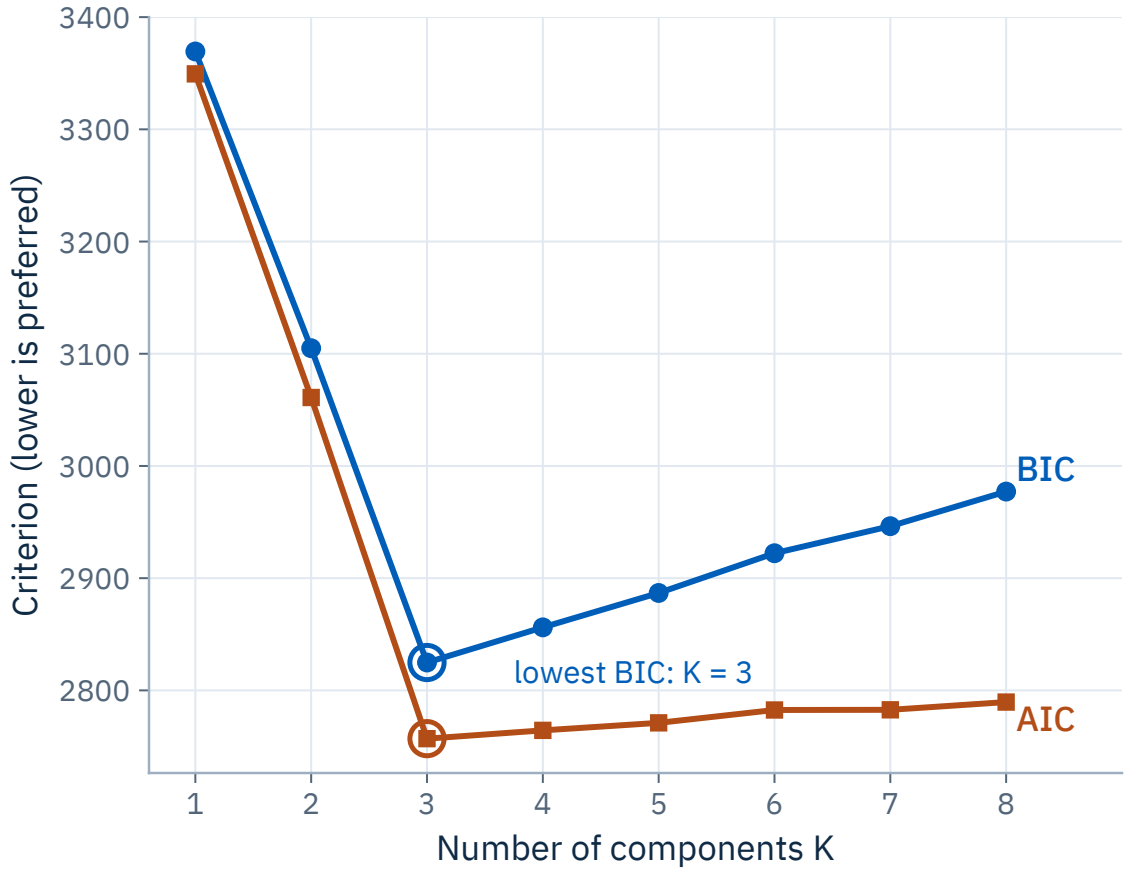


Synthetic three-component data (seed 17); GaussianMixture(K = 3, n_init = 5, random_state = 0) with each covariance_type; ellipses at 1 and 2 standard deviations.

BIC and AIC penalize model complexity

$$\mathrm{BIC} = -2 \log \hat{L} + p \log n$$

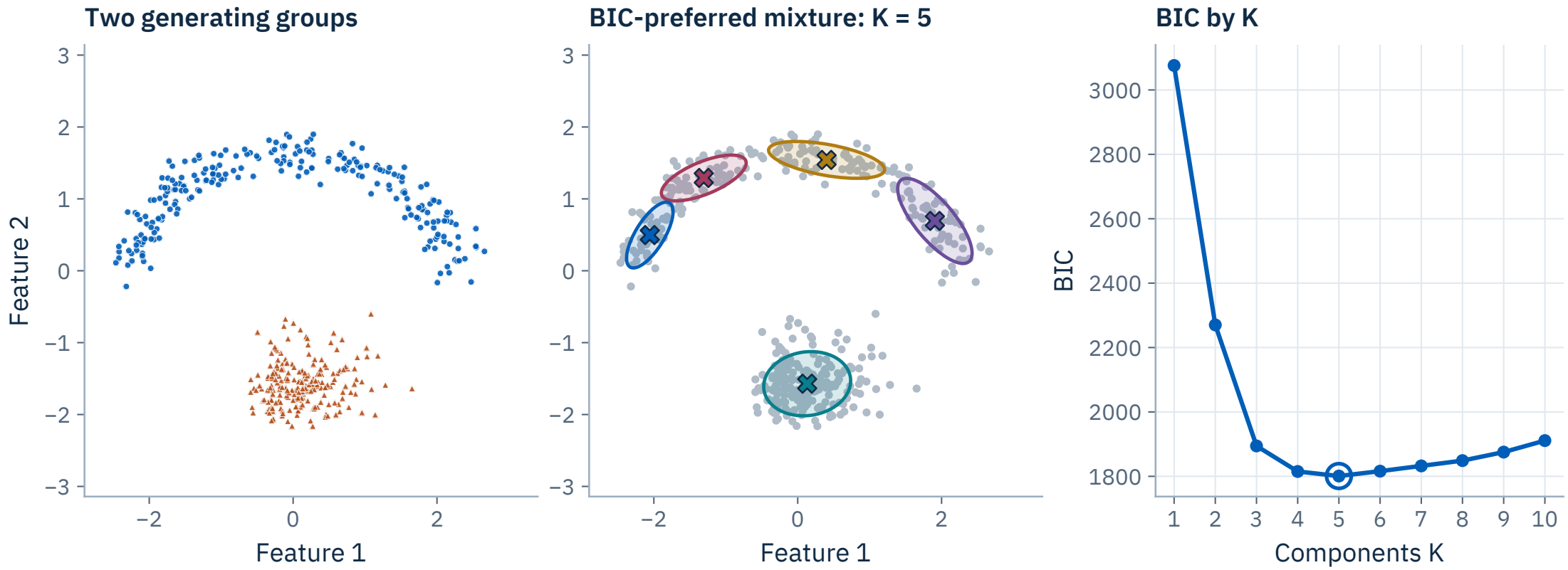
- $\hat{L}$: maximized likelihood; p: free parameters; n: observations
- Lower is preferred; compare models fitted to the same data and features
- AIC replaces $p \log n$ with $2p$, a lighter penalty
- Check stability and plausibility alongside the criterion



Same synthetic data as the covariance figure (three generating components); full-covariance mixtures, K = 1 to 8.

Mixture components are not the same as groups

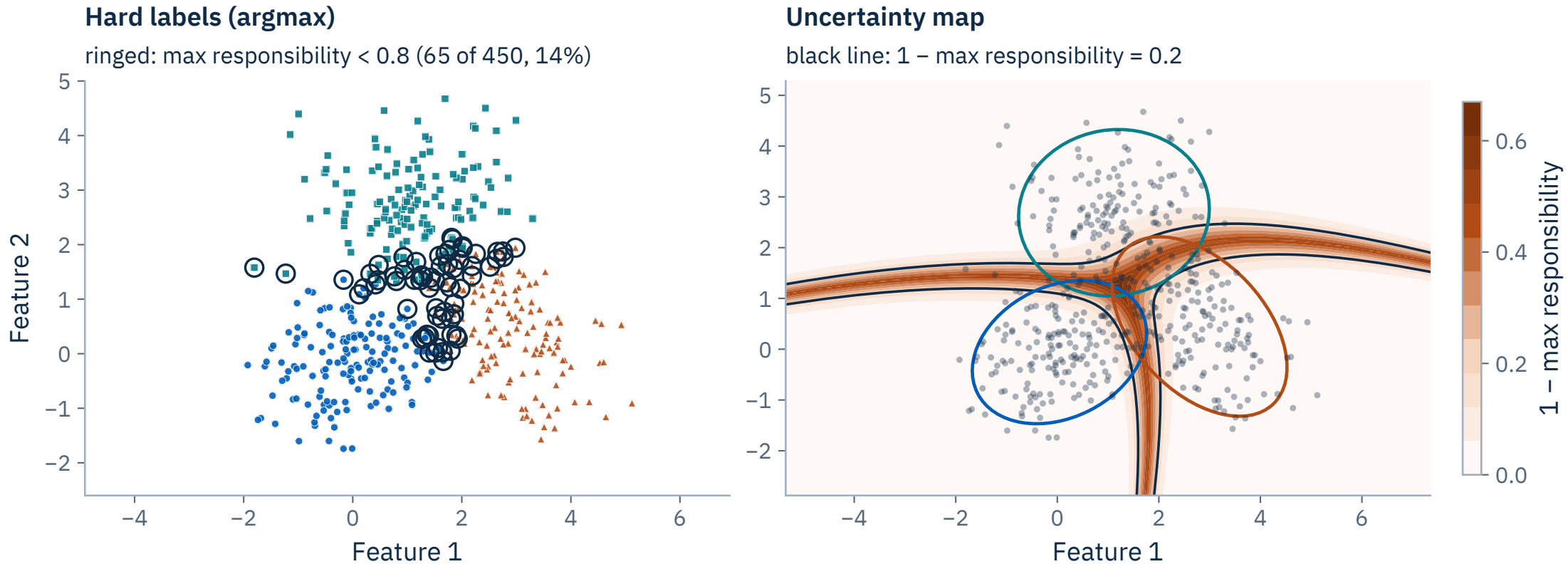
Two generating groups, but BIC prefers five components: Gaussians tile the curved group.



Synthetic: a curved group and a log-normal (skewed) group, seed 23; full-covariance mixtures, n_init = 5; ellipses at 1.5 standard deviations.

Uncertainty maps show where labels are fragile

Responsibilities stay low along boundaries, where a hard label hides ambiguity.



Synthetic overlapping groups (seed 41); GaussianMixture(K = 3, full, n_init = 5, random_state = 0); 2-SD ellipses of the fitted components.

A reproducible mixture fit

```
from sklearn.mixture import GaussianMixture
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler().fit(X_train)
Z_train = scaler.transform(X_train)
gmm = GaussianMixture(
    n_components=3, covariance_type="full",
    n_init=5, reg_covar=1e-6, random_state=42
).fit(Z_train)
membership = gmm.predict_proba(scaler.transform(X_new))
```

PRACTICE

5 min · Pairs

Count the parameters before you fit

Lab 3 fits mixtures to d = 10 standardized oxides on n = 1,684 samples.

1. How many free parameters does a full-covariance mixture with K = 8 have?
2. How many does a diagonal-covariance mixture with K = 8 have?
3. By how much must the full model raise the log-likelihood to beat diagonal on BIC?

PART 6 OF 6

06

Validation and interpretation

A grouping earns trust through several independent kinds of evidence.

- Internal indices and what they reward
- External indices when labels exist
- Stability under resampling
- Profiles, depth tracks and method choice

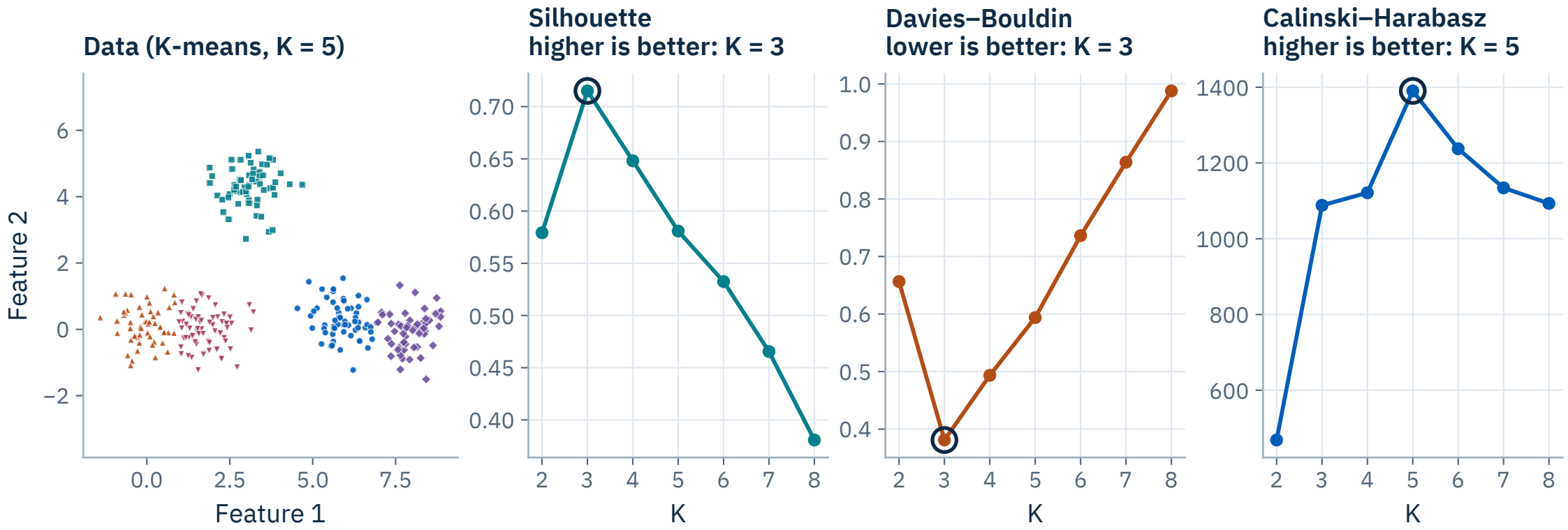
Internal indices reward different geometry

Index	Rewards	Better	Caveat
Silhouette	Points nearer their own cluster than the next	Higher, −1 to 1	Needs all pairwise distances
Davies–Bouldin	Small spread relative to centroid separation	Lower, 0 is best	Centroid-based: favours convex
Calinski–Harabasz	High between- to within-cluster variance	Higher	Favours convex, similar-size

All three use only the data and the labels, so they share the clustering's own assumptions.

Internal indices can disagree on K

Silhouette and Davies–Bouldin see three groups; Calinski–Harabasz resolves all five blobs.



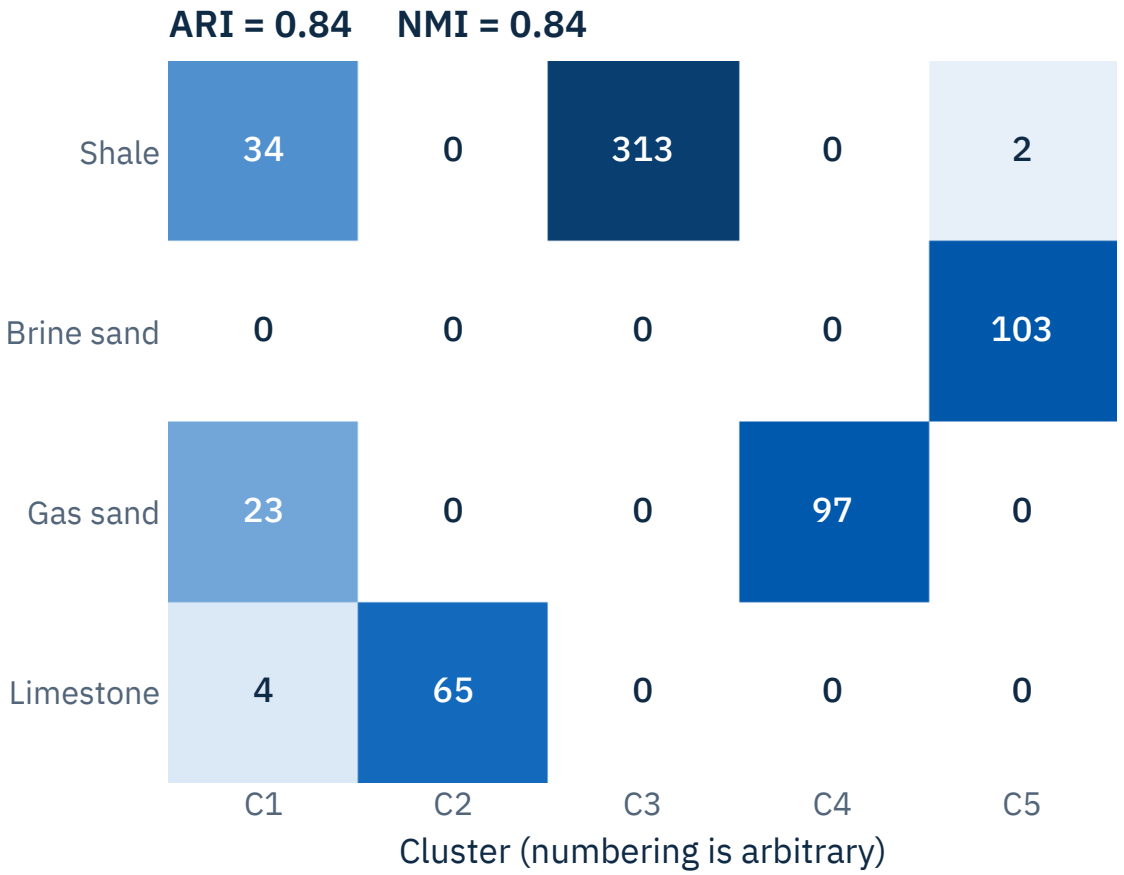
Synthetic five blobs with two close pairs (random_state = 0); K-means (n_init = 10) for K = 2 to 8; scikit-learn metric implementations.

Scores need context

Evidence	What it can establish	What it cannot establish alone
Silhouette	Separation under a selected distance	Geological validity
Adjusted Rand index	Agreement with an external labelling	Correctness of that labelling
Stability across fits	Robustness to perturbations	Usefulness for a decision
Expert review	Domain plausibility	Generalization to all new data

External indices compare clusters with reference labels

- **ARI**: pair agreement corrected for chance; 0 random, 1 identical
- NMI: shared information, 0 to 1; not chance-corrected (AMI is)
- Both ignore cluster numbering; neither says which cluster is wrong
- Use labels to evaluate, never as clustering features



Synthetic well S-1 (seed 7): reference lithology against K-means (K = 5) on standardized logs; counts of depth samples.

Stable clusters survive resampling

- Refit on bootstrap resamples; compare labels on all samples with ARI
- Keep K, features and scaling fixed
- A split that flips between runs is not a finding
- Resample by well or depth block when samples are correlated



Synthetic well S-1 (seed 7), standardized logs; K-means refitted on 30 bootstrap resamples per K; labels predicted for all samples and compared with the full-data fit.

Profile clusters in original units

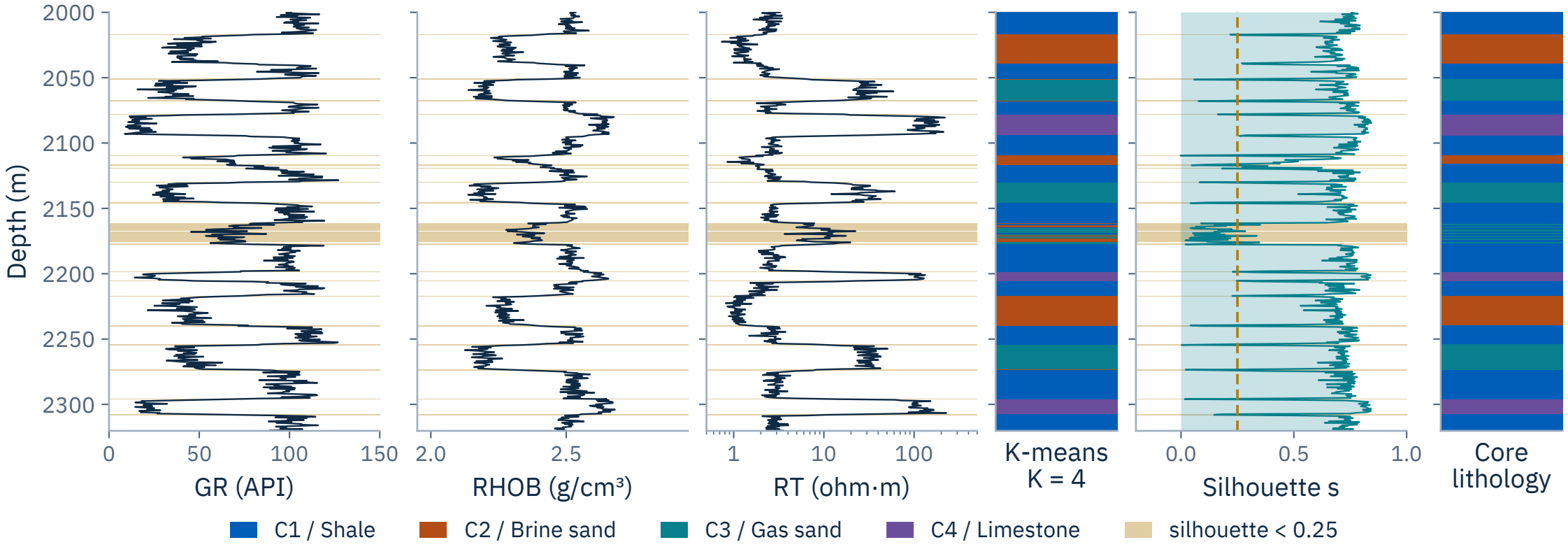
- Report cluster means or medians in physical units
- Add counts, spreads and depth ranges
- Write each reading as a hypothesis to test
- Check against core, cuttings or tests before naming

	GR (API)	RHOB (g/cm³)	NPHI (v/v)	RT (ohm·m)	hypothesis to test
C1 n=332	101	2.51	0.31	2.8	*high GR, low RT: shale-like*
C2 n=125	52	2.30	0.25	1.6	*low GR, low RT: wet-sand-like*
C3 n=115	43	2.22	0.17	25.5	*low GR, high RT, low NPHI: gas-sand-like*
C4 n=69	23	2.63	0.07	103.6	*high RHOB, low NPHI: tight-carbonate-like*

Synthetic well S-1: K-means (K = 4). Cell values are cluster means in original units (RT as geometric mean); colour shows the standardized mean.

Map clusters back onto depth

Low silhouette marks thin beds and boundaries, where one label per sample fails.



Synthetic well S-1 (seed 7): K-means (K = 4) on four standardized logs; per-sample silhouette; core lithology = simulation truth.

Other methods for other shapes and scales

Spectral clustering

- Clusters a similarity graph using its eigenvectors
- Handles non-convex shapes; memory grows with n^2

Mean shift

- Moves every point uphill to a density peak
- Finds K from a bandwidth; slow on large n

BIRCH

- One-pass tree of compact sub-cluster summaries
- Large or streaming data, then a final clustering

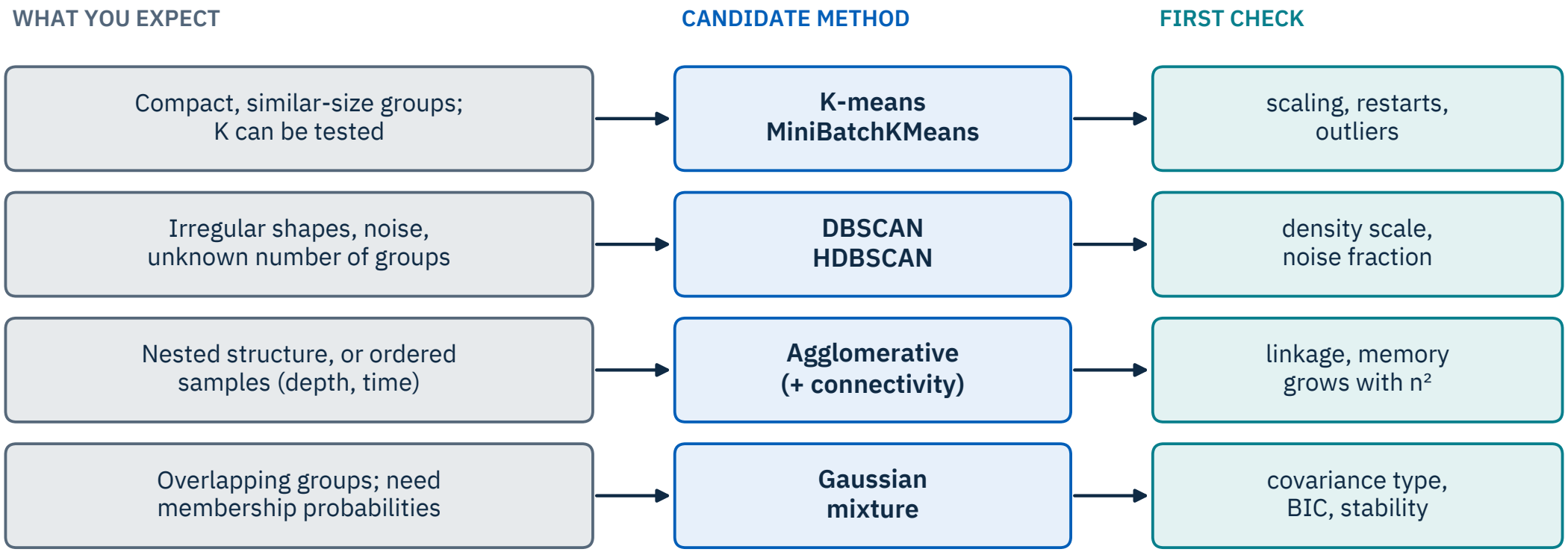
Time series with DTW

- Dynamic time warping aligns shifted profiles
- Pair with k-medoids or a hierarchy (e.g. tslearn)

◆ Choose by the shape, scale and data type you expect, as with every method today.

Select a method by the structure you expect

Start from the structure you expect, then check the method's main assumption first.



Schematic shortlist for a first attempt; not a rule.

PRACTICE

6 min · Groups

Weigh the evidence for a grouping

K-means on five logs from eight wells gives K = 5, mean silhouette 0.46. Bootstrap by well gives median ARI 0.62. Cluster 4 comes almost entirely from one well. ARI with core facies in two cored wells is 0.35.

1. Which evidence supports the grouping, and which undermines it?
2. What is the most likely explanation for cluster 4?
3. What would you do next?

LABORATORY

40 min

Lab 3: Gaussian mixtures on rock geochemistry

- [ ] Read the missing-value report (1,684 of 2,191 rows remain), then standardize.
- [ ] Fit a 3-component mixture; inspect predict and predict_proba.
- [ ] Compare BIC and AIC for K = 1–10 and refit at your chosen K.
- [ ] Crosstab components against ROCK1, compute ARI, and compare covariance types.
- [ ] Extension: the hierarchical (04) or method-comparison (05) notebook.

NOTEBOOK

`exercises/03_gmm/GMM_Rock_Decomposition_Student.ipynb`

EVIDENCE TO SUBMIT

- BIC and AIC curves with your chosen K
- A crosstab with ARI and silhouette
- One paragraph on what the components do and do not represent

DISCUSSION

Lab 3 debrief: BIC fits the density, not the labels

Covariance type, K = 8	BIC	ARI with rock type	Silhouette
Full	18,074	0.28	0.09
Tied	29,432	0.32	0.25
Diagonal	24,776	0.28	0.17
Spherical	31,000	0.30	0.19

The BIC-preferred model describes the oxide distribution best, yet agrees only weakly with the named rock types.

Key takeaways

- ✓ Similarity is a choice: features, transforms and scaling decide the clusters.
- ✓ K-means needs k-means++, several restarts and more than one kind of evidence for K.
- ✓ Density methods find shapes and noise; use a k-distance plot, or HDBSCAN.
- ✓ Hierarchies give nested partitions; connectivity turns them into zonation.
- ✓ Mixtures give soft membership; BIC counts components, not rock types.
- ✓ Validate with stability and external labels; interpret in original units and along depth.

PRACTICE

7 min · Individual

Exit check

1. Why can scaling change the scientific conclusion of a clustering?
2. Why can DBSCAN noise include valid observations?
3. What information disappears when a mixture becomes a hard partition?
4. Name two independent kinds of evidence you need before calling a cluster a facies.

References and further study

- scikit-learn User Guide: clustering, Gaussian mixture models, and clustering performance evaluation.
- K-means: Lloyd (1982), least-squares quantization; Arthur and Vassilvitskii (2007), k-means++.
- Density: Ester et al. (1996), DBSCAN; Ankerst et al. (1999), OPTICS; Campello et al. (2013), HDBSCAN.
- Hierarchy and mixtures: Ward (1963); Dempster, Laird and Rubin (1977), EM; Schwarz (1978), BIC.
- Validation: Rousseeuw (1987), silhouettes; Hubert and Arabie (1985), adjusted Rand index; Hastie, Tibshirani and Friedman, *The Elements of Statistical Learning*, chapter 14.

MACHINE LEARNING / MODULE 4

Dimensionality reduction and feature selection

Choosing, compressing and mapping many correlated measurements

Full day · 09:00–17:00 | 6 parts | 2 laboratories

Machine learning for energy applications

Today's plan

09:00	Welcome, the curse of dimensionality, feature selection	Lecture
10:30	Break	Break
10:45	PCA in depth	Lecture
11:40	Lab 1: PCA on drilling data	Lab
12:40	Lunch	Lunch
13:40	Lab 1 debrief; kernel PCA, LDA, ICA and NMF	Lecture
14:25	Manifold learning: Isomap, LLE, t-SNE and UMAP	Lecture
15:15	Break	Break
15:30	Lab 2: nonlinear maps and a fair comparison	Lab
16:30	Debrief, reduction in practice and exit check	Discussion

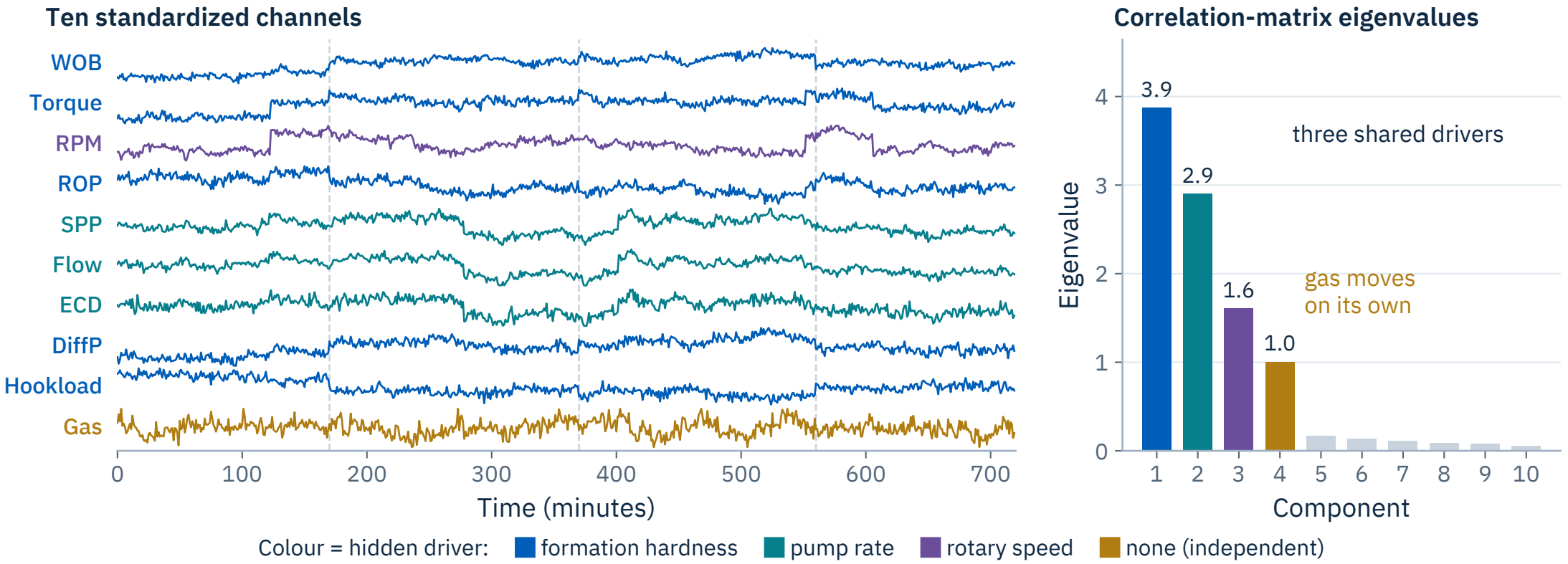
What you will be able to do

By the end of the day you will be able to:

1. Explain why many features weaken distance-based methods, and when to reduce.
2. Select features with filter, wrapper and embedded methods inside cross-validation.
3. Compute and validate a PCA: scaling, loadings, number of components, reconstruction error.
4. Choose kernel PCA, LDA, ICA or NMF for a stated goal.
5. Build t-SNE, Isomap and UMAP maps and state what they cannot show.
6. Judge a representation by trustworthiness, stability and held-out task score.

Ten drilling channels move as a few groups

Twelve hours of rig data: how many numbers per minute do you really need?



Synthetic data: three latent drivers plus noise (illustrative, seed 4). Right: eigenvalues, coloured by the driver each component follows.

PART 1 OF 6

01

Why reduce dimensions

More columns add noise and empty space before information.

- The curse of dimensionality in numbers
- Irrelevant features and nearest neighbours
- Selection versus extraction
- What each method preserves

Reduction serves four different goals

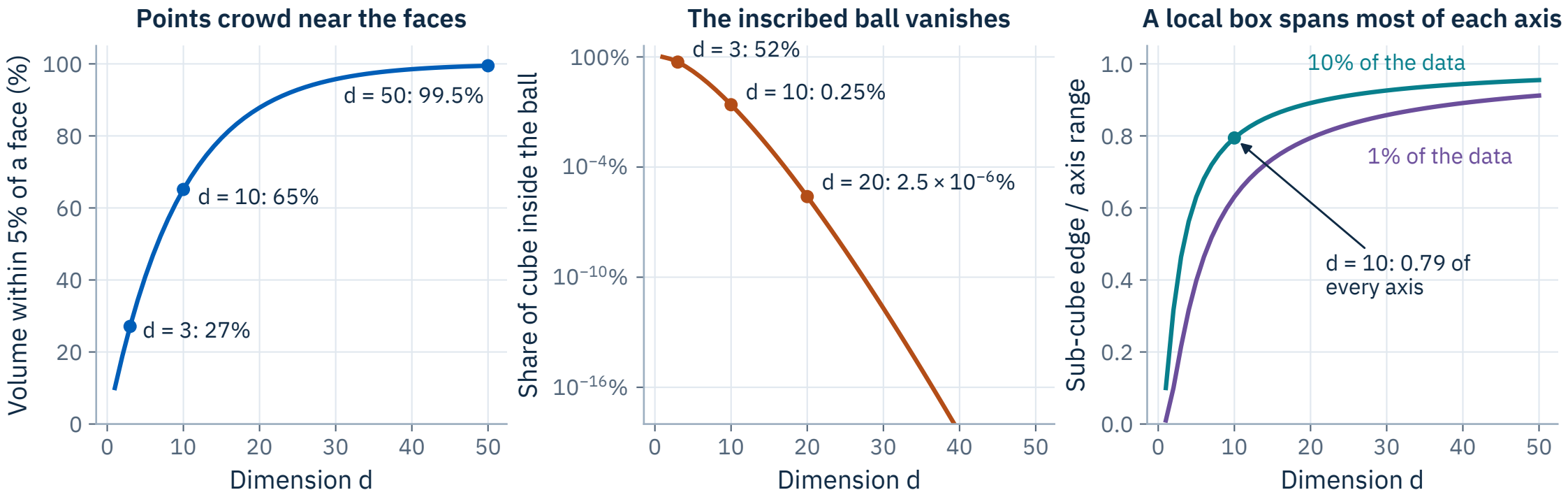
Goal	Useful output	Evidence to inspect
Visualization	Two- or three-dimensional coordinates	Neighbourhood preservation
Compression	Lower-dimensional representation	Reconstruction error
Denoising	Restricted representation	Signal preservation
Prediction	Features for a downstream estimator	Held-out task performance

Feature selection keeps original variables; feature extraction builds new ones.

◆ Name the goal before choosing a method: the goal decides which evidence counts.

High-dimensional volume concentrates near the edges

Three exact calculations for points spread uniformly in a unit cube.



Exact formulas: shell volume 1 − 0.9^d; inscribed-ball share π^(d/2) / Γ(d/2 + 1) × 0.5^d; sub-cube edge r^(1/d) for a fraction r of the data.

Keeping neighbour density needs exponentially more data

To keep 10 samples along every axis, a regular grid needs 10^d points:

10

samples for 1 feature

100,000

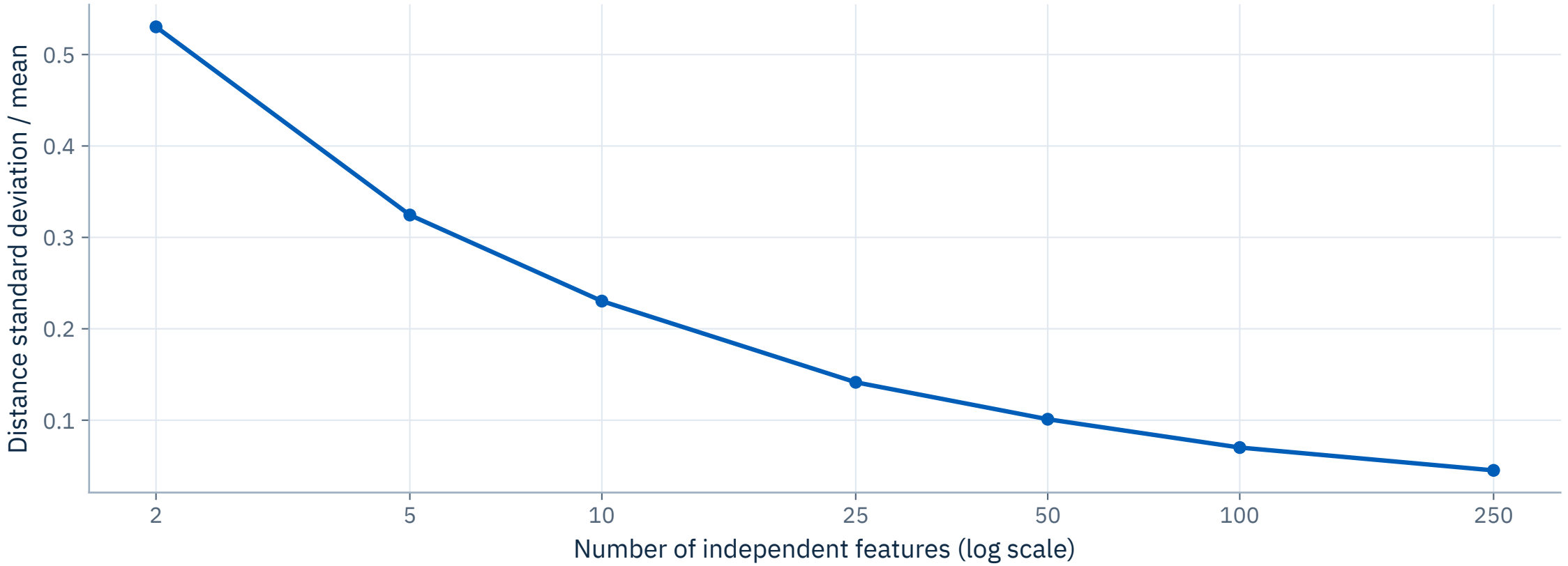
samples for 5 features

10 billion

samples for 10 features

- Real data escape this only because measurements share a few drivers and lie near lower-dimensional structure.

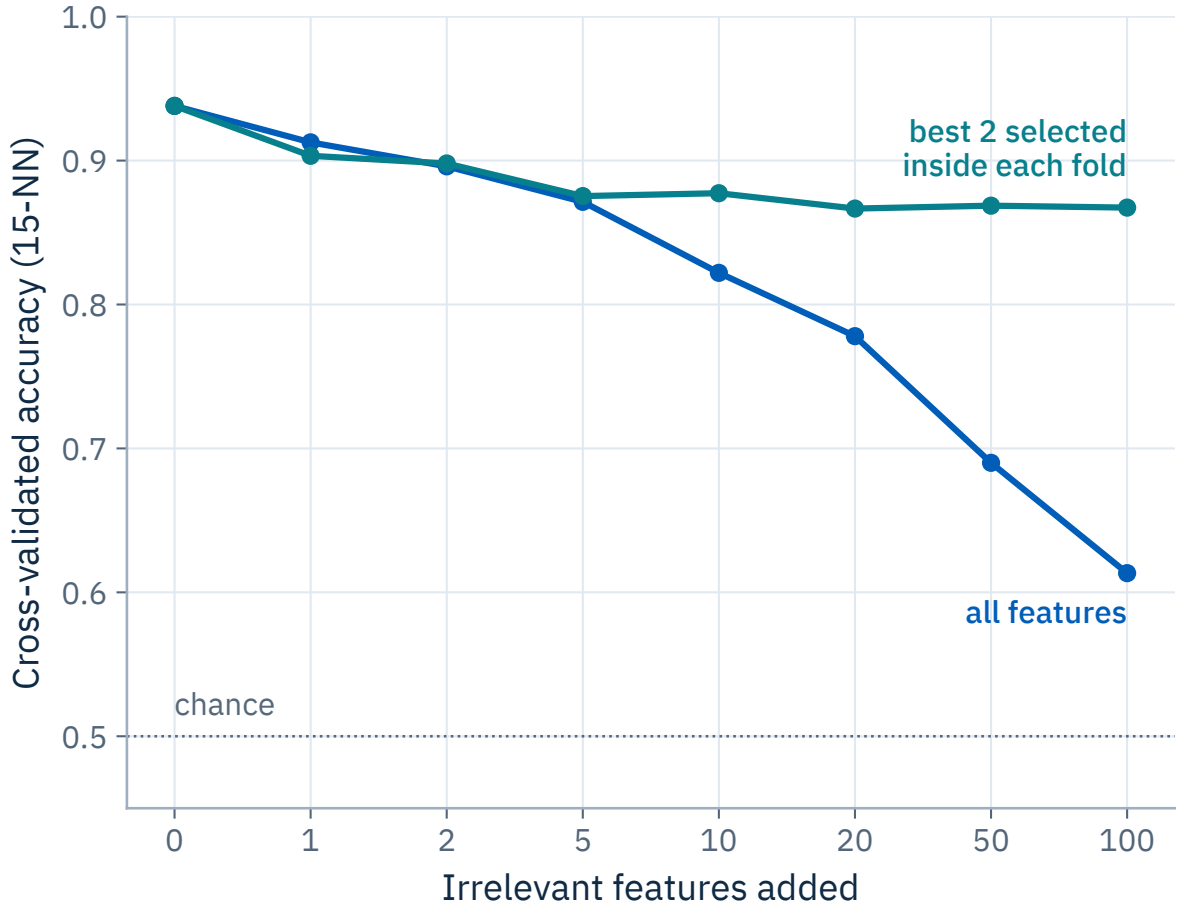
Distances can become less distinctive



Simulation with independent standard-normal features. Relative pairwise-distance spread shrinks as dimension grows.

Irrelevant features drown a nearest-neighbour signal

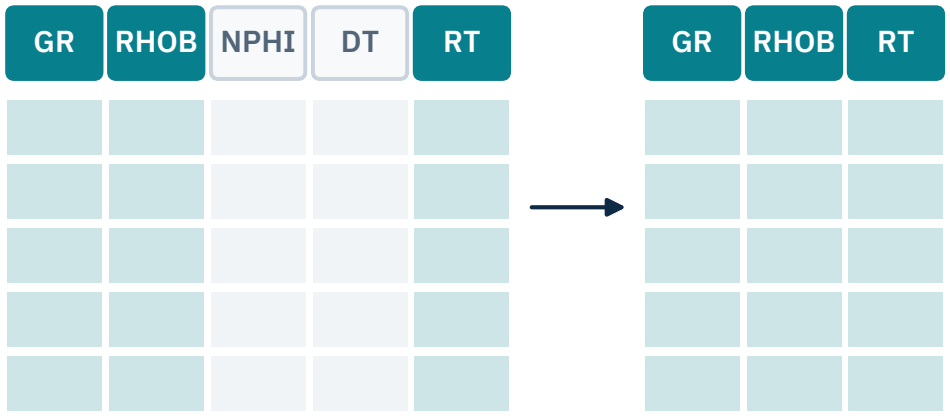
- Two informative features; pure-noise columns added in blocks
- 15-NN accuracy falls from 0.94 to 0.61 with 100 noise columns
- Selecting the best 2 features inside each fold holds 0.87



Synthetic two-class data (seeds 0–4); 5-fold CV averaged over five datasets.

Selection keeps columns; extraction builds new ones

Feature selection

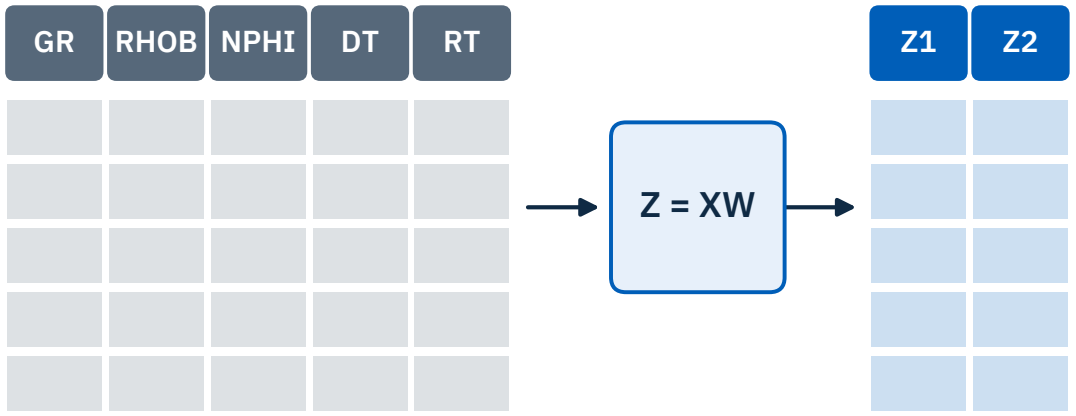


keeps names and units

unused sensors can go

A subset of the original columns

Feature extraction



every column contributes

new axes need interpretation

New columns built from all of them

Schematic. Selection returns a subset of the original logs; extraction returns new variables such as principal-component scores.

Reduction methods preserve different things

Method	Preserves	Uses labels	New observations
PCA, kernel PCA	Variance, linear or in a kernel space	No	`transform()`
LDA	Separation of known classes	Yes	`transform()`
Isomap, LLE	Geodesic or local linear structure	No	Approximate `transform()`
t-SNE	Local neighbourhoods	No	None in sklearn TSNE
UMAP	Neighbourhood graph	Optional	Fitted `transform()`

Parameter settings and preprocessing remain part of the method.

PRACTICE

4 min · Pairs

Match each energy task to a reduction goal

For each task, name the goal, choose selection or extraction, and name the evidence you would inspect:

1. Plot 25 wells' log suites to spot tool problems before interpretation
2. Store 1 Hz rig data in fewer channels, reconstructible within a tolerance
3. Predict ROP from 60 candidate surface and mud features
4. Decide which of 12 redundant sensors to keep on the next rig

PART 2 OF 6

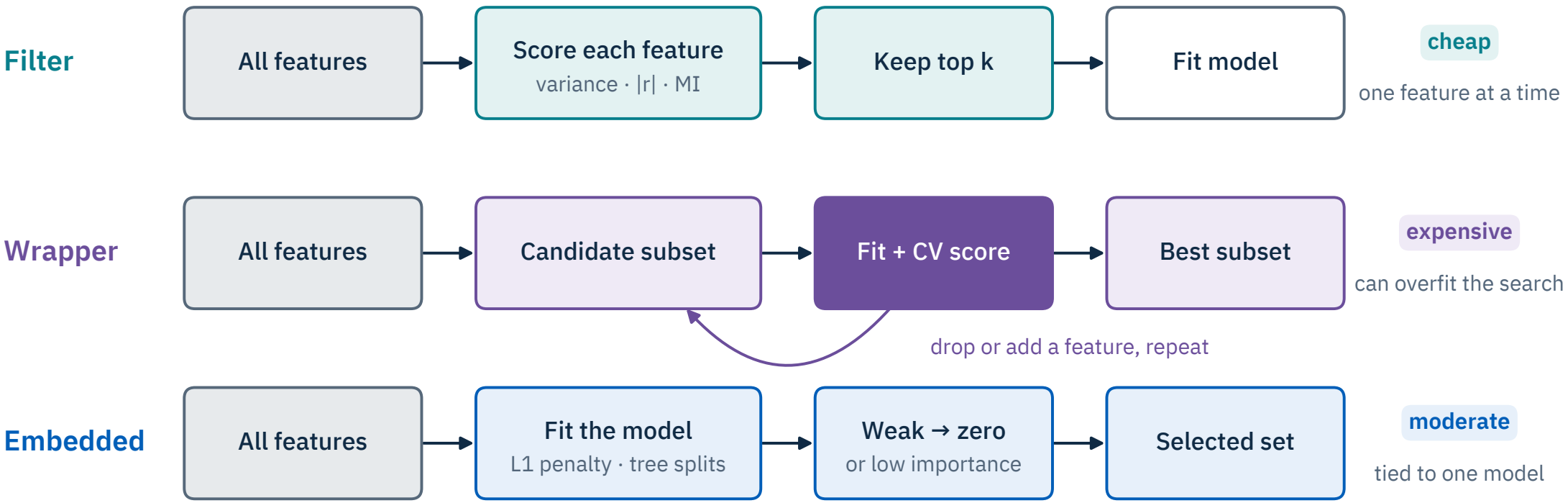
02

Feature selection

Keep the columns that matter; prove it on held-out folds.

- Filters: variance, redundancy, mutual information
- Wrappers: recursive feature elimination
- Embedded: L1 penalties and tree importance
- Leakage and the stability of selected sets

Three selection families use the model differently



Schematic. Filters score features without the final model; wrappers search subsets with it; embedded methods select while fitting it.

Filters rank features before any model is fitted

Variance threshold

- Drops constant or near-constant channels
- Scale-dependent: set it per sensor, in raw units

Redundancy

- Flags pairs of features with high $|r|$
- Keep the cheaper or more reliable sensor

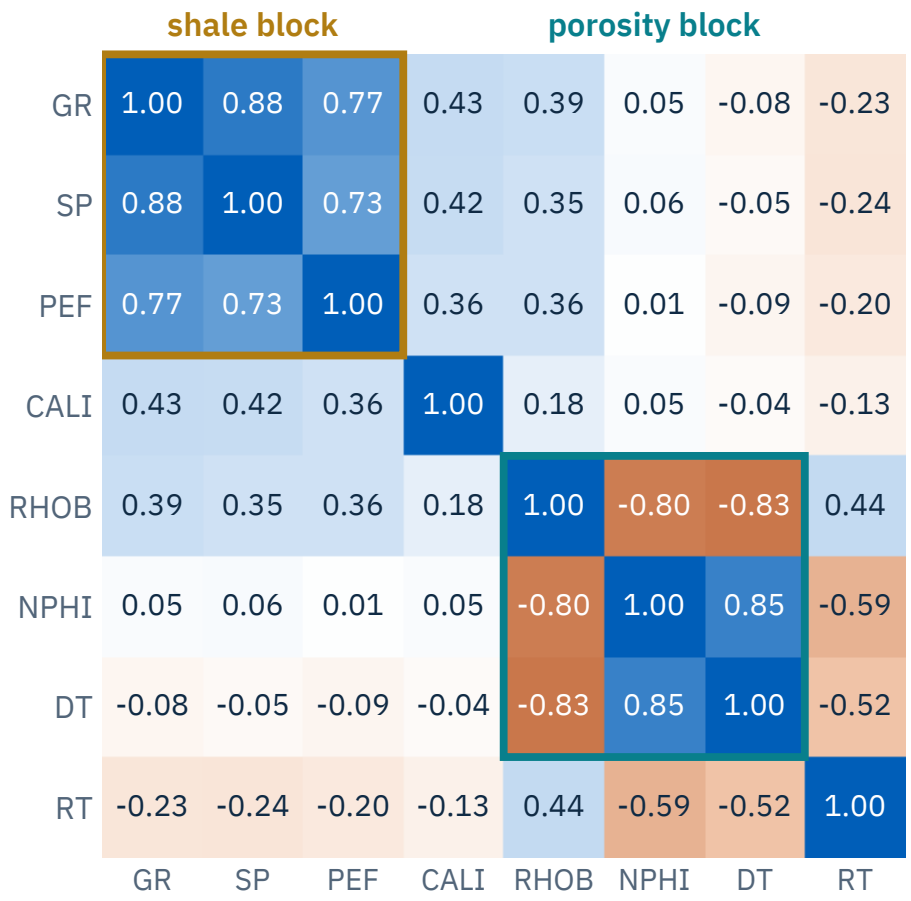
Relevance

- Scores each feature against the target
- F-test for linear links, mutual information for any

◆ Filters are fast and model-free, but they judge one feature at a time.

Correlated logs form redundant blocks

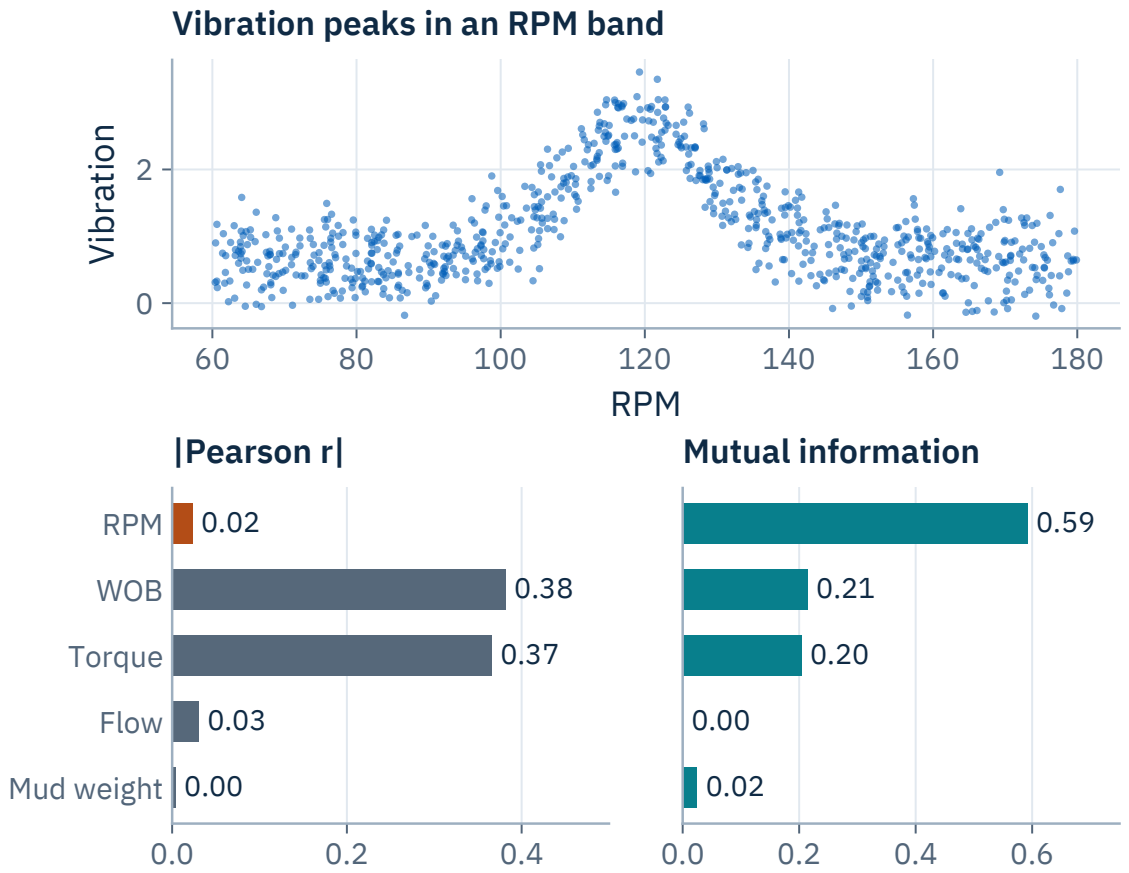
- GR, SP and PEF: $|r| \geq 0.73$, one shale signal
- RHOB, NPHI and DT: $|r| \geq 0.80$, one porosity signal
- Keep one log per block, or summarize the block with PCA
- Pearson r sees only linear, pairwise links



Synthetic log suite (seed 11); orange cells are negative correlations.

Mutual information catches nonlinear relevance

- Vibration peaks in an RPM band, yet $|r|$ for RPM is only 0.02
- Mutual information ranks RPM first (0.59): it scores any dependence
- WOB and torque score alike because they carry one signal
- Still one feature at a time: blind to redundancy and interactions



Synthetic example (seed 7): resonance peak plus a weak WOB effect. MI from the k-nearest-neighbour estimator.

Recursive elimination drops the weakest feature each round

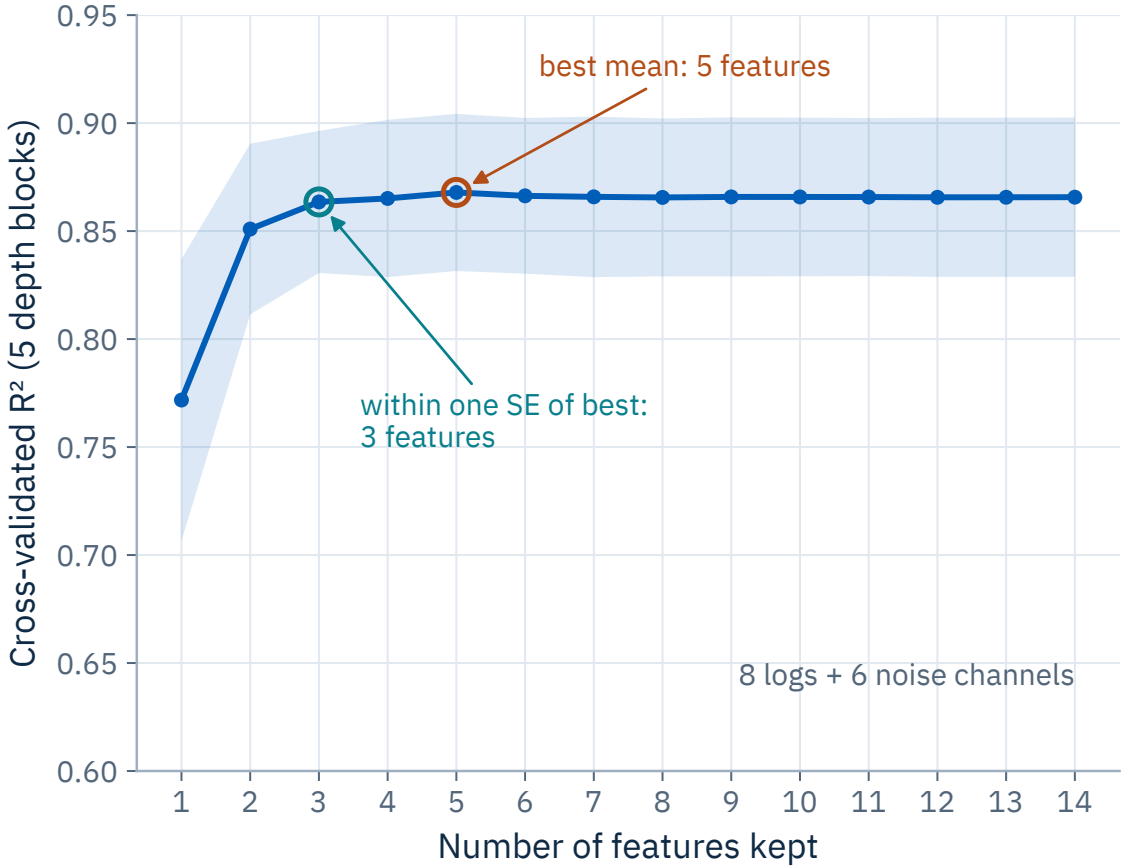
A backward search that uses the model's own ranking in every round.

1. **Fit** the model on all current features
2. **Rank** features by coefficient size or importance
3. **Drop** the weakest one (step = 1)
4. **Repeat** until one feature remains
5. **Choose** the size with the best CV score

◆ RFECV repeats the whole loop inside every training fold, so each size is scored on unseen rows.

Cross-validation chooses how many features to keep

- R^2 rises from 0.77 with one log to 0.87 with five, then flattens
- Six noise channels neither help nor clearly hurt this linear model
- Within one standard error of the best: three features
- Folds are contiguous depth blocks, not shuffled rows



RFECV with linear regression; synthetic log suite plus six noise channels; band = ±1 SD across five folds.

Wrapper selectors nest inside an outer cross-validation

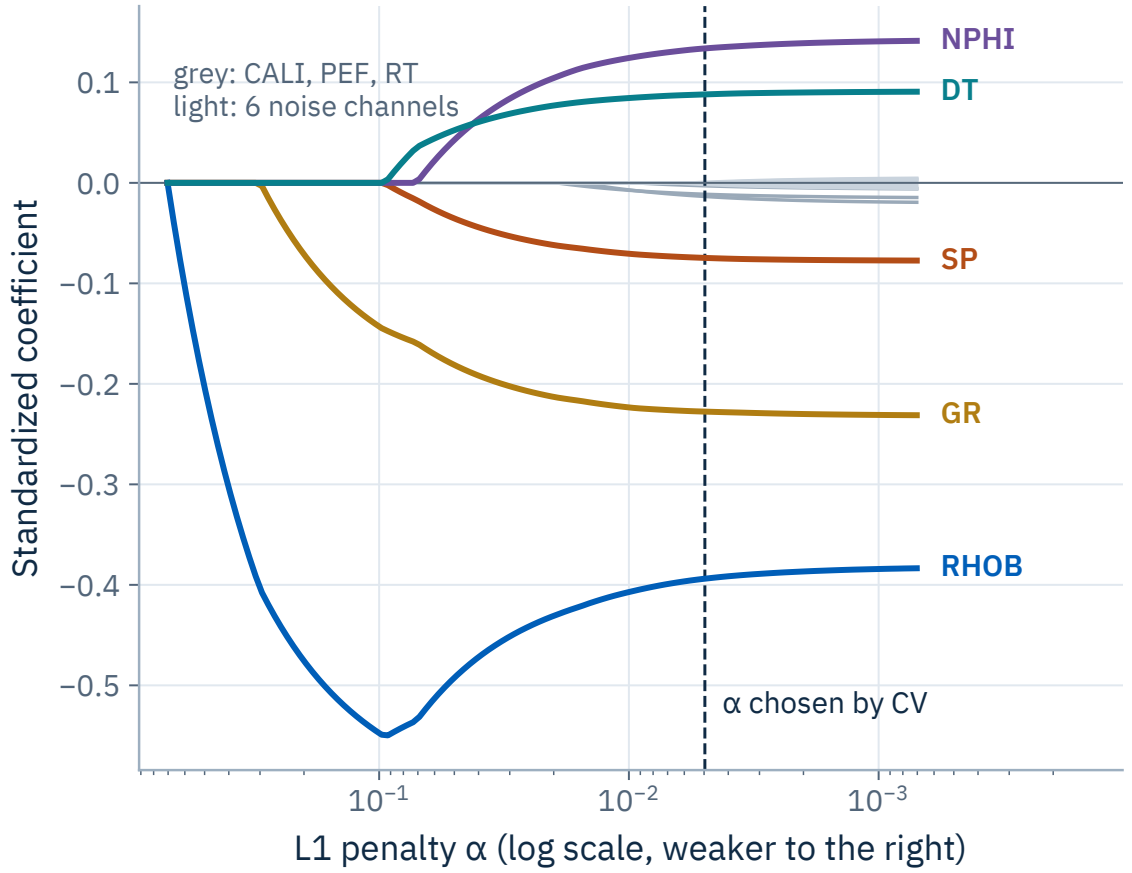
```
from sklearn.feature_selection import RFECV, SequentialFeatureSelector
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import KFold, cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

blocks = KFold(n_splits=5)            # contiguous depth blocks, no shuffle
rfe = RFECV(LinearRegression(), cv=blocks, scoring="r2")
sfs = SequentialFeatureSelector(LinearRegression(), direction="forward",
                                n_features_to_select="auto", tol=0.005,
                                cv=blocks, scoring="r2")
for selector in (rfe, sfs):
    model = make_pipeline(StandardScaler(), selector, LinearRegression())
    outer = cross_val_score(model, X, y, cv=blocks, scoring="r2")
    print(type(selector).__name__, outer.mean().round(3))
```

The inner search runs on each outer training fold only; the outer score is the honest one.

L1 penalties drive weak coefficients exactly to zero

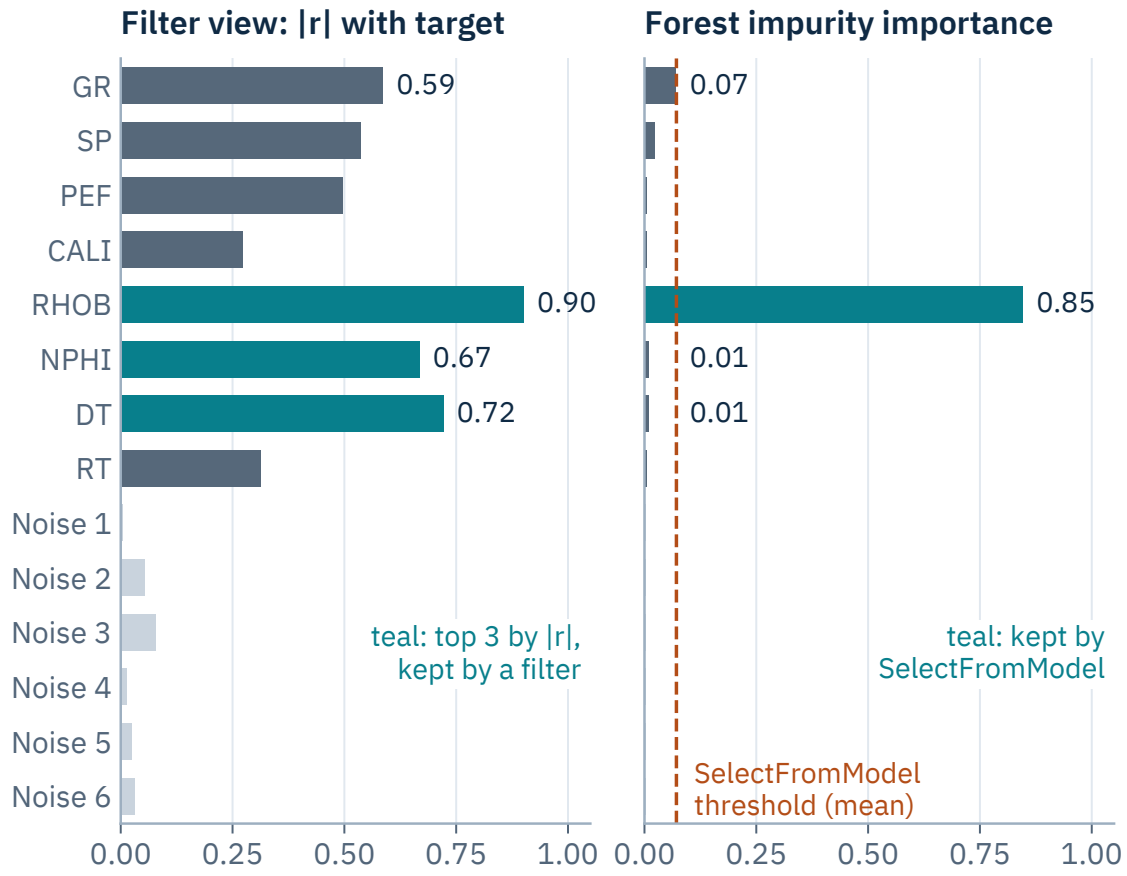
- As α shrinks, features enter: RHOB, then GR, then SP, DT and NPHI
- RHOB's coefficient shrinks once correlated NPHI and DT join
- Standardize first; pick α by CV (dashed line); see Module 1 on the lasso



Lasso path on the synthetic log suite plus six noise channels (target: log permeability).

A forest gives redundant logs almost no importance

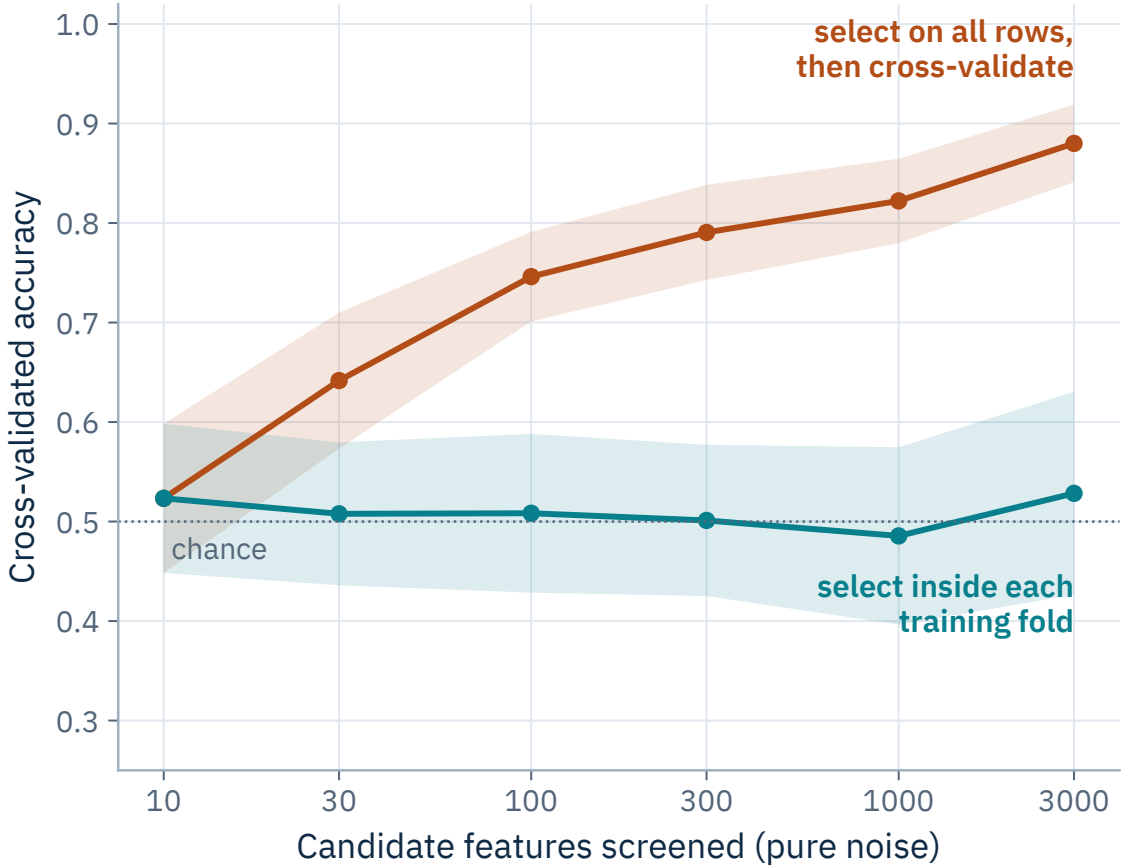
- A filter rates NPHI and DT highly: $|r| = 0.67$ and 0.72 with the target
- The forest credits RHOB with 0.85; NPHI and DT get 0.01, near noise
- SelectFromModel at the mean importance keeps only RHOB
- Redundant is not useless: without RHOB, NPHI and DT carry the signal



Random forest (300 trees, seed 0) on the synthetic log suite plus six noise channels; target log permeability.

Selection must happen inside cross-validation

- Pure noise: 60 rows, random labels, keep the 10 best-scoring features
- Selected on all rows: accuracy climbs from 0.52 to 0.88 with more candidates
- Selected inside each training fold: chance at every size



30 synthetic pure-noise datasets per point (seeds 2000–2029); SelectKBest (k = 10) and logistic regression; 5-fold CV; bands = ±1 SD.

⚠ Any step that looks at the target belongs inside the fold, as in Module 2's leakage demonstration.

The selector belongs inside the pipeline

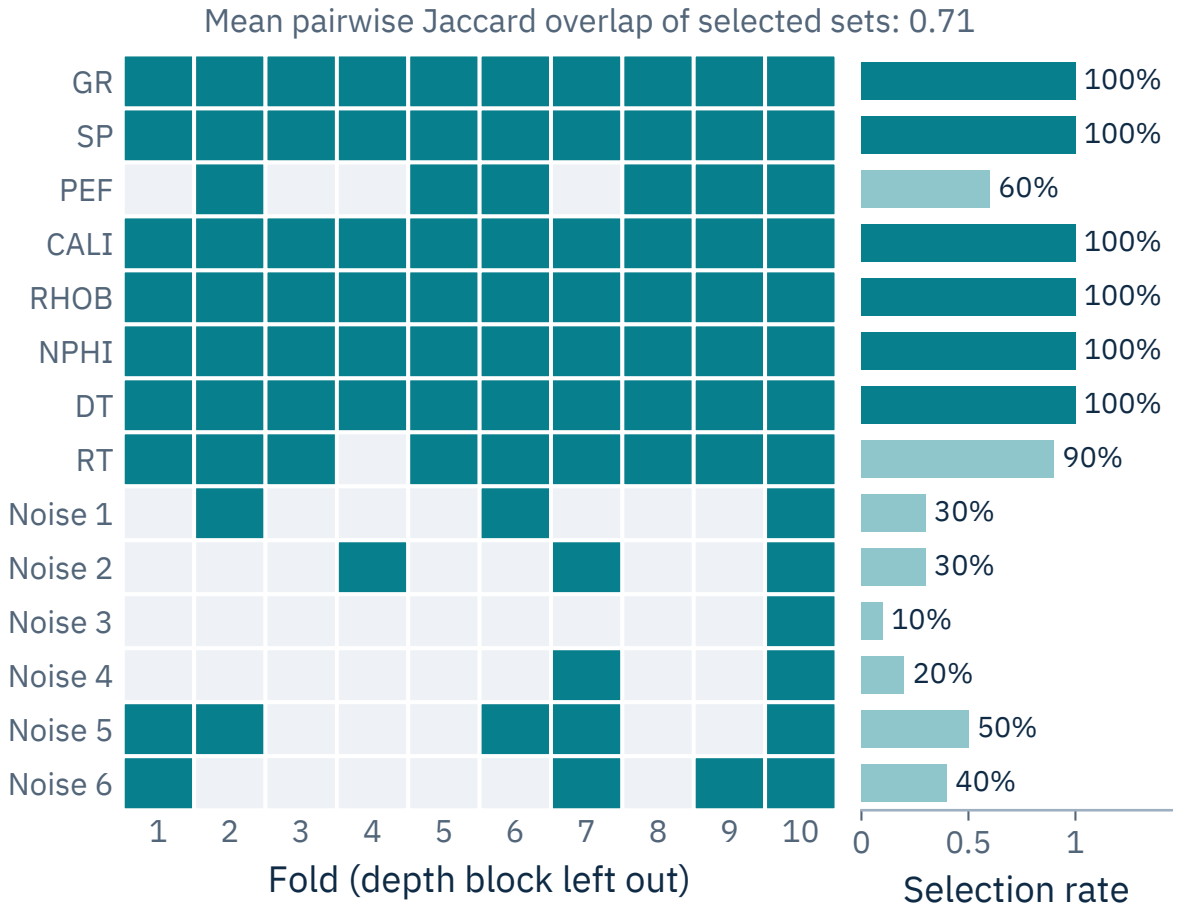
```
from functools import partial
from sklearn.feature_selection import SelectKBest, mutual_info_classif
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GroupKFold, cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

mi = partial(mutual_info_classif, random_state=0)
model = make_pipeline(StandardScaler(), SelectKBest(mi, k=10),
                      LogisticRegression(max_iter=1000))
scores = cross_val_score(model, X, y, groups=well_id,
                         cv=GroupKFold(n_splits=5), scoring="f1_macro")
print(scores.mean().round(3), scores.std().round(3))
```

Each fold re-ranks the features on its own training wells; k can be tuned in the same grid search.

Selected sets change from fold to fold

- LassoCV refitted in ten depth-block folds
- Six logs chosen in every fold; noise channels in 10–50% of folds
- Mean Jaccard overlap between fold selections: 0.71
- Report selection frequencies, not one "final" list



Synthetic log suite plus six noise channels (seeds 11, 5).

Choose a selection family by cost and blind spot

Family	Cost	Sees interactions	Main risk
Filter	Low	No	Redundant picks; misses joint effects
Wrapper	High	Through the model	Overfitting the search
Embedded	Moderate	Model-specific	Arbitrary picks among correlated features

All three need the same honest validation as the final model.

◆ A robust default: drop dead and duplicate channels first, then an embedded or wrapper method inside grouped CV.

PRACTICE

6 min · Pairs

Find the leak in this selection workflow

Thirty wells, 400 candidate drilling features, target ROP. An analyst ranks features by $|r|$ with ROP on all rows, keeps the top 25, then reports a GroupKFold score by well.

1. Where does information leak, and why does grouping by well not prevent it?
2. Restructure the workflow so that the score is honest.
3. What should the report say about the 25 selected features?

DISCUSSION

A leak-free selection workflow has four steps

The order is the point: the split comes first, and everything that learns sits inside the fold.

1. **Split**
   by well, before anything is learned
2. **Inside each training fold**
   scale, rank, select, fit
3. **Score**
   on held-out wells only
4. **Report**
   score spread and selection frequency

- Tune k in an inner loop, and keep final test wells untouched until the end.

PART 3 OF 6

03

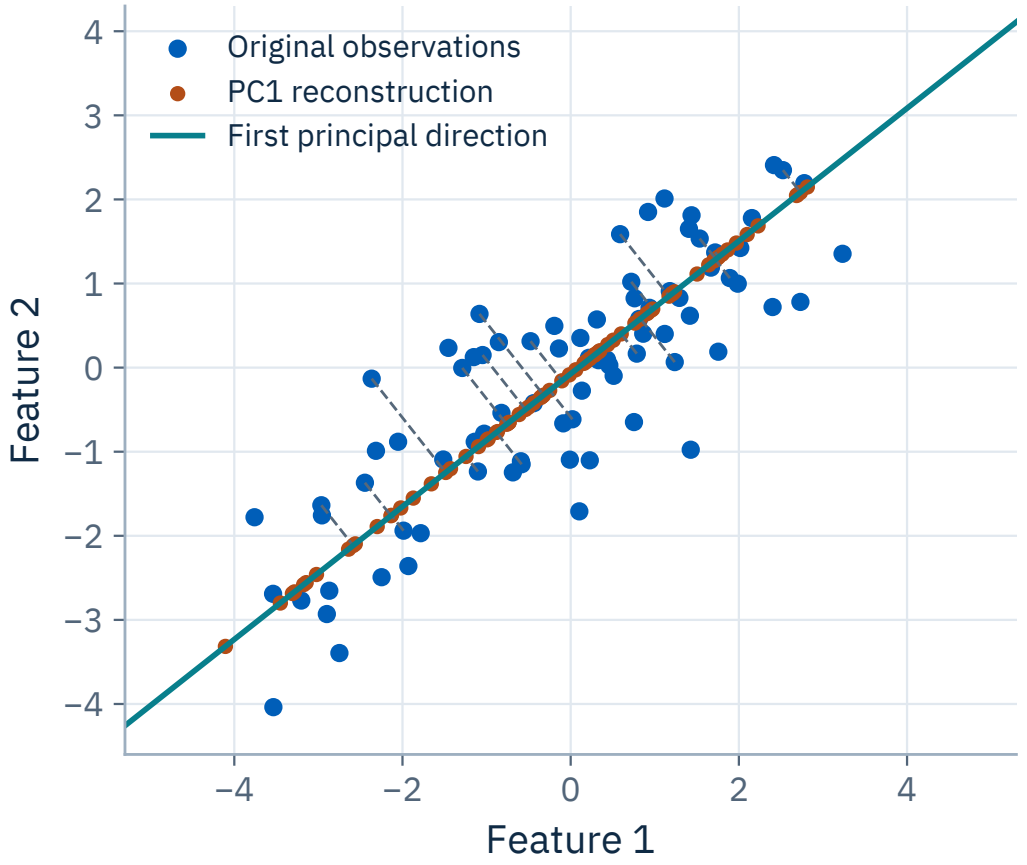
Principal components in depth

Orthogonal directions summarize variance in centred data.

- Covariance, eigenvectors and the SVD
- Scaling choices and the number of components
- Loadings, biplots and reconstruction error
- Whitening, batches and pipelines

The first principal direction keeps the most spread

- PC1 is the line along which projected points vary most
- Equivalently, it minimizes squared perpendicular distances (dashed)
- Components are orthogonal, ordered by variance, and sign-arbitrary

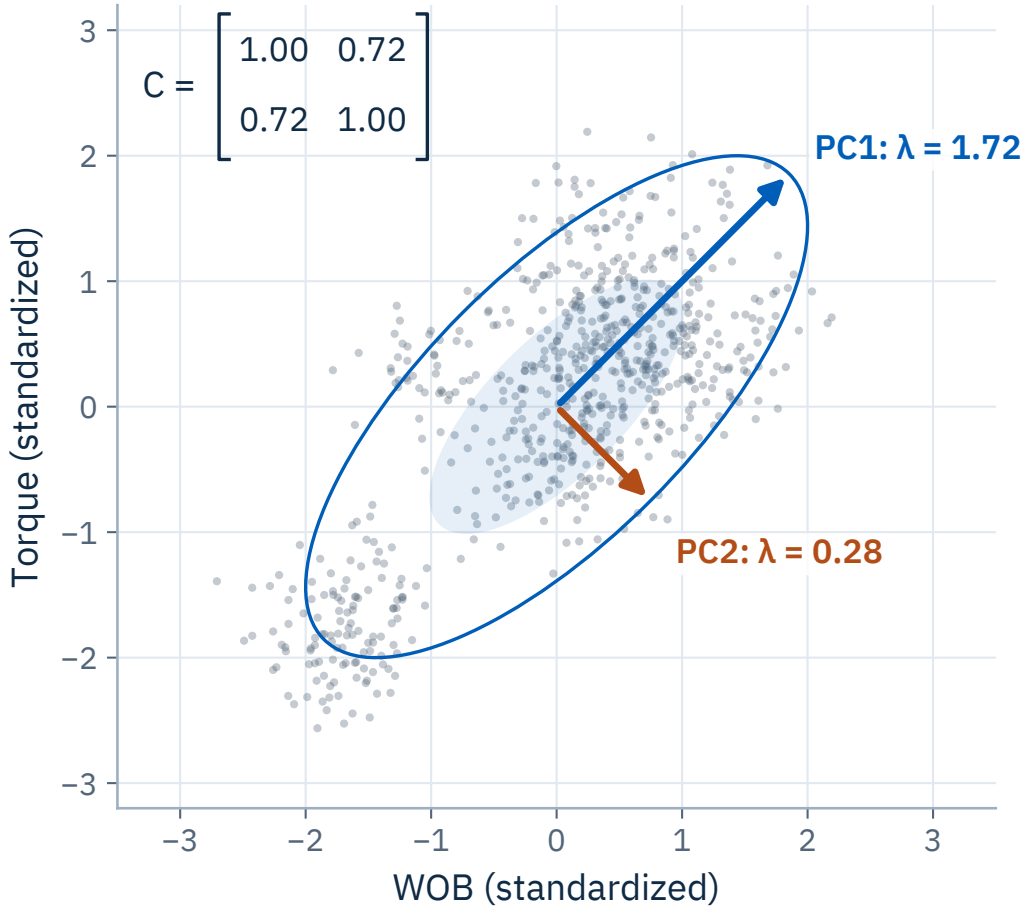


PCA computed from a synthetic correlated two-feature dataset. Dashed segments show reconstruction residuals after projection onto PC1.

The covariance matrix sets the principal axes

$$C v_j = \lambda_j v_j$$

- C: covariance of the centred data, d × d
- Eigenvectors v: the principal directions
- Eigenvalues λ: the variance along each direction
- Standardized WOB and torque: r = 0.72, so λ = 1 ± r = 1.72 and 0.28

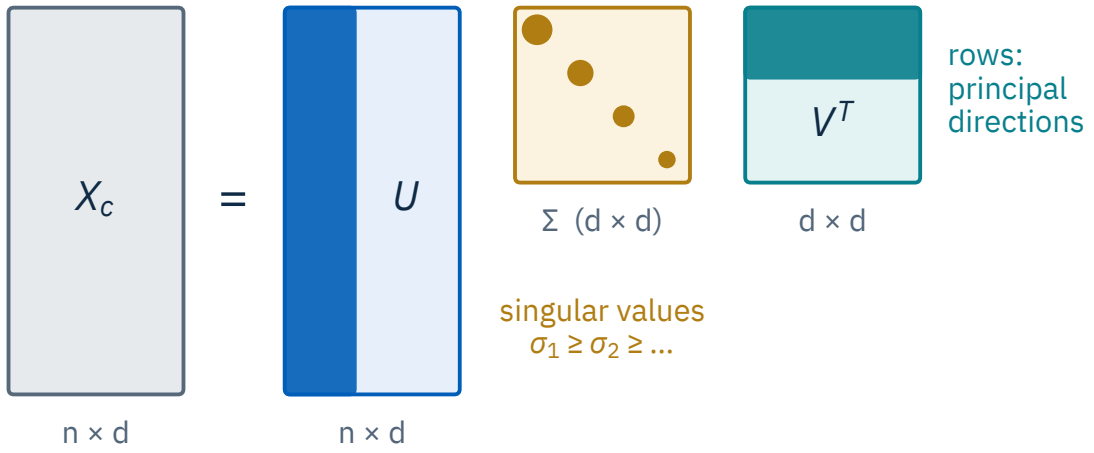


Synthetic drilling channels (seed 4); arrows have length 2√λ.

The SVD delivers directions, variances and scores

$$X_c = U\Sigma V^T, \qquad Z = X_c V_k$$

- Centred data: n observations × d features
- Rows of V^T are the principal directions
- Keeping k directions gives the n × k score matrix Z



Keep the first k components (shaded):



$$Z = X_c V_k = U_k \Sigma_k$$

scores; variance of score j: $\lambda_j = \sigma_j^2/(n-1)$

Schematic of the thin singular value decomposition.

Centring and scaling answer different questions

Standardize: correlation-matrix PCA

- Each feature contributes in unitless terms
- Default for mixed units: psi, rpm, klbf, ft/h

VS

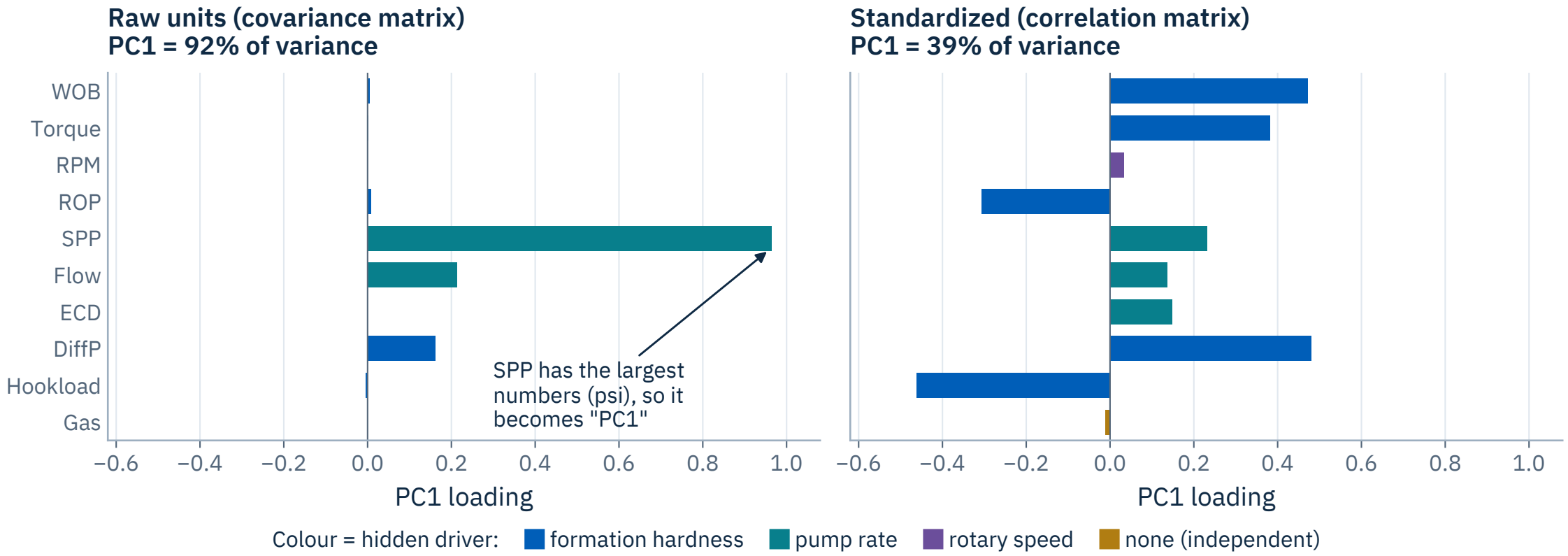
Keep units: covariance-matrix PCA

- Absolute variation is the point
- Only when units are comparable, such as one sensor type

◆ Always centre, and fit means and scales on training data only.

Standardization changes the answer

Same ten channels, same algorithm: only the scaling differs.



Synthetic drilling channels in engineering units (seed 4). Bar colours mark the driver behind each channel.

PRACTICE

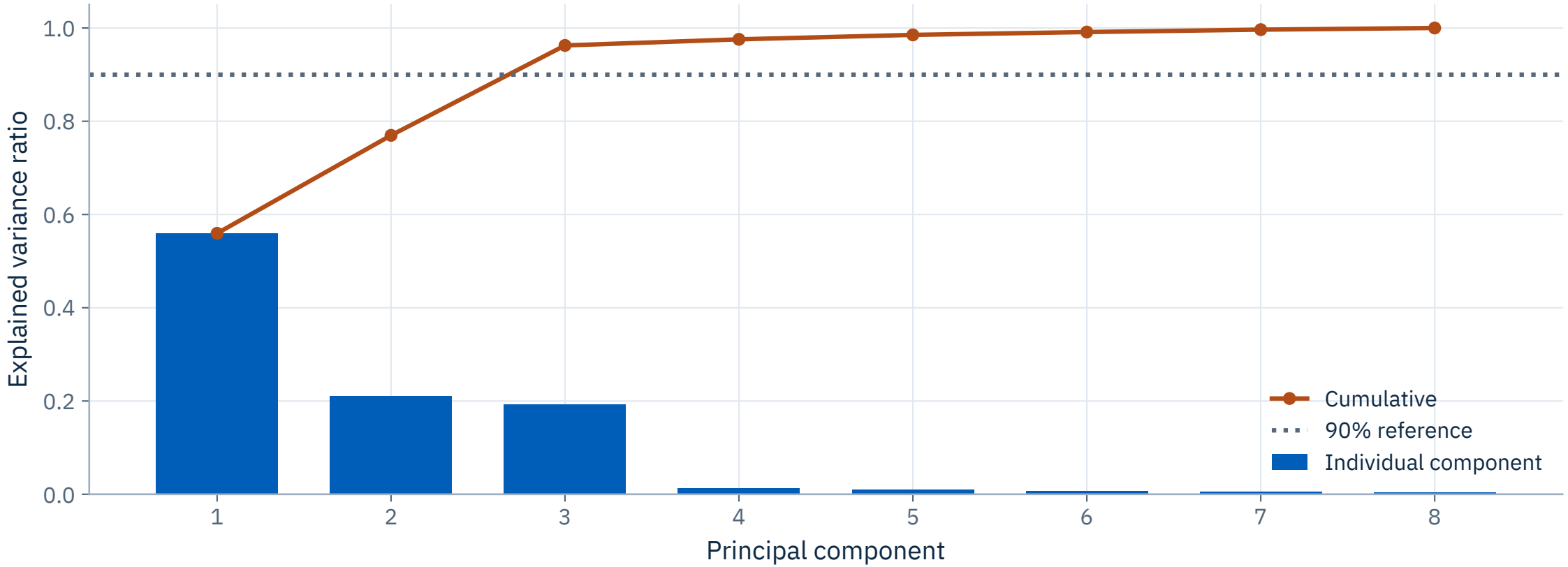
5 min · Individual

A projection by hand shows what PCA discards

A centred observation is $x = [2, 0]$. The first direction is $v = [1, 1] / \sqrt{2}$.

1. Calculate its score on this direction.
2. Reconstruct the observation using this single component.
3. Calculate the squared reconstruction error.
4. What share of the point's squared length did PC1 keep?

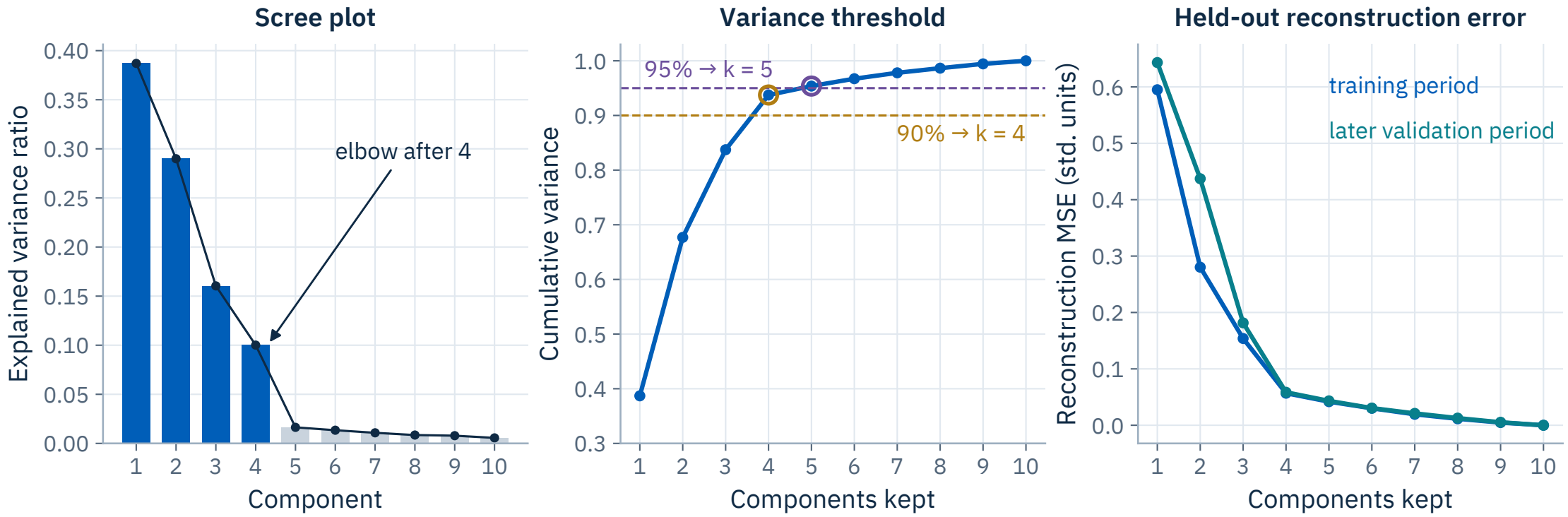
Explained variance describes retained spread



PCA on a generated correlated feature matrix. Bars show individual variance ratios; the line shows their cumulative sum.

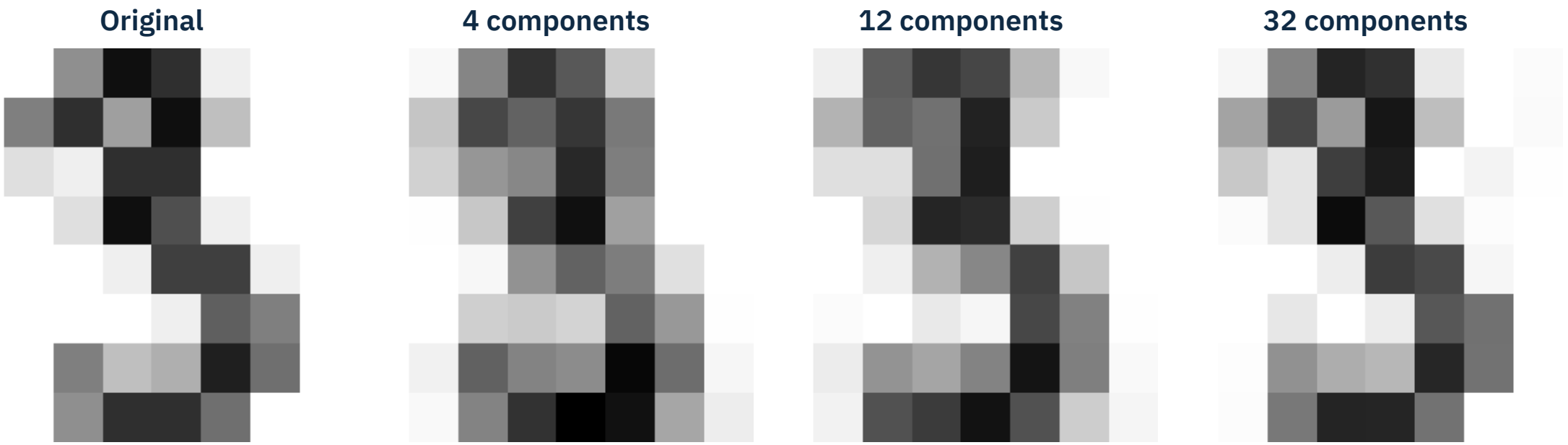
Three rules for k give related, not identical, answers

Scree elbow after 4; 90% of variance needs 4 components, 95% needs 5; held-out error flattens after 4.



Synthetic drilling channels (seed 4). Left and middle: all 720 minutes. Right: PCA fitted on minutes 0–429, validated on minutes 430–719.

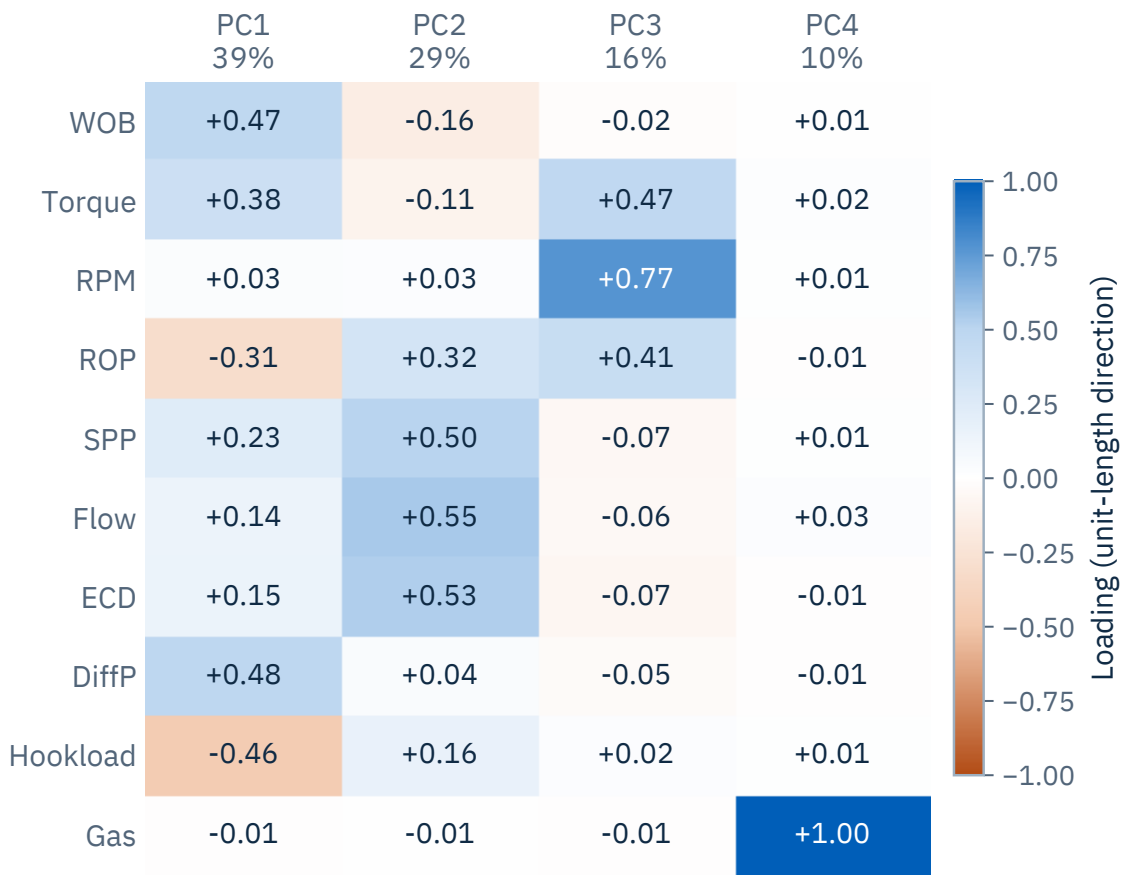
Reconstruction makes information loss visible



PCA fitted to the bundled scikit-learn digits (8 × 8 images). Linear compression, not a neural autoencoder.

Loadings connect components to features

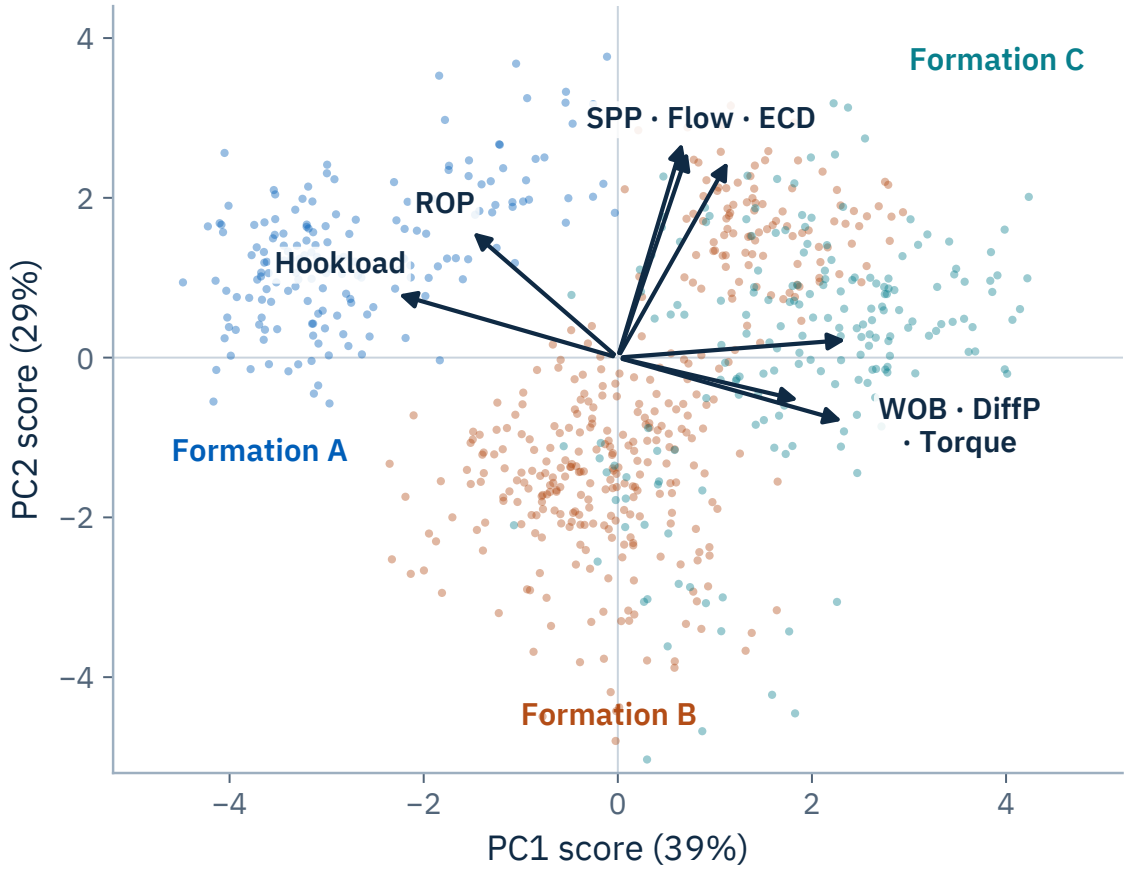
- Large |*loading*|: the feature drives that direction
- Opposite signs mark a contrast: WOB up, hook load down on PC1
- A component's overall sign can flip without changing the fit
- PC4 is gas alone: a whole component for one channel



Synthetic drilling channels (seed 4), standardized. Rows of components_, signs fixed for display.

A biplot shows observations and loadings together

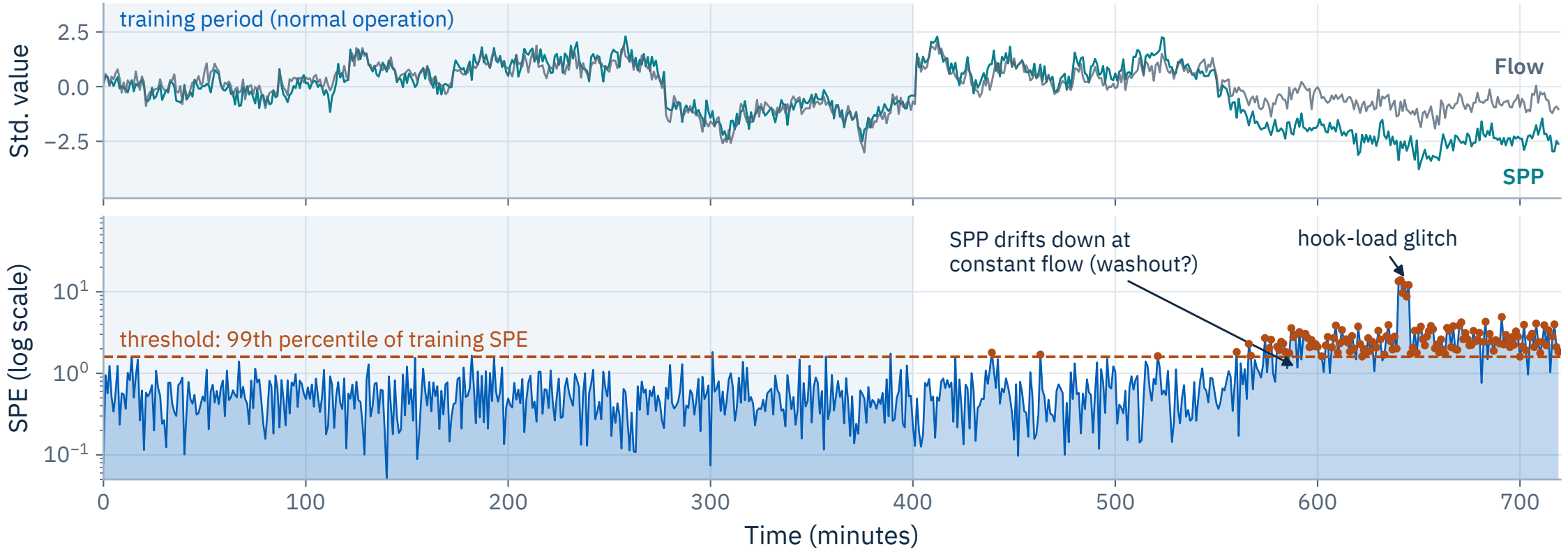
- Points: PC1–PC2 scores, coloured by formation after fitting
- Arrows: loadings, rescaled for display
- Formation A sits toward high hook load and ROP, low WOB
- Angles approximate correlations only if two PCs dominate (68% here)



Synthetic drilling channels (seed 4); arrows are loadings × 5 for display. RPM and gas load on PC3–PC4.

Reconstruction error flags unusual observations

Fit PCA on normal operation; score each new minute by its squared prediction error (SPE).



Synthetic drilling channels (seed 4) with an injected SPP drift and a hook-load glitch; PCA with k = 4 fitted on minutes 0–399.

Whitening rescales every component to unit variance

$$z_j = \frac{x_c^T v_j}{\sqrt{\lambda_j}}$$

- `PCA(whiten=True)` divides each score by the square root of its eigenvalue
- Whitened scores are uncorrelated and have equal variance
- A standard first step for ICA and some distance-based models
- It inflates noise in small components, so keep k modest

Incremental PCA fits data in batches

```python
import pandas as pd
from sklearn.decomposition import IncrementalPCA
from sklearn.preprocessing import StandardScaler

cols = ["WOB", "TORQUE", "RPM", "ROP", "SPP", "FLOW"]
scaler = StandardScaler()
for chunk in pd.read_csv("rig_1hz.csv", usecols=cols, chunksize=100_000):
    scaler.partial_fit(chunk)                    # pass 1: means and SDs

ipca = IncrementalPCA(n_components=4)
for chunk in pd.read_csv("rig_1hz.csv", usecols=cols, chunksize=100_000):
    ipca.partial_fit(scaler.transform(chunk))    # pass 2: components
print(ipca.explained_variance_ratio_.round(3))
```

Useful when a year of 1 Hz rig data does not fit in memory; check against batch PCA on a sample.

PCA belongs inside the prediction pipeline

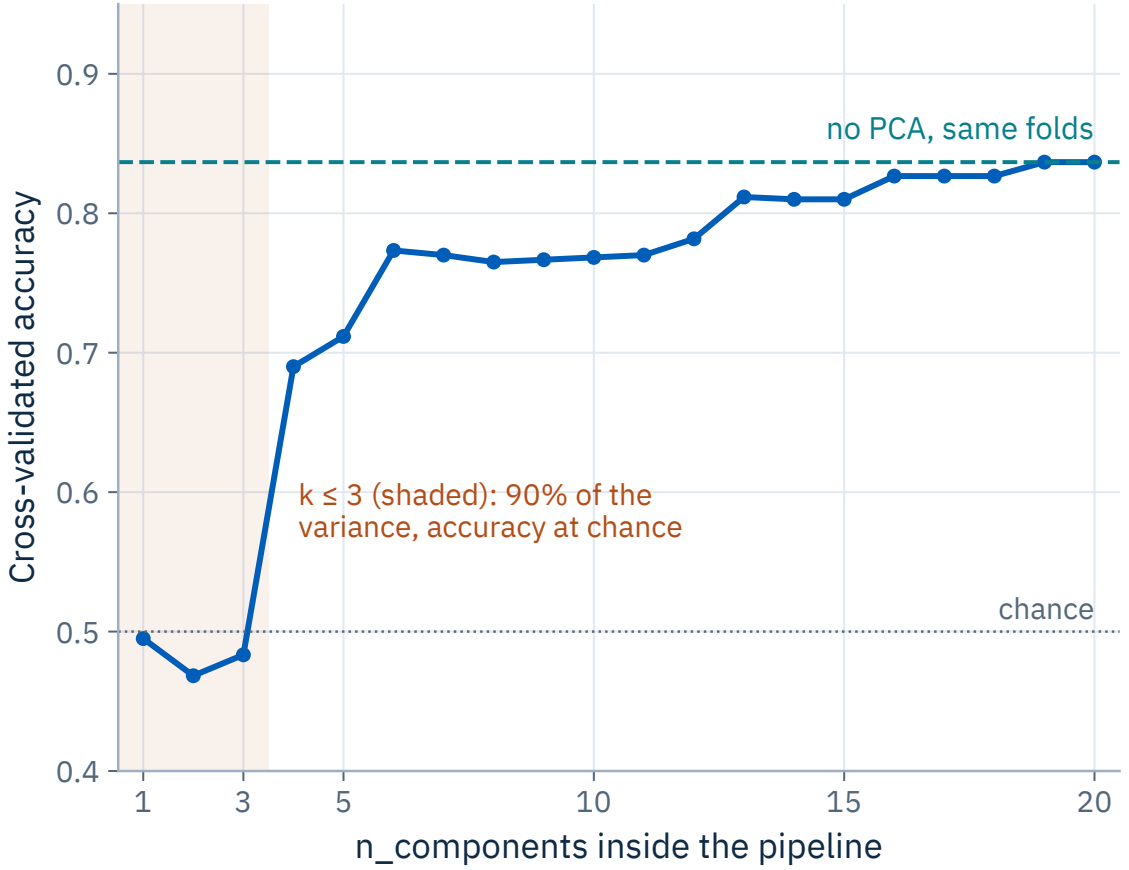
```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GridSearchCV, GroupKFold

model = make_pipeline(StandardScaler(), PCA(),
                      LogisticRegression(max_iter=1000))
grid = {"pca__n_components": [2, 4, 6, 8, 10]}
search = GridSearchCV(model, grid, cv=GroupKFold(n_splits=5),
                      scoring="f1_macro")
search.fit(X_train, y_train, groups=wells_train)
print(search.best_params_, round(search.best_score_, 3))
```

Compare with the same pipeline without PCA, on the same folds.

Tune the number of components against the task

- Three nuisance factors hold 90% of the variance and no class signal
- With k ≤ 3 the classifier sits at chance (0.47–0.50)
- It matches the no-PCA baseline (0.84) only at k = 19



Synthetic 20-feature data (seed 3); pipeline: StandardScaler, PCA, logistic regression; 5-fold CV.

PRACTICE

5 min Pairs

PCA limitations are testable

Pressure explains most variance, but a small vibration feature predicts failures.

1. Would keeping only high-variance components necessarily help classification?
2. What experiment would test this?
3. Which method from Part 2 could keep the vibration feature?

LABORATORY

60 min

Lab 1: PCA on drilling data

- Check the correlation matrix first; treat Depth as an index, not a feature
- Standardize ROP, RPM, Torque and WOB; fit PCA with all components
- Choose k from scree and cumulative plots, and justify the threshold
- Read loadings and the biplot; report reconstruction error in original units
- Extension: repeat PCA on raw units and compare PC1 with the standardized fit

NOTEBOOK

`exercises/01_pca/PCA_Drilling_Student.ipynb`

EVIDENCE TO SUBMIT

- Scree and cumulative-variance plots with your chosen k
- A loadings table with one sentence per component
- Reconstruction RMSE per feature in original units

DISCUSSION

Lab 1 debrief: this drilling table barely compresses

The course file drilling.csv, analysed as in the notebook:

28.5%

PC1 share after standardizing

4 of 4

components needed for 90% of variance

89%

PC1 share in raw units: WOB alone

- A flat spectrum is a finding: the four channels are nearly uncorrelated ($|r| \leq 0.13$), so there is little to compress.

PART 4 OF 6

04

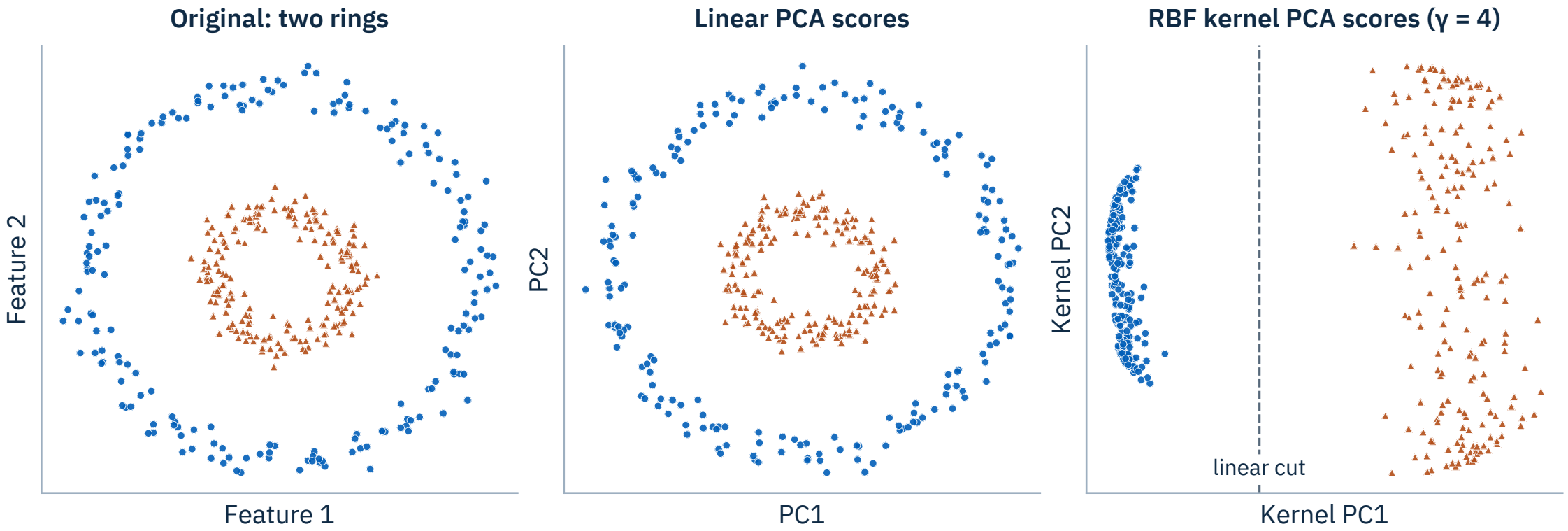
Beyond linear PCA

Change the geometry or the objective of PCA.

- Kernel PCA for curved structure
- LDA: a supervised projection
- ICA and NMF: independent sources, additive parts
- Autoencoders: a preview of the neural-network course

Kernel PCA can unfold what linear PCA cannot

PCA runs on an n × n similarity (kernel) matrix instead of the d × d covariance matrix.



scikit-learn make_circles (400 points, seed 0); RBF kernel, γ = 4. Colours show ring membership, which is never used in fitting.

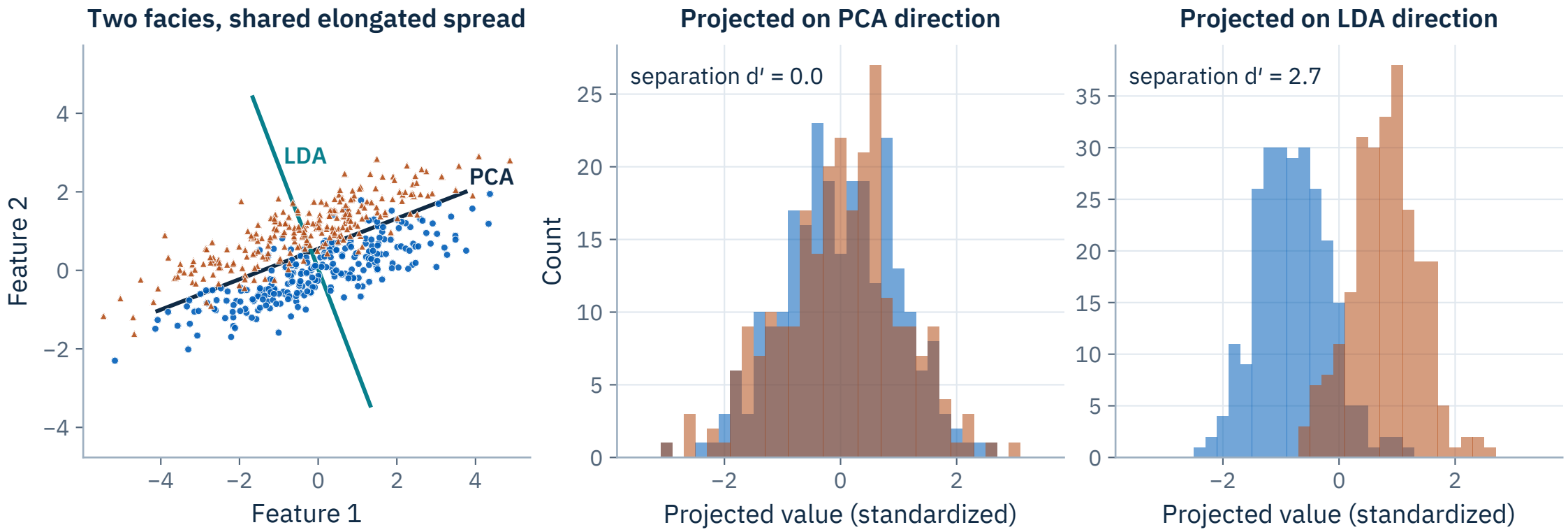
Kernel PCA trades interpretability for flexibility

	Linear PCA	Kernel PCA
Works on	d × d covariance	n × n kernel matrix
Captures	Linear directions	Curved structure, via the kernel
Memory and time	Modest for large n	Grows with n^2 and n^3
Interpretation	Loadings per feature	No loadings; approximate pre-images
New observations	Exact projection	`transform()` via kernels to training rows

◆ Tune γ against a downstream score; without one, prefer linear PCA.

LDA projects to separate labelled classes

PCA follows the largest spread; LDA follows the direction that separates the labelled classes.



Synthetic two-class data with a shared covariance (seed 2). d′ = difference in class means / pooled standard deviation.

PCA and LDA answer different questions

PCA

- Unsupervised: never sees labels
- Maximizes total projected variance
- Up to d components

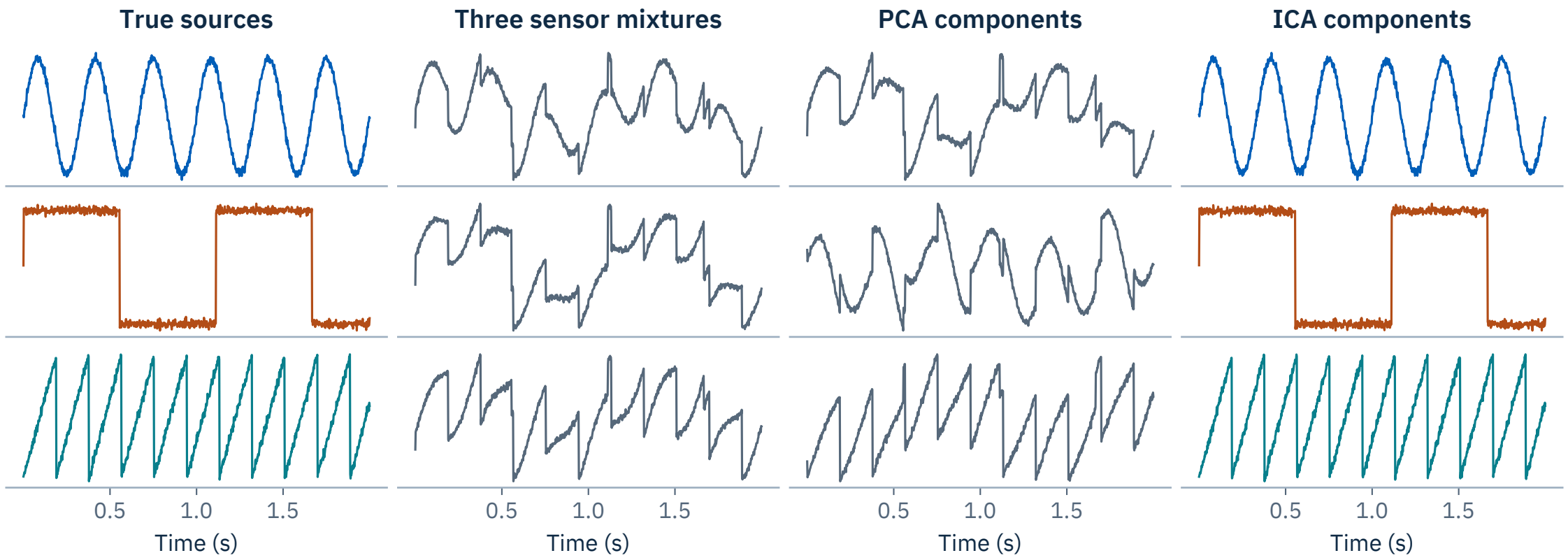
vs

LDA

- Supervised: uses class labels
- Maximizes between- over within-class variance
- At most C – 1 components for C classes

◆ LDA learns from labels, so fit it inside the training folds, like any model.

ICA unmixes sources that PCA only decorrelates



Synthetic sources mixed by a fixed 3 × 3 matrix; FastICA (seed 0). ICA rows are reordered and sign-matched for display, because ICA cannot recover order, sign or scale.

ICA works only when its assumptions hold

Assumptions

- Statistically independent sources
- At most one Gaussian source
- Linear, instantaneous mixing; sensors ≥ sources

Ambiguities

- Order, sign and scale are arbitrary
- Match components to physics after fitting

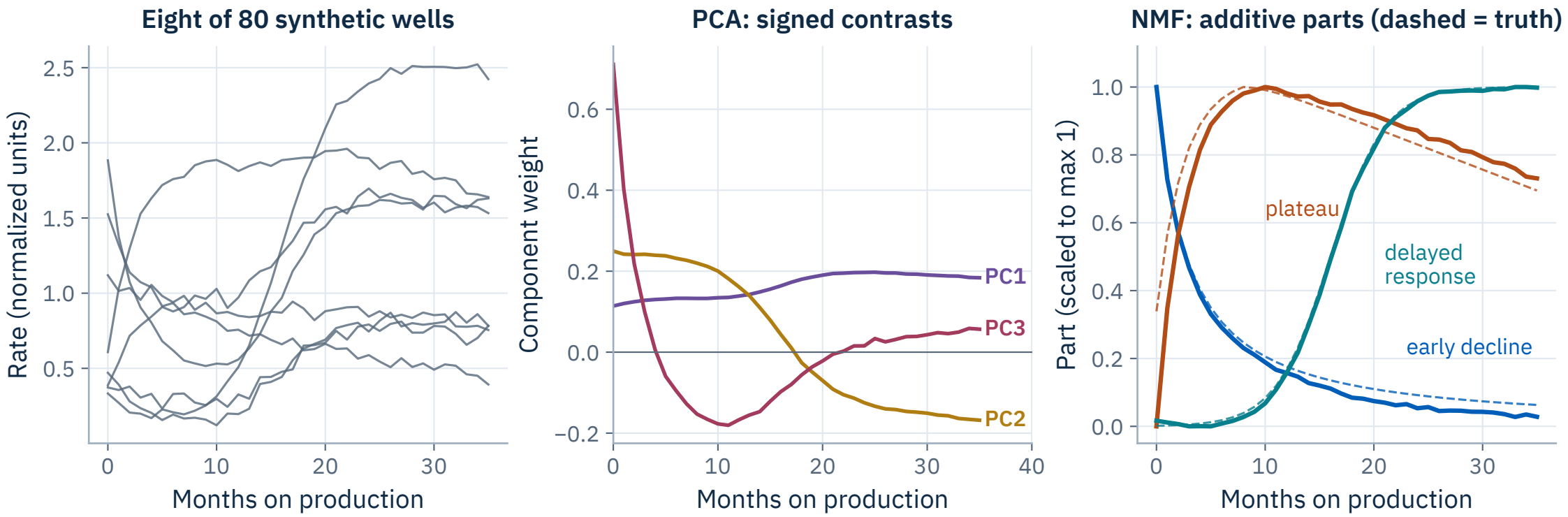
Energy uses

- Separating overlapping vibration or acoustic sources
- Removing an interference shared by many sensors

◆ FastICA whitens the data first: PCA is its first step.

NMF builds profiles from additive parts

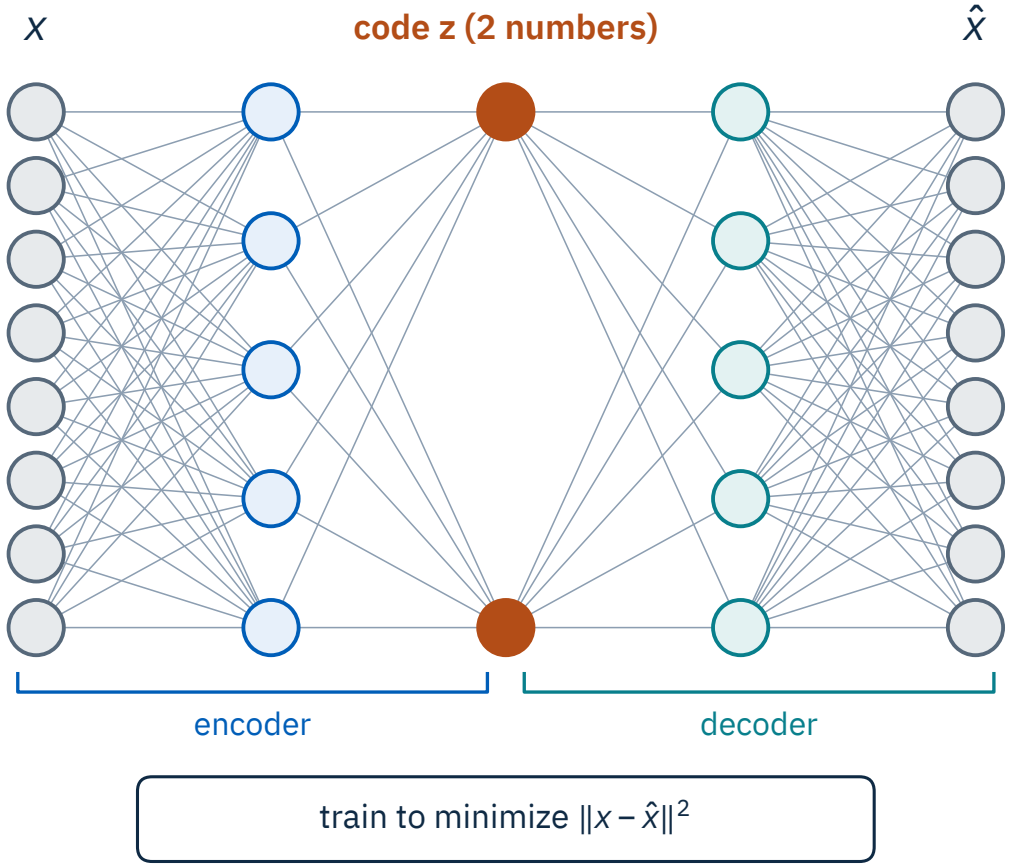
Non-negative data (rates, spectra, concentrations) give non-negative parts and weights.



Synthetic production profiles (seed 9); NMF with k = 3 (seed 0); PCA with k = 3 shown for contrast.

Autoencoders learn a nonlinear bottleneck

- Encoder: $x \rightarrow$ short code z; decoder: $z \rightarrow \hat{x}$
- Linear layers with squared error span the same subspace as PCA
- Nonlinear layers can follow curved structure, at a cost in data and care
- Reconstruction error again serves as an anomaly score



Schematic. Eight inputs, a two-number code and eight outputs.

PRACTICE

5 min · Pairs

Each goal points to a different method

Name one method for each goal, and one check you would run:

1. Summarize 20 correlated rig channels for storage within an error budget
2. A 2-D view that best separates three labelled facies for a report
3. Unmix pump and motor vibration recorded by three accelerometers
4. Describe non-negative production profiles as a few typical shapes
5. One unsupervised score that separates two concentric groups in a cross-plot

PART 5 OF 6

05

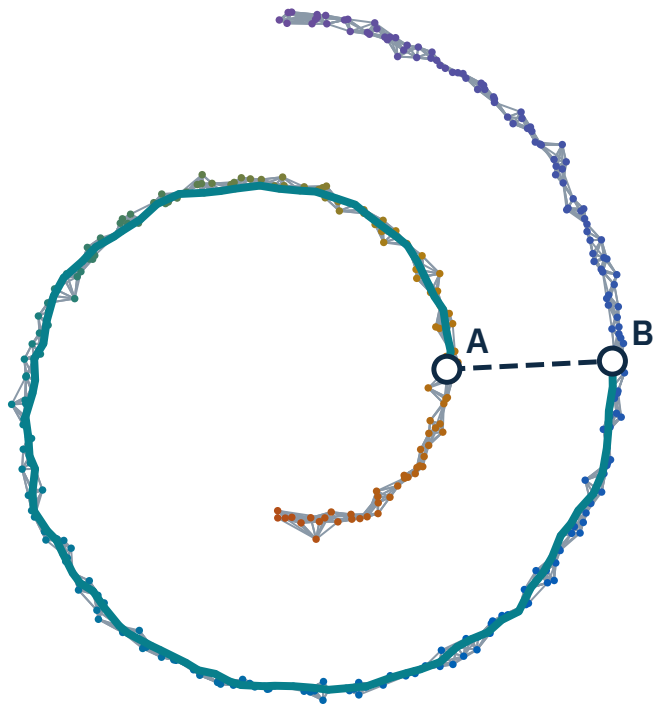
Manifold learning

An embedding is a view of relationships, not a physical map.

- Geodesic distances: Isomap and LLE
- t-SNE: neighbour probabilities
- UMAP: neighbourhood graphs
- Trustworthiness, seeds and batch effects

Curved data need distances along the surface

- A and B are 6.2 apart in a straight line, 60.3 along the data
- A k-nearest-neighbour graph approximates the surface
- Shortest paths on the graph estimate geodesic distance
- Isomap then embeds these distances with classical MDS



grey lines: graph of 7 nearest neighbours

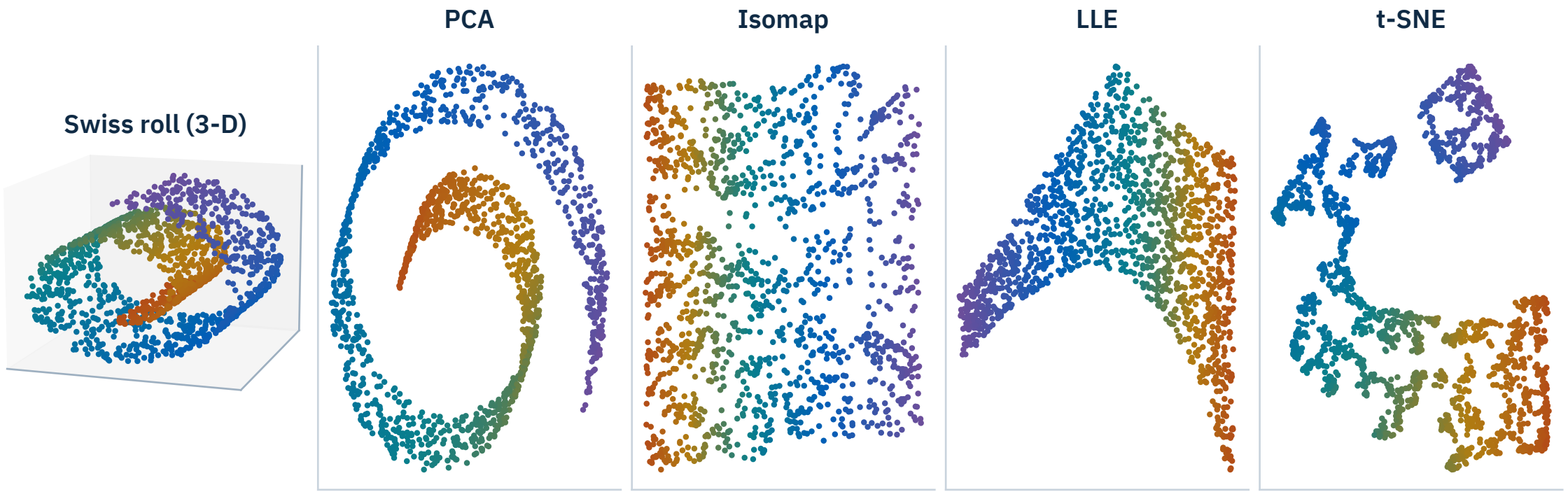
straight line A–B: 6.2

path along the graph: 60.3

Synthetic noisy spiral (seed 0, 400 points); 7-nearest-neighbour graph; scipy shortest path. Colour = position along the spiral.

Methods unroll the Swiss roll differently

Colour shows position along the roll; it is never used in fitting.



scikit-learn make_swiss_roll (1,500 points, noise 0.05, seed 0); Isomap k = 10, LLE k = 12, t-SNE perplexity 30.

Isomap and LLE keep different local facts

Isomap

- Neighbour graph, then geodesic distances
- Classical MDS on those distances
- Keeps global shape; one short-circuit edge can fold it

vs

LLE

- Rebuild each point from its neighbours
- Keep those weights in the 2-D map
- Keeps local geometry; global shape can distort

◆ Both hinge on n_neighbors: too small breaks the graph, too large short-circuits the roll.

t-SNE turns distances into neighbour probabilities

Four ideas explain every t-SNE map, and what it cannot show.

1

Similarities

Gaussian neighbourhoods in the input space

2

Heavy tail

Student-t similarities in the map

3

Optimize

move points until the two agree

4

Check

perplexity, initialization and seed

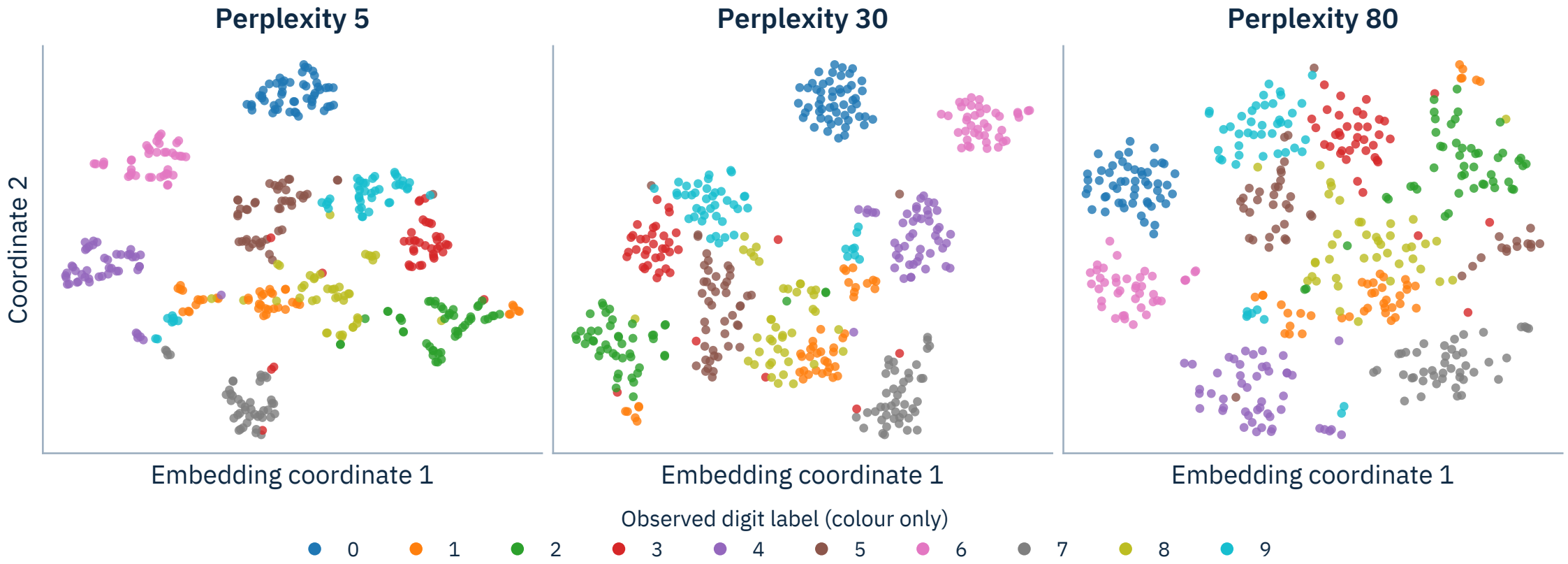
◆ Perplexity is roughly the effective number of neighbours, and must be smaller than the number of observations.

The t-SNE objective is asymmetric

$$\mathrm{KL}(P\|Q) = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

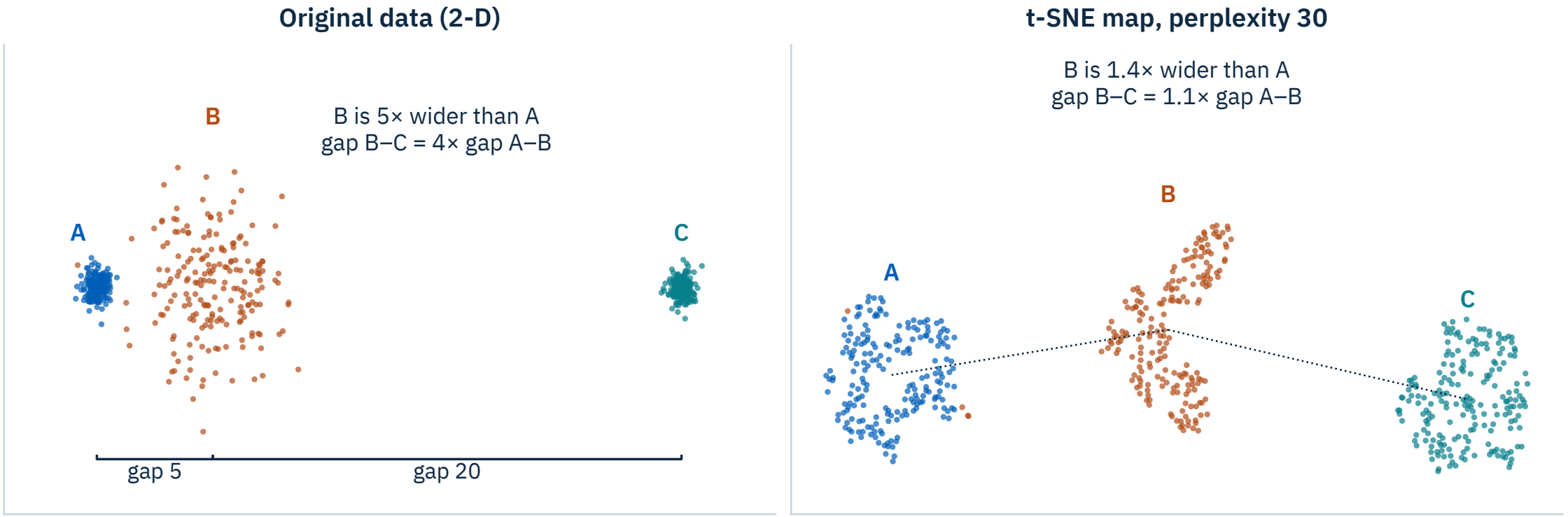
- P describes high-dimensional similarities
- Q describes similarities in the embedding
- Missing an important original neighbour incurs a strong penalty

Perplexity changes the map



t-SNE fits on the same standardized digit subset with a fixed seed. Colours show known digit labels for inspection only.

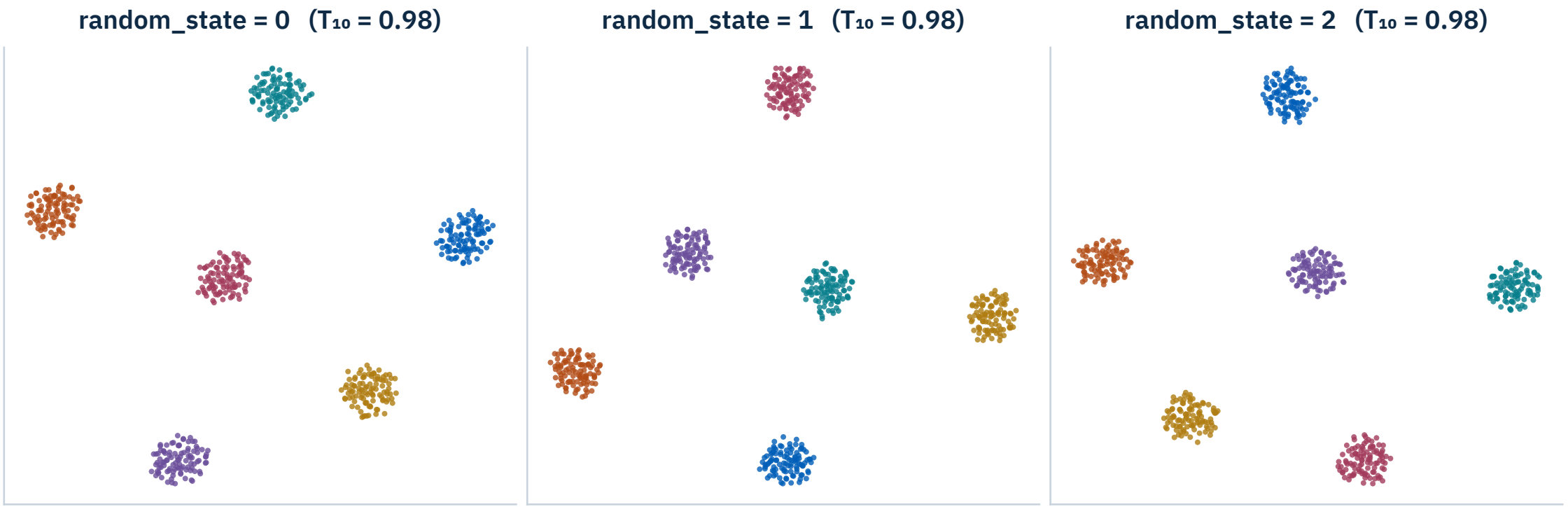
Island sizes and gaps in t-SNE are not measurements



Synthetic 2-D clusters (seed 0): spreads 0.3, 1.5 and 0.3 per axis, centres at 0, 5 and 25; t-SNE seed 0. Spread = RMS distance to the cluster centre.

Random initialization changes the layout, not the neighbours

Same data and perplexity; only random_state differs. Trustworthiness T_{10} is 0.98 in all three.



Six synthetic 10-D Gaussian clusters (make_blobs seed 11); t-SNE perplexity 30, init="random". Colours show generating clusters, not used in fitting.

One t-SNE map supports some claims, not others

✓ Supported by one map

- These points are neighbours in this fit
- Labels tend to occupy nearby areas
- A candidate group is worth checking

vs

✗ Not supported by one map

- Axis values or directions carry units
- Island area or density matches the data
- Gap length measures dissimilarity

◆ Test suspected groups back in the original feature space.

Current t-SNE code uses max_iter, not n_iter

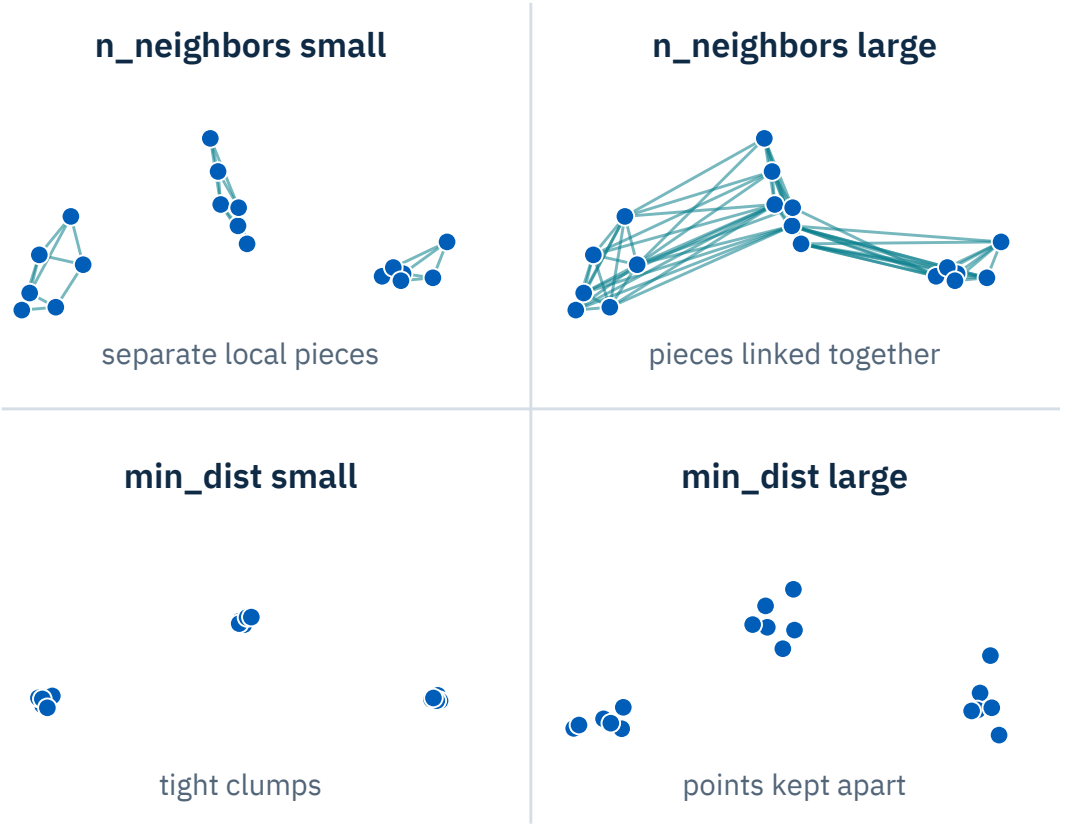
```
from sklearn.manifold import TSNE

embedding = TSNE(
    n_components=2, perplexity=30,
    init="pca", learning_rate="auto",
    max_iter=1000, random_state=42
).fit_transform(Z)
```

Z is an explicitly prepared feature matrix with more than 30 rows.

UMAP starts from a neighbourhood graph

- Build weighted local relationships in the input space
- Optimize a layout that approximates those relationships
- n_neighbors sets the neighbourhood scale of the graph
- min_dist sets how tightly points may pack in the map



Schematic, not UMAP output; umap-learn is an optional package (requirements-labs-optional.txt).

Graph cross-entropy balances attraction and separation

$$-\sum_{i<j}[v_{ij}\log w_{ij}+(1-v_{ij})\log(1-w_{ij})]$$

- v represents input-graph membership strength
- w represents membership implied by embedded distance
- Practical optimization uses sampling rather than all pairs on every step

UMAP supports an explicit future-data transform

```python
import umap

reducer = umap.UMAP(
    n_neighbors=15, min_dist=0.1,
    metric="euclidean", random_state=42
)
Z_train = reducer.fit_transform(X_train_scaled)
Z_new = reducer.transform(X_new_scaled)
```

Evaluate whether the fitted representation transfers to new wells.

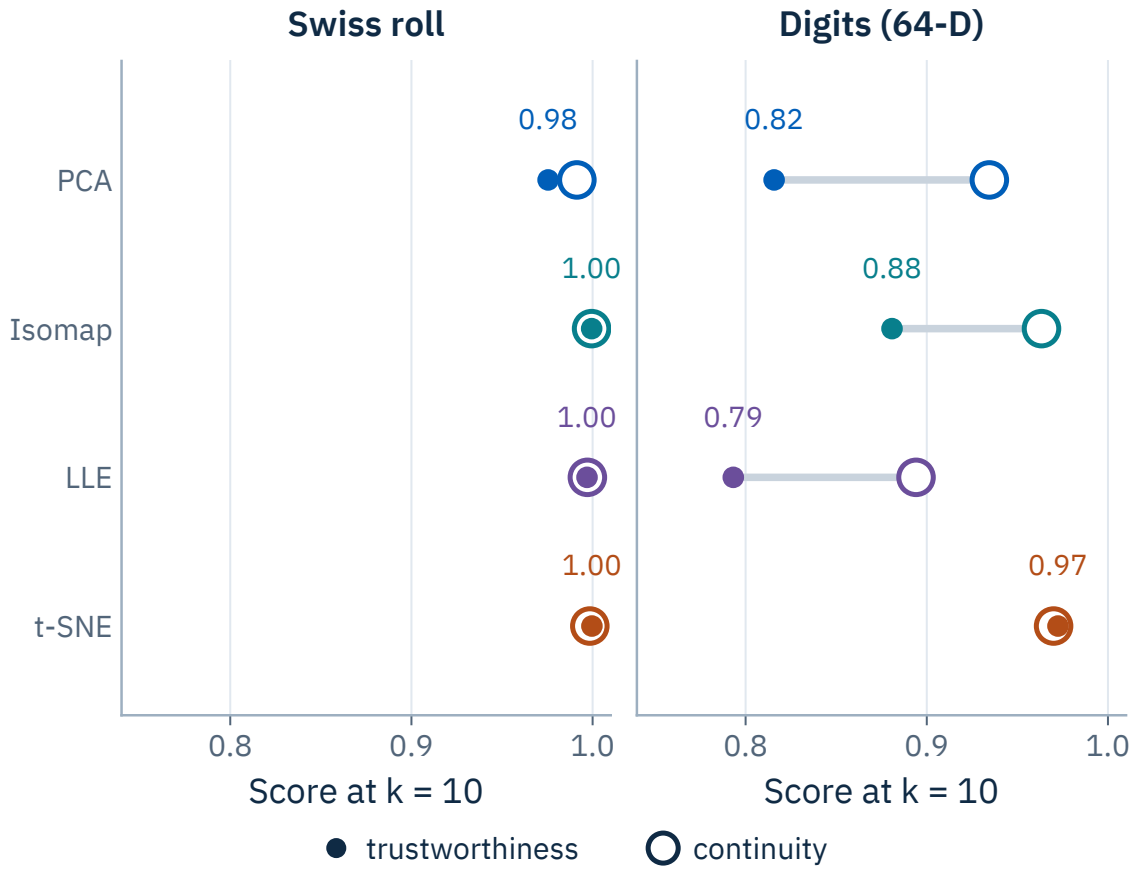
Supervised UMAP changes the evidence

- Supplying labels influences the fitted representation
- Fit it inside training folds when predicting those labels
- Evaluate on separate observations whose labels did not shape the map
- Label a supervised visualization clearly in presentations

⚠ A map shaped by the labels cannot also be the evidence that those labels separate.

Trustworthiness scores how faithful neighbourhoods are

- Trustworthiness: are map neighbours true neighbours?
- Continuity: are true neighbours kept in the map?
- Swiss roll: PCA 0.98, the other methods 0.997 or more
- Digits: t-SNE 0.97, Isomap 0.88, PCA 0.82, LLE 0.79



sklearn.manifold.trustworthiness at k = 10; continuity is the same function with the arguments swapped. Swiss roll seed 0; 600 standardized digit images (seed 42).

A comparison needs a fixed protocol

Dimension	PCA	t-SNE	UMAP
Geometry	Linear variance	Local similarities	Neighbourhood graph
Interpretation	Directions and loadings	Local visual neighbourhoods	Local visual neighbourhoods
Reproducibility	Usually stable up to sign	Check seeds and settings	Check seeds and settings
Validation	Reconstruction or task score	Neighbourhoods and stability	Neighbourhoods, stability, task score

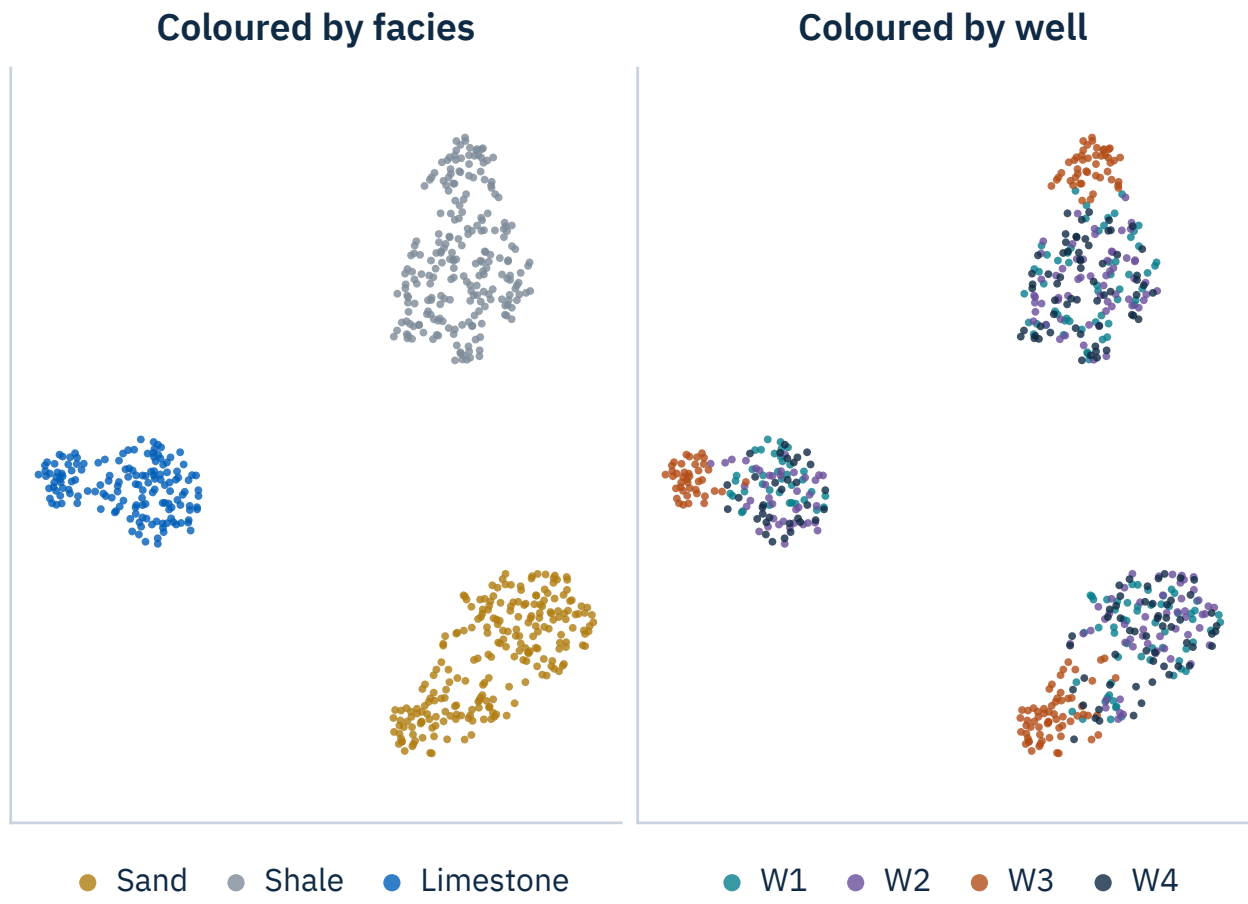
◆ Same rows, same scaling, recorded settings and at least two seeds for every method.

PRACTICE

5 min · Pairs

Colour the map by metadata before trusting it

1. What does the left panel suggest?
2. What does the right panel add?
3. Which check in the original logs comes next?
4. What would you change before clustering?



Synthetic logs from four wells (seed 21); t-SNE on standardized GR, RHOB, NPHI and PEF.

LABORATORY

60 min

Lab 2: nonlinear maps and a fair comparison

- Route A (02_tsne): production data; perplexity 5 to 50; two seeds with init="random"
- Route B (04_comparison): PCA and t-SNE on petro.csv under one protocol
- Add `trustworthiness(X, emb, n_neighbors=10)` to every map
- Colour by metadata after fitting; write one supported and one unsupported claim
- UMAP (03_umap) is optional: umap-learn is in requirements-labs-optional.txt

NOTEBOOK

`exercises/02_tsne/tSNE_Production_Student.ipynb`

EVIDENCE TO SUBMIT

- Maps at two perplexities and two seeds, each with its trustworthiness
- One supported conclusion, one unsupported interpretation, one next check

DISCUSSION

Lab 2 debrief: maps differ more than their scores do

Values computed from the course files with the notebooks' settings:

0.91 vs 0.70

petro.csv trustworthiness (k = 10): t-SNE vs PCA

0.97–0.98

production t-SNE trustworthiness at perplexity 5 to 50

0.31 vs 0.06

facies silhouette: log_0 and log_1 only vs all ten logs

- Pictures changed a lot with settings; neighbourhood scores barely moved. The facies signal lives in two logs.

PART 6 OF 6

06

Reduction in practice

Choose by goal; check results in original units.

- A goal-to-method guide
- Drilling: compression error per channel
- Logs: reconstruction error for quality control
- Production: shape versus amplitude

Choose the method from the goal

Goal	First choice	Evidence to report
Keep original sensors	Filter, wrapper or embedded selection	Selection frequency across folds
Predict a target	Selection or PCA inside the pipeline	Held-out score vs. no reduction
Compress or QC	PCA, k by held-out error	Error per channel in original units
Visualize neighbourhoods	t-SNE or UMAP, with PCA alongside	Trustworthiness, seeds, metadata colours
Unmix or find parts	ICA or NMF	Physical plausibility of components

Compression error must be read channel by channel

- With k = 3, most channels keep a quarter to a third of their SD as error
- Gas is lost completely at k = 3 (8.5 units, its full SD)
- k = 4 recovers gas: the channel that matters may carry little variance

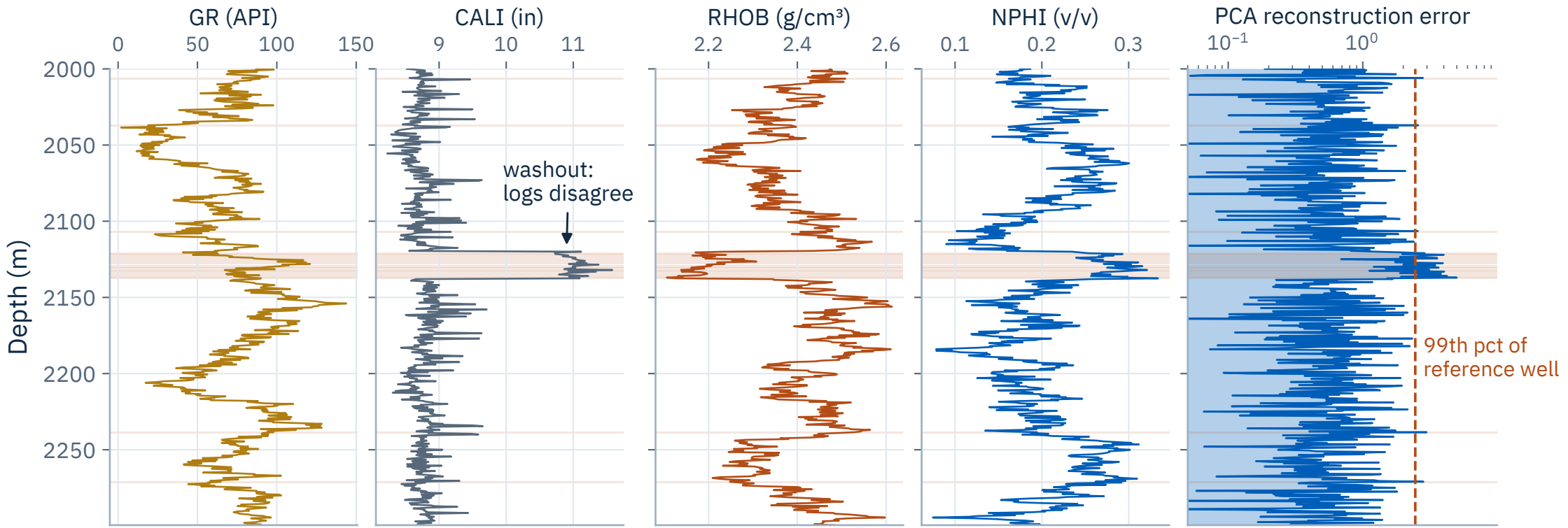
	k = 1	k = 2	k = 3	k = 4	k = 5
WOB (klbf)	1.8	1.6	1.3	1.3	1.3
Torque (kft·lbf)	1.9	1.9	0.8	0.8	0.8
RPM (rpm)	24	25	4.8	3.1	3.1
ROP (ft/h)	11	11	3.9	3.7	3.4
SPP (psi)	242	95	62	56	51
Flow (gpm)	55	20	16	14	14
ECD (ppg)	0.10	0.04	0.03	0.03	0.02
DiffP (psi)	50	33	25	25	16
Hookload (klbf)	2.4	2.2	2.0	2.0	1.9
Gas (units)	8.2	8.3	8.5	0.08	0.07

Validation RMSE in engineering units (shade = RMSE / channel SD)

Synthetic drilling channels (seed 4); PCA fitted on standardized minutes 0–429; validation RMSE on minutes 430–719 in original units.

Reconstruction error screens logs for bad-hole intervals

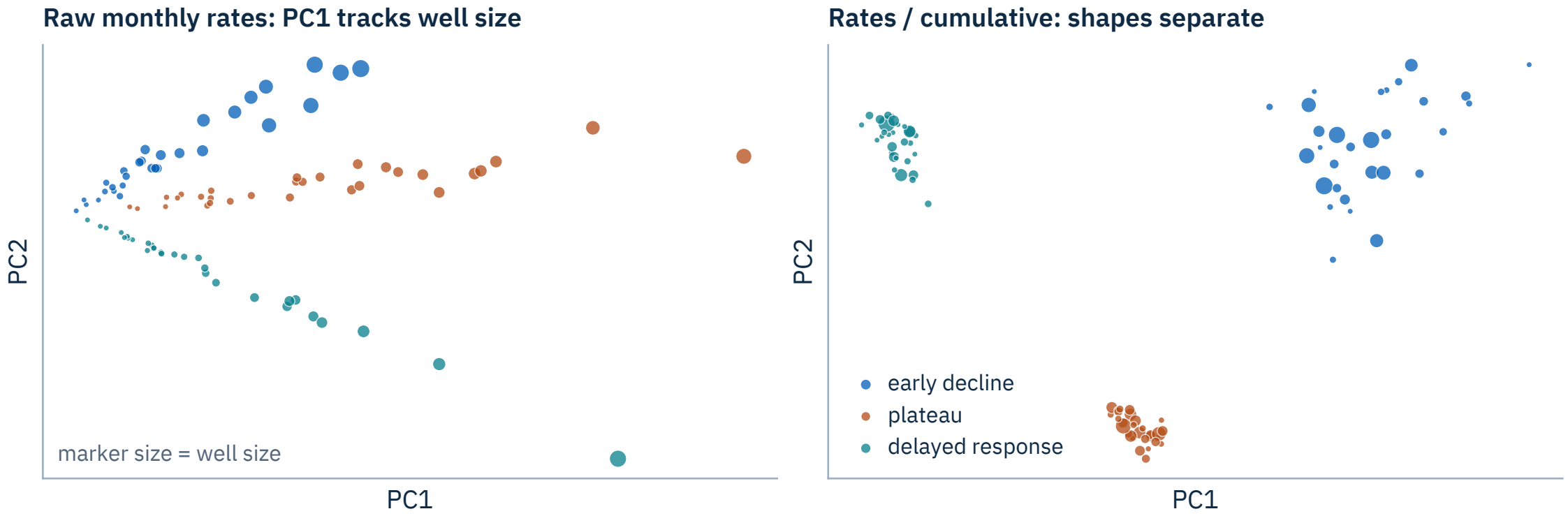
PCA fitted on a reference well; every depth of a new well is scored by its reconstruction error.



Synthetic log suites (reference seed 11, new well seed 23) with an injected washout at 2,120–2,138 m; PCA with k = 4 on the standardized reference well; shading marks depths above the threshold.

Production maps depend on shape versus amplitude

Divide each well's rates by its cumulative before PCA when shape, not size, defines similarity.



Synthetic production profiles for 90 wells (seed 13): three shape families, well sizes varying more than tenfold; colours added after fitting.

REDUCTION IN PRACTICE

Energy applications need original-unit checks

Well-log suites

- Does component structure follow lithology or tool effects?

Seismic attributes

- Do neighbourhoods survive changes in scaling and sampling?

Drilling sensors

- Operating regimes, or sensor outages and calibration drift?

Production profiles

- Does amplitude or shape define similarity?

◆ Colour maps by well, campaign or tool after fitting; never call an island a compartment without evidence.

KEY TAKEAWAYS

Reduction must earn its place in the workflow

- ✓ High dimensions dilute distances; reduce for a stated goal, not by habit.
- ✓ Select features inside cross-validation, and report how stable the choice is.
- ✓ Standardize mixed units; choose k by task score or held-out error, not a fixed 95%.
- ✓ Kernel PCA, LDA, ICA and NMF change the objective, not just the geometry.
- ✓ Read t-SNE and UMAP maps for neighbours only; check trustworthiness, seeds and metadata.
- ✓ Check reconstructions and maps in original units, per channel and per well.

PRACTICE

5 min · Individual

Five questions check today's core ideas

1. Does retaining 95% of variance guarantee retaining target information?
2. Why must SelectKBest sit inside the cross-validation pipeline?
3. Can distances between t-SNE islands be read as physical distances?
4. What must stay fixed when transforming a new well with UMAP?
5. Name one check that a 2-D map's neighbourhoods are trustworthy.

References and further study

- scikit-learn User Guide: Feature selection; Decomposing signals in components (PCA, kernel PCA, ICA, NMF); Manifold learning; Linear and quadratic discriminant analysis.
- Guyon and Elisseeff (2003), An introduction to variable and feature selection, JMLR 3. Hastie, Tibshirani and Friedman, The Elements of Statistical Learning, 2nd ed., sections 2.5 and 7.10.2, chapter 14.
- Jolliffe and Cadima (2016), Principal component analysis: a review and recent developments, Phil. Trans. R. Soc. A 374.
- Tenenbaum, de Silva and Langford (2000), Science 290; Roweis and Saul (2000), Science 290; van der Maaten and Hinton (2008), Visualizing data using t-SNE, JMLR 9; McInnes, Healy and Melville (2018), UMAP, arXiv:1802.03426.
- Hyvärinen and Oja (2000), Independent component analysis, Neural Networks 13; Lee and Seung (1999), Nature 401; Wattenberg, Viégas and Johnson (2016), How to use t-SNE effectively, Distill.

MACHINE LEARNING / MODULE 5

Rare events, anomalies, and model selection

Finding the unusual, judging rare-event evidence, and choosing models without fooling yourself

Full day · 09:00–17:00 | 7 parts | 2 laboratories | Course synthesis

Machine learning for energy applications

Today's plan

09:00	Welcome: rare events in energy operations	Lecture
09:15	Part 1: What counts as an anomaly	Lecture
09:55	Part 2: Detectors that learn normal behaviour	Lecture
10:45	Break	Break
11:00	Lab 1: statistical rules and Isolation Forest	Lab
12:05	Part 3: Evaluating rare events	Lecture
12:45	Lunch	Lunch
13:30	Part 4: Imbalance, calibration, uncertainty	Lecture
14:10	Lab 2: detector comparison and an alert policy	Lab
15:25	Break	Break
15:40	Part 5: Honest and efficient model selection	Lecture
16:05	Exercise: read a tuning log and choose a model	Exercise
16:20	Part 6: Deployment, drift, and monitoring	Lecture
16:40	Part 7: Course synthesis and exit check	Discussion

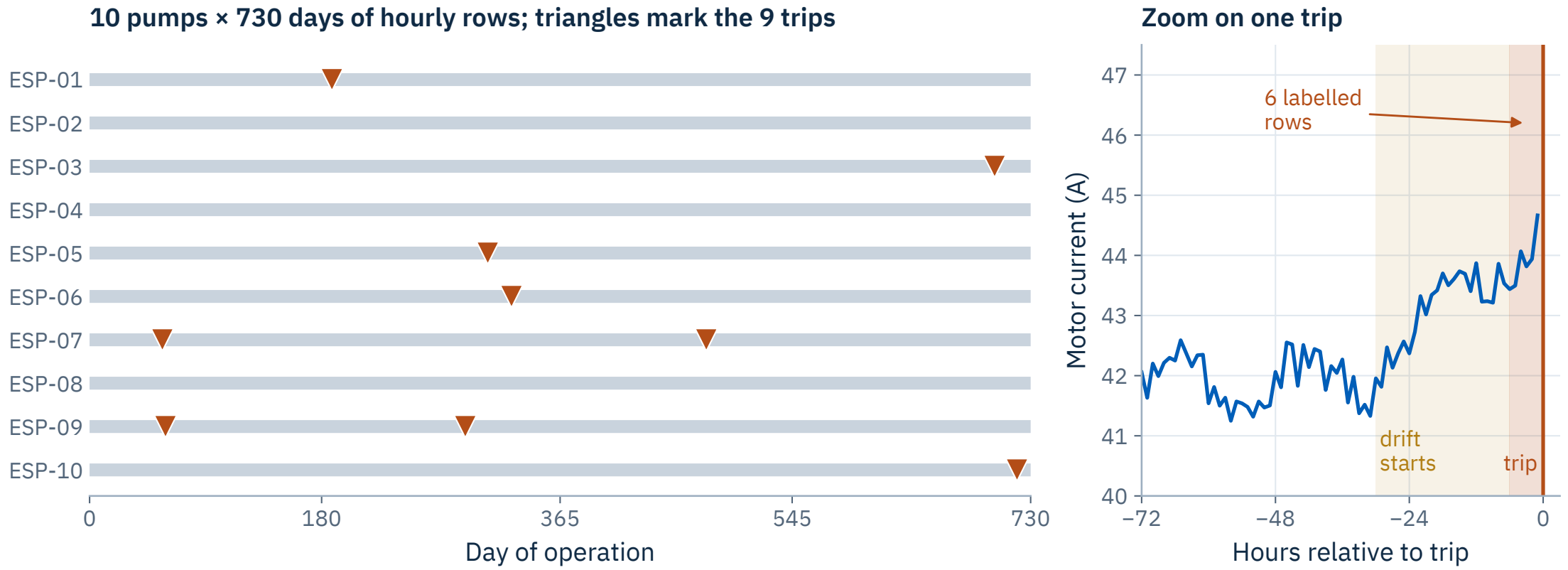
What you will be able to do by 17:00

By the end of the day you will be able to:

1. Match an anomaly problem to its setting and to its type: point, contextual, or collective.
2. Build robust statistical baselines and compare Isolation Forest, LOF, One-Class SVM, and reconstruction error.
3. Evaluate rare-event detectors with precision-recall, event-level metrics, calibration, and honest intervals.
4. Set and document an alert threshold from costs or from review capacity.
5. Run a bounded, leakage-free tuning experiment and compare models with nested CV and paired folds.
6. Plan drift monitoring and connect the full workflow of Modules 1 to 5.

Nine pump trips hide in 175,200 hourly rows

Labelling the 6 hours before each trip gives 54 positive rows: 0.03% of the data.



Synthetic fleet (seed 5): 10 pumps, 730 days, 9 trips. The zoomed current trace is illustrative, not field data.

PRACTICE

3 min · Pairs

Warm-up: judge a detector that never alerts

Same fleet: 175,200 hourly rows, of which 54 are labelled positive (6 per trip).

1. What accuracy does a model that never alerts achieve?
2. How many of the nine trips does it catch?
3. Which two numbers would you ask for instead?

PART 1 OF 7

01

What counts as an anomaly

Unusual is defined by a reference, a context, and a purpose.

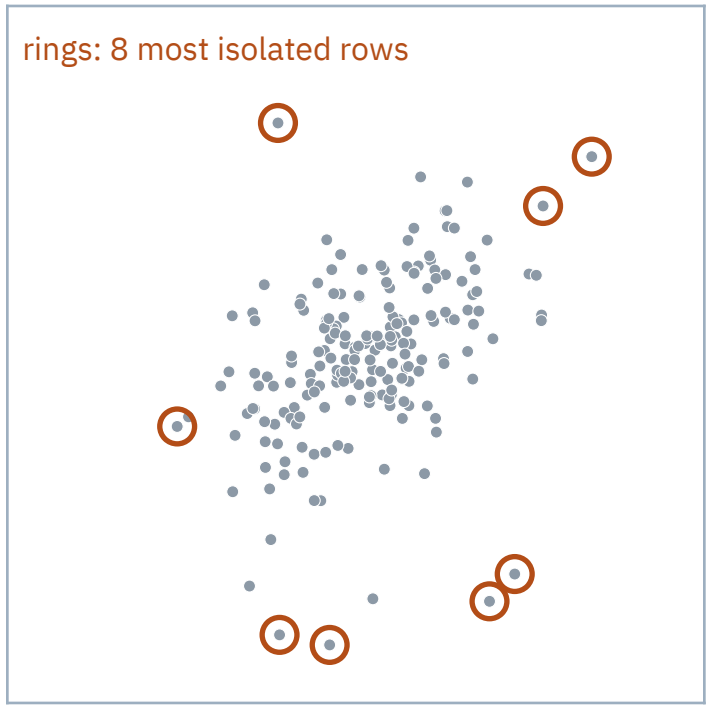
- Three meanings of anomaly and the labels behind them
- Point, contextual, and collective anomalies
- Robust statistical baselines and how they fail
- Physical rules that statistics cannot replace

Anomaly has several meanings

Setting	Training information	Typical question
Outlier detection	Mixed, mostly ordinary observations	Which training observations differ?
Novelty detection	A reference population intended to be normal	Does a new observation depart from normal?
Supervised rare-event prediction	Examples labelled by event outcome	How likely is the labelled event?

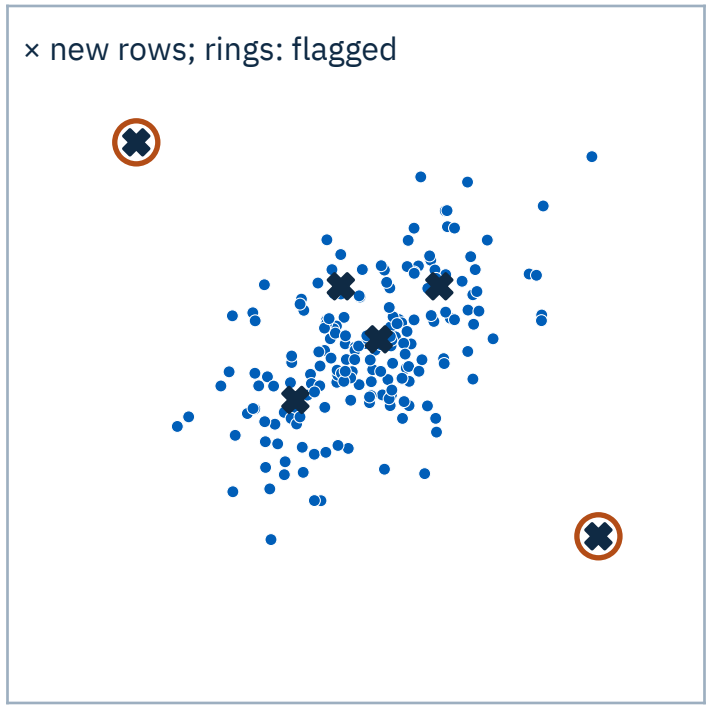
Available labels decide the learning setting

Outlier detection



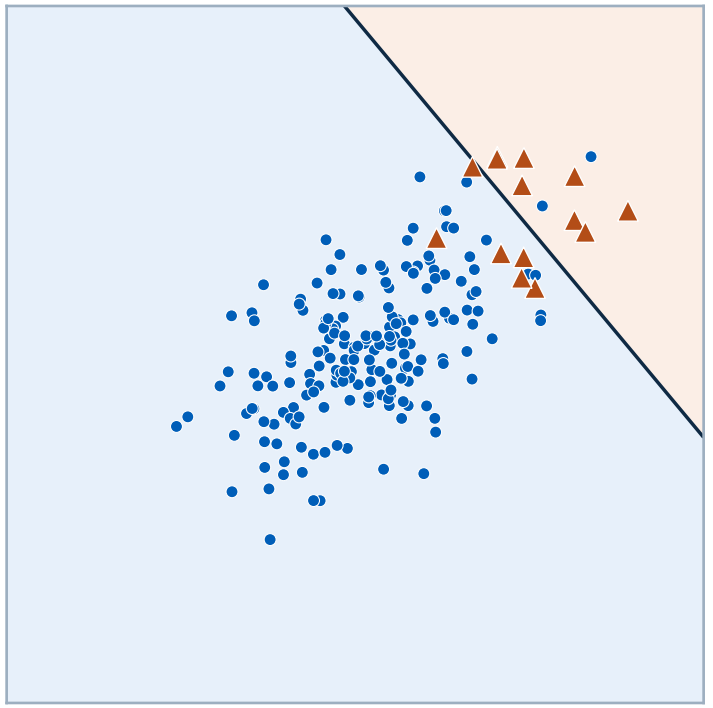
Training labels: none
(the data are contaminated)

Novelty detection



Training labels: normal only
(new rows scored later)

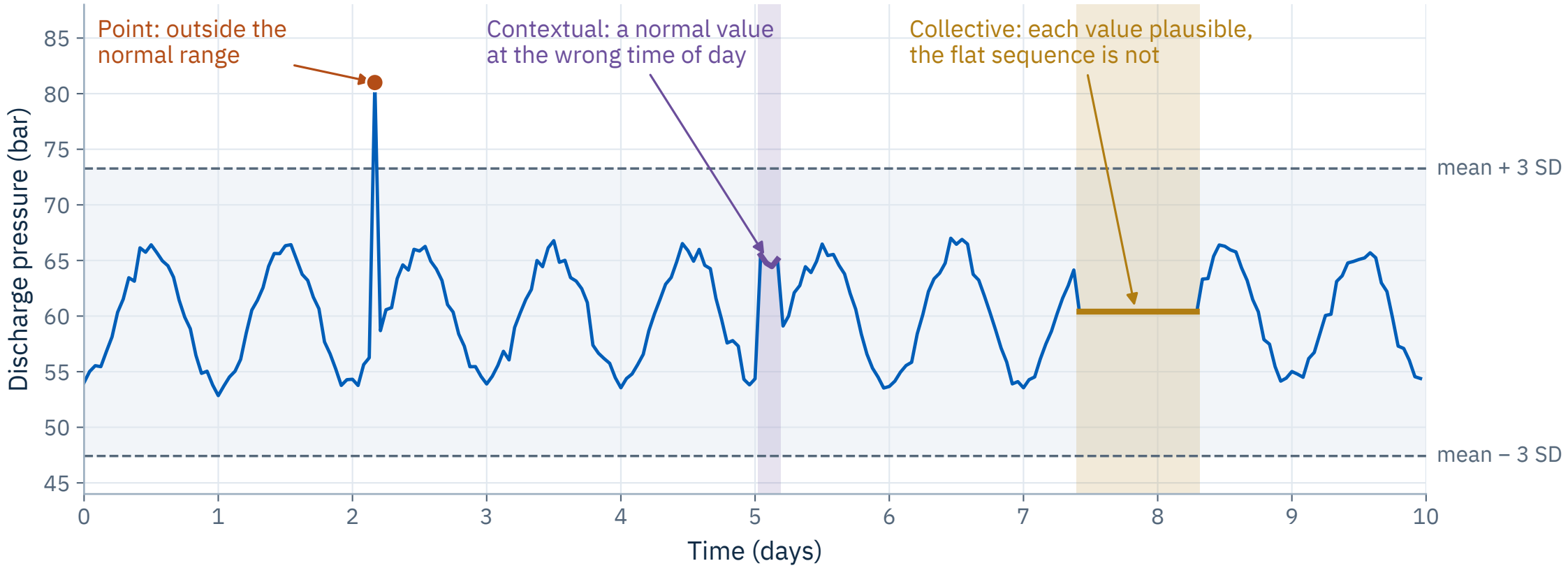
Supervised rare events



Training labels: normal and events
(14 events among 204 rows)

Synthetic 2-D data (seed 7). Left: Isolation Forest ranks a contaminated set. Middle: novelty LOF scores new rows against a clean reference. Right: logistic regression on labelled events.

Anomalies come as points, contexts, or collectives



Synthetic hourly discharge pressure (seed 11) with an injected spike, four peak-level hours at trough time, and a 22-hour flat segment.

Unusual is not the same as faulty

Rare is a property of the data; faulty is a property of the equipment.

What an anomaly score says

- This row is rare relative to the reference data
- A ranking under one model's idea of normal
- Depends on features, scaling, and settings

vs

What a fault diagnosis needs

- A physical mechanism and an engineering review
- A confirmed outcome, with its timing
- Evidence that the pattern recurs across assets

◆ Report detections as candidates for review, not as faults.

Statistical rules provide a transparent baseline

Start every anomaly project with rules you can explain line by line:

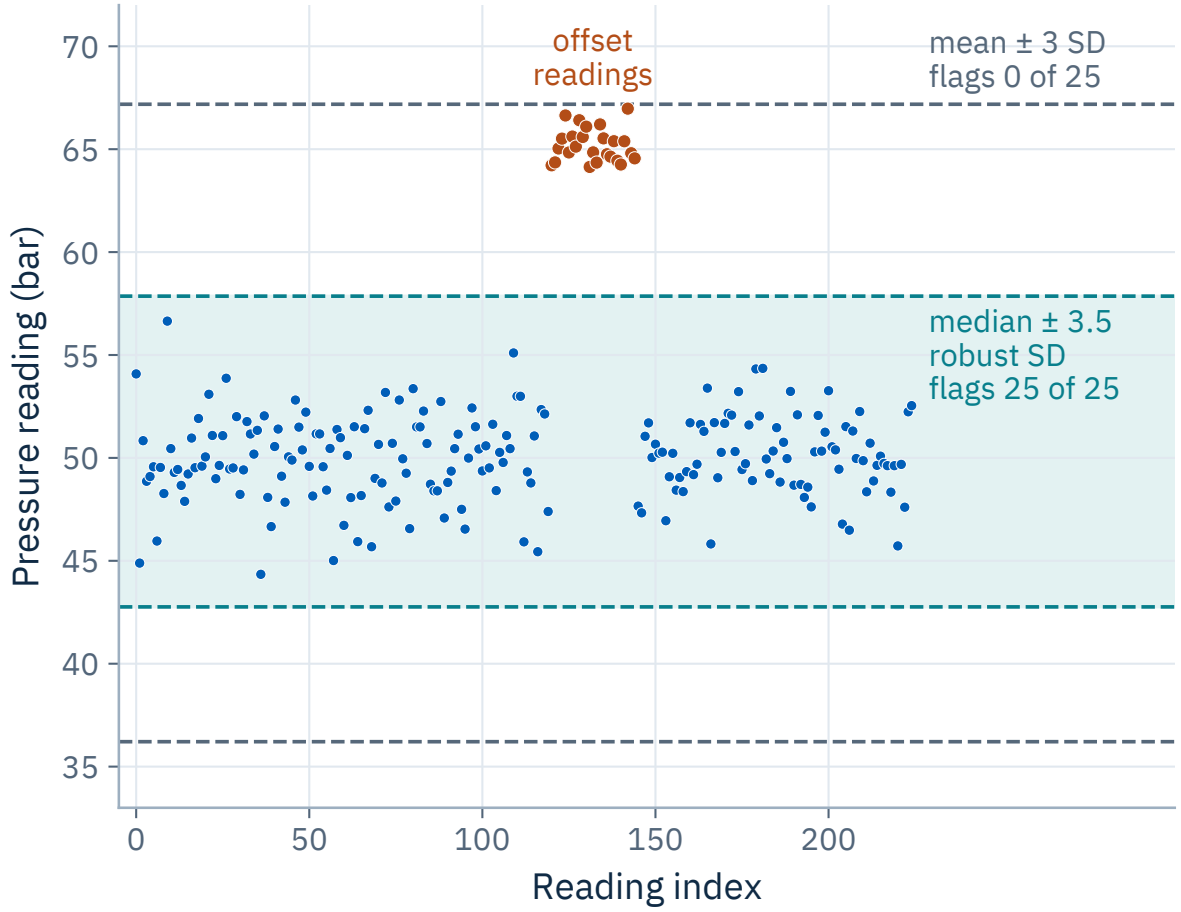
1. **Plot first** — each variable over time or depth, coloured by operating regime
2. **Describe robustly** — median and MAD per regime, not one global mean and SD
3. **Write physical rules** — ranges, rates of change, flatlines, channel consistency
4. **Flag and rank** — robust z beyond 3.5, fences per facies, rule violations
5. **Review before naming faults** — an engineer labels flagged and unflagged samples

◆ Treat every rule as an explicit, reviewable assumption.

Outliers inflate the statistics meant to catch them

$$z = \frac{x - \bar{x}}{s} \qquad z_{\text{robust}} = \frac{x - \text{median}}{1.4826\,\text{MAD}}$$

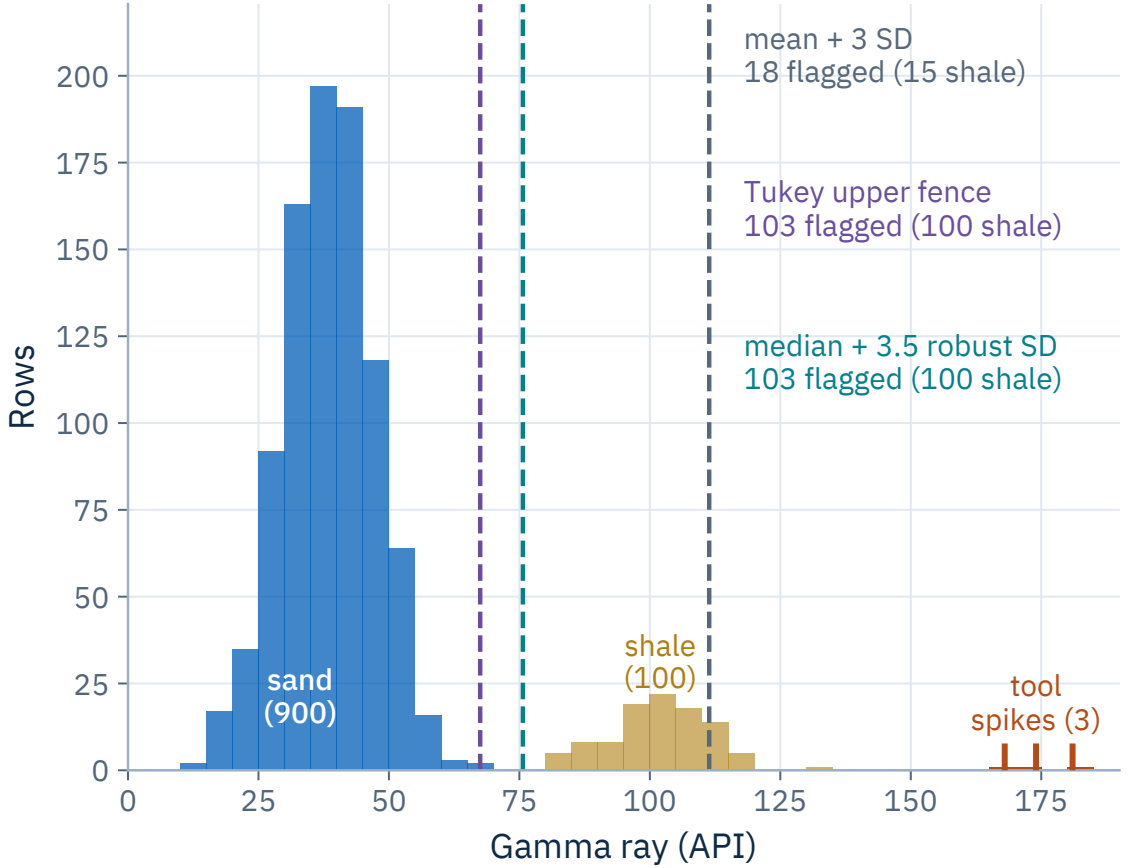
- 25 offset readings raise the SD from about 2 to 5.2 bar
- Classical |z| > 3 flags none of them: masking
- Median and MAD barely move; robust |z| > 3.5 flags all 25



Synthetic readings (seed 3): 200 ordinary N(50, 2) and 25 offset N(65, 0.8).

Univariate fences assume a single population

- Tukey fences: below Q1 – 1.5 IQR or above Q3 + 1.5 IQR
- A 10% shale population sits beyond both robust fences
- 100 of the 103 robust flags are valid shale, not tool error
- Set fences per facies or regime, or use multivariate methods

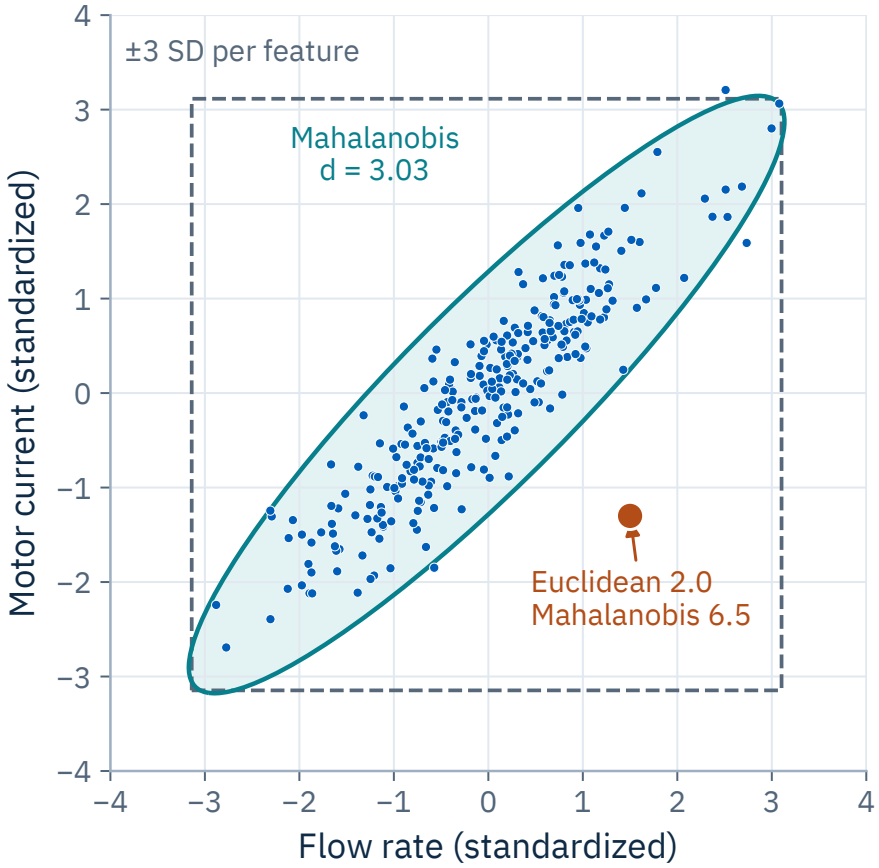


Synthetic GR-like mixture (seed 21): 900 sand rows N(38, 9), 100 shale rows N(102, 9), three tool spikes.

Joint outliers hide inside univariate limits

$$d_M(x) = \sqrt{(x-\mu)^T \Sigma^{-1} (x-\mu)}$$

- Each value of the orange point is within ±3 SD of its feature
- It breaks the flow–current relationship: $d_M = 6.5$
- For Gaussian data with two features, the 99% cut-off is 3.03
- Estimate μ and Σ robustly so outliers cannot mask themselves



Synthetic correlated features (seed 4): correlation 0.9, 300 rows; covariance estimated from the sample.

Physical rules catch what statistics cannot

Range

- Values beyond physical or tool limits
- e.g. negative flow, NPHI above 1

Rate of change

- Jumps faster than the process allows
- Spikes and dropouts in one curve

Flatline

- Stuck sensors repeat one value
- Rolling variance near zero

Consistency

- Related channels must agree
- e.g. caliper vs bit size, flow balance

◆ Rules encode engineering knowledge; keep them separate from learned scores.

PRACTICE

5 min · Individual

Compute a robust z-score by hand

Seven intake-pressure readings (bar): 48, 49, 50, 50, 51, 52, 71.

1. The mean is 53.0 and the SD is 8.04. Is 71 beyond $|z| = 3$?
2. Compute the median, the MAD, and the robust z of 71.
3. Can any reading in a sample of seven exceed $|z| = 3$?

PART 2 OF 7

02

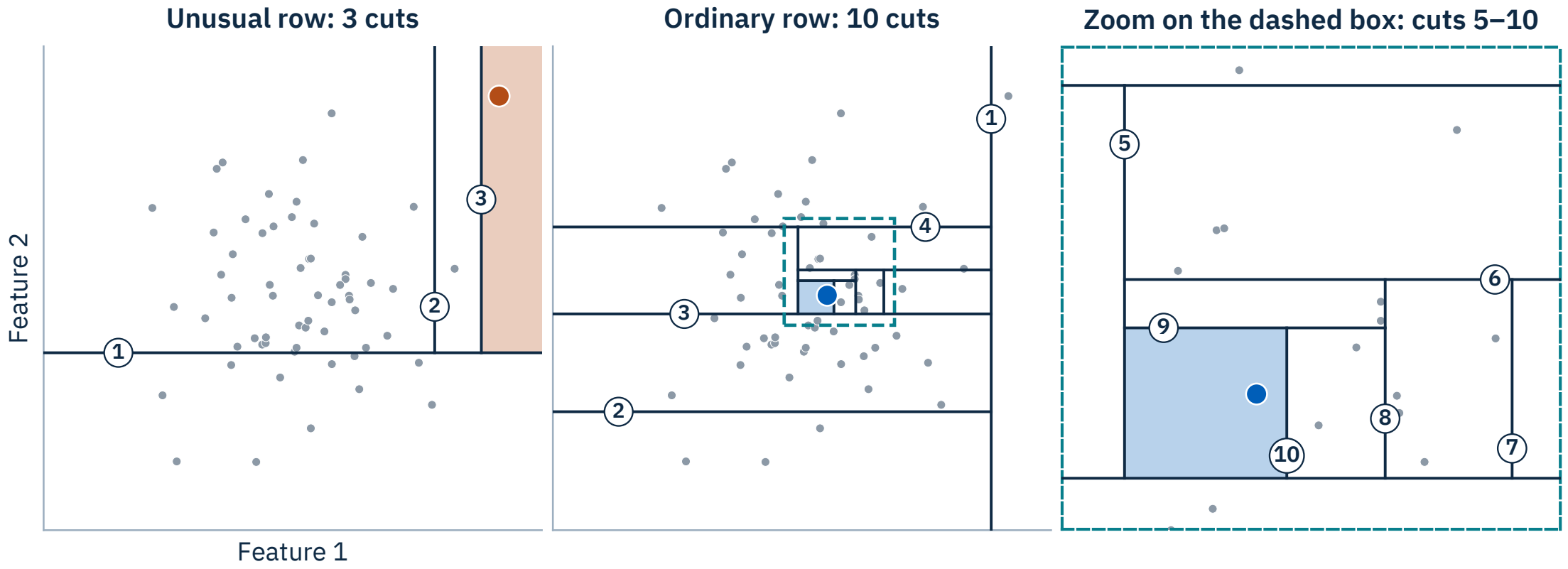
Detectors that learn normal behaviour

Each detector encodes a different idea of "far from normal".

- Isolation: how quickly random cuts separate a row
- Local density: LOF and the neighbour count
- Boundaries: One-Class SVM
- Reconstruction and forecast residuals for correlated and time-series data

Random cuts isolate unusual points in few steps

Each cut: a random feature, and a random value inside the current cell's range.

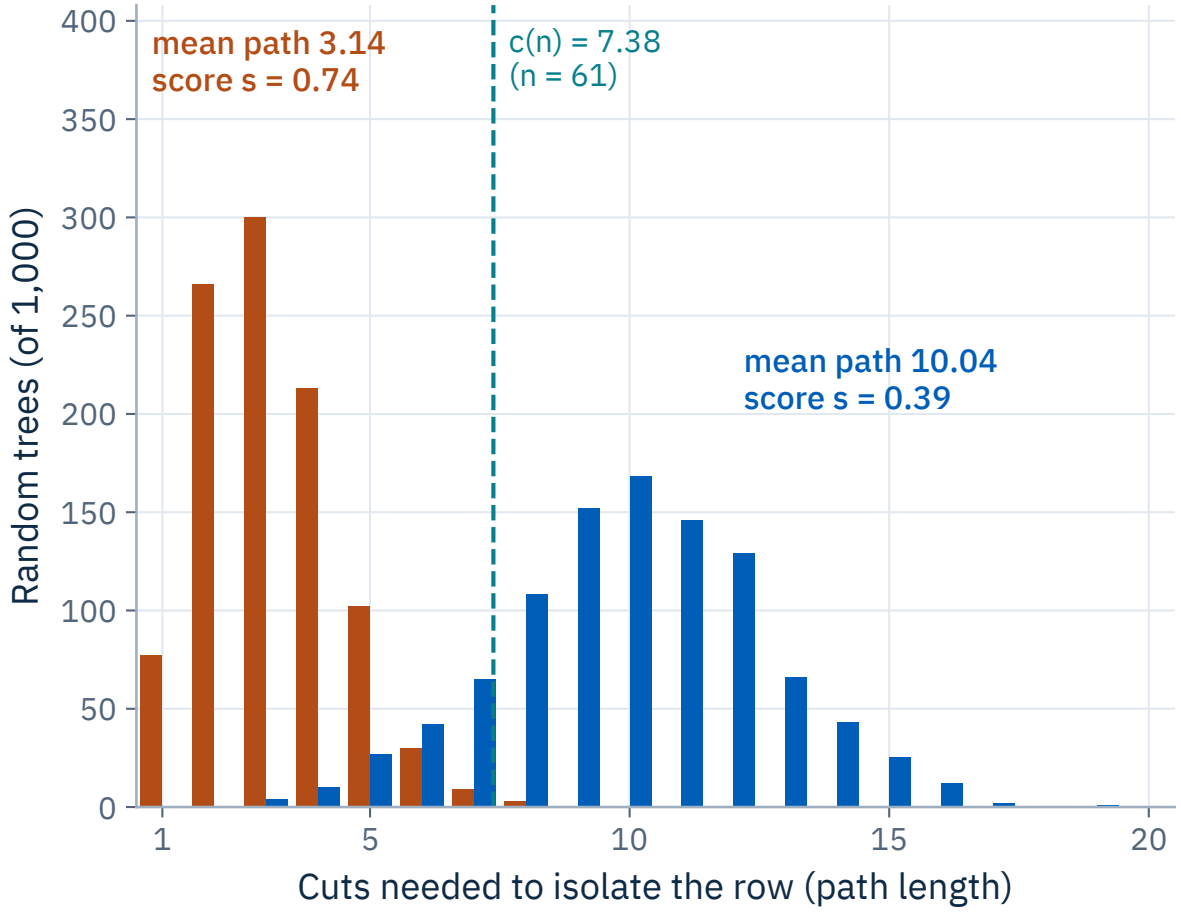


Synthetic 2-D sample (seed 2): 60 ordinary rows and one unusual row; one random isolation path for each highlighted row.

Isolation Forest turns short paths into high scores

$$s(x, n) = 2^{-E[h(x)]/c(n)}$$

- Average the path length $h(x)$ over many random trees
- $c(n)$ normalizes by the average path for n rows
- s near 1: isolated quickly; well below 0.5: ordinary
- Investigate ranked rows; validate any alert threshold

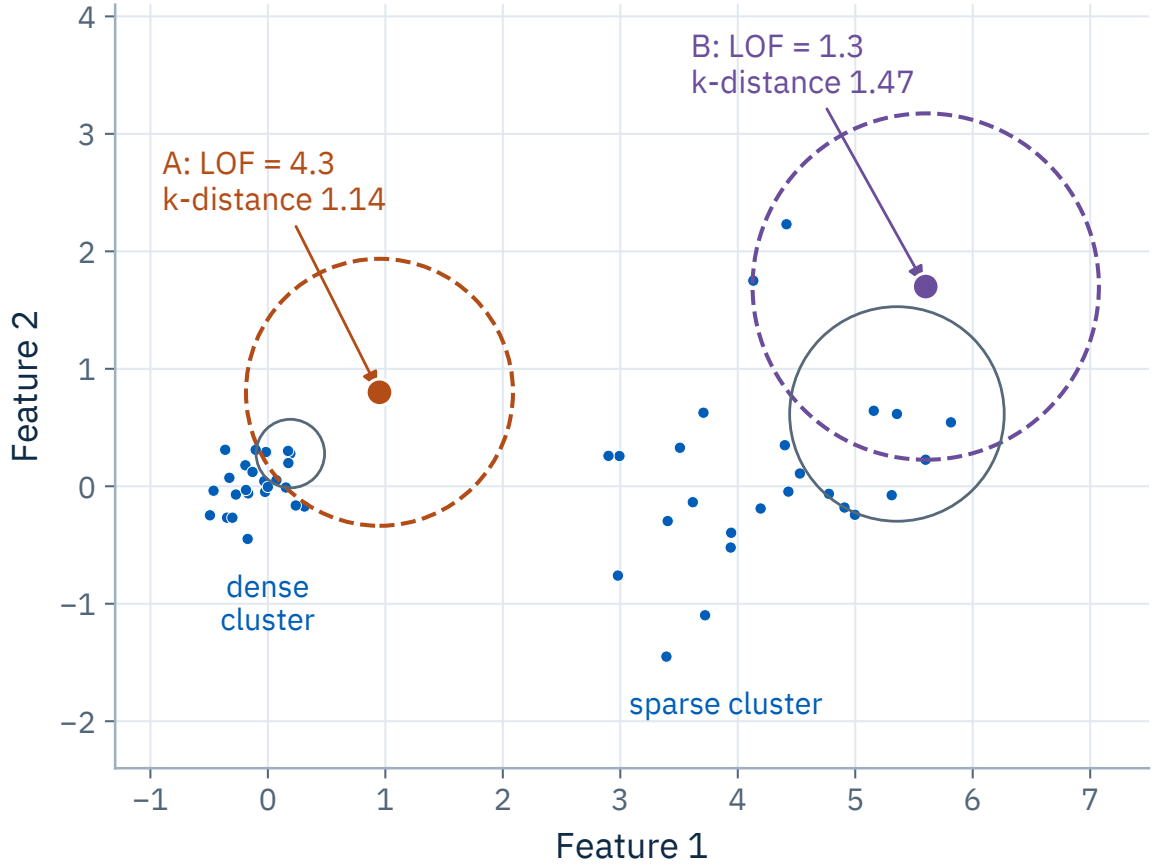


c(61) = 7.38 for the 61-row sample; 1,000 random paths per highlighted row.

Local Outlier Factor compares neighbourhoods

$$\mathrm{LOF}_k(p) = \frac{1}{k} \sum_{o \in N_k(p)} \frac{\mathrm{lrd}_k(o)}{\mathrm{lrd}_k(p)}$$

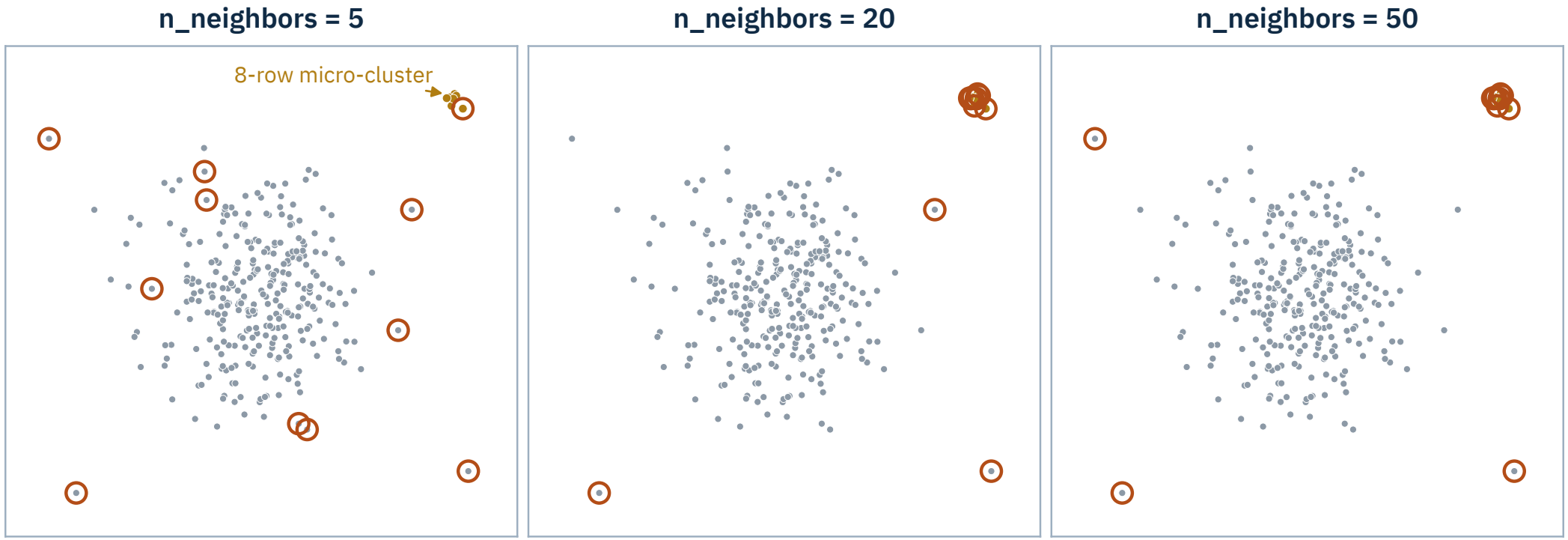
- lrd: inverse of the mean reachability distance to the k neighbours
- LOF near 1: as dense as its neighbours; well above 1: sparser
- A is closer to its neighbours than B, yet A is the local outlier
- Scaling and the neighbour count change the result



Synthetic clusters (seed 8); LocalOutlierFactor with n_neighbors = 6. Dashed circles: k-distances of A and B; grey circles: one neighbour of each.

The neighbour count decides what counts as local

Set n_neighbors above the size of any group of rows that should still count as unusual.

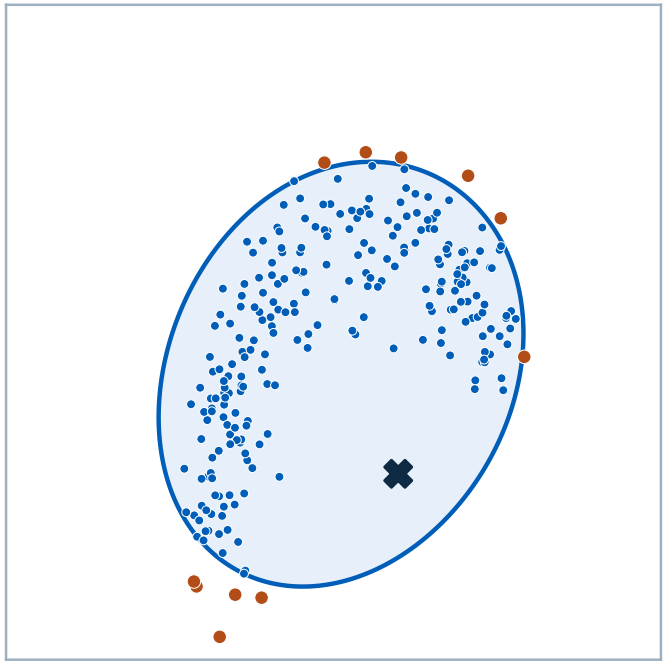


Synthetic data (seed 12): 300 ordinary rows, an 8-row micro-cluster, and 3 isolated rows; rings mark the 11 highest LOF scores.

One-Class SVM estimates a boundary

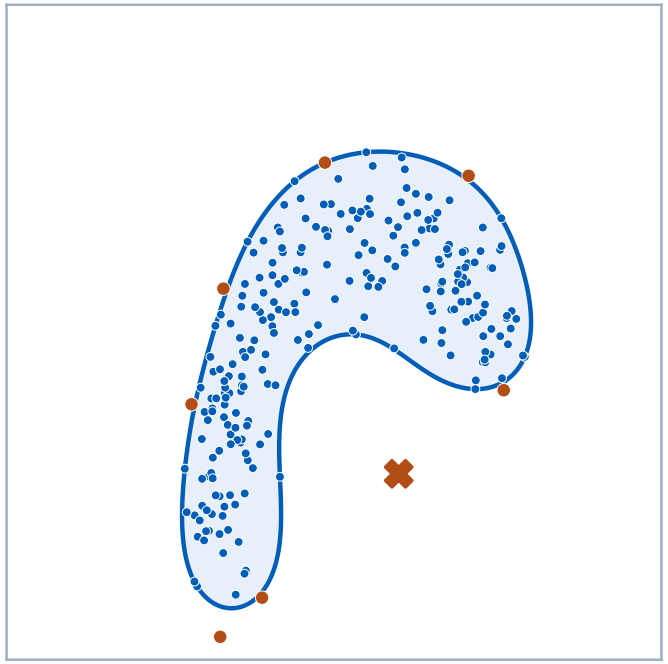
nu caps the share of training rows outside; gamma sets how tightly it wraps.

gamma = 0.05



new ordinary rows flagged: 7.9%
interior probe (X) flagged: no

gamma = 0.5



new ordinary rows flagged: 8.6%
interior probe (X) flagged: yes

gamma = 5

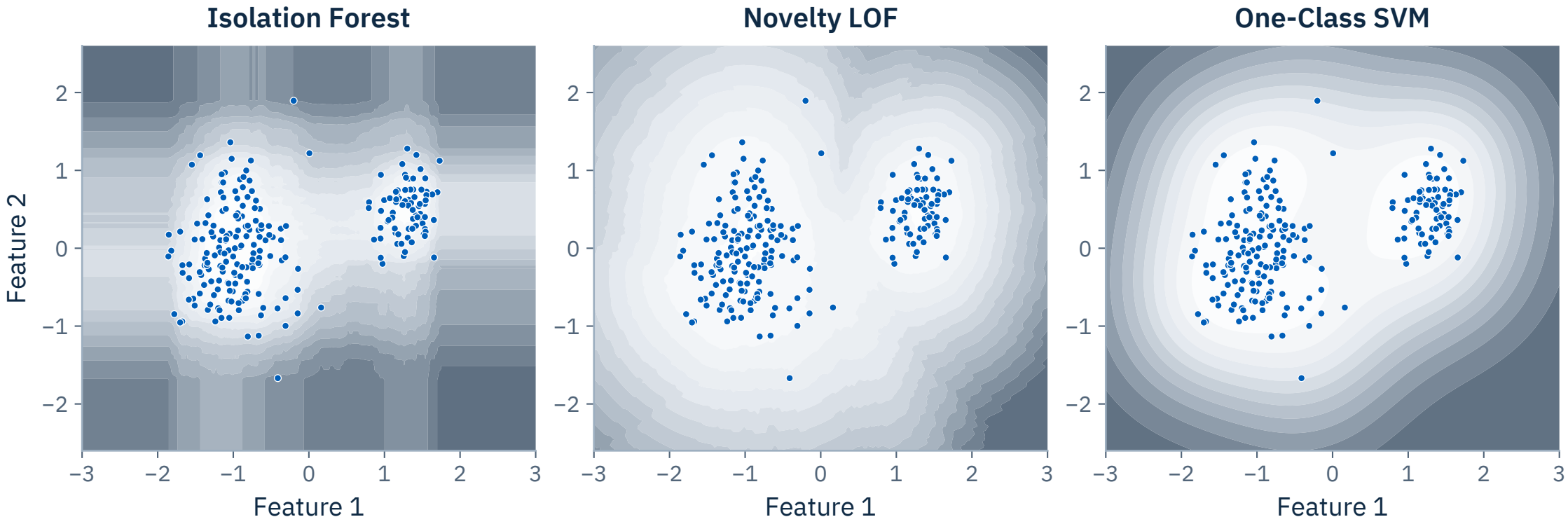


new ordinary rows flagged: 23.8%
interior probe (X) flagged: yes

Synthetic arc-shaped reference data (seed 6, standardized); RBF kernel, nu = 0.05. Flag rates measured on 2,000 new rows from the same generator.

Methods rank unusual observations differently

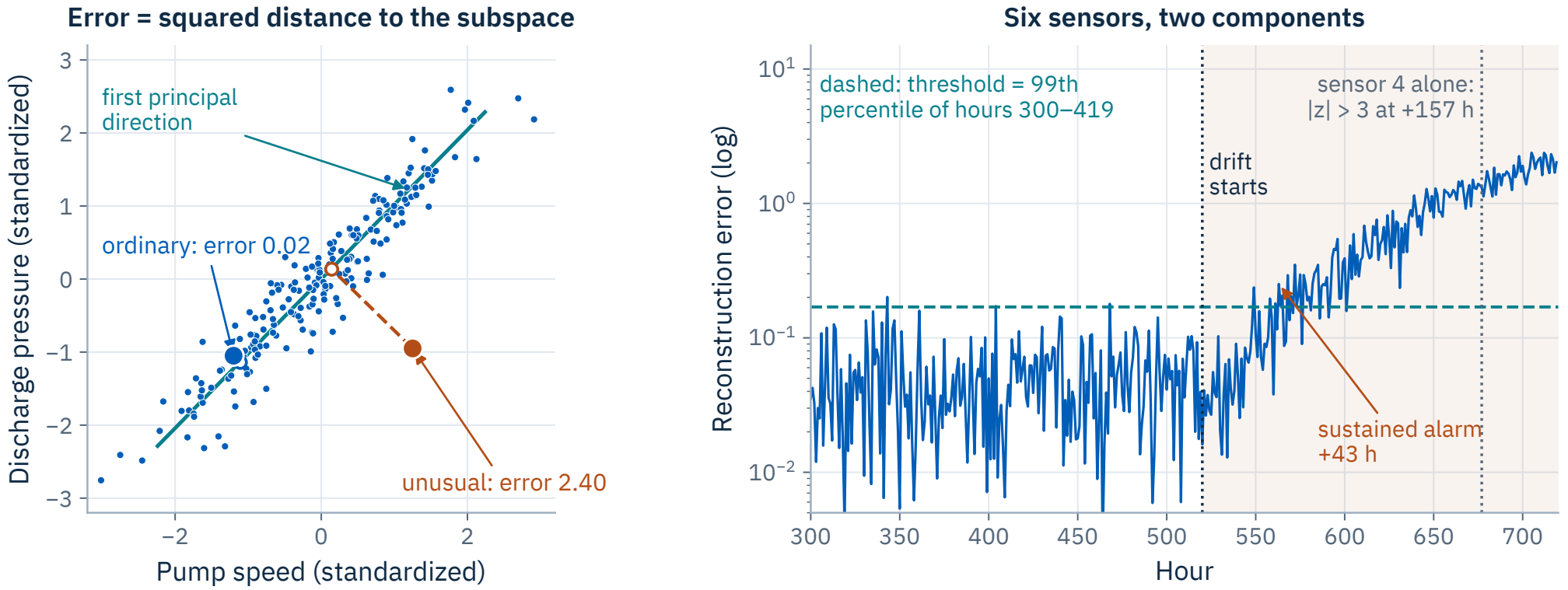
Darker background = more unusual under each model; blue points: the reference rows



Three detectors trained on the same synthetic reference set (seed 42). Backgrounds show model scores, not fault probabilities.

Reconstruction error flags broken relationships

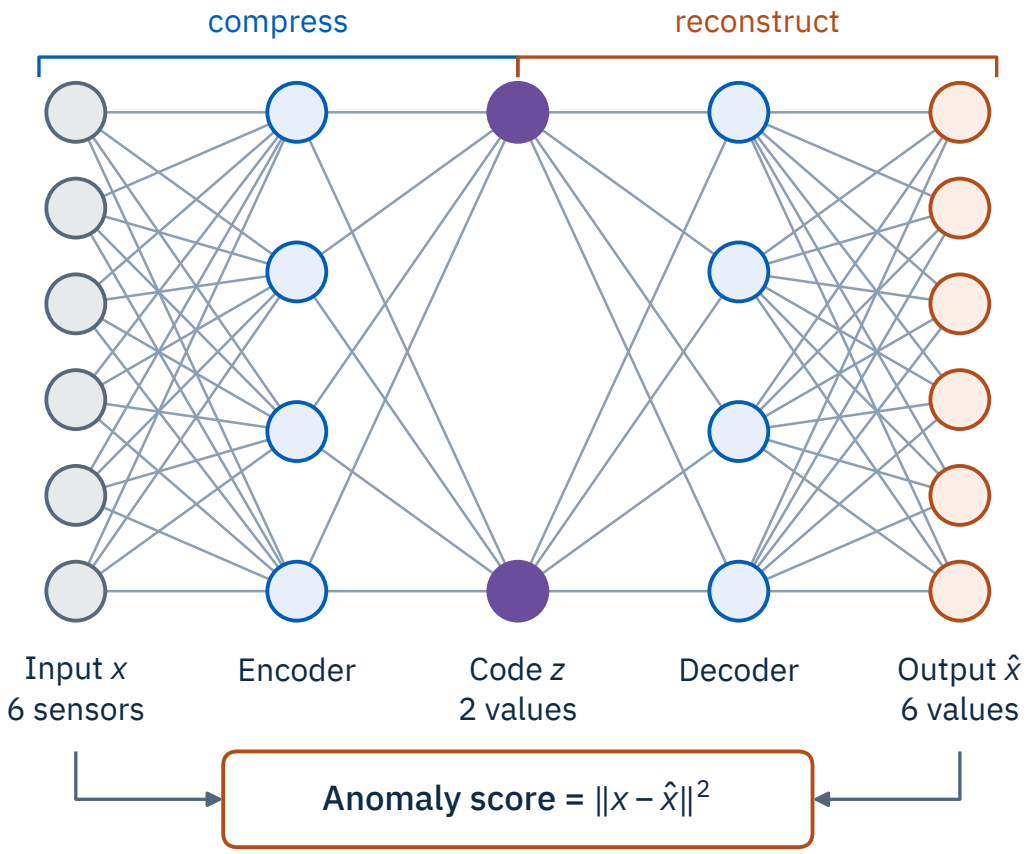
PCA (Module 4) keeps the correlations; rows that break them reconstruct badly.



Left: synthetic correlated pair (seed 13). Right: six synthetic channels (seed 32); PCA fitted on hours 0–299, threshold from hours 300–419, sensor 4 drifts from hour 520.

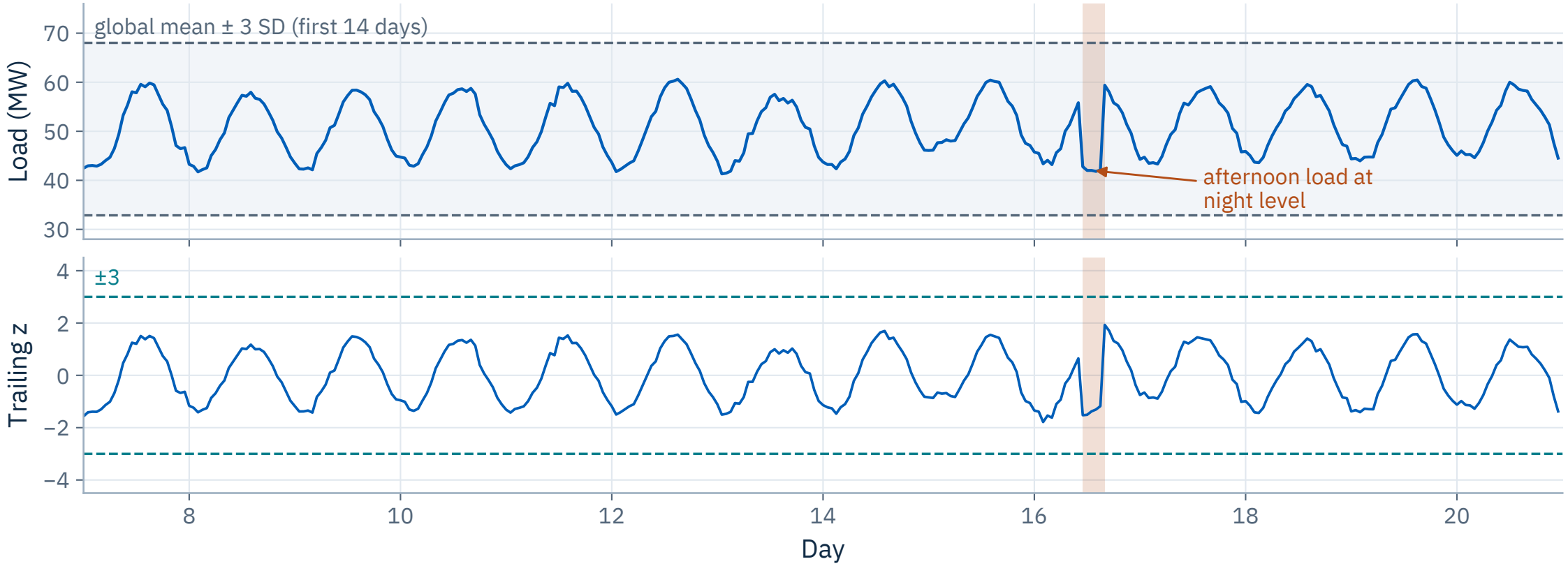
Autoencoders learn nonlinear reconstructions

- The encoder compresses; the decoder rebuilds the input
- Train on normal operation only; score by reconstruction error
- A linear autoencoder with squared error spans the PCA subspace
- Needs more data and tuning: see the neural network course



Schematic undercomplete autoencoder (6-4-2-4-6 units).

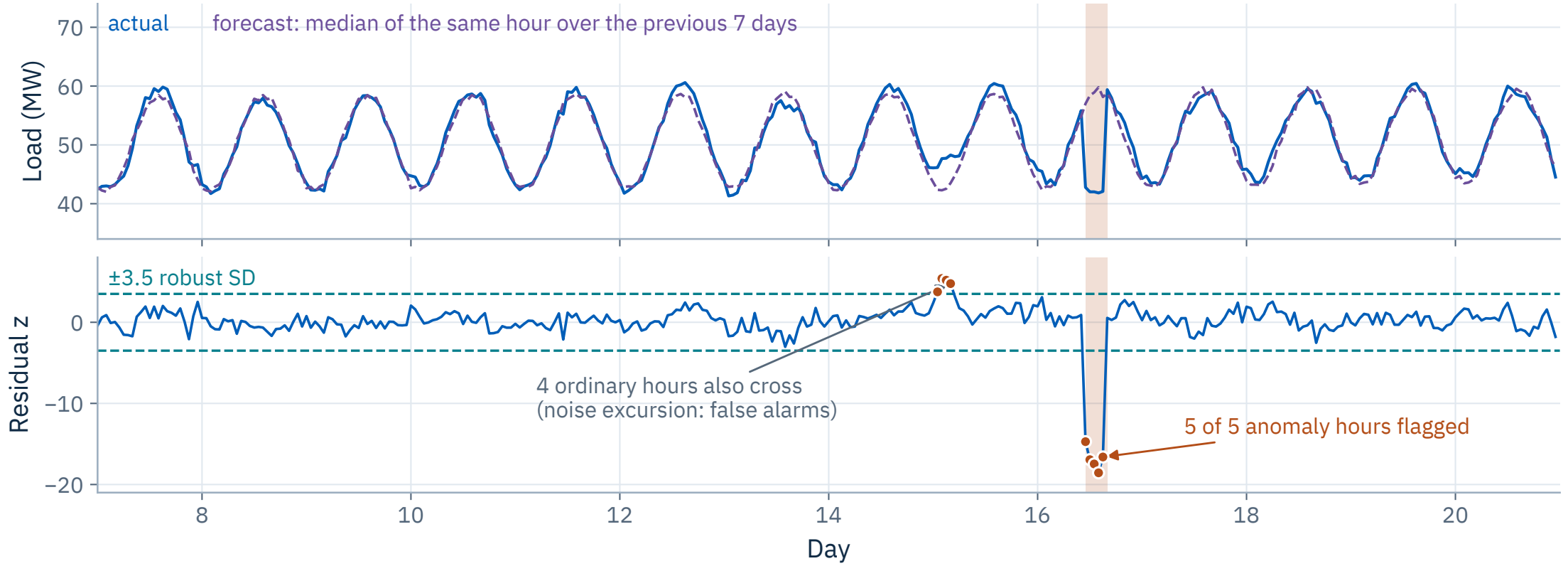
Global limits and rolling z-scores miss daily context



Synthetic hourly load (seed 17); five afternoon hours near day 16.5 set to night level. Top: mean ± 3 SD of the first 14 days. Bottom: z-score against the trailing 24 hours.

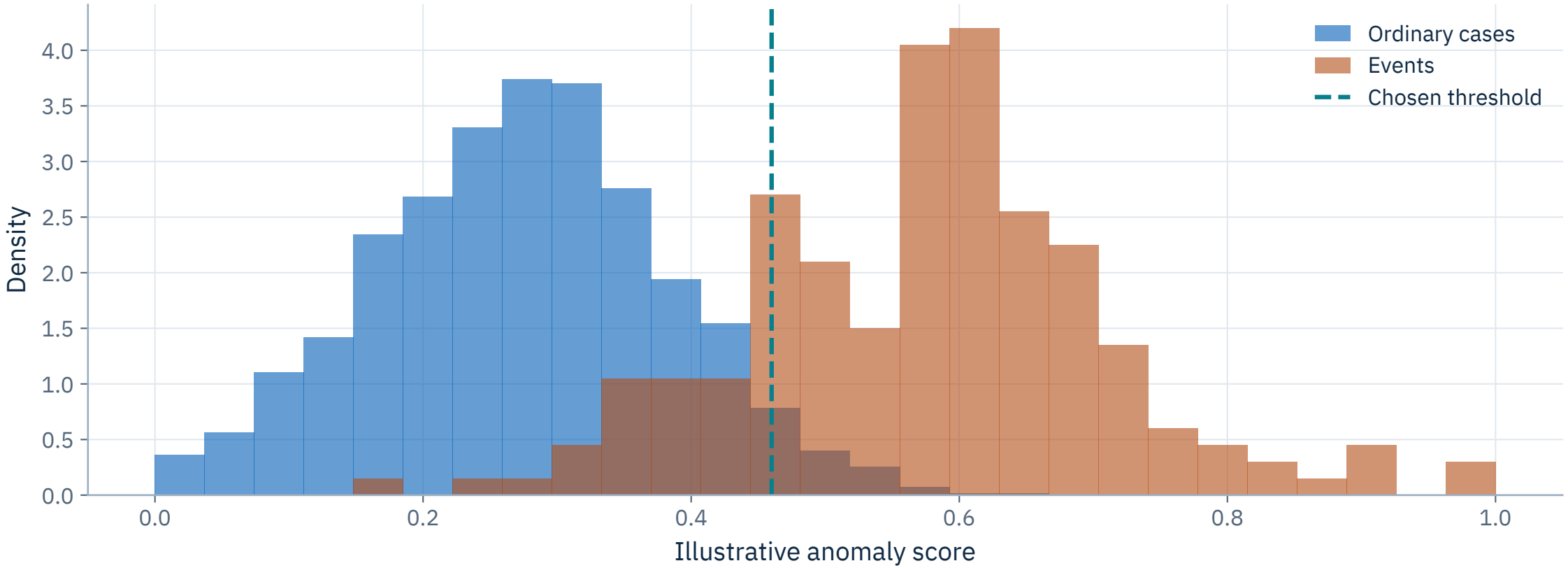
Forecast residuals expose contextual anomalies

Compare each hour with what that hour usually looks like, then threshold the residual.



Same series. Forecast: median of the same hour over the previous 7 days; residuals scaled by the median and MAD of days 7–14; threshold 3.5.

A score and an alert are separate outputs



Simulated score distributions (seed 42), not a fitted detector, with a chosen threshold. The overlap creates both missed events and false alarms.

Score signs differ across scikit-learn detectors

Estimator	Lower score_samples means	predict for outliers	Watch for
IsolationForest	More abnormal	−1	decision_function subtracts offset_
LocalOutlierFactor	More abnormal	−1	Only with novelty=True; else fit_predict
OneClassSVM	More abnormal	−1	Scale features; tune nu and gamma
EllipticEnvelope	More abnormal	−1	Assumes one elliptical cloud

Convert to "higher = more unusual" once, in one place, and test it.

Combine detectors by rank, not by raw score

```
from scipy.stats import rankdata

scores = {                          # higher = more unusual
    "iforest": -iso.score_samples(X),
    "lof": -lof.negative_outlier_factor_,
    "ocsvm": -ocsvm.decision_function(X),
}
pct = {k: rankdata(v) / len(v)   # 0-1 percentile ranks
       for k, v in scores.items()}
combined = np.mean(list(pct.values()), axis=0)
review = np.argsort(-combined)[:50]   # top 50 to review
```

- Raw scores differ in scale and sign across methods
- Percentile ranks put them on one 0–1 scale
- The average rank is a review order, not a probability
- Keep each method's rank to explain why a row was flagged

Fit the detector on a reference period only

```
import numpy as np
from sklearn.ensemble import IsolationForest
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

ref = df[df["time"] < cutoff]      # reviewed, ordinary period
new = df[df["time"] >= cutoff]

det = make_pipeline(StandardScaler(),
                    IsolationForest(n_estimators=300, random_state=0))
det.fit(ref[features])

s_ref = -det.score_samples(ref[features])   # higher = more unusual
threshold = np.quantile(s_ref, 0.995)        # a start; validate it
alerts = -det.score_samples(new[features]) > threshold
```

PRACTICE

4 min · Pairs

Contamination is not a discovered prevalence

An analyst sets contamination = 0.02 and reports that 2% of the equipment is faulty.

1. What did the parameter actually control?
2. What evidence is missing from the conclusion?

Choose a first detector by its assumptions

Method	Assumes	Good first use	Watch for
Robust z / IQR	One population per feature	Single-sensor quality control	Mixtures, context
Mahalanobis	One elliptical cloud	Correlated channels	Masking: fit robustly
Isolation Forest	Anomalies few and different	Tabular baseline	Irrelevant features
LOF	Local density matters	Clusters of different density	n_neighbors, scaling
One-Class SVM	Compact normal region	Clean reference, modest size	gamma, nu, runtime
PCA reconstruction	Normal rows near a subspace	Many correlated sensors	Nonlinear relationships

LABORATORY

65 min

Lab 1: statistical rules and Isolation Forest

- Part 1: load, plot one well, drop incomplete rows; record the rows and wells lost
- Part 2: flag |z| > 3 and IQR outliers; compare the two sets
- Part 3: fit Isolation Forest (contamination 0.05) and plot its score track
- Sweep contamination from 0.01 to 0.10 and explain what changes

NOTEBOOK

`exercises/anomaly_detection/Anomaly_Detection_Student.ipynb`

EVIDENCE TO SUBMIT

- Row and well counts before and after cleaning
- z-score versus IQR overlap counts, with one reviewed interval
- One sentence on what contamination controls

DISCUSSION

Cleaning silently removed two of five wells

Reference run of the Lab 1 notebook (scikit-learn 1.7):

27,845
rows loaded

5,493
rows complete in the six curves

2 of 5
wells lost: DT missing in all rows

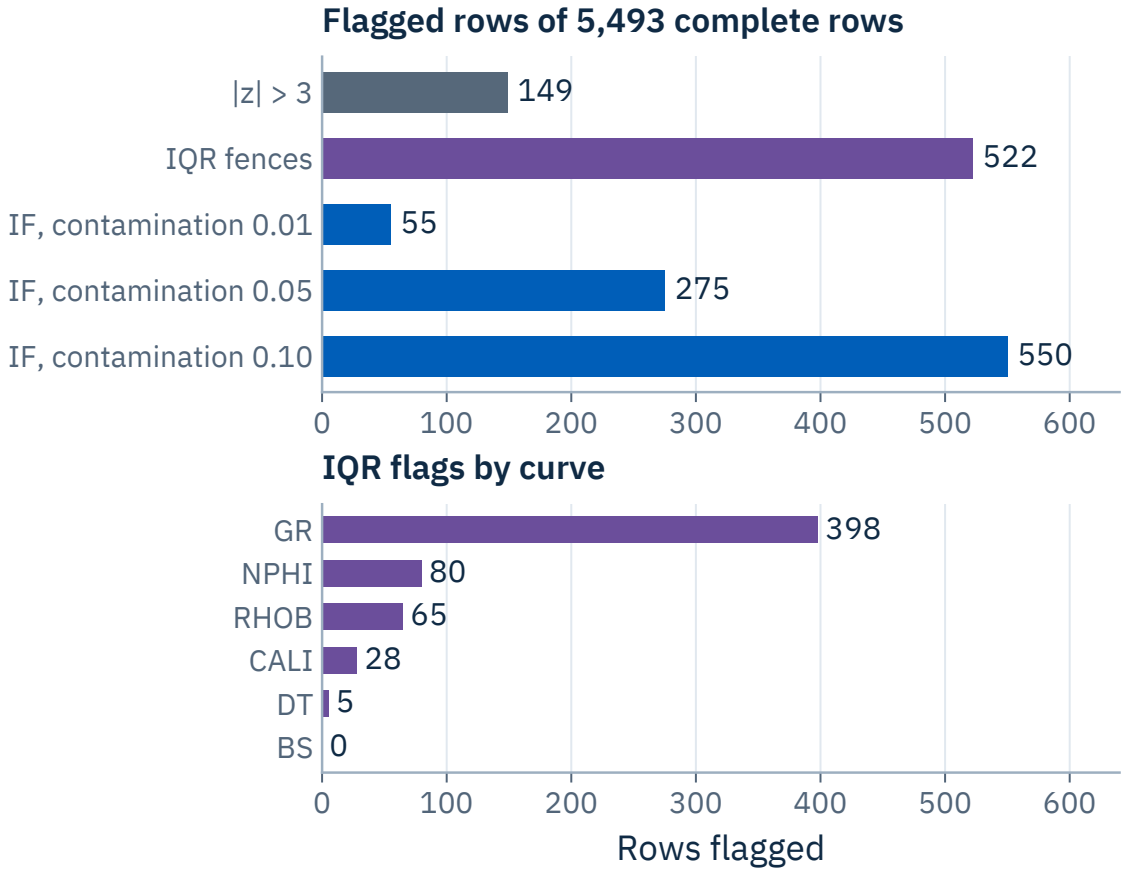
0
spread in BS: constant 8.5 in

- Record what cleaning removed before interpreting any flag.

DISCUSSION

Rules disagree, and contamination fixes the count

- Every |z| > 3 row is also an IQR row; IQR adds 373 more
- 398 IQR flags are high GR: shale-rich rows (median VSH 0.79)
- Isolation Forest flags exactly 1%, 5%, and 10% of the rows
- A higher contamination only lowers the threshold on one ranking



Computed from well_logs_data.csv (course data): 5,493 complete rows; IsolationForest(n_estimators=100, random_state=42).

PART 3 OF 7

03

Evaluating rare events

Useful alerts require denominators, thresholds, and event definitions.

- Base rates and the precision of alerts
- ROC versus precision-recall under imbalance
- Thresholds from costs and from review capacity
- Event-level metrics, lead time, and alert fatigue

PRACTICE

4 min | Individual

Prevalence changes the meaning of a false alarm rate

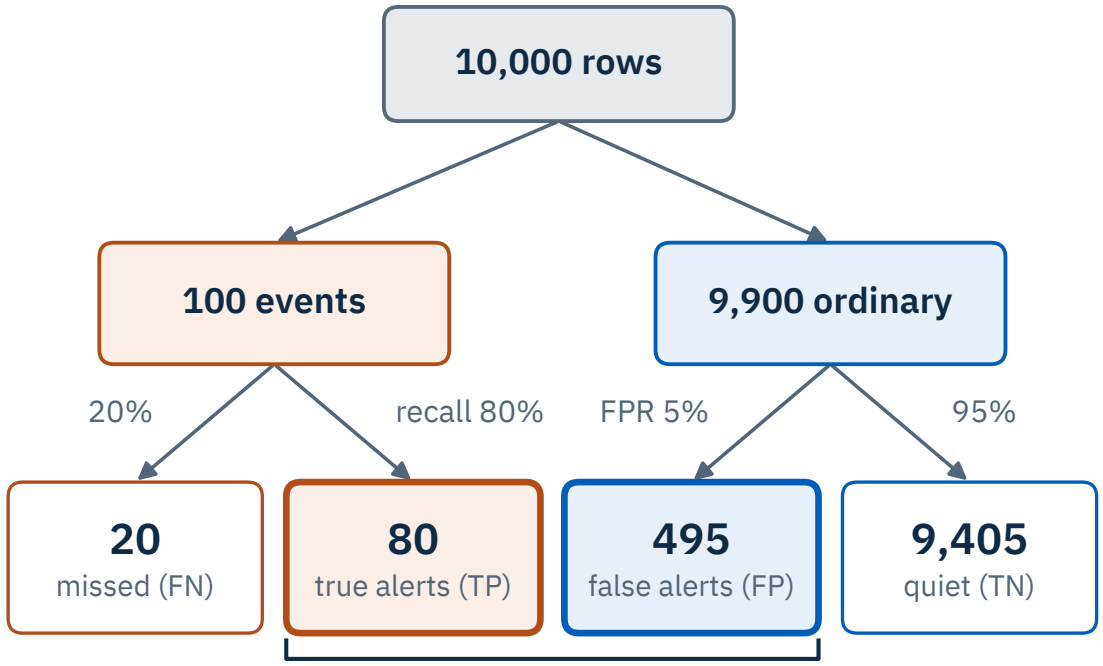
In 10,000 observations, 100 are events. A detector has 80% recall and a 5% false-positive rate.

1. How many true alerts and false alerts occur?
2. What fraction of alerts are true events?

DISCUSSION

Most alerts are false when events are rare

- 80 true alerts and 495 false alerts: precision 13.9%
- False alerts outnumber true ones about six to one
- The same detector at 10% prevalence: precision 64%
- Quote precision at the prevalence of deployment



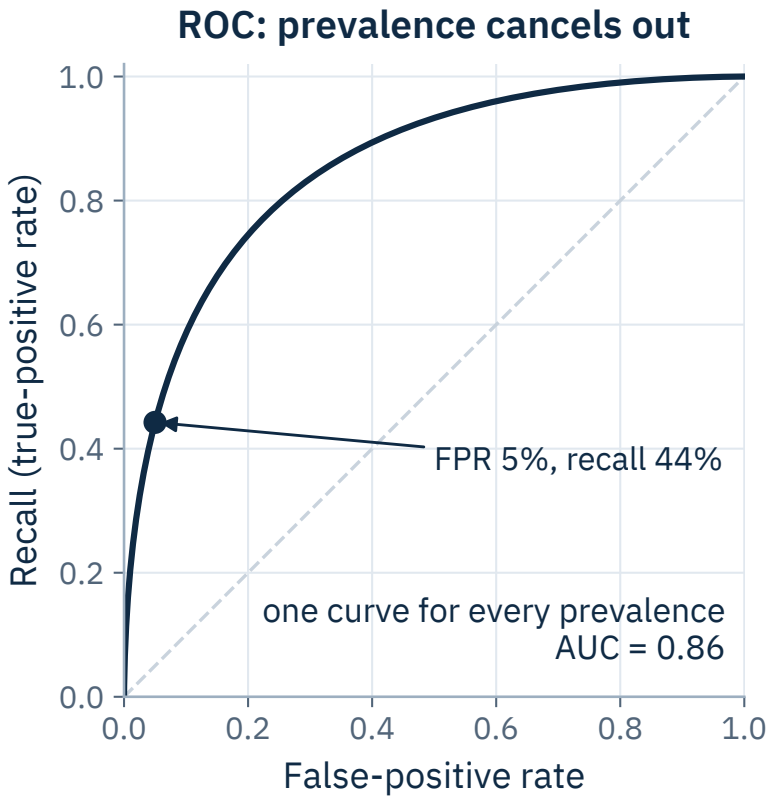
Alerts: 80 + 495 = 575

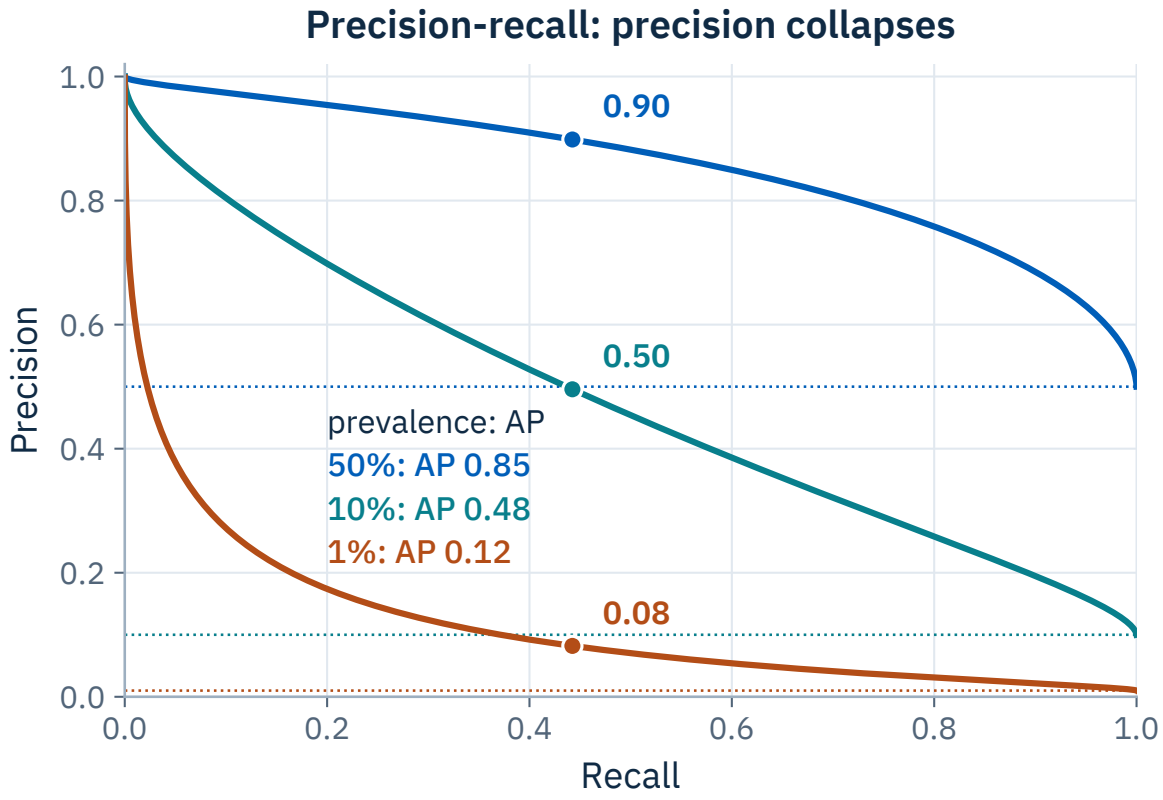
Precision = 80 / 575 = 13.9%

Exact arithmetic for 10,000 rows, 1% prevalence, recall 80%, false-positive rate 5%.

ROC curves hide what prevalence does to precision

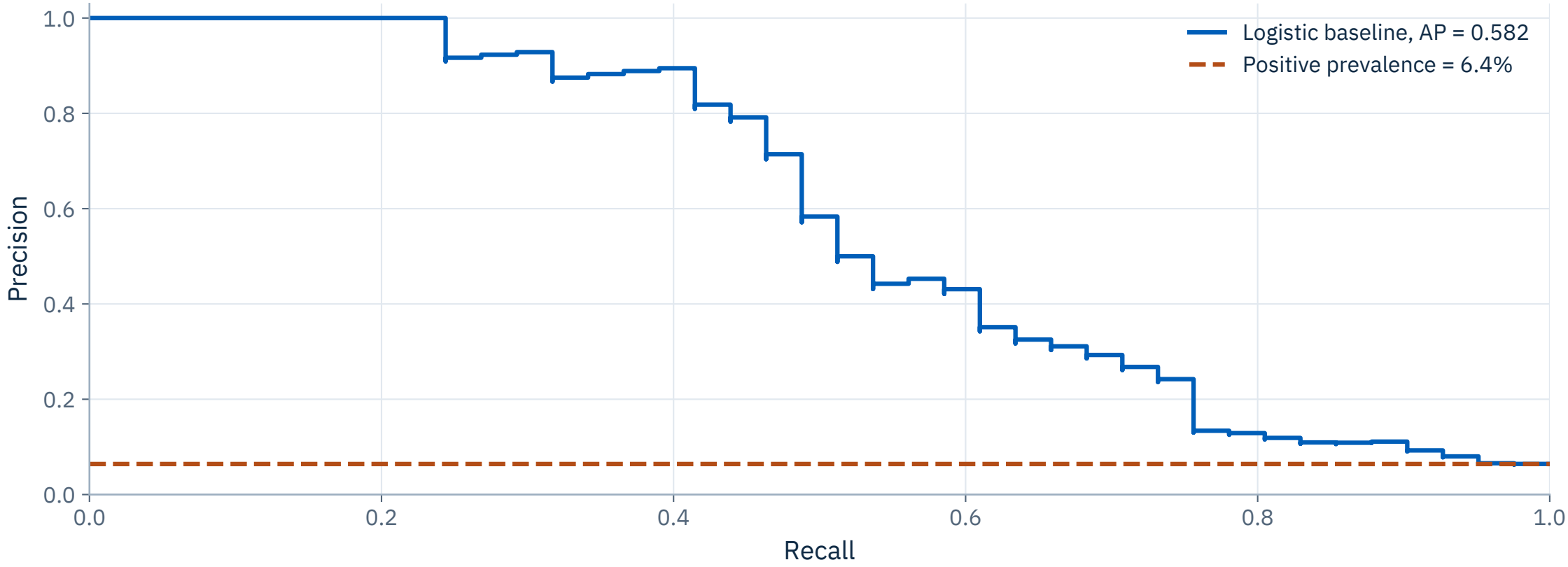
One detector, three prevalences: one ROC curve, three precision-recall curves.





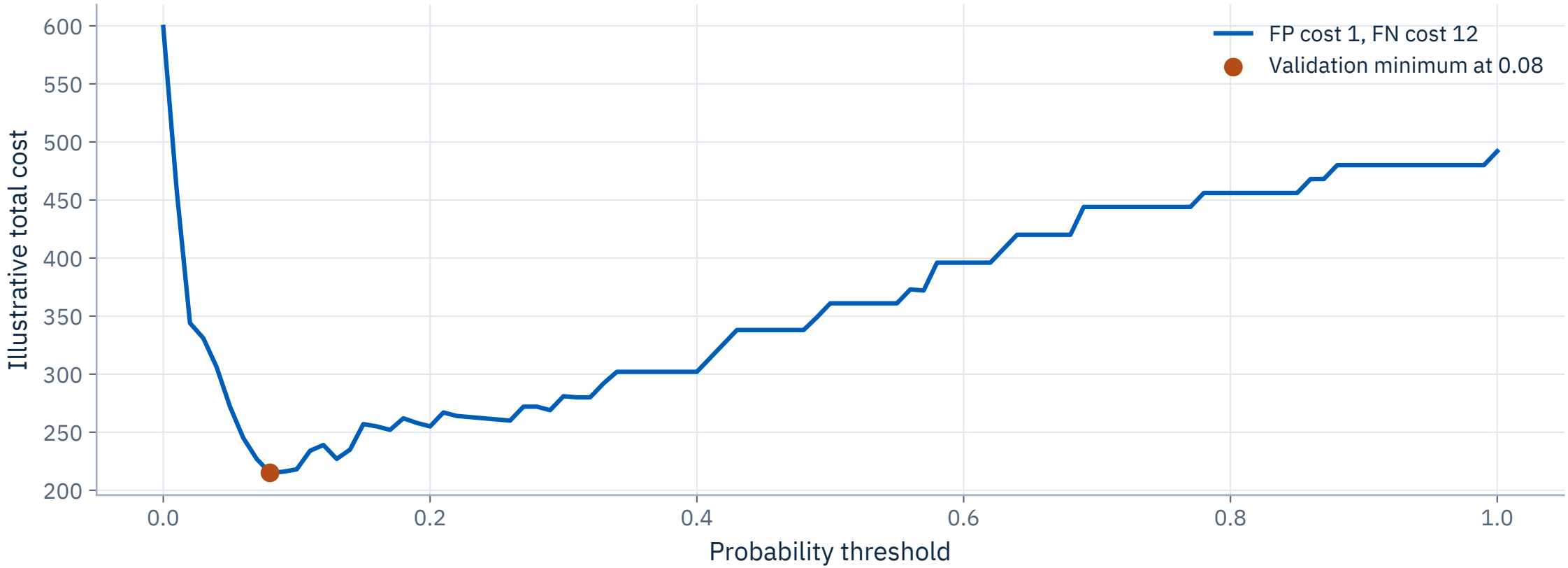
Exact binormal scores: ordinary N(0, 1), events N(1.5, 1). Dots mark the threshold with a 5% false-positive rate (recall 0.44) and the resulting precision.

Precision-recall exposes the alert trade-off



Logistic regression on a synthetic rare-event validation set (make_classification, seed 42). The dashed line is the observed positive prevalence; AP summarizes ranking across thresholds.

The threshold belongs to the decision



Cost computed on the same synthetic validation set (seed 42) with an illustrative cost ratio of 12 to 1. This is not an operational cost estimate.

With calibrated probabilities, costs set the threshold

$$t^* = \frac{C_{FP}}{C_{FP} + C_{FN}}$$

- Alert when the expected miss cost exceeds the false-alarm cost
- Costs 1 and 12 give $t^* = 1/13 \approx 0.077$
- Close to the validation minimum of 0.08 on the previous slide
- Valid only if the probabilities are calibrated: see Part 4

◆ Review capacity can override costs: set the threshold to an alert rate the team can act on.

Events differ from individual rows

Define the event

- Start, end, and grouping of each labelled event
- A warning window agreed before scoring

Score the event

- Event recall and useful warning time
- No credit for alerts after onset

Count false alerts per exposure

- Per asset-day or per pump-month
- Not per row

State the alert rules

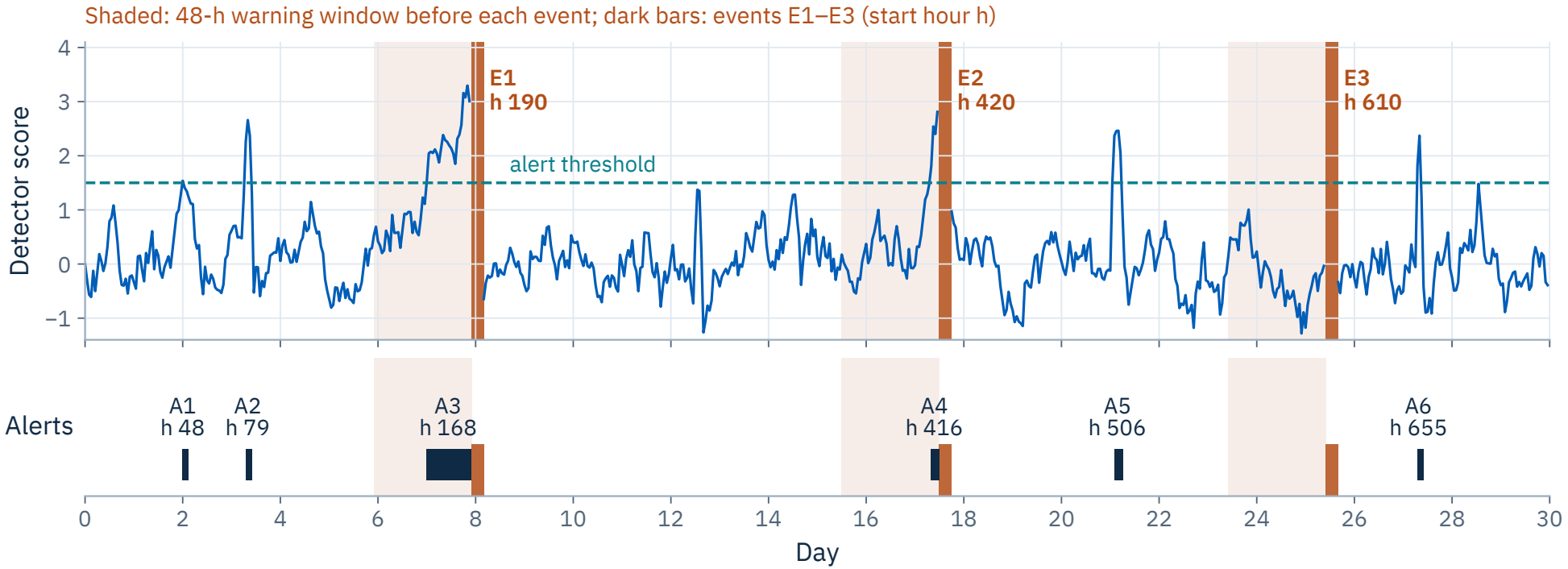
- Persistence, cooldown, deduplication
- Applied the same way in validation

PRACTICE

6 min · Pairs

Score an alert log at the event level

1. Which events were warned within 48 h, and with what lead time?
2. How many false alert episodes occurred per day?



Synthetic 30-day detector score (seed 22). Alert rows above the threshold are merged into episodes A1–A6 when gaps are 6 h or less; h = start hour.

DISCUSSION

Event metrics answer the operator's questions

Same alerts, counted by events instead of rows:

2 of 3

events warned within 48 h

22 h, 4 h

lead times for E1 and E2

0.13

false alert episodes per day

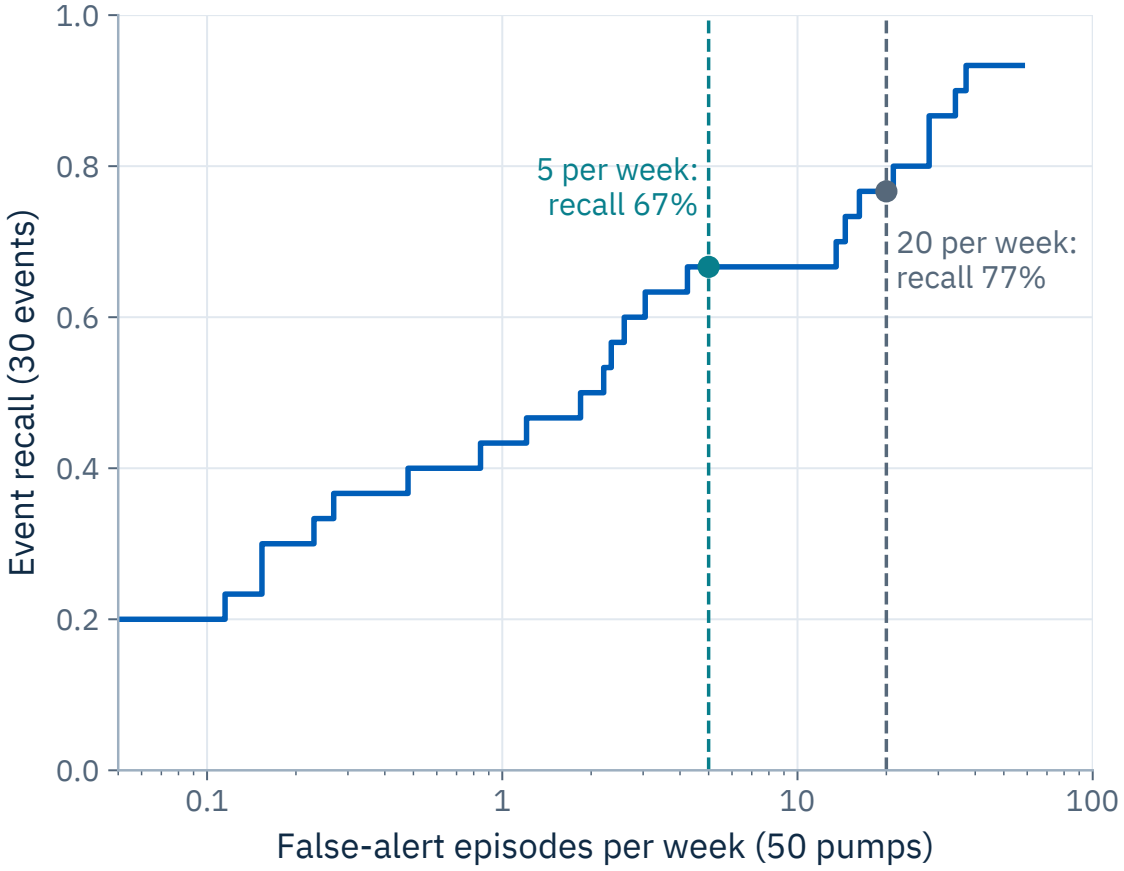
18%

row-level recall of the same alerts

- Row recall of 18% sounds poor; two of three events warned is the operational answer.

Alert capacity chooses the operating point

- Plot event recall against false alerts per week, not per row
- A team that reviews 5 false alerts a week gets 67% recall here
- Quadrupling the load to 20 a week buys only 77%
- Unreviewed alerts teach people to ignore the system



Synthetic fleet (seed 41): 50 pumps over 365 days, 30 events with precursors of random strength; alert episodes are runs of alert days outside warning windows.

Without labels, judge a detector by stratified review

Score band	Rows	Confirmed in a sample of 30	Estimated problems
Top: flagged	275	12	110
Middle	1,000	3	100
Bottom	4,218	1	141

Hypothetical review counts. Precision in the flagged band is 12 of 30 (40%), yet it holds only 110 of about 351 estimated problems (31%).

- Sample every band, unflagged rows included; review blind; weight each band by its size.

PART 4 OF 7

04

Imbalance, calibration, uncertainty

Rebalancing changes probabilities; few events limit claims.

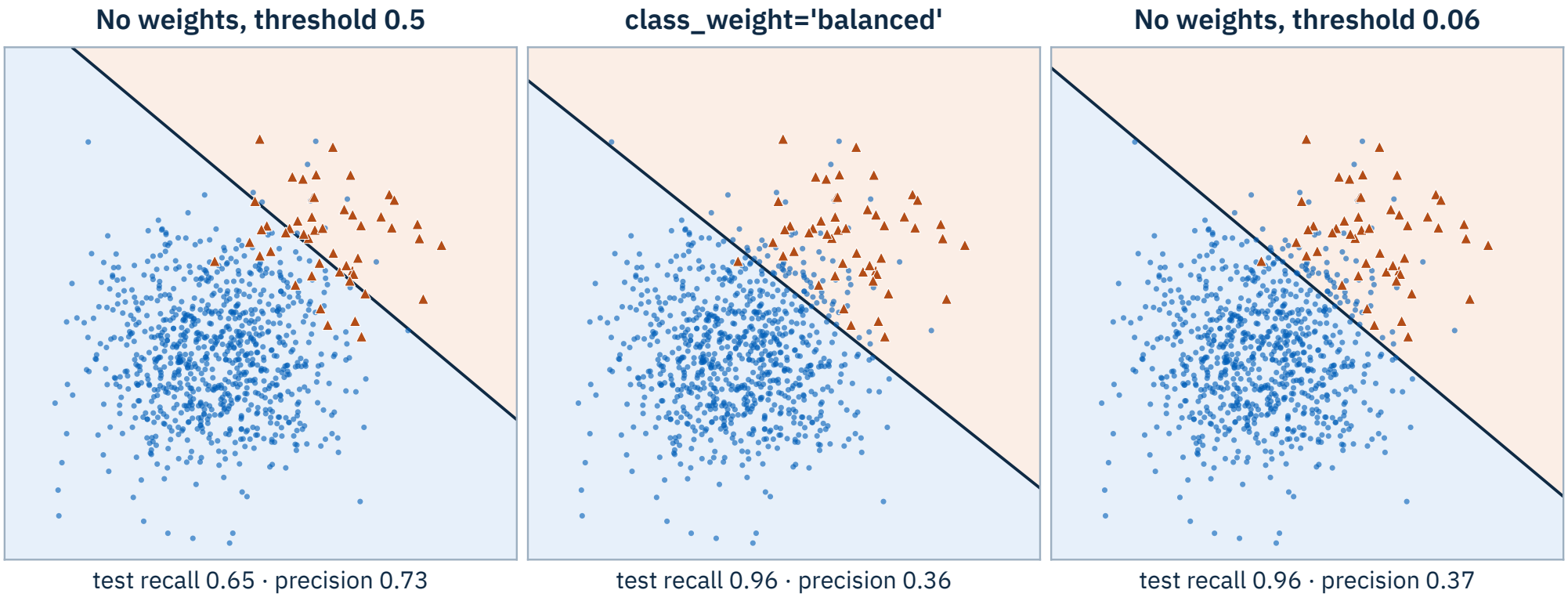
- Class weights, resampling, and SMOTE inside folds
- Calibration: reliability diagrams and recalibration
- Prior correction after undersampling
- Honest intervals with ten events

Each rebalancing strategy trades one risk for another

Strategy	Changes	Watch for
Class-weighted loss	Relative training penalty	Probability calibration changes
Random undersampling	Majority examples discarded	Loss of useful variation
Random oversampling	Minority examples repeated	Overfitting repeated cases
SMOTE	Interpolated minority features	Implausible combinations

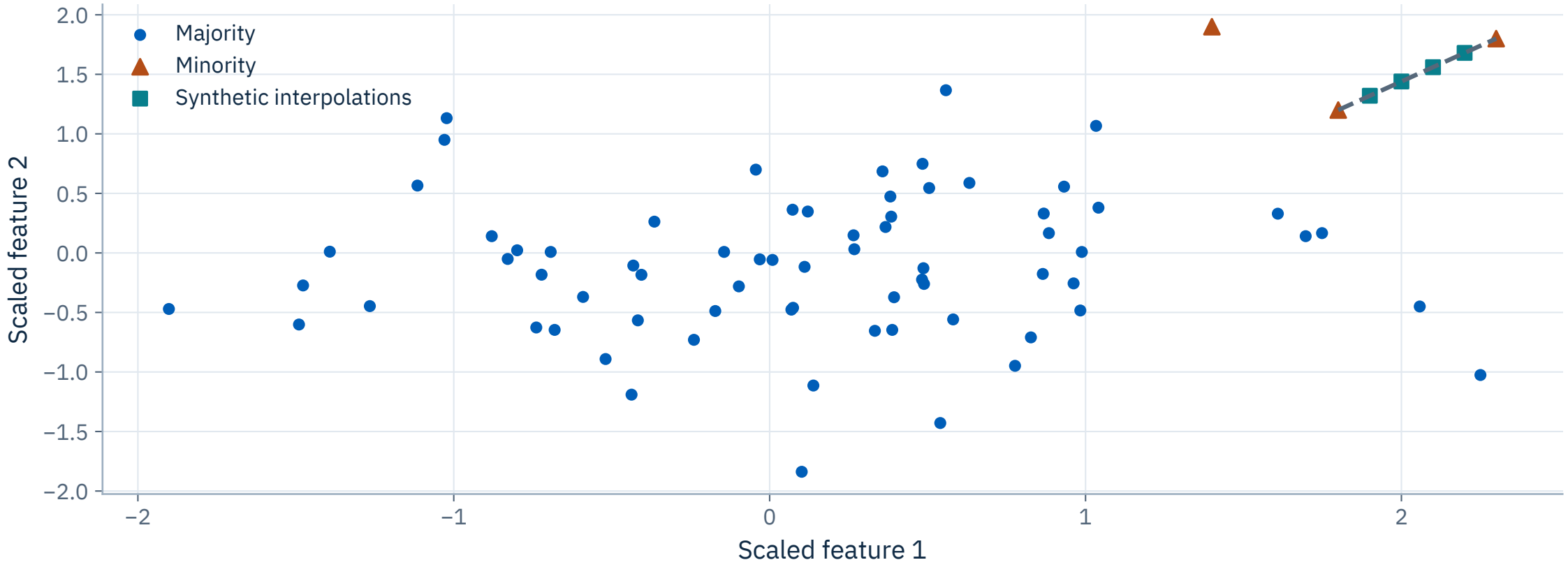
Class weights shift the decision boundary

Balanced weights move the boundary much as a lower threshold would.



Synthetic 2-D data (seed 9), 5% events, 50/50 stratified split; logistic regression. Right: unweighted model with the threshold that matches the balanced model's training recall.

SMOTE interpolates in a chosen feature space



Synthetic points (seed 42): one SMOTE-style interpolation between two minority observations. The segment is mathematical interpolation, not evidence of physical feasibility.

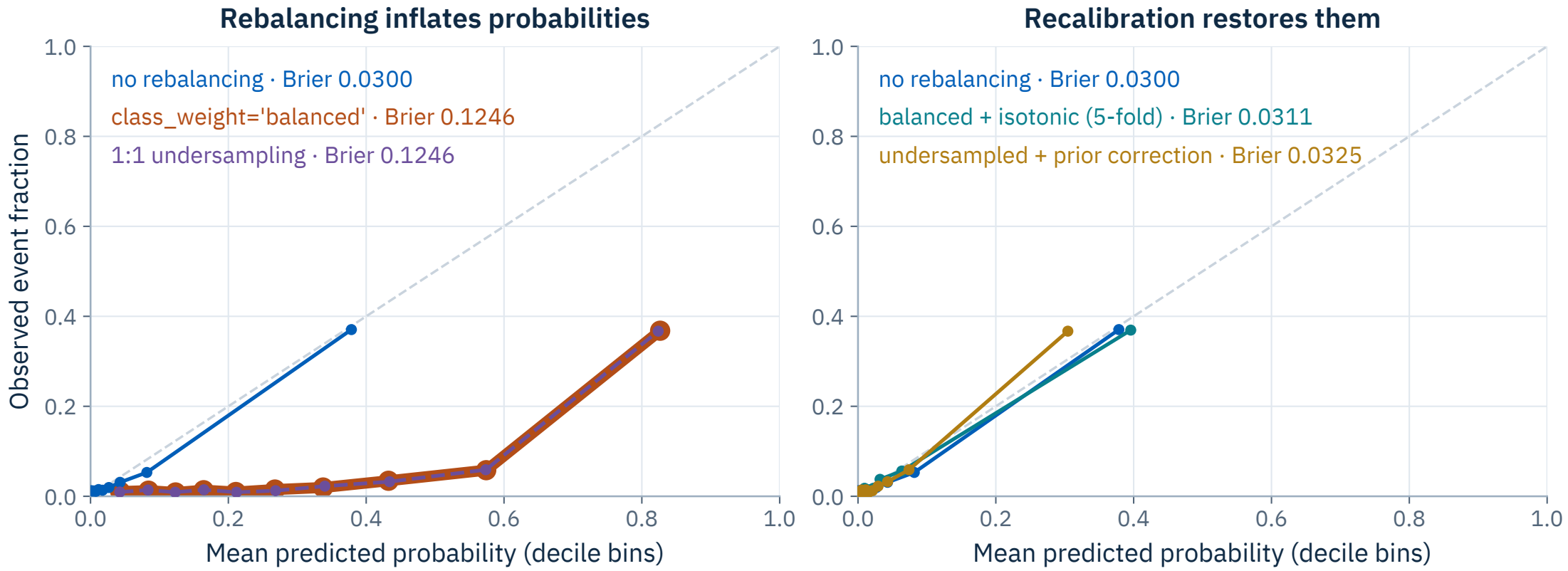
Resampling must happen inside training folds

```python
from imblearn.pipeline import Pipeline
from imblearn.over_sampling import SMOTE
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

model = Pipeline([
    ("scale", StandardScaler()),
    ("sample", SMOTE(random_state=42)),
    ("model", LogisticRegression(max_iter=1000))
])
```

Reweighting and undersampling distort probabilities

Mean probability 0.31 against an event rate of 0.055; the ranking barely moves.



Synthetic data (make_classification, seed 5, 40,000 rows, 5.5% events); logistic regression; reliability curves on the test half in decile bins.

Recalibrate on rows the model did not train on

```python
from sklearn.calibration import CalibratedClassifierCV
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import brier_score_loss
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

lr = LogisticRegression(class_weight="balanced",
                        max_iter=1000)
base = make_pipeline(StandardScaler(), lr)
cal = CalibratedClassifierCV(base, method="isotonic",
                             cv=5)
cal.fit(X_train, y_train)

p = cal.predict_proba(X_test)[:, 1]
print(brier_score_loss(y_test, p))
```

- Each fold fits the model, then a calibrator on held-out rows
- method="sigmoid" is Platt scaling: needs fewer events
- Isotonic is more flexible but needs more events
- Check a reliability plot and the Brier score on test rows

Undersampling needs a prior correction

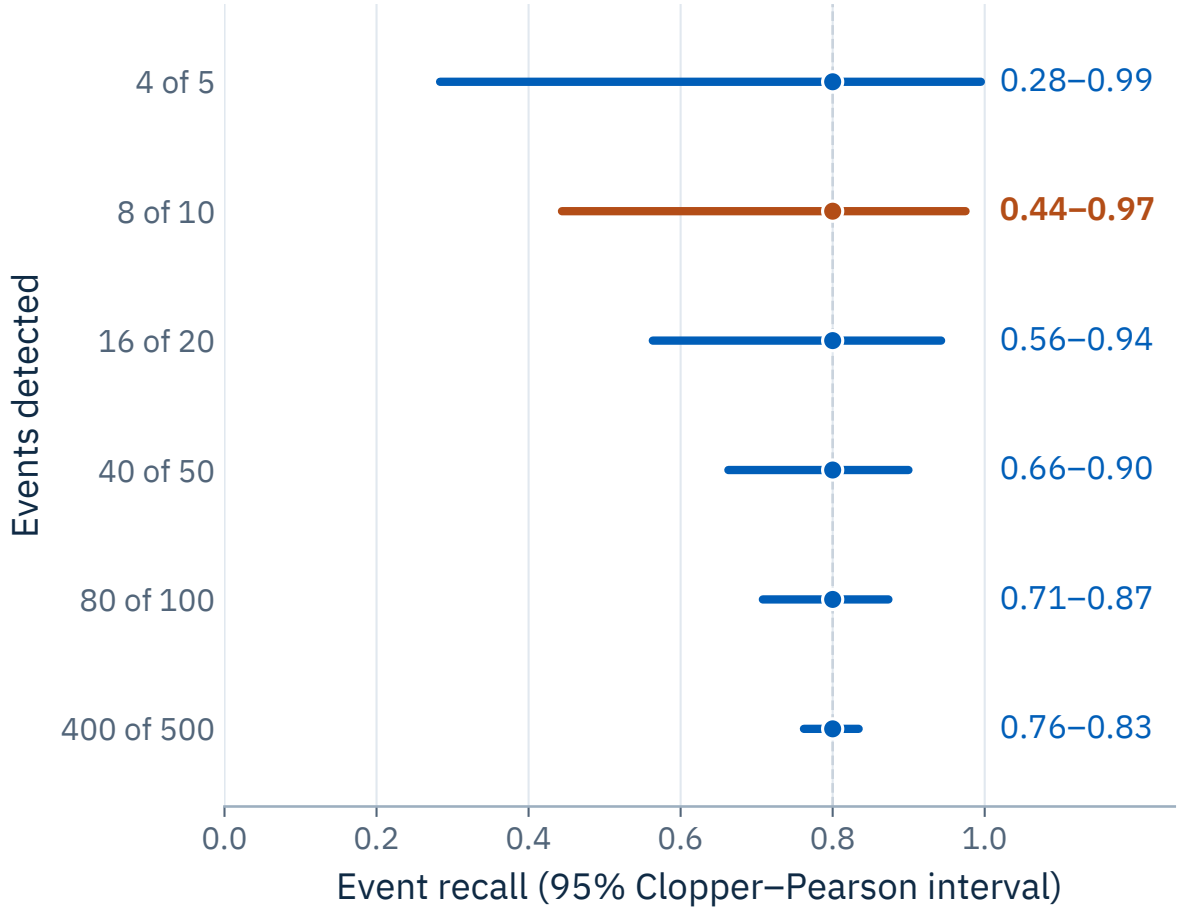
$$p = \frac{\beta p_s}{\beta p_s - p_s + 1}$$

- p_s: probability from the model trained on undersampled data
- β: fraction of majority rows kept (0.058 here)
- Restores the base rate: Brier 0.1246 falls to 0.0325
- Assumes the kept majority rows were sampled at random

◆ For class-weighted models there is no such shortcut: recalibrate on held-out rows.

Few labelled events require honest uncertainty

- Count independent events, not just labelled rows
- Review a sample of alerts and of non-alerts
- Resample whole events or assets, not rows
- State what the labels cannot estimate



Exact Clopper–Pearson 95% intervals for an observed recall of 80%.

PRACTICE

4 min · Pairs

Eight of ten events detected: what can you claim?

A detector warned of 8 of the 10 pump trips in the validation year.

1. Is "recall = 80%" a fair summary for the report?
2. What is the 95% interval, and what would 80 of 100 give?
3. Name one way the ten trips might not be independent.

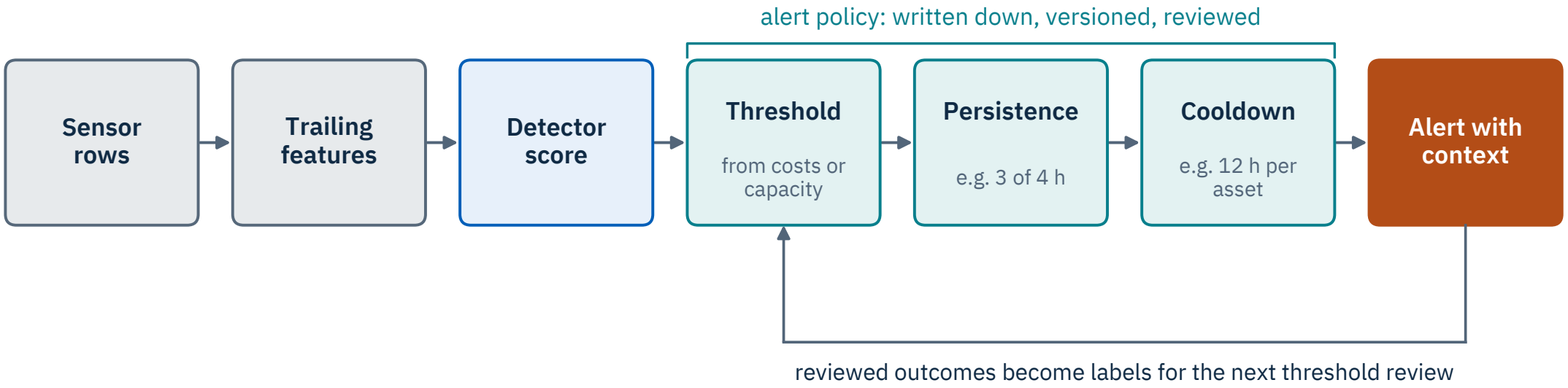
KEY TAKEAWAYS

Rare-event evidence checklist

- ✓ Keep the deployment prevalence in validation and test sets
- ✓ Report precision-recall and event-level metrics, not accuracy
- ✓ Freeze the threshold before the test; justify it by cost or capacity
- ✓ Check calibration after any weighting or resampling
- ✓ Quote intervals based on the number of independent events

An alert policy turns scores into accountable actions

Lab 2 ends with a written policy: each box in this chain needs a stated choice and its evidence.



Schematic. Persistence and cooldown values are illustrative choices, not recommendations.

LABORATORY

75 min

Lab 2: compare detectors and write an alert policy

- [ ] Part 4: LOF with n_neighbors from 5 to 100; compare flagged sets, not counts
- [ ] Part 5: One-Class SVM (nu = 0.05); plot its score track
- [ ] Part 6: overlap matrix, votes, and a rank-averaged review list
- [ ] Write a half-page alert policy with a threshold rationale

NOTEBOOK

`exercises/anomaly_detection/Anomaly_Detection_Student.ipynb`

EVIDENCE TO SUBMIT

- Overlap matrix of the five methods
- Jaccard overlap of LOF sets across n_neighbors
- Alert policy: score, threshold, rules, capacity, limits

Detectors agree on a core and differ at the margins

- IF and One-Class SVM share 173 rows; LOF overlaps least
- LOF always flags 275 rows, but which rows changes with n_neighbors
- 24 rows are flagged by all five methods; 322 by two or more
- Agreement is not validation: no label confirms any flag yet

	|z| > 3	IQR	IF	LOF	OCSVM
|z| > 3	149	149	105	26	84
IQR	149	522	164	48	132
IF	105	164	275	68	173
LOF	26	48	68	275	74
OCSVM	84	132	173	74	279

diagonal: rows flagged by each method;
off-diagonal: rows flagged by both

Computed from well_logs_data.csv (course data), 5,493 complete rows, lab settings; scikit-learn 1.7.

DISCUSSION

A defensible alert policy shows its evidence

Same detector, same data: only the second policy can be checked and improved.

✗ Weak policy

- "Flag the top 5% of Isolation Forest scores"
- No unit, no capacity, no review loop
- Reports that 5% of the depths are bad

vs

✓ Defensible policy

- Score, threshold, and why: cost or capacity
- Persistence, cooldown, routing, and owner
- Evidence, known limits, and a review date

◆ A policy is defensible when a colleague could check every number in it.

PART 5 OF 7

05

Honest and efficient model selection

Every choice made on validation data is part of the model.

- Tuning plans, grid search, and random search
- Successive halving and Bayesian optimization
- Selection bias and nested cross-validation
- Comparing models on paired folds; the one-SE rule

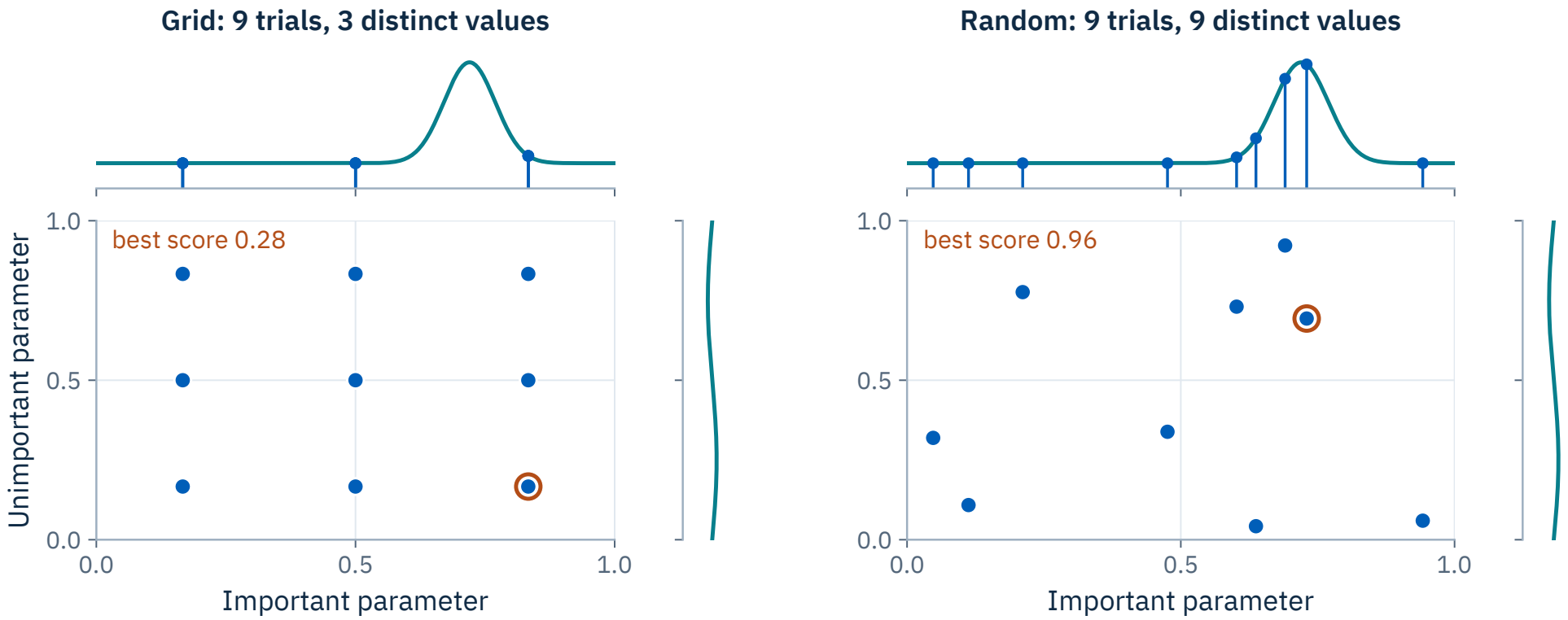
A tuning plan defines the experiment

1. **Fix the question** — target, prediction-time features, and the split policy
2. **Choose the yardstick** — one primary metric plus practical constraints
3. **Bound the search** — candidate space, log scales, compute budget, stopping rule
4. **Keep a ledger** — record every trial, including failures and reruns
5. **Protect the test** — the final partition is used once, at the end

◆ The ledger shows how many candidates were tried, which is what the winner's score must be judged against.

Random search covers the parameters that matter

With 9 trials, a 3 × 3 grid tries 3 values of the important parameter; random search tries 9.



Synthetic objective with a narrow optimum on the important axis (teal curves). One random draw (seed 40); 9 random trials land within 0.05 of the optimum with probability 1 − 0.9 to the 9th power = 0.61.

Random search explores a bounded space

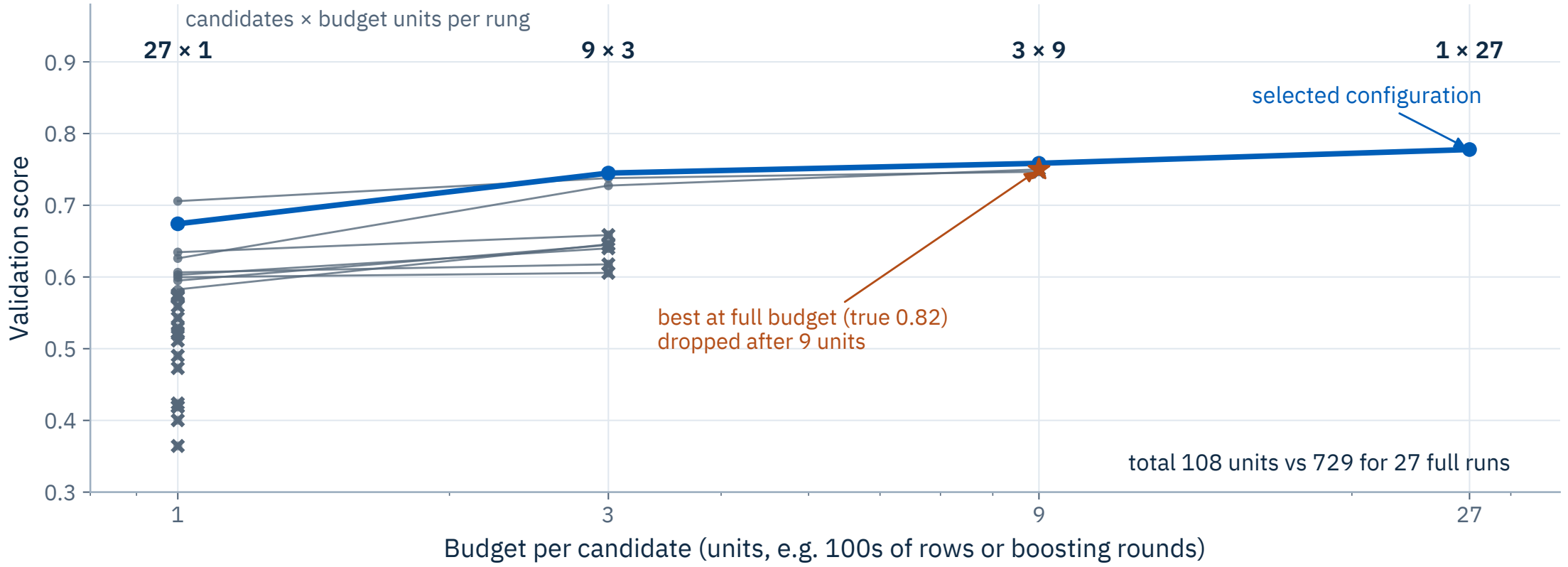
```
from scipy.stats import loguniform
from sklearn.model_selection import RandomizedSearchCV

search = RandomizedSearchCV(
    model, {"model__C": loguniform(1e-3, 1e2)},
    n_iter=12, scoring="average_precision",
    cv=cv, random_state=42, n_jobs=-1
)
search.fit(X_train, y_train)
```

Pass groups to fit when cv requires asset or well groups.

Successive halving spends budget on survivors

Start many candidates cheaply, keep the best third, and triple the budget of the survivors.

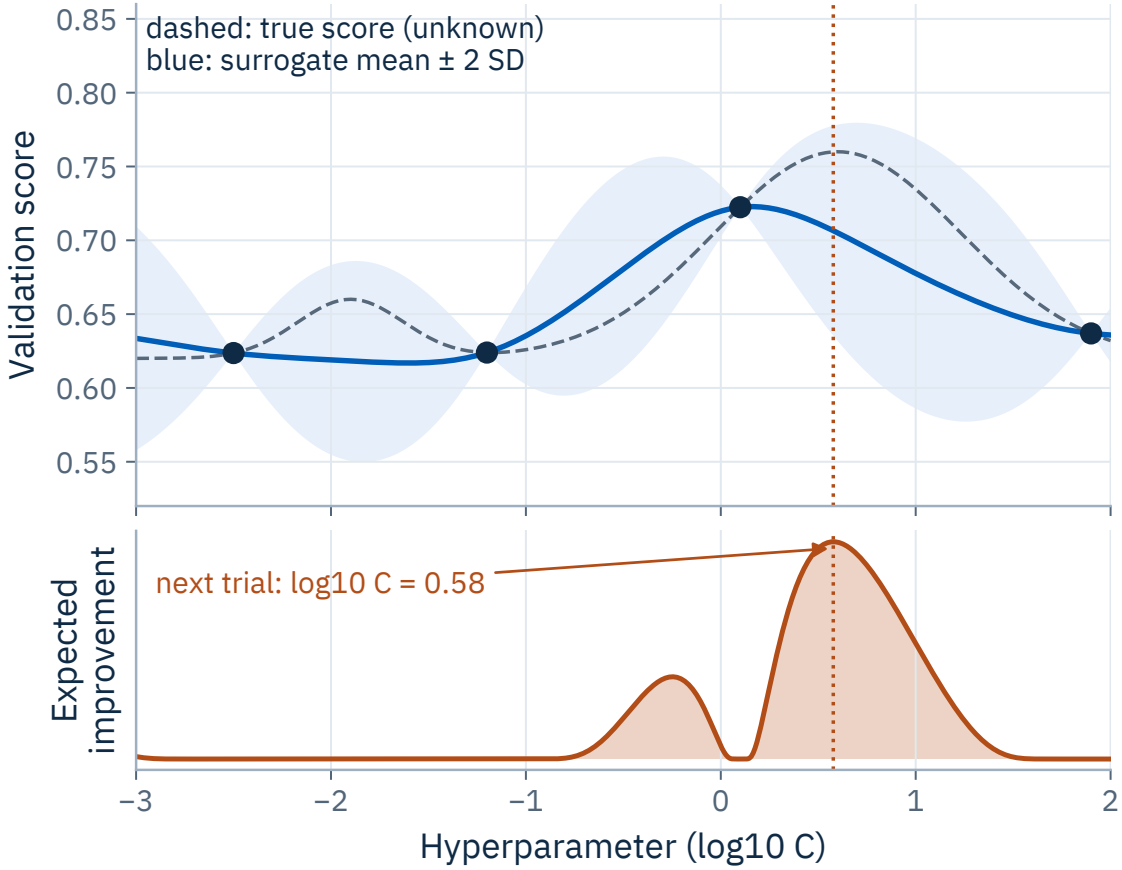


Simulated learning curves for 27 candidates (seed 8); the top third is kept at each rung (factor 3).

Bayesian optimization uses earlier trial results

$$\mathrm{EI}(x) = E[\max(f(x) - f^{+}, 0)]$$

- A surrogate estimates performance across the search space
- The acquisition rule balances promising and uncertain settings
- Each completed trial updates the next choice
- Keep the same validation design; record the whole budget



Synthetic 1-D objective; scikit-learn GaussianProcessRegressor (Matern kernel, fixed hyperparameters) after four trials.

The best of many noisy scores is biased upward

50 candidate settings, all truly equal, each scored with CV noise of SD 0.02:

0.600

true AP of every candidate

0.645

expected best observed AP

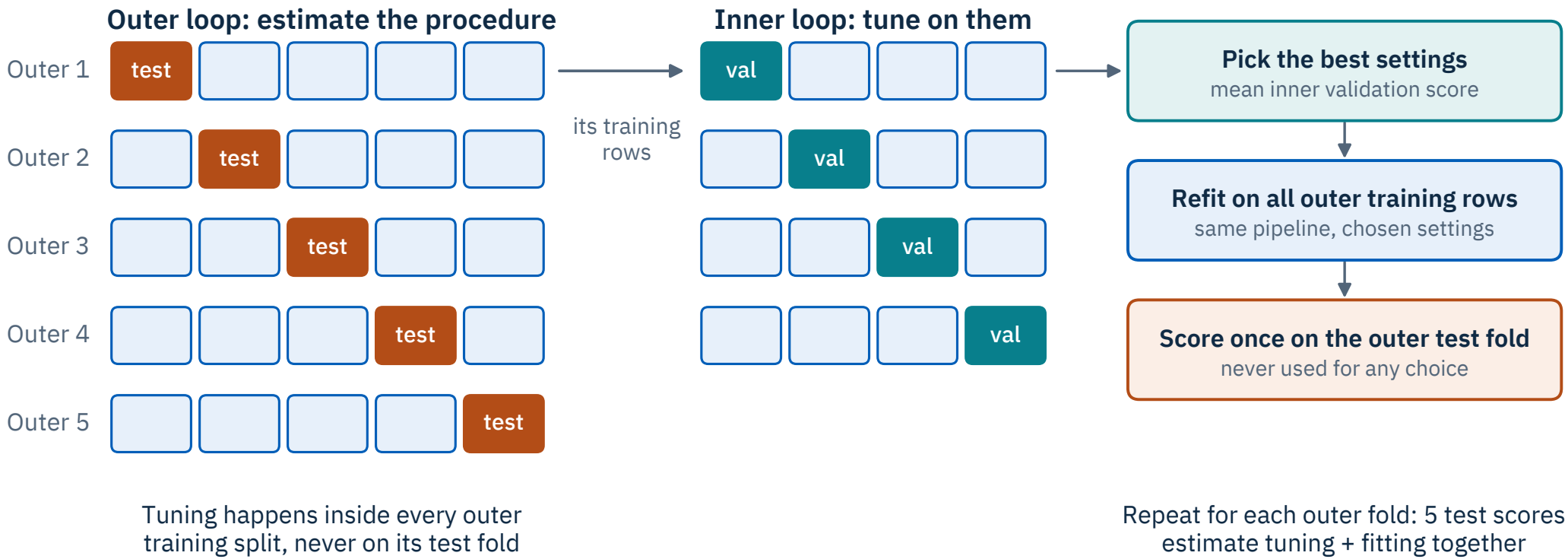
+0.045

optimism from selection alone

◆ Report the chosen model's score from data that played no part in choosing it.

Nested cross-validation scores the selection procedure

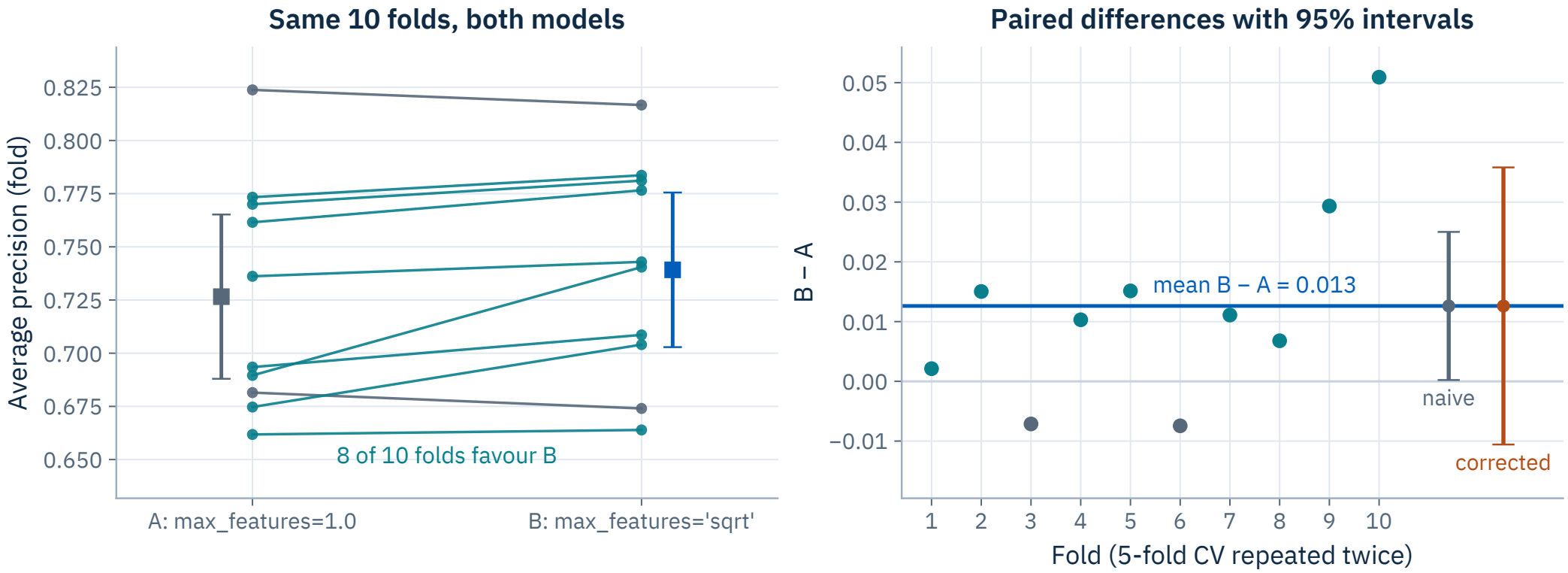
The outer score estimates how well tuning plus fitting generalizes, not one fixed model.



Schematic: five outer folds; inside each outer training split, four inner folds choose the settings.

Compare models on paired folds, with a corrected test

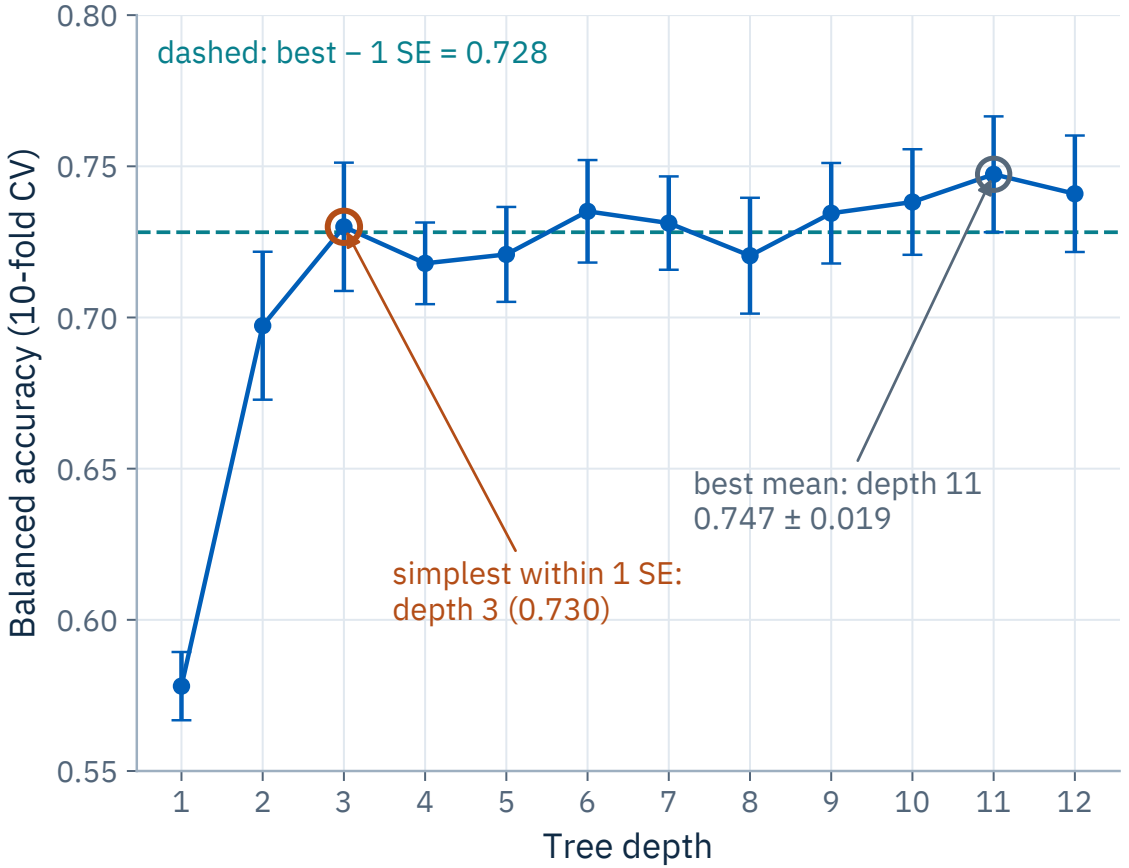
The means' error bars overlap, yet B beats A in 8 of 10 paired folds.



Synthetic data (seed 4, 1,200 rows); RepeatedStratifiedKFold 5 × 2; two random forests differing only in max_features. Corrected interval: Nadeau–Bengio variance, test/train ratio 1/4.

Prefer the simplest model within one standard error

- Find the best mean score and its standard error
- Choose the simplest model within one SE of that best
- Here depth 3 (0.730) instead of depth 11 (0.747)
- A heuristic: fold scores are correlated, so the SE is approximate



Synthetic data (seed 20, 800 rows); DecisionTreeClassifier; 10-fold stratified CV; bars show ±1 standard error.

PRACTICE

8 min · Pairs

Read a tuning log and choose a model

Fold	Depth 3	Depth 6	Depth 12
1	0.61	0.66	0.67
2	0.58	0.62	0.61
3	0.64	0.69	0.70
4	0.55	0.60	0.62
5	0.62	0.68	0.70

1. Which depth has the best mean average precision?
2. Apply the one-SE rule (the SE of depth 12 is 0.019).
3. Is depth 12 clearly better than depth 6 on paired folds?

DISCUSSION

One standard error favours depth 6 in the log

0.660

best mean (depth 12)

0.641

best mean minus one SE

depth 6

simplest within one SE (0.650)

t = 1.8

depth 12 vs 6, naive paired

◆ A 0.01 gain in 4 of 5 folds is weak evidence; decide on cost and simplicity.

PART 6 OF 7

06

Deployment, drift, and monitoring

A validated model is a hypothesis about the future.

- Features that exist at prediction time
- Data drift versus concept drift
- PSI and KS for a shifted input
- The lifecycle loop and a monitoring case

PRACTICE

4 min · Pairs

Feature timing determines whether a forecast is real

A 30-minute warning model uses a centred 30-minute rolling mean, which includes the next 15 minutes.

1. Is that feature available at the forecast origin?
2. How would you repair the feature and the validation design?

Energy features must exist at prediction time

Well logs

- Unit checks and valid transforms
- Within-well gradients, missingness flags

Sensors

- Trailing variability, rates of change
- Operating-state indicators

Production

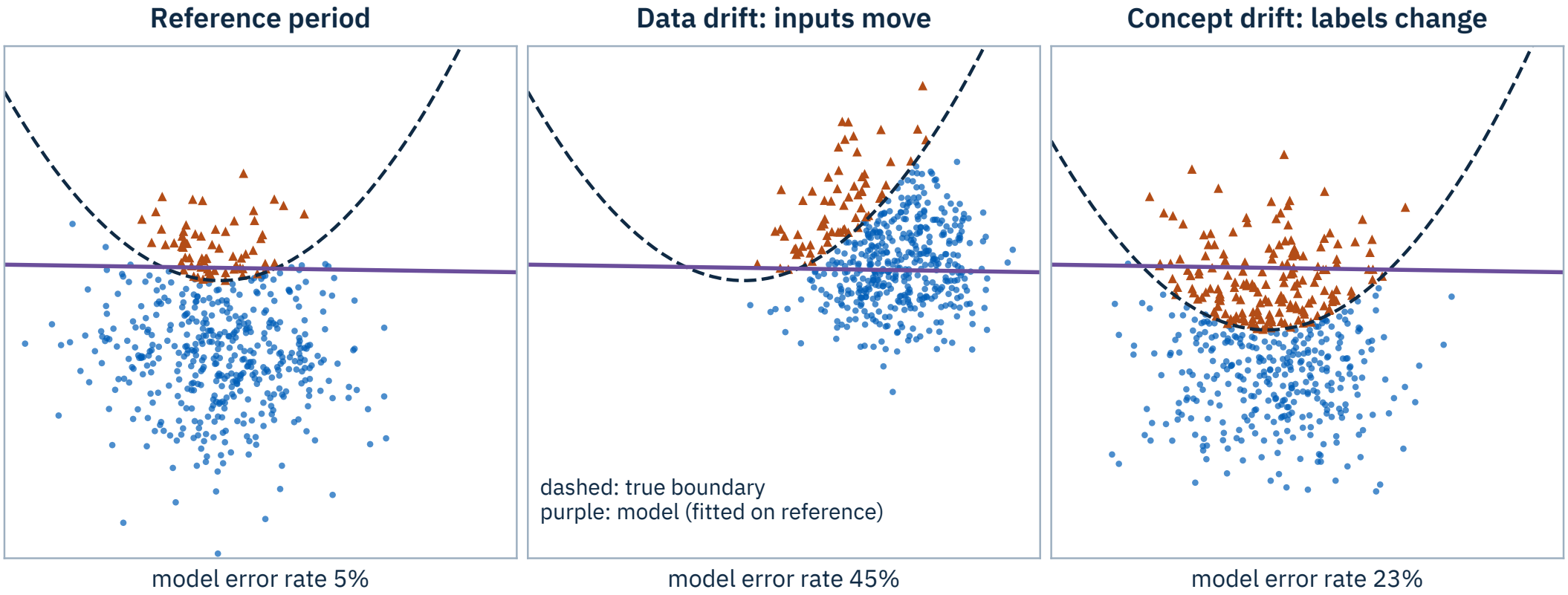
- Lagged rates, downtime indicators
- Covariates known at forecast time

Validation

- Test new features on the intended holdout
- Never carry windows across asset IDs

Data drift and concept drift need different responses

Data drift: P(x) moves. Concept drift: P(y | x) changes. Either breaks a validated model.

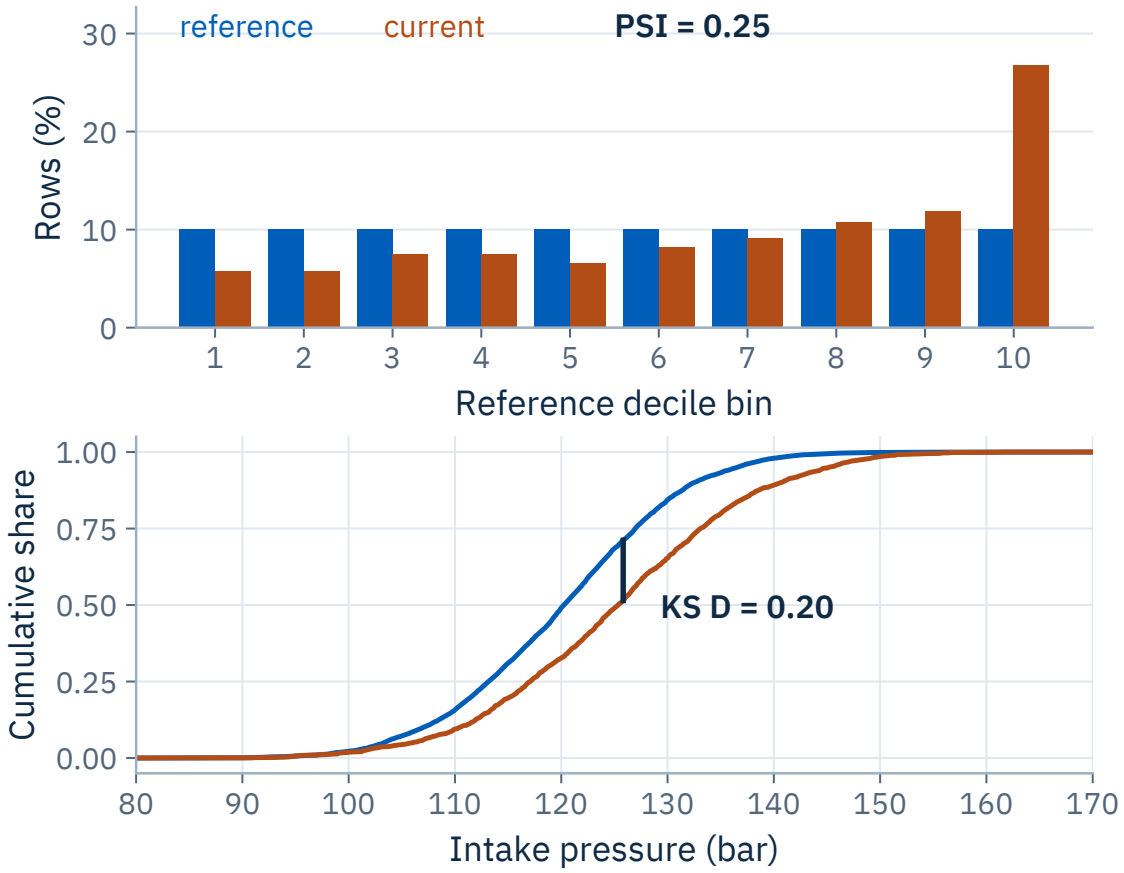


Synthetic data (seed 14): events when x2 > 0.25 x1² + c; logistic regression fitted on the reference period only. Error rates on 500 rows per panel.

PSI and KS quantify a shifted input

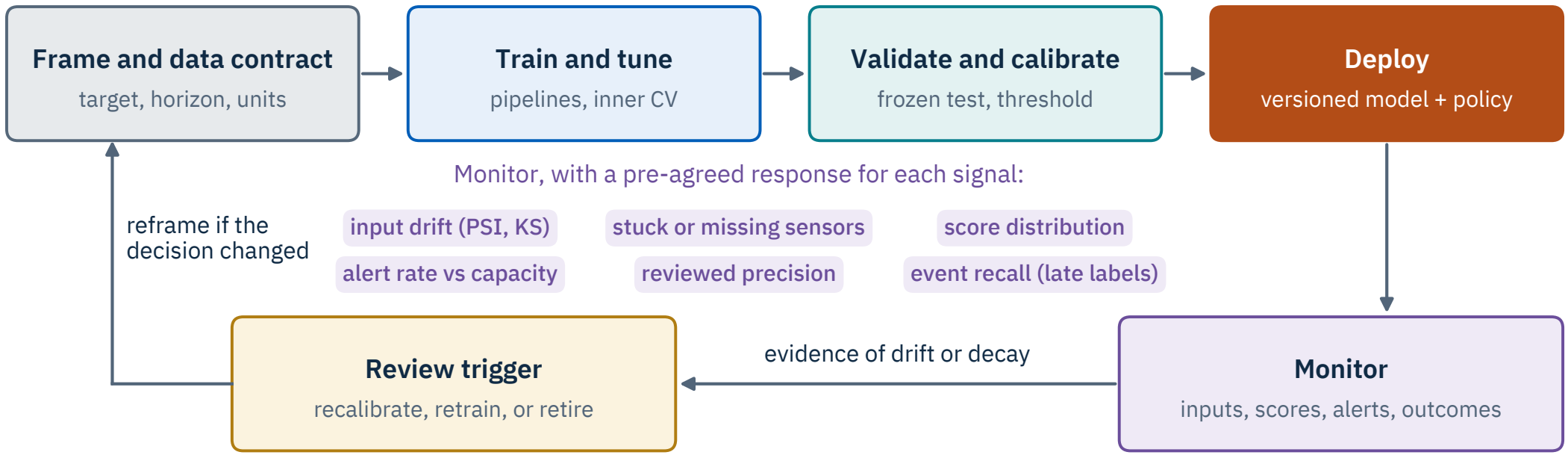
$$\mathrm{PSI} = \sum_i (a_i - e_i) \ln \frac{a_i}{e_i}$$

- e_i and a_i: reference and current shares in bin i
- Here PSI = 0.25 and KS D = 0.20 for a 5-bar shift
- With 1,500 rows the KS p-value is below 10^{-39}
- Set alert levels from PSI between past stable periods



Synthetic intake pressure (seed 15): reference N(120, 10), 5,000 rows; current N(125, 12), 1,500 rows.

Monitoring closes the model lifecycle loop



Schematic model lifecycle with a monitoring feedback loop.

A monitoring case is an experiment, not a promise

Proposed use	Baseline	Evaluation question
ESP monitoring	Rules or logistic classifier	Event recall at a manageable alert rate
Pipeline investigation	Flow-balance residual	False alerts per unit exposure
Well-log quality review	Physical ranges and robust scores	Expert-reviewed ranking quality
Wind turbine performance	Binned power-curve residual	Share of alerts confirmed on inspection
Solar inverter monitoring	Comparison with neighbouring inverters	Confirmed losses per review hour

◆ Each row is a hypothesis to test on your own data; no field performance is claimed.

PART 7 OF 7

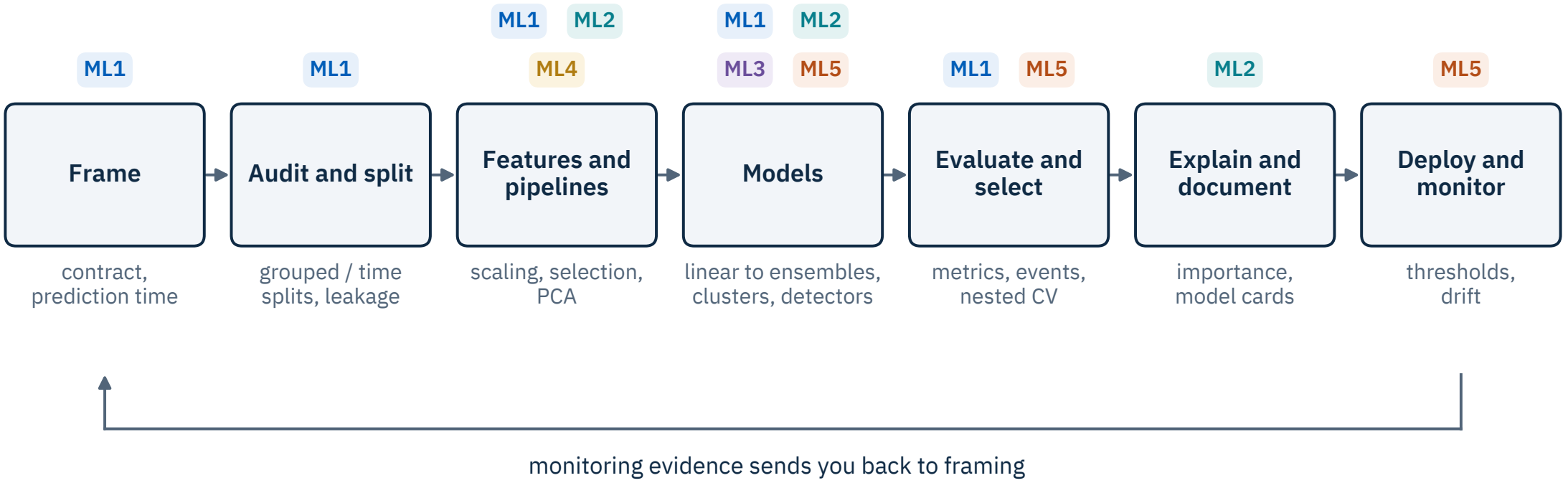
07

Course synthesis

Five modules, one workflow: from a question to a monitored decision.

- The end-to-end workflow and where each module fits
- A synthesis case
- A final review across Modules 1 to 5
- Key takeaways and the exit check

Five modules form one workflow



Schematic of the course workflow; chips show which module teaches each stage.

PRACTICE

4 min · Pairs

Course synthesis: plan a new well-data project

You receive a new well dataset and a request for a high-accuracy model.

1. State the first decision you must clarify.
2. Describe the baseline and the evaluation design.
3. Name the evidence needed before recommending deployment.

PRACTICE

5 min · Groups

Final review: find the flaw in each plan

1. Rows from 40 wells are split at random; the test R^2 is 0.93.
2. A scaler and PCA are fitted on all rows before cross-validation.
3. k-means runs on raw GR (API) and RHOB (g/cc).
4. SMOTE is applied before the train/test split; recall is 0.95.
5. The alert threshold is tuned on the test set that reports precision.

Key takeaways

- ✓ An anomaly score ranks rows; an alert policy turns ranks into decisions
- ✓ Robust baselines and physical rules come before complex detectors
- ✓ Evaluate rare events at the event level and at the real prevalence
- ✓ Rebalancing changes probabilities: recalibrate before using costs
- ✓ Tune inside the validation design; nested CV scores the procedure
- ✓ Monitor inputs, outputs, and outcomes, with the response agreed in advance

PRACTICE

3 min | Individual

Exit check

1. Why can a detector with 80% recall and a 5% false-positive rate produce mostly false alerts?
2. What does nested cross-validation estimate that a tuned CV score does not?
3. Name one monitoring signal and the action it should trigger.

References and further study

- Liu, Ting, Zhou (2008). Isolation Forest. IEEE International Conference on Data Mining.
- Breunig, Kriegel, Ng, Sander (2000). LOF: identifying density-based local outliers. ACM SIGMOD.
- Schölkopf et al. (2001). Estimating the support of a high-dimensional distribution. *Neural Computation* 13(7).
- Chandola, Banerjee, Kumar (2009). Anomaly detection: a survey. *ACM Computing Surveys* 41(3).
- Saito, Rehmsmeier (2015). The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLoS ONE* 10(3).
- Bergstra, Bengio (2012). Random search for hyper-parameter optimization. *JMLR* 13.
- Cawley, Talbot (2010). On over-fitting in model selection and subsequent selection bias in performance evaluation. *JMLR* 11.
- scikit-learn User Guide: outlier detection, calibration, tuning; imbalanced-learn: common pitfalls.